

YAMAHA ROBOT CONTROLLER

RCX series

Programming Manual

ENGLISH 

YAMAHA MOTOR CO., LTD.

IM Operations

882 Soude, Naka-ku, Hamamatsu, Shizuoka 435-0054.Japan
URL <http://www.yamaha-motor.jp/robot/index.html>

E76-Ver. 2.11

Introduction

Our sincere thanks for your purchase of this YAMAHA robot controller.

This manual describes robot program commands and related information for using YAMAHA RCX series robot controllers. Be sure to read this manual carefully as well as related manuals and comply with their instructions for using the YAMAHA robot controllers safely and correctly.

For details on how to operate YAMAHA robot controllers, refer to the separate controller user's manual that comes with the YAMAHA robot controller.

Applicable controllers: RCX240, RCX141, RCX142, RCX40, RCX221 and RCX222

Model names as used in this manual include the following controllers.

RCX240 ... Includes RCX240, RCX141, RCX142 and RCX40 (4-axis controllers)

RCX14x ... Includes RCX141, RCX142 and RCX40 (4-axis controllers excluding RCX240)*

RCX22x ... Includes RCX221 and RCX222 (2-axis controllers)

* Here, "RCX14x" does not include RCX240 and is used when there is a difference between the RCX240 and other 4-axis controllers due to differences in software versions.

Safety precautions (Be sure to read before using)

Before using the YAMAHA robot controller, be sure to read this manual and related manuals, and follow their instructions to use the robot controller safely and correctly. Warning and caution items listed in this manual relate to YAMAHA robot controllers. When this robot controller is used in a robot controller system, please take appropriate safety measures as required by the user's individual system.

This manual classifies safety caution items and operating points into the following levels, along with symbols for signal words "WARNING", "CAUTION" and "NOTE".



WARNING

"WARNING" indicates a potentially hazardous situation which, if not avoided, could result in death or serious injury.



CAUTION

"CAUTION" indicates a potentially hazardous situation which, if not avoided, could result in minor or moderate injury or damage to the equipment or software.



NOTE

Explains key points in the operation in a simple and clear manner.

Note that the items classified into "CAUTION" might result in serious injury depending on the situation or environmental conditions. So always comply with CAUTION and WARNING instructions since these are essential to maintain safety.

Keep this manual carefully so that the operator can refer to it when needed. Also make sure that this manual reaches the end user.

[System design precautions]



CAUTION

When the program execution stops before it is complete, the program re-executes the command that has stopped. Keep this point in mind when re-executing the program, for example, when using an arch motion with the MOVE command, a relative movement command such as the MOVEI or DRIVEI command, or a communication command such as the SEND command.

This manual does not constitute a concession of rights or a guarantee of industrial rights. Please acknowledge that we bear no liability whatsoever for conflicts with industrial rights arising from the contents of this manual.

2010 YAMAHA MOTOR CO., LTD.

Contents

1. The YAMAHA Robot Language	1
2. Characters	2
3. Program Names	3
4. Identifiers.....	7
5. Command Statement Format	8
6. Constants	9
6.1 Character constants	9
6.2 Numeric constants	9
6.2.1 Integer constants	9
6.2.2 Real constants	9
7. Variables	10
7.1 Valid range of variables	11
7.1.1 Valid range of dynamic variables	11
7.1.2 Valid range of static variables	11
7.1.3 Valid range of dynamic array variables	11
7.2 Character variables	12
7.3 Numeric variables	12
7.3.1 Integer variables	12
7.3.2 Real variables	12
7.4 Array variables	12
7.5 Clearing variables	13
7.5.1 Clearing dynamic variables	13
7.5.2 Clearing static variables	13
8. Other Variables	14
9. Expressions and Operations	24
9.1 Arithmetic operations	24
9.1.1 Arithmetic operators	24
9.1.2 Relational operators	24
9.1.3 Logic operations	25
9.1.4 Priority of arithmetic operation	25
9.1.5 Data format conversion	26
9.2 Character string operations	26
9.2.1 Character string connection	26
9.2.2 Character string comparison	26
9.3 Point data format	27
9.3.1 Joint coordinate format	27
9.3.2 Cartesian coordinate format	27
9.4 DI/DO conditional expressions	27
10. Multiple Robot Control	28
10.1 Overview	28
10.2 Command list for each group	29

11. Command Statements	30
ABSRST statement	30
ACCEL statement (Acceleration setting statement for main group)	31
ACCEL2 statement (Acceleration setting statement for sub group)	32
ARCH statement (Arch position setting statement for main group)	33
ARCH2 statement (Arch position setting statement for sub group)	34
ASPEED statement (Automatic movement speed setting statement for main group)	35
ASPEED2 statement (Automatic movement speed setting statement for sub group)	36
AXWGHT statement (Axis tip weight setting statement for main group)	37
AXWGHT2 statement (Axis tip weight setting statement for sub group)	38
CALL statement	39
CHGPRI statement	40
CUT statement	41
DECEL statement (Deceleration setting statement for main group)	42
DECEL2 statement (Deceleration setting statement for sub group)	43
DECLARE statement	44
DEF FN statement	46
DELAY statement	47
DIM statement (Array variable declaration statement)	48
DO statement (Parallel output)	49
DRIVE statement	50
DRIVE2 statement	55
DRIVEI statement	60
DRIVEI2 statement	63
EXIT FOR statement	66
EXIT SUB statement	67
EXIT TASK statement	68
FOR statement, NEXT statement	69
GOSUB statement, RETURN statement	70
GOTO statement	71
HALT statement	72
HAND definition statement, CHANGE statement (Main robot hand selection)	73
HAND2 definition statement, CHANGE2 statement (Sub robot hand selection)	77
HOLD statement	81
IF statement	82
INPUT statement	84
LET statement (Assignment statement)	85
LO statement (Arm lock output)	87
MO statement (Internal output)	88
MOVE statement (Absolute position movement command)	89

MOVE2 statement (Absolute position movement command)	98
MOVE1 statement (Relative position movement command)	107
MOVE12 statement (Relative position movement command)	110
ON ERROR GOTO statement	113
ON to GOTO statement	114
ON to GOSUB statement	115
ONLINE statement, OFFLINE statement	116
ORGORD statement (Return-to-origin sequence setting statement for main group)	117
ORGORD2 statement (Return-to-origin sequence setting statement for sub group)	118
ORIGIN statement	119
OUT statement	120
OUTPOS statement (OUT position setting statement for main group)	121
OUTPOS2 statement (OUT position setting statement for sub group)	122
PATH-related statements	123
What is a PATH function?	123
PATH statement (PATH-related statements)	125
PATH END statement (PATH-related statements)	129
PATH SET statement (PATH-related statements)	130
PATH START statement (PATH-related statements)	132
PDEF statement	133
PMOVE statement (Pallet movement command)	134
PMOVE2 statement (Pallet movement command)	138
PRINT statement	142
Pn (Point definition statement)	143
REM (Comment statement)	144
RESET statement	145
RESTART statement	146
RESUME statement	147
RIGHTY statement, LEFTY statement	148
RIGHTY2 statement, LEFTY2 statement	149
Sn (Shift coordinate definition statement)	150
SELECT CASE statement, END SELECT statement	151
SEND statement	152
SERVO statement	154
SERVO2 statement	155
SET statement	156
SHARED statement	157
SHIFT statement (Shift coordinate setting statement for main robot)	158
SHIFT2 statement (Shift coordinate setting statement for sub robot)	159
SO statement (Serial output)	160
SPEED statement (Speed setting statement for main group)	161
SPEED2 statement (Speed setting statement for sub group)	162

START statement	163
SUB statement, END SUB statement	164
SUSPEND statement	166
SWI statement	167
TO statement	168
TOLE statement (Tolerance setting statement for main group)	169
TOLE2 statement (Tolerance setting statement for sub group)	170
TORQUE statement	171
TORQUE2 statement	172
TRQTIME statement	173
TRQTIME2 statement	174
WAIT statement	175
WEIGHT statement (Tip weight parameter setting statement for main robot)	176
WEIGHT2 statement (Tip weight parameter setting statement for sub robot)	177
WHILE statement, WEND statement	178
Label statement	179
12. Functions	180
12.1 Arithmetic functions	180
12.2 Character string functions	199
12.3 Point functions	202
13. Multi-tasking	205
13.1 Outline	205
13.2 Task definition	205
13.3 Task status and transition	206
13.4 Starting tasks	207
13.5 Task scheduling	207
13.6 Condition wait in task	208
13.7 Suspending and restarting tasks	209
13.8 Deleting tasks	209
13.9 Stopping tasks	210
13.10 Multi-task program example	211
13.11 Sharing the data	212
14. Data file description	213
14.1 Program file	214
14.1.1 All programs	214
14.1.2 One program	215
14.2 Point file	216
14.2.1 All points	216
14.2.2 One point	217
14.3 Point comment file	218
14.3.1 All point comments	218
14.3.2 Individual point comment	219
14.4 Parameter file	220
14.4.1 All parameters	220
14.4.2 One parameter	222
14.5 Shift coordinate definition file	223
14.5.1 All shift data	223

14.5.2	One shift definition	224
14.6	Hand definition file	225
14.6.1	All hand data	225
14.6.2	One hand definition	226
14.7	Pallet definition file	227
14.7.1	All pallet definitions	227
14.7.2	One pallet definition	229
14.8	All file	230
14.8.1	All files	230
14.9	Program directory file	231
14.9.1	Entire program directory	231
14.9.2	One program	232
14.10	Parameter directory file	233
14.10.1	Entire parameter directory	233
14.11	Variable file	234
14.11.1	All variables	234
14.11.2	One variable	236
14.12	Constant file	237
14.12.1	One character string	237
14.13	Array variable file	238
14.13.1	All array variables	238
14.13.2	One array variable	240
14.14	DI file	241
14.14.1	All DI information	241
14.14.2	One DI port	242
14.15	DO file	243
14.15.1	All DO information	243
14.15.2	One DO port	244
14.16	MO file	245
14.16.1	All MO information	245
14.16.2	One MO port	246
14.17	LO file	247
14.17.1	All LO information	247
14.17.2	One LO port	248
14.18	TO file	249
14.18.1	All TO information	249
14.18.2	One TO port	250
14.19	SI file	251
14.19.1	All SI information	251
14.19.2	One SI port	252
14.20	SO file	253
14.20.1	All SO information	253
14.20.2	One SO port	254
14.21	Error message history file	255
14.21.1	All error message history	255
14.22	Machine reference file	256
14.22.1	All machine reference data	256
14.23	EOF file	257
14.23.1	EOF data	257
14.24	Serial port communication file	258
14.24.1	Serial port communication file	258
14.25	SIW file	259
14.25.1	All SIW	259
14.25.2	One SIW data	260

14.26	SOW file	261
14.26.1	All SOW	261
14.26.2	One SOW data	262
14.27	Ethernet port communication file	263
14.27.1	Ethernet port communication file	263
15.	User program examples	264
15.1	Basic operation	264
15.1.1	Directly writing point data in program	264
15.1.2	Using point numbers	265
15.1.3	Using shift coordinates	266
15.1.4	Palletizing	267
15.1.4.1	Utilizing the shift coordinates	267
15.1.4.2	Utilizing pallet movement	268
15.1.5	DI/DO (digital input and output) operation	270
15.2	Application	271
15.2.1	Pick and place between 2 points	271
15.2.2	Palletizing	273
15.2.3	Pick and place of stacked parts	275
15.2.4	Parts inspection (Multi-tasking example)	277
15.2.5	Sealing	280
15.2.6	Connection to an external device through RS-232C (example 1)	281
15.2.7	Connection to an external device through RS-232C (example 2)	282
16.	Sequence function	283
16.1	Creating a sequence program	283
16.1.1	Programming method	283
16.1.2	Compiling	284
16.2	Executing a sequence program	285
16.2.1	Sequence program STEP execution	286
16.3	Creating a sequence program	286
16.3.1	Assignment statements used with sequence program	286
16.3.2	Input/output variables used in sequence program	287
16.3.3	Timer definition statement	289
16.3.4	Logical operators used with sequence program	289
16.3.5	Priority of logic operations	290
17.	Online commands	291
17.1	Key operation	293
17.1.1	Changing the mode	293
17.1.2	AUTO mode operation	293
17.1.2.1	Program execution	293
17.1.2.2	Setting a break point	294
17.1.2.3	Switching the execution task	295
17.1.3	MANUAL mode operation	295
17.1.3.1	Changing the MANUAL mode speed	295
17.1.3.2	Absolute reset	296
17.1.3.3	Return-to-origin operation	296
17.1.3.4	Manual movement (inching)	297
17.1.3.5	Manual movement (jog)	297
17.1.3.6	Point data teaching	298
17.2	Utility operation	299
17.2.1	Acquiring the program execution status	299
17.2.2	Copy	299
17.2.2.1	Copying a program	299
17.2.2.2	Copying point data	300
17.2.2.3	Copying point comments	300
17.2.3	Erase	301
17.2.3.1	Erasing a program	301
17.2.3.2	Erasing point data	301

17.2.3.3	Erasing point comments	302
17.2.3.4	Erasing pallet data	302
17.2.4	Rename	303
17.2.5	Changing the attribute	303
17.2.6	Initialize	304
17.2.6.1	Initializing the memory	304
17.2.6.2	Initializing the communication port	305
17.2.6.3	Initializing the error log	305
17.2.7	Setting the display language	305
17.2.8	Setting the coordinates and units in MANUAL mode	306
17.2.9	Clearing the MPB/RPB error message	306
17.2.10	Setting the UTILITY mode	307
17.2.10.1	Setting the access level	307
17.2.10.2	Setting the execution level	307
17.2.10.3	Setting the sequence program execution flag	308
17.2.10.4	Setting the SCARA robot hand system	308
17.2.10.5	Resetting the internal emergency stop flag	309
17.2.11	Checking and setting the date	309
17.2.12	Checking and setting the time	310
17.3	Data handling	311
17.3.1	Acquiring the display language	311
17.3.2	Acquiring the access level	311
17.3.3	Acquiring the arm status	312
17.3.4	Acquiring the break point status	312
17.3.5	Acquiring the controller configuration status	313
17.3.6	Acquiring the execution level	313
17.3.7	Acquiring the mode status	314
17.3.8	Acquiring the message	314
17.3.9	Acquiring return-to-origin status	315
17.3.10	Acquiring the absolute reset status	316
17.3.11	Acquiring the servo status	316
17.3.12	Acquiring the sequence program execution status	317
17.3.13	Acquiring the speed setting status	317
17.3.14	Acquiring the point coordinates and units	318
17.3.15	Acquiring the version information	318
17.3.16	Acquiring the current positions	319
17.3.16.1	Acquiring the current positions on pulse unit coordinates	319
17.3.16.2	Acquiring the current positions on XY coordinates	319
17.3.17	Acquiring the tasks in RUN or SUSPEND status	320
17.3.18	Acquiring the tasks operation status	320
17.3.19	Acquiring the shift status	321
17.3.20	Acquiring the hand status	321
17.3.21	Acquiring the remaining memory capacity	322
17.3.22	Acquiring the emergency stop status	322
17.3.23	Acquiring the error status by self-diagnosis	323
17.3.24	Acquiring the option slot status	324
17.3.25	Acquiring various values	325
17.3.25.1	Acquiring the value of a numerical expression	325
17.3.25.2	Acquiring the value of a character string expression	325
17.3.25.3	Acquiring the value of a point expression	326
17.3.25.4	Acquiring the value of a shift expression	326
17.3.26	Data readout processing	327
17.3.27	Data write processing	328
17.4	Executing the robot language independently	329
17.4.1	Switching the program	329
17.4.2	Other robot language command processing	330
17.5	Control codes	330
17.5.1	Interrupting the command execution	330

18. IO commands	331
18.1 IO command format	331
18.2 Sending and receiving IO commands	332
18.3 IO command list	333
18.4 IO command description	334
18.4.1 MOVE command	334
18.4.1.1 PTP designation	334
18.4.1.2 Linear interpolation	334
18.4.2 MOVEI command	335
18.4.2.1 PTP designation	335
18.4.3 Pallet movement command	335
18.4.3.1 PTP designation	335
18.4.4 Jog movement command	336
18.4.5 Inching movement command	336
18.4.6 Point teaching command	337
18.4.7 Absolute reset movement command	337
18.4.8 Absolute reset command	337
18.4.9 Return-to-origin command	338
18.4.10 Servo command	338
18.4.11 Manual movement speed change command	339
18.4.12 Auto movement speed change command	339
18.4.13 Program speed change command	339
18.4.14 Shift designation change command	340
18.4.15 Hand designation change command	340
18.4.16 Arm designation change command	340
18.4.17 Point display unit designation command	340
Appendix	341
A. Reserved word list	341
B. Robot language lists	342

1. The YAMAHA Robot Language

The YAMAHA robot language was developed by Yamaha Motor Co., Ltd. IM Company for simple and efficient programming to control YAMAHA industrial robots. The YAMAHA robot language is similar to BASIC (Beginner's All-purpose Symbolic Instruction Code) and makes even complex robot movements easy to program. This manual explains how to write robot control programs with the YAMAHA robot language, including actual examples on how its commands are used.

2. Characters

The YAMAHA robot language uses the following characters and symbols.

Alphabetic characters

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z

Numbers

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Symbols

() [] + - * / ^ = < > & | ~ _ % ! # \$: ; , . " ' @ ?



NOTE

*Katakana (Japanese phonetic characters) cannot be entered from a programming box (MPB/RPB).
Katakana can be used when communicating with a host computer (if it handles katakana).*

katakana (Japanese phonetic characters)

3. Program Names

Each program to be created in the robot controller must have its own name. The same program name cannot be given to other programs. Program names can be up to 8 characters consisting of a combination of alphanumeric characters and underscores (_).

Functions and examples of program names having special meaning are shown below.

- a) FUNCTION
- b) SEQUENCE
- c) _SELECT
- d) COMMON

a) FUNCTION

Functions:

Pressing the **USER** key in “PROGRAM” mode or “MANUAL” mode allows the user function to be used.

When used in “PROGRAM” mode, commands (MOVE, GOTO, etc.) frequently used during program editing can be entered with the function keys.

When used in “MANUAL” mode, DO output is available with the function keys without running the program.

Program example:

```
' FOR MANUAL MODE
* M_F1:'DO(20)ALTERNATE
  DO(20)=~DO(20)
* M_F2:'DO(21)ALTERNATE
  DO(21)=~DO(21)
:
* M_F6:'DO(25)MOMENTARY
  DO(25)=1
  DO(25)=0
:
' FOR PROGRAM MODE
* P_F1:'MOVE P,
* P_F6:'MOVE L,
* P_F2:'GOTO *
```

For more details, see the YAMAHA robot controller user's manual.

3. Program Names

b) SEQUENCE

Functions:

As distinct from the robot programs, this program processes the robot input/output (DI, DO, MO, LO, TO, SI, SO) signals in fixed cycles. The cycle is determined by the program capacity.

Using this function allows the controller to operate as if it had a built-in PLC (programmable logic controller).

Program example:

```
DO(20)=~DO(20)
DO(25)=DI(21) AND DI(22)
MO(26)=DO(26) OR DO(25)
:
```

For more details, see "16. Sequence function" in this manual.

c) _SELECT

Functions:

The system will always select a certain program if available when the robot program is reset. This function is for selecting a program by DI input and that program is then always selected when reset.

Differences in processing by each type of reset:

- When reset from the teaching pendant, the system awaits a response to a query to switch the program to "_SELECT".
- When reset by the HALT command in a program, dedicated DI (reset signal) or online command, the system switches to the "_SELECT" program.
- When the execution level is selected so that the program is reset at power ON, the program resets when power is turned on and then switches to the "_SELECT" program.

Program example:

A program is selected according to the value input from DI3().

When DI3() is 0, the system repeatedly monitors the DI input.

When DI3() is from 1 to 3, the matching program is selected.

When DI3() is other than the above cases, the system quits the program that is currently running.

```
ON ERROR GOTO *ER1
*ST:
  SELECT CASE DI3( )
    CASE 0
      GOTO *ST
    CASE 1
      SWI <PART1>
    CASE 2
      SWI <PART2>
    CASE 3
      SWI <PART3>
    CASE ELSE
      GOTO *FIN
  END SELECT
  GOTO *ST
*FIN:
  HALT
*ER1:
  IF ERR=&H0303 THEN *NEXT_L
  ON ERROR GOTO 0
*NEXT_L:
  RESUME NEXT
```



NOTE

- Using an ON ERROR statement allows running the program in a loop not ending in an error even without the program name specified by a SWI statement.
- An error code issued during execution of the program is input into a variable ERR. "ERR=&0303" means "Program doesn't exist".

See the description of each command in this manual for details.

3. Program Names

d) COMMON

Functions:

Using two or more robot programs for the same processing is usually a waste of programming area. So the COMMON program can be used to perform the same task in multiple robot programs.

Program examples:

```
Program name: SAMPLE1
  DECLARE SUB *DISTANCE(A!,B!,C!)
  DECLARE *AREA
  X!=2.5
  Y!=1.2
  CALL *DISTANCE(X!,Y!,REF C!)
  GOSUB *AREA
  PRINT C!,Z!
  HALT
```

```
Program name: SAMPLE2
  DECLARE SUB *DISTANCE(A!,B!,C!)
  DECLARE *AREA
  X!=5.5
  Y!=0.2
  CALL *DISTANCE(X!,Y!,REF C!)
  GOSUB *AREA
  PRINT C!,Z!
  HALT
```

```
Program name: COMMON
  SUB *DISTANCE(A!,B!,C!)
    C!=SQR(A!^2+B!^2)
  END SUB
  *AREA:
    Z!=X!*Y!
  RETURN
```

See the description of each command in this manual for details.

4. Identifiers

The groups of characters used to express the names of labels, variables, procedures and so on are referred to as “identifiers.”

Identifier length can be up to 16 characters composed of a combination of alphanumeric characters and underscores (_). Identifiers must begin with an alphabetic character. In the case of label names, the character following an asterisk (*) can be a number. If an identifier exceeds 16 characters, the characters from the 17th on are ignored and deleted. A maximum of 500 identifiers can be used.

Examples: LOOP, SUBROUTINE, GET_DATA

5. Command Statement Format

One robot language command must be written on a single line within 75 characters and arranged in the format shown below:



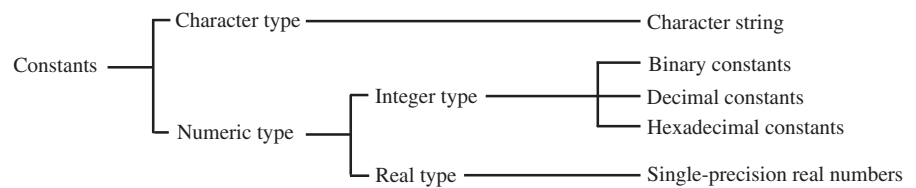
NOTE

- Items enclosed in [] can be omitted.
- Items enclosed in < > must be written in a specific format.
- Items not enclosed in < > should be written directly as shown.
- Items surrounded by | | are selectable.
- Labels can be omitted from the command. All labels must begin with an asterisk (*) and end with a colon (:).
- Operands may be unnecessary for some commands.
- Programs are executed in order from top to bottom unless a branching instruction is given.

```
[<label>:] <statement> [<operand>]
```

6. Constants

Constants are basically classified as follows:



6.1 Character constants

Character constants are character string data of up to 75 bytes surrounded by double quotation marks ("). Character strings may include upper case alphabetic characters, numbers, symbols and katakana (Japanese phonetic characters). To include a double quotation mark in a string, enter two double quotation marks in succession.

Examples:

```
"YAMAHA ROBOT"  
"EXAMPLE OF""A""  
PRINT "COMPLETED"  
"YAMAHA ROBOT"
```

6.2 Numeric constants

6.2.1 Integer constants

1. Decimal constants

Integers from -1,073,741,824 to 1,073,741,823 may be used.

2. Binary constants

Unsigned binary numbers of 8 bits or less may be used. The prefix "&B" is attached to the number to define it as a binary number.

&B0 (0) to &B11111111 (255)

3. Hexadecimal constants

Signed hexadecimal numbers of 32 bits or less may be used. The prefix "&H" is attached to the number to define it as a hexadecimal number.

&H80000000 (-2,147,483,648) to &H7FFFFFFF (2,147,483,647)

6.2.2 Real constants

1. Single-precision real numbers

Real numbers from -999999.9 to +999999.9 may be used (7 digits including integers and decimals). For example, ".0000001" may be used.

2. Single-precision real numbers in exponent form

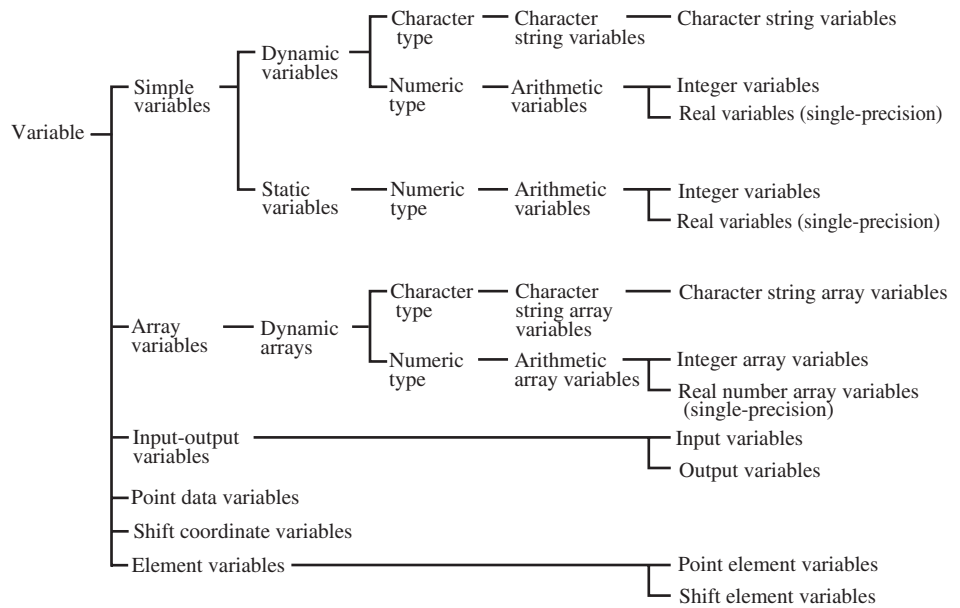
Numbers from -1.0×10^{38} to $+1.0 \times 10^{38}$ may be used. Mantissas should be 7 digits or less, including integers and decimals.

Examples:

```
-1.23456E-12  
3.14E0  
1.E5
```

7. Variables

Variables are classified as follows:



Variables with the same names as reserved words and variables starting with FN, DIn, DOn, MOn, LOn, TOn, SIn, SOOn, Pn, Sn or Hn (n=numerical value) cannot be used.

Examples:

```

COUNT .....permitted
ABS .....not permitted
FNAME .....not permitted
S91 .....not permitted
  
```

Character spelling (identifier) used for variables must begin with an alphabetic character.

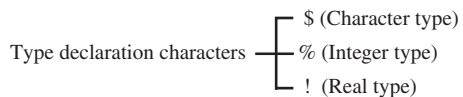
Examples:

```

COUNT .....permitted
COUNT123 .....permitted
2COUNT .....not permitted
  
```

For details on identifiers used for variables, refer to "4. Identifiers".

The type of variable is specified by the type declaration character attached at the end of the variable name. If no type declaration character is attached, the variable is viewed as a real type.



Examples:

```

CNT0% ..... Integer variable
CNT1 ..... Real variable
STR1$ ..... Character variable
ACT%(1) ..... Integer array
  
```

Variables using the same identifier are recognized to be different from each other by the type of each variable.

Examples:

```

ASP_DEF%..... Integer variable } ASP_DEF% and ASP_DEF are different variables.
ASP_DEF..... Real variable

ASP_DEF!..... Real variable } ASP_DEF! and ASP_DEF are the same variables.
ASP_DEF..... Real variable
  
```

Names of static variables are predetermined as follows:

Integer type S G I n (n: 0 to 7)

Real type S G R n (n: 0 to 7)

7.1 Valid range of variables

7.1.1 Valid range of dynamic variables

Dynamic variables are classified into global variables and local variables according to their declaration position in the program.

- Dynamic global variables are declared outside of sub-procedures (outside of program areas enclosed by a SUB statement and END SUB statement).
- Dynamic local variables are declared within sub-procedures and are valid only in these sub-procedures.

7.1.2 Valid range of static variables

Static variables are always valid in the entire program regardless of program statements.

7.1.3 Valid range of dynamic array variables

Dynamic array variables are classified into global array variables and local array variables according to their declaration position in the program.

- Dynamic global array variables are declared outside of sub-procedures (outside of program areas enclosed by a SUB statement and END SUB statement).
- Dynamic local array variables are declared within sub-procedures and are valid only in these sub-procedures.

An array variable can express multiple elements. The elements of an array can be integers or subscript expressions following the variable name (see below). The length of an array variable is defined by the DIM statement. The actual number of array elements will be the DIM statement subscript number plus 1, as subscripts begin with 0.

Format :

$\langle \text{variable name} \rangle \left[\begin{array}{c} \% \\ ! \\ \$ \end{array} \right] (\langle \text{expression} \rangle, [\langle \text{expression} \rangle, \langle \text{expression} \rangle])$
--

Examples:

C%(2) Integer variable

N!(1,2) Real variable

R1\$(A) Character variable

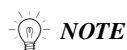
The length of an array variable that can be declared with the DIM statement depends on the program size.

7.2 Character variables

Character variables and character array elements can handle a character string of up to 75 characters. Character strings may include alphabetic characters, numbers, symbols and katakana (Japanese phonetic characters).

Examples:

```
R1$      = "YAMAHA"
R2$(2)   = R1$ + " MOTOR"
```



NOTE

Integers from -1,073,741,824 to 1,073,741,823 can be expressed in signed hexadecimal numbers from &HC0000000 to &H3FFFFFFF.

7.3 Numeric variables

7.3.1 Integer variables

Integer variables and integer array elements can handle an integer from -1,073,741,824 to 1,073,741,823.

Examples:

```
R1%      = 10
R2%(2)   = R1% + 10000
```

7.3.2 Real variables

Real variables and real array elements can handle a real number from -1.0×10^{38} to 1.0×10^{38} .

Examples:

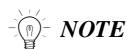
```
R1!      = 10.31
R2!(2)= R1% + 1.98E3
```

7.4 Array variables

An array variable can be used to reference multiple elements. Each element in an array is referenced by its index or subscript in <expression> (see below). The length of an array variable is declared by using the DIM statement. The actual number of elements will be the number of the DIM statement subscripts plus 1, as subscripts begin with 0. The subscripts can be used in up to three dimensions. All arrays are dynamic variables.

Format :

<variable name> [	%	]	(<expression>, [<expression>, <expression>])
	!		
	\$		



NOTE

The length of an array variable that can be declared with the DIM statement depends on the program size.

Examples:

```
A%(1) ..... Integer array variable
DATA!(1,10,3) ..... Single-precision real number array variable
STRING$(10) ..... Character array variable
```


7.5 Clearing variables

7.5.1 Clearing dynamic variables

In the cases below, numeric variables are cleared to zero, and character variables are cleared to a null string. Array variables are cleared in the same way.

- When the program was edited in PROGRAM mode.
- When the program was switched.
- When compiling was performed in PROGRAM mode.
- When the program was compiled in AUTO mode.
- When the program was reset in AUTO mode.
- When dedicated input signal DI15 (program reset input) was turned on while the program was stopped in AUTO mode.
- When either of the following was initialized in SYSTEM mode.
 1. Program memory (SYSTEM>INIT>MEMORY>PROGRAM)
 2. Entire memory (SYSTEM>INIT>MEMORY>ALL)
- When the SWI command was executed in AUTO mode.
- When any of the following online commands was executed.
@RESET, @INIT PGM, @INIT MEM, @INIT ALL, @SWI
- When the SWI statement was executed in the program.
- When the HALT statement was executed in the program.



CAUTION

Definitions of dynamic variables are cleared when the program was edited in PROGRAM mode.

7.5.2 Clearing static variables

In the cases below, integer variables and real variables are cleared to zero.

- When the following was initialized in SYSTEM mode.
Entire memory (SYSTEM>INIT>MEMORY>ALL)
- When any of the following online commands was executed.
@INIT MEM, @INIT ALL

8. Other Variables



CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified with point variables.

1. Point data variable

This variable specifies a point data number with a numeric constant or expression. A point data number is expressed with a “P” followed by a number of 4 digits or less, or an expression surrounded by brackets ([]). Point numbers from 0 to 9999 can be specified with point variables.

Format :

Pnnnn or P ”[”<expression>”]”

n = 0 to 9

Each bracket in quotation marks (" ") must be written. (Brackets are not used to indicate an item that may be omitted.)

Examples:

P0,P110
P[A],P[START_POINT],P[A(10)]

2. Shift coordinate variable

This variable specifies a shift coordinate number with a numeric constant or expression. A shift coordinate number is expressed with an “S” followed by a 1-digit number or an expression surrounded by brackets ([]).

Format :

Sn or S ”[”<expression>”]”

n = 0 to 9

Each bracket in quotation marks (" ") must be written. (Brackets are not used to indicate an item that may be omitted.)

Examples:

S1
S[A],S[BASE],S[A(10)]

3. Point element variable

This variable handles point data for each axis and hand system flag.

Format :

LOCx (<point expression>)

x : X, Y, Z, R, A, B (axis definition), F (hand system flag definition)

Examples:

A(1)=LOCX(P10)
----- The X-axis data of P10 is assigned to array variable A(1).
LOCZ(P[A])=100.0
----- The Z-axis data of P[A] is set to 100.0.
LOCF(P100)=1
----- Sets the P100 hand system flag to RIGHTY (a right-handed system).
(The P100 point data must be in "mm" units.)



CAUTION

Hand system flags are only available from software version 8.08 onwards.



NOTE

*The hand system flag is enabled when the point data unit is set to the "mm" units.
Note that the hand system flag is enabled only for the SCARA robot.
The hand system flag value may be 0 (no designation), 1 (right-handed system) or 2 (left-handed system).*

4. Shift element variable

This variable is used with shift data for each element.

Format :

LOCx (<shift expression>)

x : X, Y, Z, R

Examples:

A(1)=LOCX(S1)

----- The X data of S1 is assigned to array variable A(1).

LOCX(S[A])=45.0

----- The R data of S[A] is set to 45.0°.

5. Parallel input variable

This variable is used to indicate the status of parallel input signals.

Format 1:

DIm ([b, . . . , b])

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

DI (mb, . . . , mb)

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

Examples:

A%=DI1()

----- Input status of ports DI(17) to DI(10) is assigned to variable A%.

A%=DI5(7,4,0)

----- Input status of DI(57), DI(54) and DI(50) is assigned to variable A%.

(If all above signals are 1(ON), then A%=7.)

A%=DI(27,15,10)

----- Input status of DI(27), DI(15) and DI(10) is assigned to variable A%.

(If all above signals except DI(10) are 1 (ON), then A%=6.)

**NOTE**

Bits must be specified in ascending order from the right.

A "0" is entered if there is no actual input board.

6. Parallel output variable

This variable is used to specify the parallel output signals and indicate the output status.

Format 1:

DOm ([b, . . . , b])

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

DO (mb, . . . , mb)

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7



NOTE

Bits must be specified in ascending order from the right.
External output is unavailable if there is no output board.

Examples:

A%=DO2()

----- Output status of DO(27) to DO(20) is assigned to variable A%.

A%=DO5(7,4,0)

----- Output status of DO(57), DO(54) and DO(50) is assigned to variable A%.

(If all above signals are 1(ON), then A%=7.)

A%=DO(37,25,20)

----- Output status of DO(37), DO(25) and DO(20) is assigned to variable A%.

(If all above signals except DO(20) are 1 (ON), then A%=6.)

7. Internal output variable

This variable is used to exchange signals with a sequence program.

The contents of this variable can be changed and referred to as needed.

Ports 0 and 1 are used for dedicated internal output variables that can only be referred to.

- 1) Port 0 indicates the status of origin sensors for axes 1 to 8 (in order from bit 0).

Each bit sets to “1” when the origin sensor turns ON, and to “0” when OFF.

- 2) Port 1 indicates the HOLD status of axes 1 to 8 (in order from bit 0).

Each bit sets to “1” when the axis is in HOLD status, and to “0” when not.

Being in HOLD status means that the axis movement is stopped and positioned within the target point tolerance while the servo is still turned ON.

When the servo turns OFF, the HOLD status is released.

Axes not being used are set to “1”.

Format 1:

MOm ([b, . . . , b])

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

MO (mb, . . . , mb)

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

Examples:

A=MO2 ()

----- Internal output status of MO(27) to MO(20) is assigned to variable A.

A=MO5(7,4,0)

----- Internal output status of MO(57), MO(54) and MO(50) is assigned to variable A.
(If all above signals are 1 (ON), then A=7.)

A=MO(37,25,20)

----- Internal output status of MO(37), MO(25) and MO(20) is assigned to variable A.
(If all above signals except MO(25) are 1 (ON), then A=5.)



NOTE

Bits must be specified in ascending order from the right.

8. Arm lock output variable

This variable is used to prohibit axis movement.

The contents of this variable can be output and referred to as needed.

There is only 1 port, and bits 0 to 7 respectively correspond to axes 1 to 8.

When this variable is ON, movement on the corresponding axis is prohibited.

Format 1:

LOm ([b, ···, b])

m : port number 0

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

LO (mb, ···, mb)

m : port number 0

b : bit definition 0 to 7

Examples:

A%=LO0()

----- Arm lock status of LO(07) to LO(00) is assigned to variable A%.

A%=LO0(7,4,0)

----- Arm lock status of LO(07), LO(04) and LO(00) is assigned to variable A%.

(If all above signals are 1 (ON), then A%=7.)

A%=LO0(06,04,01)

----- Arm lock status of LO(06), LO(04) and LO(01) is assigned to variable A%.

(If all above signals except LO(01) are 1 (ON), then A%=6.)



NOTE

Bits must be specified in ascending order from the right.

9. Timer output variable

This variable is used in the timer function of a sequence program.

The contents of this variable can be changed and referred to as needed.

Timer function can be used only in the sequence program. If this variable is output in a normal program, it is an internal output like the MO variable.

Format 1:

```
TOm ( [ b, . . . , b ] )
```

m : port number 0

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

```
TO ( mb, . . . , mb )
```

m : port number 0

b : bit definition 0 to 7

Examples:

A%=TO0()

----- Status of TO(07) to TO(00) is assigned to variable A%.

A%=TO0(7,4,0)

----- Status of TO(07), TO(04) and TO(00) is assigned to variable A%.

(If all above signals are 1 (ON), then A%=7.)

A%=TO(06,04,01)

----- Status of TO(06), TO(04) and TO(01) is assigned to variable A%.

(If all above signals except TO(01) are 1 (ON), then A%=6.)

**NOTE**

Bits must be specified in ascending order from the right.

10. Serial input variable

This variable is used to indicate the status of serial input signals.

Format 1:

```
SIm ( [ b, . . . , b ] )
```

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

```
SI ( mb, . . . , mb )
```

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

Examples:

A%=SI1()

----- Input status of ports SI(17) to SI(10) is assigned to variable A%.

A%=SI5(7,4,0)

----- Input status of SI(57), SI(54) and SI(50) is assigned to variable A%.

(If all above signals are 1(ON), then A%=7.)

A%=SI(27,15,10)

----- Input status of SI(27), SI(15) and SI(10) is assigned to variable A%.

(If all above signals except SI(10) are 1 (ON), then A%=6.)

WAIT SI(21)=1

----- Waits until SI(21) sets to 1 (ON).



NOTE

Bits must be specified in ascending order from the right.

A "0" is entered if there is no actual serial board.

11. Serial output variable

This variable is used to define the serial output signals and indicate the output status.

Format 1:

`SOM ( [ b, ..., b ] )`

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

If the bit definition is omitted, bits 0 to 7 are all selected.

Format 2:

`SO ( mb, ..., mb )`

m : port number 0 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

**NOTE**

Bits must be specified in ascending order from the right.
External output is unavailable if there is no serial board.

Examples:

`A%=SO2()`

----- Output status of SO(27) to SO(20) is assigned to variable A%.

`A%=SO5(7,4,0)`

----- Output status of SO(57), SO(54) and SO(50) is assigned to variable A%.

(If all above signals are 1(ON), then A%=7.)

`A%=SO(37,25,20)`

----- Output status of SO(37), SO(25) and SO(20) is assigned to variable A%.

(If all above signals except SO(25) are 1 (ON), then A%=5.)

8. Other Variables



CAUTION

Serial word input is only available from software version 8.08 onwards.



NOTE

*The information is handled as unsigned word data.
0 is input if the serial board does not actually exist.*



CAUTION

Serial double word input is only available from software version 8.08 onwards.



NOTE

*The information is handled as signed double word data.
0 is input if the serial board does not actually exist.
An error will occur if the value is not within the acquisition range (&H80000000 to &HBFFFFFFF, &H40000000 to &H7FFFFFFF.)*

12. Serial word input

This variable indicates the status of the serial input word information.

Format 1:

SIW (m)

m : Port No. 2 to 15

The acquisition range is 0 (&H0000) to 65535 (&HFFFF).

Example:

A%=SIW (2)

----- The input state from SIW (2) is assigned to variable A%.

A%=SIW (15)

----- The input state from SIW (15) is assigned to variable A%.

13. Serial double word input

This variable indicates the state of the serial input word information as a double word.

Format 1:

SID (m)

m : Port No. 2, 4, 6, 8, 10, 12, 14

The acquisition range is -1073741824 (&HC00000000) to 1073741823 (&H3FFFFFFF).

Example:

A%=SID (2)

----- The input state from SIW (2) , SIW (3) is assigned to variable A%.

A%=SID (14)

----- The input state from SIW (14), SIW (15) is assigned to variable A%.

 CAUTION

Serial word output is only available from software version 8.08 onwards.

 NOTE

The information is handled as unsigned word data.

If a serial board does not actually exist, the information is not output externally.

If a value exceeding the output range is assigned, the low-order 2-byte information is output.

14. Serial word output

This variable outputs the serial output word information, and indicates the output status.

Format 1:

SOW (m)

m : Port No. 2 to 15

The output range is 0 (&H0000) to 65535 (&HFFFF).

Note that if a negative value is output, the low-order word information will be output after being converted to hexadecimal.

Example:

A%=SOW (2)

----- The output status from SOW (2) is assigned to variable A%.

SOW (15)=A%

----- The contents of variable A% are assigned in SOW (15).
If the variable A% value exceeds the output range, the low-order word information will be assigned.

SOW (15)=-255

----- The contents of -255 (&HFFFFFF01) are assigned to SOW (15).

-255 is a negative value, so the low-order word information (&HFF01) will be assigned.

 CAUTION

Serial double word output is only available from software version 8.08 onwards.

 NOTE

The information is handled as signed double word data.

If a serial board does not actually exist, the information is not output externally.

An error will occur if the value is not within the output range (&H80000000 to &HBFFFFFFF, &H40000000 to &H7FFFFFFF.)

15. Serial double word output

This variable outputs the serial output word information status as a double word, and indicates the output status.

Format 1:

SOD (m)

m : Port No. 2, 4, 6, 8, 10, 12, 14

The output range is -1073741824 (&HC0000000) to 1073741823 (&H3FFFFFFF).

Note that if a negative value is output, the low-order word information will be output after being converted to hexadecimal.

Example:

A%=SOD (2)

----- The input status from SOW (2), SOW (3) is assigned to variable A%.

SOD (14)=A%

----- The contents of variable A% are assigned in SOD (14).

9. Expressions and Operations

9.1 Arithmetic operations

9.1.1 Arithmetic operators

$\wedge$	Exponent operation
$-$	Minus sign
$*, /$	Multiplication and division
$+, -$	Addition and subtraction
MOD	Remainder

When the values used in remainder calculations are real numbers, they are converted into integers (all decimal fractions are truncated) and the calculation is then performed with the integers. The result is the remainder of a division operation using the integers.

Examples:

```
A=15 MOD 2
----- This becomes: A=1 (15/2 = 7 .... 1)
A=17.34 MOD 5.98
----- This becomes: A=2 (17/5 = 3 .... 2)
```

9.1.2 Relational operators

$=$	Equal to
$<>, ><$	Not equal to
$<$	Less than
$>$	Greater than
$<=, <=$	Less than or equal to
$>=, >=$	Greater than or equal to

Relational operators are used to compare 2 values. If the result is true, a “-1” is obtained. If it is false, a “0” is obtained.

Example:

```
A=10>5
----- Since 10 > 5 is true, A = -1.
```



NOTE

The expected result might not always be maintained if equivalent relational operators were used with real variables or real array variables.

Examples:

```
A = 2
B = SQR (A!)
IF A! = B!*B ! THEN
:
```

In this case, A! will be unequal to B!*B!.

9.1.3 Logic operations

NOT, ~	Logical NOT
AND, &	Logical AND
OR,	Logical OR
XOR	Exclusive OR

Logic operators are used to manipulate 1 or 2 values bit by bit. For example, the status of an I/O port can be manipulated. Depending on the logic operation performed, the results generated are either “0” or “1”. Logic operations with real numbers convert the values into integers before they are executed.

Examples:

A%=NOT 13.05

----- Each bit of 13 is reversed, and the result “-14” is assigned to A%.

A%=3 AND 10

----- The logical product of 3 and 10 is calculated (“1” is obtained when both bits are “1”), and the result is assigned to A%, so A% becomes 2.

A%=3 OR 10

----- The logical sum of 3 and 10 is calculated (“1” is obtained when either bit is “1”), and the result is assigned to A%, so A% becomes 11.

A%=3 XOR 10

----- The exclusive OR of 3 and 10 is calculated (“1” is obtained when both bits are different from each other), and the result is assigned to A%, so A% becomes 9.

9.1.4 Priority of arithmetic operation

1. Expressions included in parentheses
2. Functions, variables
3. ^ (exponents)
4. Independent “+” and “-” signs (monominal operators)
5. * (multiplication), / (division)
6. MOD
7. + (addition), - (subtraction)
8. Relational operators
9. NOT, ~ (Logical NOT)
10. AND, & (logical AND)
11. OR, |, XOR (Logical OR, exclusive OR)

Operations are performed in the above order of priority. When two operations of equal priority appear in the same statement, the operations are executed in order from left to right.

9.1.5 Data format conversion

Data format is converted in cases where two values of different formats are involved in the same operation.

- 1) When a real number is assigned to an integer, decimal places are rounded off.

Example: $A \% = 125.67$ ----- $A \% = 126$

- 2) When integers and real numbers are involved in the same operation, the result becomes a real number.

Example: $A (0) = 125 * 0.25$ ----- $A (0) = 31.25$

- 3) When an integer is divided by an integer, the result is an integer.

Example: $A (0) = 100 / 3$ ----- $A (0) = 33$

9.2 Character string operations

9.2.1 Character string connection

Character strings may be combined by using the “+” sign.

Examples:

```
A$="YAMAHA"  
B$="ROBOT"  
C$="LANGUAGE"  
D$="MOUNTER"  
E$=A$+" "+B$+" "+C$  
F$=A$+" "+D$  
PRINT E$  
PRINT F$
```

Results: YAMAHA ROBOT LANGUAGE
YAMAHA MOUNTER

9.2.2 Character string comparison

Characters can be compared with the same relational operators as used for numeric values. In the case of character strings, the comparison is performed from the beginning of each string, character by character. If all characters match in both strings, they are considered to be equal. Even if only one character in the string differs from its corresponding character in the other string, then the string with the larger (higher) character code is treated as the larger string.

If one string is shorter than the other, it is judged to be the string of lesser value.

All examples below are “true”.

Examples:

```
"AA"<"AB"  
"X&">"X#"  
"DESK"<"DESKS"
```

Character string comparison can be used to find out the contents of character strings, or to sort character strings into alphabetical order.

 CAUTION

In controllers whose software version is earlier than 8.28, the range of point numbers is from 0 to 4000.

9.3 Point data format

There are two types of point data formats: joint coordinate format and Cartesian coordinate format.

Point numbers are in the range of 0 to 9999.

9.3.1 Joint coordinate format

$\pm n n n n n n n$ (same for X, Y, Z, R, A, B axes)

This is a decimal integer constant of 7 digits or less with a plus or minus sign, and can be specified from -6144000 to 6144000. (Unit: pulses)

9.3.2 Cartesian coordinate format

$\pm n n n . n n$ to $\pm n n n n n n n$. (same for X, Y, Z, R, A, B axes)

This is a decimal fraction of a total of 7 digits including 2 or less decimal places. (Unit: mm or degrees)

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the end of the data. If a value other than 1 or 2 is set, or if no value is designated, 0 will be set to indicate that no hand system flag is set.

1: RIGHTY (right-handed system) 2: LEFTY (left-handed system)

 NOTE

Plus (+) signs can be omitted.

 CAUTION

Hand system flags are only available from software version 8.08 onwards.

9.4 DI/DO conditional expressions

DI/DO conditional expressions may be used to set conditions for WAIT statements and STOPON options in MOVE statements. Numeric constants, variables and arithmetic operators that may be used with DI/DO conditional expressions are shown below.

- a. Constant
Decimal integer constant, binary integer constant, hexadecimal integer constant
- b. Variables
Global integer type, global real number type, input/output type
- c. Operators
Relational operators, logic operators
- d. Operation priority
 1. Relational operators
 2. NOT, ~
 3. AND, &
 4. OR, |, XOR

Example:

WAIT DI(31)=1 OR DI(34)=1

----- The program waits until either DI31 or DI34 turns ON.

10. Multiple Robot Control

10.1 Overview

The YAMAHA robot controller can be used to control multiple robots.

The multitask function also enables multiple robots to move asynchronously. To use this function, settings for two robots or settings for auxiliary axes must be made in the system prior to shipment.

A robot axis is classified into one of the groups below.

Main group (4 axes)

Main group (2 axes) + sub group (2 axes)

A main group is composed of one main robot and main auxiliary axes, and a sub group is composed of one sub robot and sub auxiliary axes.

When using one robot without auxiliary axis, settings are made only for the main group robot.

When no settings have been made for main auxiliary axes and sub auxiliary axes, the main group is composed only of the one main robot, and the sub group is composed only of the one sub robot.

Main group — Main robot (Number of axes: 1 to 4)
(Number of axes: 1 to 4) — Main auxiliary axis (Number of axes: 1 to 4)

Sub group — Sub robot (Number of axes: 1 or 2)
(Number of axes: 1 or 2) — Sub auxiliary axis (Number of axes: 1 or 2)

10.2 Command list for each group



NOTE

1. The **MOVE** (**MOVE2**) and **MOVEI** (**MOVEI2**) commands are used to move a main robot (a sub robot).

An axis specified as an auxiliary axis cannot be moved with the **MOVE** (**MOVE2**), **MOVEI** (**MOVEI2**) and **PMOVE** (**PMOVE2**) commands. Use the **DRIVE** (**DRIVE2**) or **DRIVEI** (**DRIVEI2**) command to move it.

2. When specifying all axes with the **SERVO** or **SERVO2** command, the servos of all axes in the main group and sub group can be switched **ON** or **OFF**.

The special commands and functions for robot movement and coordinate control are shown below.

Classification	Main group	Sub group
Robot movement	DRIVE, DRIVEI, MOVE, MOVEI, PMOVE, SERVO, WAIT ARM	DRIVE2, DRIVEI2, MOVE2, MOVEI2, PMOVE2, SERVO2, WAIT ARM2
Coordinate control	CHANGE, HAND, LEFTY/RIGHTY, SHIFT	CHANGE2, HAND2, LEFTY2/RIGHTY2, SHIFT2
Status change	ACCEL, ARCH, ASPEED, AXWGHT, DECEL, ORGORD, OUTPOS, SPEED, TOLE, WEIGHT.	ACCEL2, ARCH2, ASPEED2, AXWGHT2, DECEL2, ORGORD2, OUTPOS2, SPEED2, TOLE2, WEIGHT2
Point operation	JTOXY, WHERE, WHRXY, XYTOJ	JTOXY2, WHERE2, WHRXY2, XYTOJ2
Parameter reference	ACCEL, ARCH, AXWGHT, DECEL, ORGORD, OUTPOS, TOLE, WEIGHT	ACCEL2, ARCH2, AXWGHT2, DECEL2, ORGORD2, OUTPOS2, TOLE2, WEIGHT2
Status reference	ABSRPOS, ARMCND, ARMTYPE, MCHREF	ABSRPOS2, ARMCND2, ARMTYPE2, MCHREF2
PATH control	PATH, PATH END, PATH SET, PATH START	
Torque control	DRIVE (with torque limit setting option), TORQUE, TRQSTS, TRQTIME	DRIVE2 (with torque limit setting option), TORQUE2, TRQSTS2, RQTIME2

11. Command Statements

ABSRST statement

Format:

ABSRST

Explanation:

This statement executes return-to-origin along the robot absolute motor axes. Return-to-origin will fail if the robot stops en route.

In the case of two-robot setting, first the main robot group return-to-origin is run, then sub robot group run and finally absolute reset performed.



NOTE

- This command applies to axes whose return-to-origin method is not specified as "MARK".
- This command cannot be executed if return-to-origin is incomplete for an axis whose return-to-origin method is specified as "MARK".

Example:

ABSRST

----- Performs return-to-origin along absolute motor axes.

Related commands: ORIGIN, ORGORD, ORGORD2, MCHREF, MCHREF2

ACCEL statement (Acceleration setting statement for main group)

Format 1:

```
ACCEL <expression>
```

Format 2:

```
ACCEL (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression> and <expression 2> must be from 1 to 100. (Unit: %)



NOTE

- This command changes the acceleration parameters for the main axes and auxiliary axes of the robot.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the acceleration coefficient of the main group parameters to the value specified in <expression>.

Format 1 changes all axes in the main group. Format 2 changes the acceleration coefficient of the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A=50
ACCEL A
ACCEL(3)=100
'CYCLE WITH INCREASING ACCELERATION
FOR A=10 TO 100 STEP 10
  ACCEL A
  MOVE P,P0
  MOVE P,P1
NEXT A
HALT "END TEST"
```

ACCEL2 statement (Acceleration setting statement for sub group)

Format 1:

```
ACCEL2 <expression>
```

Format 2:

```
ACCEL2 (<expression 1>) = <expression 2>
```



NOTE

- This statement can be used only when the sub group is set in the system generation.
- If an axis that is set to “no axis” in the system generation is specified, then an error message “Specification mismatch” appears and execution of the command will stop.

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression> and <expression 2> must be from 1 to 100. (Unit: %)

Explanation:

This command changes the acceleration coefficient of the sub group parameter to the value specified in <expression>.

Format 1 changes all axes in the sub group. Format 2 changes the acceleration coefficient of the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A=50
ACCEL2 A
ACCEL2(3)=100
'CYCLE WITH INCREASING ACCELERATION
FOR A=10 TO 100 STEP 10
  ACCEL2 A
  MOVE2 P,P0
  MOVE2 P,P1
NEXT A
HALT "END TEST"
```

ARCH statement (Arch position setting statement for main group)

Format 1:

```
ARCH <expression>
```

Format 2:

```
ARCH (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression> and <expression 2> must be from 1 to 6144000. (Unit: pulses)



NOTE

If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the arch position parameter for the main group to the value specified in <expression>. Format 1 changes all axes of the main group. Format 2 changes the arch position parameter for the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH INCREASING ARCH POSITION
DIM SAV(3)
GOSUB *SAVE_ARCH
FOR A=1000 TO 10000 STEP 1000
  GOSUB *CHANGE_ARCH
  MOVE P,P0,Z=0
  DO3(0)=1          ----- Chuck (or gripper) closes.
  MOVE P,P1,Z=0
  DO3(0)=0          ----- Chuck (or gripper) opens.
NEXT A
GOSUB *RESTORE_ARCH
HALT
*CHANGE_ARCH:
  FOR B=1 TO 4
    ARCH(B)=A
  NEXT B
  RETURN
*SAVE_ARCH:
  FOR B=1 TO 4
    SAV(B-1)=ARCH(B)
  NEXT B
  RETURN
*RESTORE_ARCH:
  FOR B=1 TO 4
    ARCH(B)=SAV(B-1)
  NEXT B
  RETURN
```

ARCH2 statement (Arch position setting statement for sub group)

Format 1:

```
ARCH2 <expression>
```

Format 2:

```
ARCH2 (<expression 1>) = <expression 2>
```



NOTE

- This statement can be used only when the sub group is set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression> and <expression 2> must be from 1 to 6144000. (Unit: pulses)

Explanation:

This statement changes the arch position parameter for the sub group to the value specified in <expression>. Format 1 changes all axes of the sub group. Format 2 changes the arch position parameter for the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH INCREASING ARCH POSITION
DIM SAV(3)
GOSUB *SAVE_ARCH
FOR A=1000 TO 10000 STEP 1000
  GOSUB *CHANGE_ARCH
  MOVE2 P,P0,Z=0
  DO3(0)=1           ----- Chuck (or gripper) closes.
  MOVE2 P,P1,Z=0
  DO3(0)=0           ----- Chuck (or gripper) opens.
NEXT A
GOSUB *RESTORE_ARCH
HALT
*CHANGE_ARCH:
  FOR B=1 TO 4
    ARCH2(B)=A
  NEXT B
  RETURN
*SAVE_ARCH:
  FOR B=1 TO 4
    SAV(B-1)=ARCH2(B)
  NEXT B
  RETURN
*RESTORE_ARCH:
  FOR B=1 TO 4
    ARCH2(B)=SAV(B-1)
  NEXT B
  RETURN
```

ASPEED statement

(Automatic movement speed setting statement for main group)

Format:

```
ASPEED <expression>
```

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

This statement changes the automatic movement speed for the main group to the value specified in <expression>.

Example:

```
SPEED 70
ASPEED 100
MOVE P,P0
```

----- Moves at 70% (=100*70) of maximum speed from the current position to P0.

```
ASPEED 50
MOVE P,P1
```

----- Moves at 35% (=50*70) of maximum speed from the current position to P1.

```
MOVE P,P2,S=10
```

----- Moves at 5% (=50*10) of maximum speed from the current position to P2.

```
HALT
```

Related commands: ASPEED2, SPEED, SPEED2



NOTE

- This statement changes all the movement speed parameters for the main axes and auxiliary axes of the robot.
- Robot operating speed is determined by the product of the automatic movement speed (specified by the MPB/RPB or ASPEED command) and the program speed (specified by the SPEED command or the SPEED option for movement commands).

Example:

When the automatic movement speed is 80% and the program speed is 50%, then: Robot operating speed = $80\% \times 50\% = 40\%$

- If the speed is specified by the DSPEED option in the DRIVE or DRIVEI command, the operating speed is determined only by the DSPEED option without using the automatic movement speed and SPEED command.

ASPEED2 statement (Automatic movement speed setting statement for sub group)

Format:

ASPEED2 <expression>

The value of <expression> must be from 1 to 100. (Unit: %)

NOTE

- This statement changes all the movement speed parameters for the main axes and auxiliary axes of the robot.
- Robot operating speed is determined by the product of the automatic movement speed (specified by the MPB/RPB or ASPEED2 command) and the program speed (specified by the SPEED2 command or the SPEED option for movement commands).
- Example:**
When the automatic movement speed is 80% and the program speed is 50%, then: Robot operating speed = $80\% \times 50\% = 40\%$
- If the speed is specified by the DSPEED option in the DRIVE2 or DRIVEI2 command, the operating speed is determined only by the DSPEED option without using the automatic movement speed and SPEED2 command.
- This statement can be used only when the sub group is set in the system generation.

Explanation:

This statement changes the automatic movement speed for the sub group to the value specified in <expression>.

Example:

```
SPEED2 70
ASPEED2 100
MOVE2 P,P0
```

----- Moves at 70% ($=100 \times 70$) of maximum speed from the current position to P0.

```
ASPEED2 50
MOVE2 P,P1
```

----- Moves at 35% ($=50 \times 70$) of maximum speed from the current position to P1.

```
MOVE2 P,P2,S=10
```

----- Moves at 5% ($=50 \times 10$) of maximum speed from the current position to P2.

Related commands: ASPEED, SPEED, SPEED2

AXWGHT statement (Axis tip weight setting statement for main group)

Format:

```
AXWGHT (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The range of <expression 2> differs depending on the selected robot model.



NOTE

- This statement changes the axis tip weight parameter of a specified axis.
- This statement is valid only for "MULTI" axes or auxiliary axes of the main robot.
- Robot type and auxiliary axes are preset prior to shipping.

Explanation:

This statement changes the axis tip weight parameter for the main group axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A=5
B=0
C=AXWGHT(1)      ----- Escape
AXWGHT(1)=A
DRIVE(1,P0)
AXWGHT(1)=B
DRIVE(1,P1)
AXWGHT(1)=C      ----- Restore
HALT
```

Related commands: WEIGHT, WEIGHT2, AXWGHT2

AXWGHT2 statement (Axis tip weight setting statement for sub group)

Format:

`AXWGHT2 (<expression 1>) = <expression 2>`



NOTE

- This statement changes the axis tip weight parameter of a specified axis.
- This statement is valid only for "MULTI" axes or auxiliary axes of the sub robot.
- Robot type and auxiliary axes are preset prior to shipping.

The value of <expression 1> must be from 1 to 4 (axis number).

The range of <expression 2> differs depending on the selected robot model.

Explanation:

This statement changes the axis tip weight parameter for the sub group axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A=5
B=0
C=AXWGHT2(1)      -----Escape
AXWGHT2(1)=A
DRIVE2(1,P0)
AXWGHT2(1)=B
DRIVE2(1,P1)
AXWGHT2(1)=C      -----Restore
HALT
```

Related commands: WEIGHT, WEIGHT2, AXWGHT

CALL statement

Format:

```
CALL <label> [( <actual argument> [, <actual argument> , ... ] )]
```

NOTE

- When a value is passed on to the sub-procedure, the value of the real argument will not change even if it is changed in the sub-procedure.
- When a reference is passed on to the sub-procedure, the value of the real argument will also change if it is changed in the sub-procedure.

NOTE

- CALL statements containing one actual argument can be used up to 15 times in succession.
Note that the number of times will drop if commands containing a stack such as an IF statement or GOSUB statement are used, or depending on the number of arguments in the CALL statement.
- To end a sub-procedure which has been called with the CALL statement, always use the END SUB statement. If another statement such as GOTO is used to jump out of the sub-procedure, an error such as "Stack overflow" may occur.

Explanation:

This statement calls up sub-procedures defined by the SUB and END SUB statements. The <label> is the name of the sub-procedure defined by the SUB statement. The arguments used here are defined as follows:

1. When a constant or expression is specified as an actual argument, its value is passed on to the sub-procedure.
2. When an element of variable or array variable is specified as an actual argument, its value is passed on to the sub-procedure but can also be passed on as a reference by adding "REF" to the head of the real argument.
3. When an entire array variable (array name followed by parentheses) is specified as an argument, then this is passed along as a reference.

Example 1:

```
X%=4
Y%=5
CALL *COMPARE( REF X%, REF Y% )
HALT
'SUB ROUTINE: COMPARE
SUB *COMPARE( A%, B% )
  IF A% < B% THEN
    TEMP%=A%
    A%=B%
    B%=TEMP%
  ENDIF
END SUB
```

Example 2:

```
I = 1
CALL *TEST( I )
HALT
'SUB ROUTINE: TEST
SUB *TEST
  X = X + 1
  IF X < 15 THEN
    CALL *TEST( X )
  ENDIF
END SUB
```

Related commands: SUB, END SUB, CALL, DECLARE, EXIT SUB

CHGPRI statement



NOTE

- The priority of task 1 (main task) is 32.
- The smaller the priority number, the higher the priority level.
- When a task with a higher priority is set "READY", all tasks with lower priority levels also stay "READY".

Format:

```
CHGPRI Tn, p
```

n : Task number 2 to 8.

p : Task priority 17 to 47

Explanation:

This statement changes the priority (p) of the specified task (n).

Example:

```
START *SUBTASK,T2,33
*ST:
  MOVE P,P0,P1
  IF DI(20) = 1 THEN
    CHGPRI T2,32
  ELSE
    CHGPRI T2,33
  ENDIF
GOTO *ST
HALT
'SUBTASK ROUTINE
*SUBTASK:
  IF LOCZ(WHERE) > 10000 THEN
    DO(20) = 1
    GOTO *SUBTASK
  ENDIF
  DO(20) = 0
GOTO *SUBTASK
EXIT TASK
```

Related commands: CUT, EXIT TASK, RESTART, SUSPEND, START

CUT statement

Format:

CUT T_n

n : task number 2 to 8

Explanation:

This statement terminates a task currently being executed or temporarily stopped.
This statement cannot terminate its own task.

Example:

```
'TASK1 ROUTINE
*ST:
  MO(20) = 0
  START *SUBTASK2,T2
  MOVE P,P0
  MOVE P,P1
  WAIT MO(20) = 1
  CUT T2
GOTO *ST
HALT
'TASK2 ROUTINE
*SUBTASK2:
  P100=JTOXY(WHERE)
  IF LOCZ(P100) >= 100.0 THEN
    MO(20) = 1
  ELSE
    DELAY 100
  ENDIF
GOTO *SUBTASK2
EXIT TASK
```

Related commands: EXIT TASK, CUT, RESTART, START, SUSPEND



CAUTION

The **DECEL** statement is only available from software version 8.15 onwards.

DECEL statement (Deceleration setting statement for main group)

Format 1:

```
DECEL <expression>
```

Format 2:

```
DECEL (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression> and <expression 2> must be from 1 to 100. (Unit: %)



NOTE

- The deceleration set in the axis parameters for the robot configuration axis and auxiliary axis is changed.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the deceleration rate of the main group parameter to the value specified in <expression>.

Format 1 changes all axes in the main group. Format 2 changes the deceleration rate of the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A =50
DECEL A
DECEL(3)=100
'CYCLE WITH INCREASING DECELERATION
FOR A =10 TO 100 STEP 10
  DECEL A
  MOVE P ,P0
  MOVE P ,P1
NEXT A
HALT "END TEST"
```

**CAUTION**

The **DECEL2** statement is only available from software version 8.15 onwards.

DECEL2 statement (Deceleration setting statement for sub group)

Format 1:

```
DECEL2 <expression>
```

Format 2:

```
DECEL2 (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression> and <expression 2> must be from 1 to 100. (Unit: %)

**NOTE**

- The deceleration set in the axis parameters for the robot configuration axis and auxiliary axis is changed.
- This statement can be used only when the sub group is set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the deceleration rate of the sub group parameter to the value specified in <expression>.

Format 1 changes all axes in the sub group. Format 2 changes the deceleration rate of the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
A =50
DECEL2 A
DECEL2(3)=100
'CYCLE WITH INCREASING DECELERATION
FOR A =10 TO 100 STEP 10
  DECEL2 A
  MOVE2 P ,P0
  MOVE2 P ,P1
NEXT A
HALT "END TEST"
```

DECLARE statement

Format 1:

```
DECLARE <label> [, <label>...]
```

<label>: Label defined by an external program.

Format 2:

```
DECLARE SUB <label> [( <dummy argument> [, <dummy argument>]... ) ]
```

<label> : Name of sub-procedure defined by an external program.

<dummy argument> : Argument for sub-procedure. This format is significant only for the number of arguments and data type.



NOTE

- External programs can only use "COMMON".
- Only the GOSUB, CALL, ON ~ GOSUB statements can use external labels.

Explanation:

This statement declares that a label or sub-procedure is in an external program. In the case of a sub-procedure, the data type of the argument is also checked.

The DECLARE statement can be written in programs other than the COMMON program (but cannot be defined in a sub-procedure) and is effective over the entire program.

Example:

```
Labels shared with an external program.
Program name: DIST1
'=====
'   MAIN PROGRAM
'=====
  DECLARE *DISTANCE,*AREA
  X!=2.5
  Y!=1.2
  GOSUB *DISTANCE
  GOSUB *AREA
  HALT
Program name: COMMON
'=====
'   'COMMON'PROGRAM
'=====
  *DISTANCE:
    PRINT X!^2+Y!^2
    RETURN
  *AREA:
    PRINT X!*Y!
    RETURN
```


Example:

Sub-procedures shared with an external program

Program name: DIST2

```

=====
'  MAIN PROGRAM
=====
  DECLARE SUB *DISTANCE(X!,Y!,D!)
  DECLARE SUB *AREA(X!,Y!,A!)
  CALL *DISTANCE(2.5,1.2,REF D!)
  PRINT D!
  CALL *AREA(2.5,1.2,REF A!)
  PRINT A!
  HALT
Program name: COMMON
=====
'  'COMMON' PROGRAM
=====
  SUB *DISTANCE(X!,Y!,D!)
  D!=X!^2+Y!^2
  END SUB
  SUB *AREA(X!,Y!,A!)
  A!=X!*Y!
  END SUB

```

Related commands: CALL, EXIT SUB, GOSUB, ON~GOSUB, SUB, END SUB

DEF FN statement

Format:

```
DEF FN <name> [ |%| ] [ (<dummy argument>, [< dummy argument >...]) ] = <function definition expression>
```



NOTE

- You may specify numeral variables or character type variables in the <dummy arguments>.
- If a variable used in the <function definition expression> is not included in the list of arguments, the value assigned to that particular variable is used for the calculation.
- Be sure to put a space between "DEF" and "FN", otherwise "DEFFN" will be viewed as a variable.
- The DEF FN statement cannot be used within procedures.
- Definition by the DEF FN statement must be declared before inserting statements using functions.

Explanation:

The DEF FN statement defines functions available to the user. The functions defined here can be called in the format of "FN name (variable)".

The name can be up to 16 characters including the characters "FN".

The <dummy arguments> are the names of the variables used in the <function definition expression>. The names of these variables are effective only when the <function definition expression> is evaluated. There may be other variables with the same name in the program. When calling a function that uses a <dummy argument>, specify the constant, variable or expression that has the same type as the <dummy argument> type.

Example:

```
DEF FNPAI=3.141592
DEF FNASIN(X)=ATN(X/SQR(-X^2+1))
.
.
A=FNASIN(B)*10
```

DELAY statement

Format:

```
DELAY <expression>
```

The value of <expression> must be from 1 to 3600000. (Unit: ms)

Explanation:

The DELAY statement allows the program to wait for the period of time (milliseconds) specified by the <expression> before executing the next command. The minimum waiting time is 10 milliseconds.

Example:

```
DELAY 3500
```

----- Waits for 3500ms (3.5 seconds).

```
DELAY A*10
```

DIM statement (Array variable declaration statement)

Format:

```
DIM <array definition> [, <array definition>, .....]
```

Array definition:

```
<name> [ |%| ] ( <constant> [, <constant> [, <constant>] ] )
          |
          !
          $
```

The value of <constant> must be a positive integer from 0 to 32767.

Explanation:

This statement declares the name and length (number of elements) of an array variable. A maximum of 3 dimensions may be used for the array subscripts. Multiple arrays can be declared in one line by differentiating them with a comma (,).

Examples:

```
DIM A%(10)
```

----- Defines an integer array variable A%(0) to A%(10). (Number of elements: 11)

```
DIM B(2,3,4)
```

----- Defines a real array variable B(0, 0, 0) to B(2, 3, 4). (Number of elements: 60)

```
DIM C%(2,2),D!(10)
```

----- Defines an integer array C%(0,0) to C%(2,2) and a real array D!(0) to D!(10).



NOTE

- The size of each dimension in an array is equal to the DIM constant +1. Each element of an array variable begins with 0.
- An error "Memory full" may occur depending on the size of each dimension defined in an array.

DO statement (Parallel output)

Format:

[LET]	DOm ([b, ..., b])	= <expression>
	DO (mb, ..., mb)	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

The value of <expression> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

Explanation:

This statement outputs the specified value to the DO ports.



NOTE

- Output to the DO0 () and DO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO ports not connected to hardware.

Examples:

DO2() = &B10111000

----- DO (27, 25, 24, 23) are turned ON, and DO (26, 22, 21, 20) are turned OFF.

DO2(6,5,1) = &B010

----- DO (25) is turned ON, and DO (26, 21) are turned OFF.

DO3() = 15

----- DO (33, 32, 31, 30) are turned ON, and DO (37, 36, 35, 34) are turned OFF.

DO(37,35,27,20) = A

----- The contents of the 4 lower bits obtained when variable A is converted to an integer are output to DO (37, 35, 27, 20), respectively.

Related commands: RESET, SET

DRIVE statement

Format:

```
DRIVE (<expression 1>, <expression 2> ) [, (<expression 1>, <expression 2> ) ...] [, option]
      <point expression> <point expression>
```

Explanation:

The DRIVE statement executes an absolute movement of each axis in the main group, including auxiliary axes in the main group.

Movement type : PTP movement on specified axis
 Point data setting : Direct coordinate data input, point definition
 Options : Speed setting, STOPON condition setting, torque limit setting, XY setting

Movement type

· PTP (point to point) movement on specified axis

PTP movement begins after positioning on all axes specified in <expression 1> is complete (within the tolerance range), and the command terminates when the specified axes enter the OUT position range. When two or more axes are specified, they will reach their target positions simultaneously.

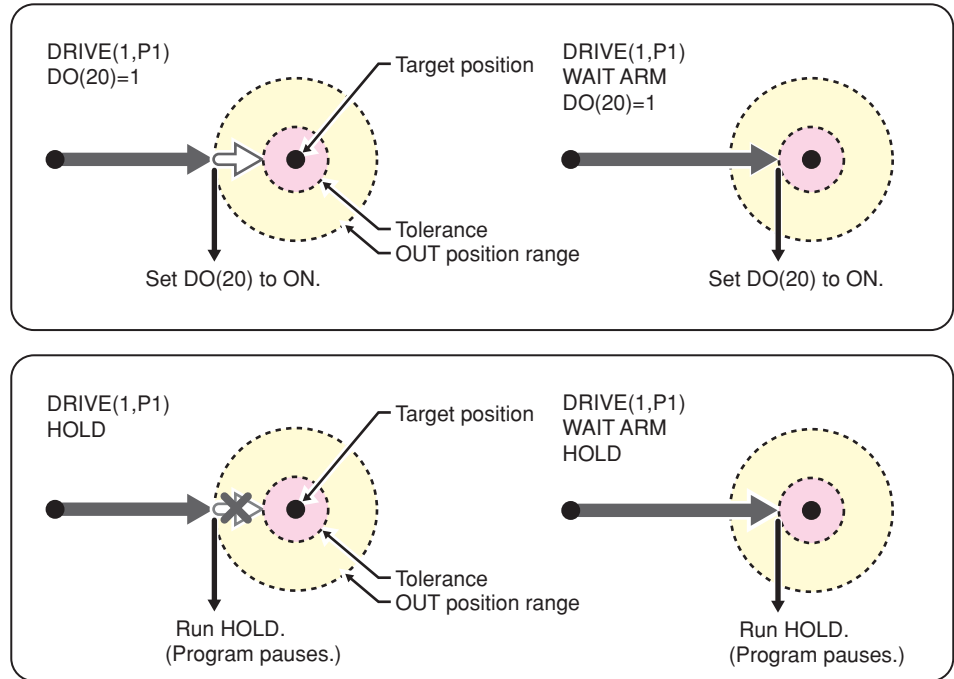
Cautions for next command following DRIVE command

If the next command following the DRIVE command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.



Example:

`DRIVE(1,P0)`

----- Axis 1 moves from the current position to the point specified by P0.

Point data setting

· Direct coordinate data input

Directly specify the coordinate data in <expression 2> with an integer or real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number containing a decimal point, the units are viewed as “mm” or “deg” and the robot will move to a position converted for each axis relative to the 0 pulse position.

Examples:

`DRIVE(1,10000)`

----- Axis 1 moves from the current position to the “10000 pulses” position.

`DRIVE(4,90.00)`

----- Axis 4 moves from the current position to the point at +90 degrees in the plus direction. (When axis 4 is a rotating axis.)

· Point definition

Specify the point expression in <expression 2>, while using the data for the axis specified in <expression 1>.

The robot moves to the position determined by the units used for point expression. When the units are viewed as “mm” or “deg”, the robot moves to the position converted for each axis relative to the 0 pulse position.

Examples:

`DRIVE(1,P1)`

----- Axis 1 moves from the current position to the position specified by P1.

`DRIVE(4,P90)`

----- Axis 4 moves from the current position to the position specified by P90 (deg) relative to the 0 pulse position. (When axis 4 is a rotating axis.)



NOTE

If point data is specified with both integers and real numbers in the same statement, all axes move in “mm” or “deg” units.

11. Command Statements

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified DRIVE statement.

Example:

DRIVE(1,10000),S=10

----- Axis 1 moves from the current position to the "10000 pulses" position using a program speed of 10%.

Format 2:

DSPEED	= <expression>
DS	

The value of <expression> must be from 0.01 to 100.00. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operation speed is determined only by the speed specified here, without using the automatic movement speed.

If this option is used with the torque limit setting option, then the operating speed will be 10% of the speed specified in <expression>.

This option is enabled only for the specified DRIVE statement.

Example:

DRIVE(1,P10),DS=0.2

----- Axis 1 moves from the current position to the position specified by P10 at 0.2% of maximum speed.

DRIVE(3,P1),T=(20,15),DSPEED=1

----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 at 0.1% of maximum speed (pushing action).

· STOPON condition setting

Format:

STOPON <conditional expression>

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

DRIVE(1,10000),STOPON DI(20)=1

----- Axis 1 moves from the current position to the "10000 pulses" position and stops when the condition "DI (20) = 1" is met.



NOTE

· Speed setting (DSPEED) is enabled only for the following software versions.

RCX14x Ver. 8.71 onwards,
RCX22x Ver. 9.18 onwards

· SPEED option and DSPEED option cannot be used together.



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,
RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

 CAUTION

- The torque offset setting is only available from software version 8.18 onwards.
- The torque limit setting range differs depending on the robot model. Setting a torque limit higher than the maximum level may cause robot malfunctions or breakdowns.
- If the specified torque limit is too small, the axis might not move.

 NOTE

For details on the current command monitor, refer to "13. Monitor mode" in Chapter 4 of the user's manual for each controller.

· Torque limit setting

Format 1:

$$T = \langle \text{expression 1} \rangle$$

Format 2:

$$T = (\langle \text{expression 1} \rangle [\langle \text{expression 2} \rangle])$$

The value of $\langle \text{expression 1} \rangle$ is the torque limit specified in percent of rated torque and must be from 1 to 100. (Unit: %)

The value of $\langle \text{expression 2} \rangle$ is the torque offset and must be from -100 to 100.

Explanation:

Moves the axis while under torque control.

The maximum torque at this point is limited to a value calculated from: Rated torque $\times \langle \text{expression 1} \rangle / 100$.

The value of $\langle \text{expression 2} \rangle$ determines the torque offset. Specify this offset when controlling the torque on an axis such as a vertical axis to which a fixed load is constantly applied. Torque offset value is set to 0 if $\langle \text{expression 2} \rangle$ is omitted and in Format 1.

The torque offset specified in $\langle \text{expression 2} \rangle$ is the torque command value (maximum torque ratio) constantly generated on the axis. Display it on the current command monitor while the axis is stopped with the servo turned on, and specify the torque value displayed at that point.

When the DRIVE statement is executed with this option specified, the robot moves to the target position while controlling the torque by changing the maximum torque on the axis to a value specified in $\langle \text{expression 1} \rangle$. The movement speed at this point is limited to a maximum of 10% of normal operating speed. No errors will occur even if the robot contacts an obstacle during movement and the axis torque (thrust) will not exceed the limit value.

 CAUTION

- Conditions for ending this command (mentioned on the right) are only valid from software version 8.45 onwards. On earlier version controllers, this command ends in the following cases.
 - The axis has reached the target position at the point when the time required to move to the target position has elapsed.
 - The axis torque has already reached the limit value for more than 1 second at the point when the time required to move to the target position has elapsed.
 - The torque has reached the limit value at the point when the time required to move to the target position has elapsed, and this condition has continued for 1 second.
- The timing that enables a maximum torque instruction changed by the TORQUE statement (mentioned on the right) is only valid from driver version 2.17 onwards. On earlier version controllers, the maximum torque instruction is enabled at the point when it was changed by the TORQUE statement.

This command ends when the axis has reached the target position or when the time (time-out period) specified by the TRQTIME statement has elapsed while the axis torque (thrust) has reached the limit value. When this command has ended due to a time-out during which the axis torque has reached its limit value, then 1 is set in the TRQSTS function.

The maximum torque limit and torque control condition continues even after this command has ended. This is the same in cases where the robot is stopped such as by an interlock while executing this command.

Torque control is released by an axis operation such as switching servo on/off switching or executing the MOVE command. To release the maximum torque limit, set a new maximum torque instruction by using the TORQUE statement. Note, however, that the maximum torque instruction is not enabled immediately. It will be enabled after the next movement command such as a MOVE or DRIVE statement is executed.

The maximum torque limit is released in the following cases, regardless of whether the TORQUE statement is executed or not.

- When controller power is turned on.
- When servo is turned off.
- When return-to-origin or absolute reset (except for mark method) is performed.
- When parameters are changed or initialized.

The following restrictions apply when specifying this option.

- Two or more axes cannot be specified with this option.
- Maximum movement speed is set to 10% of normal operating speed.
- Manual movement is not possible under torque control after this command was executed.

11. Command Statements

Example:

```
TRQTIME(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 seconds.
DRIVE(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 (pushing action).
IF TRQSTS(3)=1 THEN
----- Checks if time-out is up.
  DO(21)=1
----- Time-out is up (pushing is complete).
                                     (Result is output to DO(21) in this example.)
ELSE
  DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to complete pushing.)
                                     (Result is output to DO(21) in this example.)
ENDIF
TORQUE(3)=100
----- Returns the maximum torque instruction to original value (100%).
DRIVE(3,P0)
----- Ends the torque limit and torque control, and moves to P0.
```

Related commands: TORQUE, TRQTIME, TRQSTS functions

· XY setting

Format:

XY

Explanation:

This option moves the specified multiple axes to the position specified on the XY coordinates.

The following are limitation items when using this command option.

1. Axes specified in <expression 1> must include axes 1 and 2.
2. Robots that can be specified with this option are SCARA robots and Cartesian robots.
3. Point setting must be in “mm” or “deg” units (real number setting).

Examples:

```
DRIVE(1,P100),(2,P100),(4,P100),XY
----- Axes 1, 2 and 4 move from the current position to the position on the XY coordinates specified by P100.
```

DRIVE2 statement

Format:

DRIVE2 (<expression 1> , <expression 2>) [, (<expression 1> , <expression 2>) ...] [, option]
<point expression> <point expression>

Explanation:

The DRIVE2 statement executes an absolute movement of each axis in the sub group, including auxiliary axes in the sub group.

Movement type : PTP movement on specified axis
 Point data setting : Direct coordinate data input, point definition
 Options : Speed setting, STOPON condition setting, torque limit setting, XY setting

Movement type

· PTP (point to point) movement on specified axis

PTP movement begins after positioning on all axes specified in <expression 1> is complete (within the tolerance range), and the command terminates when the specified axes enter the OUT position range. When two or more axes are specified, they will reach their target positions simultaneously.

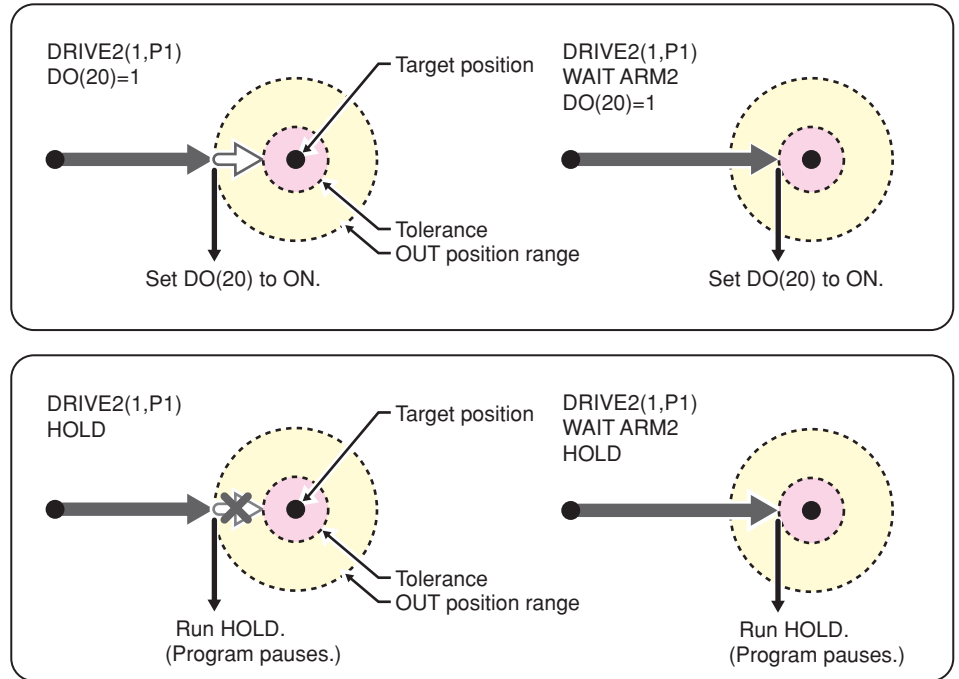
Cautions for next command following DRIVE2 command

If the next command following the DRIVE2 command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

`DRIVE2(1,P0)`

----- Axis 1 moves from the current position to the point specified by P0.

Point data setting

· Direct coordinate data input

Directly specify the coordinate data in <expression 2> with an integer or real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number containing a decimal point, the units are viewed as “mm” or “deg” and the robot will move to a position converted for each axis relative to the 0 pulse position.

Examples:

`DRIVEI2(1,10000)`

----- Axis 1 moves from the current position to the “10000 pulses” position.

`DRIVEI2(2,90.00)`

----- Axis 4 moves from the current position to the point at +90 degrees in the plus direction. (When axis 4 is a rotating axis.)

· Point definition

Specify the point expression in <expression 2>, while using the data for the axis specified in <expression 1>.

The robot moves to the position determined by the units used for point expression. When the units are viewed as “mm” or “deg”, the robot moves to the position converted for each axis relative to the 0 pulse position.

Examples:

`DRIVE2(1,P1)`

----- Axis 1 moves from the current position to the position specified by P1.

`DRIVE2(4, P90)`

----- Axis 2 moves from the current position to the position specified (degrees) by P90 relative to the 0 pulse position. (When axis 2 is a rotating axis.)



NOTE

If point data is specified with both integers then all real numbers in the same statement, then all axes move in “mm” or “deg” units.

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified DRIVE2 statement.

Example:

DRIVE2(1,10000),S=10

----- Axis 1 moves from the current position to the “10000 pulses” position using a program speed of 10%.

Format 2:

DSPEED	= <expression>
DS	

The value of <expression> must be from 0.01 to 100.00. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operation speed is determined only by the speed specified here, without using the automatic movement speed.

If this option is used with the torque limit setting option, then the operating speed will be 10% of the speed specified in <expression>.

This option is enabled only for the specified DRIVE2 statement.

Example:

DRIVE2(1,P10),DS=0.2

----- Axis 1 moves from the current position to the position specified by P10 at 0.2% of maximum speed.

DRIVE2(3,P1),T=(20,15),DSPEED=1

----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 at 0.1% of maximum speed (pushing action).

· STOPON condition setting

Format:

STOPON <conditional expression>

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

DRIVE2(1,10000),STOPON DI(20)=1

----- Axis 1 moves from the current position to the “10000 pulses” position and stops when the condition “DI (20) = 1” is met.

**NOTE**

· Speed setting (DSPEED) is enabled only for the following software versions.

RCX14x Ver. 8.71 onwards,

RCX22x Ver. 9.18 onwards

· SPEED option and DSPEED option cannot be used together.

**NOTE**

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

CAUTION

- The torque offset setting is only available from software version 8.18 onwards.
- The torque limit setting range differs depending on the robot model. Setting a torque limit higher than the maximum level may cause robot malfunctions or breakdowns.
- If the specified torque limit is too small, the axis might not move.

NOTE

For details on the current command monitor, refer to "13. Monitor mode" in Chapter 4 of the user's manual for each controller.

CAUTION

- Conditions for ending this command (mentioned on the right) are only valid from software version 8.45 onwards. On earlier version controllers, this command ends in the following cases.
 - The axis has reached the target position at the point when the time required to move to the target position has elapsed.
 - The axis torque has already reached the limit value for more than 1 second at the point when the time required to move to the target position has elapsed.
 - The torque has reached the limit value at the point when the time required to move to the target position has elapsed, and this condition has continued for 1 second.
- The timing that enables a maximum torque instruction changed by the TORQUE2 statement (mentioned on the right) is only valid from driver version 2.17 onwards. On earlier version controllers, the maximum torque instruction is enabled at the point when it was changed by the TORQUE2 statement.

- Torque limit setting

Format 1:

$T = \langle \text{expression 1} \rangle$

Format 2:

$T = (\langle \text{expression 1} \rangle [\langle \text{expression 2} \rangle])$

The value of $\langle \text{expression 1} \rangle$ is the torque limit specified in percent of rated torque and must be from 1 to 100. (Unit: %)

The value of $\langle \text{expression 2} \rangle$ is the torque offset and must be from -100 to 100.

Explanation:

Moves the axis while under torque control.

The maximum torque at this point is limited to a value calculated from: Rated torque $\times \langle \text{expression 1} \rangle / 100$.

The value of $\langle \text{expression 2} \rangle$ determines the torque offset. Specify this offset when controlling the torque on an axis such as a vertical axis to which a fixed load is constantly applied. Torque offset value is set to 0 if $\langle \text{expression 2} \rangle$ is omitted and in Format 1.

The torque offset specified in $\langle \text{expression 2} \rangle$ is the torque command value (maximum torque ratio) constantly generated on the axis. Display it on the current command monitor while the axis is stopped with the servo turned on, and specify the torque value displayed at that point.

When the DRIVE2 statement is executed with this option specified, the robot moves to the target position while controlling the torque by changing the maximum torque on the axis to a value specified in $\langle \text{expression 1} \rangle$. The movement speed at this point is limited to a maximum of 10% of normal operating speed. No errors will occur even if the robot contacts an obstacle during movement and the axis torque (thrust) will not exceed the limit value.

This command ends when the axis has reached the target position or when the time (time-out period) specified by the TRQTIME2 statement has elapsed while the axis torque (thrust) has reached the limit value. When this command has ended due to a time-out during which the axis torque has reached its limit value, then 1 is set in the TRQSTS2 function.

The maximum torque limit and torque control condition continues even after this command has ended. This is the same in cases where the robot is stopped such as by an interlock while executing this command.

Torque control is released by an axis operation such as switching servo on/off switching or executing the MOVE2 command. To release the maximum torque limit, set a new maximum torque instruction by using the TORQUE2 statement. Note, however, that the maximum torque instruction is not enabled immediately. It will be enabled after the next movement command such as a MOVE2 or DRIVE2 statement is executed.

The maximum torque limit is released in the following cases, regardless of whether the TORQUE2 statement is executed or not.

- When controller power is turned on.
- When servo is turned off.
- When return-to-origin or absolute reset (except for mark method) is performed.
- When parameters are changed or initialized.

The following restrictions apply when specifying this option.

- Two or more axes cannot be specified with this option.
- Maximum movement speed is set to 10% of normal operating speed.
- Manual movement is not possible under torque control after this command was executed.

Example:

```

TRQTIME2(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 sec-
ons.
DRIVE2(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and
the torque offset to 15, and moves axis 3 from the current
position to the point specified by P1 (pushing action).
IF TRQSTS2(3)=1 THEN
----- Checks if time-out is up.
DO(21)=1
----- Time-out is up (pushing is complete).
(Result is output to DO(21) in this example.)
ELSE
DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to
complete pushing.)
(Result is output to DO(21) in this example.)
ENDIF
TORQUE2(3)=100
----- Returns the maximum torque instruction to original value
(100%).
DRIVE2(3,P0)
----- Ends the torque limit and torque control, and moves to P0.

```

Related commands: TORQUE2, TRQTIME2, TRQSTS2 functions

· XY setting

Format:

XY

Explanation:

This option moves the specified multiple axes to the position specified on the XY coordinates.

Comply with the following conditions when using this command option.

1. Axes specified in <expression 1> must include axes 1 and 2.
2. Robots that can be specified with this option are SCARA robots and Cartesian robots.
3. Point setting must be in “mm” or “deg” units (real number setting).

Examples:

```

DRIVE2(1,P100),(2,P100), XY
----- Axes 1 and 2 move from the current position to the position
on the XY coordinates specified by P100.

```

DRIVEI statement

Format:

```
DRIVEI (<expression 1>, <expression 2> | <point expression> ) [, (<expression 1>, <expression 2> | <point expression> ) ...] [, option]
```

NOTE

- If the DRIVEI statement is interrupted and then re-executed in the controller versions shown below, the resumed motion can occur either to the original target position, or to a new target position referenced to the current position (stop position), in accordance with the "MOVEI/DRIVEI start position" setting selected at the other parameters.
For details, refer to the controller user's manual.
RCX14x Ver. 8.66 onwards
RCX22x Ver. 9.13 onwards
- The other parameters default "MOVEI/DRIVEI start position" setting is Keep (motion to the original target position when DRIVEI is interrupted and then re-executed).
- In controller versions prior to those shown above, motion occurs to a new target position referenced to the current position when DRIVEI is interrupted and then re-executed.

Explanation:

The DRIVEI statement executes a relative movement of each axis in the main group, including auxiliary axes in the main group.

Movement type : PTP movement on specified axis
Point data setting : Direct coordinate data input, point definition
Options : Speed setting, STOPON condition setting

Movement type

- PTP (point to point) movement on specified axis

PTP movement begins after positioning on all axes specified in <expression 1> is complete (within the tolerance range), and the command terminates when the specified axes enter the OUT position range. When two or more axes are specified, they will reach their target positions simultaneously.

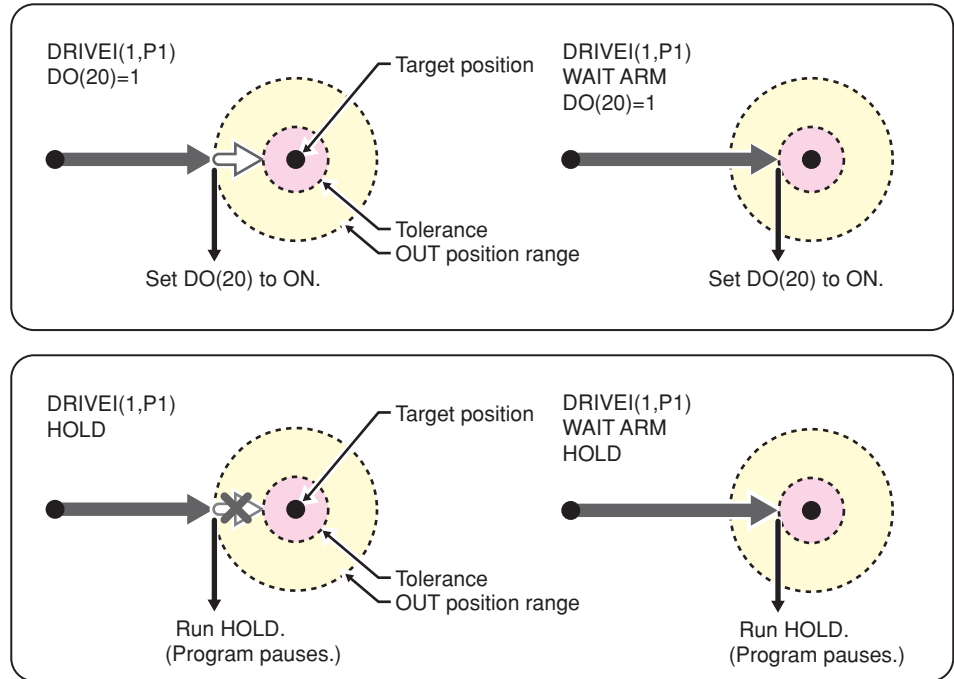
Cautions for next command following DRIVEI command

If the next command following the DRIVEI command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.



Example:

`DRIVEI(1,P0)`

----- Axis 1 moves a distance specified by P0 from the current position.

Point data setting

· Direct coordinate data input

Directly specify the coordinate data in <expression 2> with an integer or real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number containing a decimal point, the units are viewed as “mm” or “deg” and the robot will move a certain distance converted for each axis.

Examples:

`DRIVEI(1,10000)`

----- Axis 1 moves a distance “+10000 pulses” from the current position.

`DRIVEI(4,90.00)`

----- Axis 4 moves through “+90 degrees” from the current position. (When axis 4 is a rotating axis.)

· Point definition

Specify the point expression in <expression 2>, while using the data for the axis specified in <expression 1>.

The robot moves to the position determined by the units used for point expression. When the units are viewed as “mm” or “deg”, the robot moves a certain distance converted for each axis.

Examples:

`DRIVEI(1, P1)`

----- Axis 1 moves a distance specified by P1 from the current position.

`DRIVEI(4, P90)`

----- Axis 4 moves a distance (degrees) specified by P90 from the current position. (When axis 4 is a rotating axis.)



NOTE

If point data is specified with both integers and real numbers in the same statement, then all axes move in “mm” or “deg” units.

11. Command Statements

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified DRIVEI statement.

Example:

DRIVEI(1,10000),S=10

----- Axis 1 moves a distance “+10000 pulses” from the current position using a program speed of 10%.

Format 2:

DSPEED	= <expression>
DS	

The value of <expression> must be from 0.01 to 100.00. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operation speed is determined only by the speed specified here, without using the automatic movement speed.

If this option is used with the torque limit setting option, then the operating speed will be 10% of the speed specified in <expression>.

This option is enabled only for the specified DRIVEI statement.

Example:

DRIVEI(1,P10),DS=0.2

----- Axis 1 moves from the current position to the position specified by P10 at 0.2% of maximum speed.

· STOPON condition setting

Format:

STOPON <conditional expression>

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

DRIVEI(1,10000),STOPON DI(20)=1

----- Axis 1 moves a distance “+10000 pulses” from the current position and stops when the condition “DI (20) = 1” is met.



NOTE

· Speed setting (DSPEED) is enabled only for the following software versions.

RCX14x Ver. 8.71 onwards,

RCX22x Ver. 9.18 onwards

· SPEED option and DSPEED option cannot be used together.



CAUTION

When execution of the DRIVEI statement is halted with the STOPON option, executing the DRIVEI statement again on the same axis moves the robot the specified distance from the stopped position.

When executing consecutive relative movement commands, take precautions to prevent the moving sections of the robot from bumping or binding on each other.



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

DRIVEI2 statement

Format:

```
DRIVEI2 (<expression 1> , <expression 2> | <point expression> ) [ , (<expression 1> , <expression 2> | <point expression> ) ... ] [ , option ]
```

NOTE

- If the DRIVEI2 statement is interrupted and then re-executed in the controller versions shown below, the resumed motion can occur either to the original target position, or to a new target position referenced to the current position (stop position), in accordance with the "MOVEI/DRIVEI start position" setting selected at the other parameters.
For details, refer to the controller user's manual.
RCX14x Ver. 8.66 onwards
RCX22x Ver. 9.13 onwards
- The other parameters default "MOVEI/DRIVEI start position" setting is Keep (motion to the original target position when DRIVEI2 is interrupted and then re-executed).
- In controller versions prior to those shown above, motion occurs to a new target position referenced to the current position when DRIVEI2 is interrupted and then re-executed.

Explanation:

The DRIVEI2 statement executes a relative movement of each axis in the sub group, including auxiliary axes in the sub group.

- Movement type : PTP movement on specified axis
- Point data setting : Direct coordinate data input, point definition
- Options : Speed setting, STOPON condition setting

Movement type

- PTP (point to point) movement on specified axis

PTP movement begins after positioning on all axes specified in <expression 1> is complete (within the tolerance range), and the command terminates when the specified axes enter the OUT position range. When two or more axes are specified, they will reach their target positions simultaneously.

Cautions for next command following DRIVEI2 command

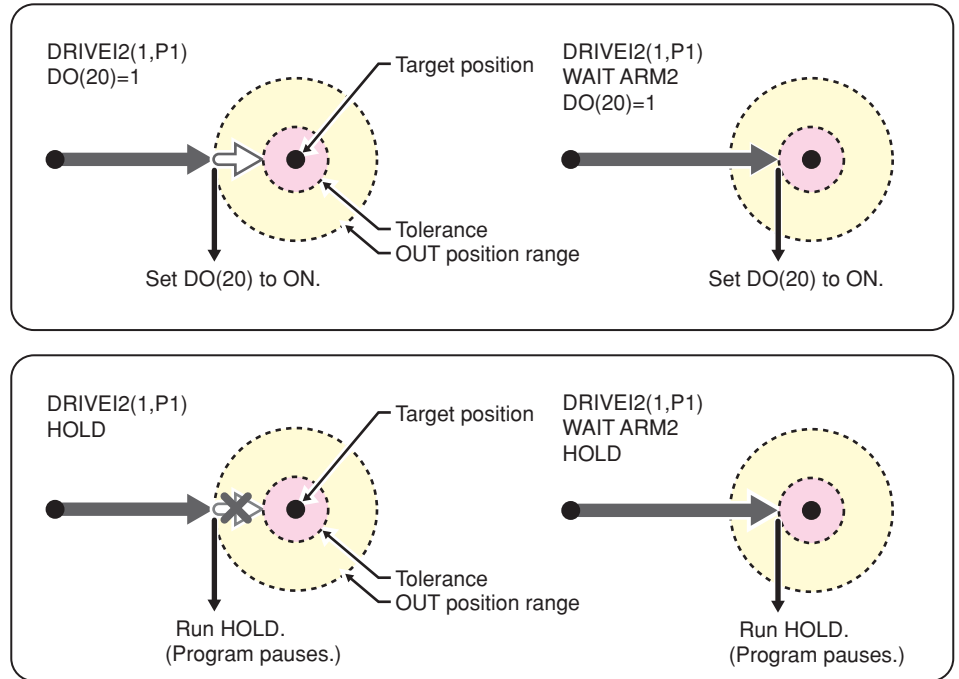
If the next command following the DRIVEI2 command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.

11. Command Statements



Example:

`DRIVEI2(1,P0)`

----- Axis 1 moves a distance specified by P0 from the current position.

Point data setting

· Direct coordinate data input

Directly specify the coordinate data in <expression 2> with an integer or real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number containing a decimal point, the units are viewed as “mm” or “deg” and the robot will move a certain distance converted for each axis.

Examples:

`DRIVEI2(1,10000)`

----- Axis 1 moves from the current position to the “+10000 pulses” position.

`DRIVEI2(2,90.00)`

----- Axis 2 moves from the current position to the “+90 degrees” position. (When axis 2 is a rotating axis.)

· Point definition

Specify the point expression in <expression 2>, while using the data for the axis specified in <expression 1>.

The robot moves to the position determined by the units used for point expression. When the units are viewed as “mm” or “deg”, the robot moves a certain distance converted for each axis.

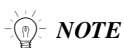
Examples:

`DRIVEI2(1,P1)`

----- Axis 1 moves a distance specified by P1 from the current position.

`DRIVEI2(2,P90)`

----- Axis 2 moves a distance specified by P90 (degrees) from the current position. (When axis 2 is a rotating axis.)



NOTE

If point data is specified with both integers and real numbers in the same statement, then all axes move in “mm” or “deg” units.

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified DRIVEI2 statement.

Example:

DRIVEI2(1,10000),S=10

----- Axis 1 moves a distance “+10000 pulses” from the current position using a program speed of 10%.

Format 2:

DSPEED	= <expression>
DS	

The value of <expression> must be from 0.01 to 100.00. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operation speed is determined only by the speed specified here, without using the automatic movement speed.

If this option is used with the torque limit setting option, then the operating speed will be 10% of the speed specified in <expression>.

This option is enabled only for the specified DRIVEI2 statement.

Example:

DRIVEI2(1,P10),DS=0.2

----- Axis 1 moves from the current position to the position specified by P10 at 0.2% of maximum speed.

· STOPON condition setting

Format:

STOPON <conditional expression>

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

DRIVEI2(1,10000),STOPON DI(20)=1

----- Axis 1 moves a distance “+10000 pulses” from the current position, and stops when the condition “DI (20) = 1” is met.

**NOTE**

· Speed setting (DSPEED) is enabled only for the following software versions.

RCX14x Ver. 8.71 onwards,

RCX22x Ver. 9.18 onwards

· SPEED option and DSPEED option cannot be used together.

**CAUTION**

When execution of the DRIVEI2 statement is halted with the STOPON option, executing the DRIVEI2 statement again on the same axis moves the robot the specified distance from the stopped position.

When executing consecutive relative movement commands, take precautions to prevent the moving sections of the robot from bumping or binding on each other.

**NOTE**

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the “TRUE condition” parameter of “Other parameters” (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining “true” and “false” (-1 is “true” and 0 is “false”, or a value other than 0 is “true” and 0 is “false”). Refer to the controller user’s manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is “true” and a value other than -1 is “false”.

EXIT FOR statement

Format:

EXIT FOR

Explanation:

The EXIT FOR statement terminates the loop of the FOR and NEXT statements and then jumps to the command following the NEXT statement.

This statement is valid only between the FOR and NEXT statements.

Example:

```
*ST:
WAIT DI(20)=1
FOR A%=101 TO 109
    MOVE P,P100,Z=0
    DO(20)=1
    MOVE P,P[A%],Z=0
    DO(20)=0
    IF DI(20)=0 THEN EXIT FOR
NEXT A%
GOTO *ST
HALT
```



NOTE

The loop of the FOR and NEXT statements will end when the FOR statement condition is satisfied or the EXIT FOR statement is executed. If another statement such as GOTO is used to jump out of the loop, an error such as “Stack overflow” may occur.

Related commands: FOR, NEXT

EXIT SUB statement

Format:

```
EXIT SUB
```

Explanation:

The EXIT SUB statement terminates the sub-procedure defined by the SUB and END SUB statements and then jumps to the next command in the CALL statement that called up the sub-procedure.

This statement is valid only within the sub-procedure defined by the SUB and END SUB statements.

Example:

```
'MAIN ROUTINE
CALL *SORT2(REF X%,REF Y%)
HALT
'SUB ROUTINE: SORT
SUB *SORT2(X%, Y%)
  IF X%>=Y% THEN EXIT SUB
  TMP%=Y%
  Y%=X%
  X%=TMP%
END SUB
```



NOTE

To end the sub-procedure defined by the SUB and END SUB statements, use the END SUB statement or EXIT SUB statement. If another statement such as GOTO is used to jump out of the sub-procedure, an error such as "Stack overflow" may occur.

Related commands: CALL, SUB, END SUB

EXIT TASK statement

Format:

EXIT TASK

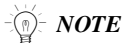
Explanation:

This statement terminates its own task currently being executed.

This statement is valid for all tasks other than task 1.

Example:

```
'TASK1 ROUTINE
*ST:
  MO(20)=0
  START *SUBTASK2,T2
  MOVE P,P0,P1
  WAIT MO(20)=1
GOTO *ST
HALT
'TASK2 ROUTINE
*SUBTASK2:
  P100=JTOXY(WHERE)
  IF LOCZ(P100)>=100.0 THEN
    MO(20)=1
    EXIT TASK
  ENDIF
  DELAY 100
GOTO *SUBTASK2
EXIT TASK
```



NOTE

*If a task that has started as a subtask jumps to another task processing routine with a statement such as **GO TO**, that processing routine is then executed as this subtask processing.*

Related commands: CUT, RESTART, START, SUSPEND, CHGPRI

FOR statement, NEXT statement

Format:

```
FOR <control variable> = <expression 1> TO <expression 2> [STEP <expression 3>]
    <command block>
NEXT [<control variable>]
```



NOTE

- If the value of <expression 3> in the FOR statement is 1, then STEP <expression 3> can be omitted.
- The FOR and NEXT statements must always be used together as a set.

Explanation:

These statements repeatedly execute commands between the FOR and NEXT statements for the number of times equal to the difference between <expression 1> and <expression 2>, while changing the value of <control variable> in steps specified by <expression 3>. The value of <expression 3> may be either positive or negative. The value of <control variable> must be a numeric <simple variable> or <array variable>.

Examples:

```
'CYCLE WITH CYCLE NUMBER OUTPUT TO DISPLAY
FOR A=1 TO 10
    MOVE P,P0
    MOVE P,P1
    MOVE P,P2
    PRINT "CYCLE NUMBER=";A
NEXT A
HALT
```

Related command: EXIT FOR

GOSUB statement, RETURN statement

Format:

```
GOSUB    <label>
GO SUB   |
          <command block>
<label> :
          <command block>
RETURN
```

Explanation:

The GOSUB statement makes a jump to the subroutine of <label> defined by the GOSUB statement.

When a RETURN statement is found in the subroutine, a jump is made to the next line in the GOSUB statement.



NOTE

- The GOSUB statement can be used up to 29 times in succession. Note that the number of times will drop if commands containing a stack such as an IF statement or CALL statement are used.
- When a jump to a subroutine was made with the GOSUB statement, always use the RETURN statement to end the subroutine. If another statement such as GOTO is used to jump out of the subroutine, an error such as "Stack overflow" may occur.

Example:

```
*ST:
  MOVE P,P0
  GOSUB *CLOSEHAND
  MOVE P,P1
  GOSUB *OPENHAND
GOTO *ST
HALT
'SUB ROUTINE
*CLOSEHAND:
  DO(20) = 1
RETURN
*OPENHAND:
  DO(20) = 0
RETURN
```

GOTO statement

Format:

GOTO	<label>
GO TO	

Explanation:

This statement makes an unconditional jump to the line specified in <label>.

Example:

```
'MAIN ROUTINE
*ST:
  MOVE P,P0,P1
  IF DI(20) = 1 THEN
    GOTO *FIN
  ENDIF
GOTO *ST
*FIN:
HALT
```

HALT statement

Format:

```
HALT [ |<expression> |
      |<character string> | ]
```

CAUTION

In PTP movement during movement commands such as MOVE and DRIVE, the next command is executed when the axis enters within the OUT position range. So if there is a HALT command right after a PTP movement command, that HALT command is executed before the axis arrives within the target position tolerance range, stopping the program and also resetting it. Likewise, in interpolation movement during MOVE/MOVE2 commands, the next command is executed right after movement starts. So if there is a HALT command right after the interpolation movement command during MOVE/MOVE2, then that HALT command is executed right after movement starts, stopping the program and also resetting it. In either of the above cases, use the WAIT ARM/WAIT ARM2 command if you want to execute the HALT command after the axis arrives within the target position tolerance range.

Explanation:

This statement stops the program and resets it. The program will run again the beginning when restarted.

Examples:

```
'MAIN ROUTINE
*ST:
  MOVE P,P0,P1
  IF DI(20) = 1 THEN
    GOTO *FIN
  ENDIF
GOTO *ST
*FIN:
  HALT "PROGRAM FIN"
```

NOTE

- If an <expression> or <character string> is written in the statement, the contents of the <expression> or <character string> are displayed on the MPB/RPB screen.
- If a program name "_SELECT" exists among the programs that were created, then the selected program will switch to "_SELECT".

HAND definition statement, CHANGE statement (Main robot hand selection)

These statements are used to define or switch the main robot hands.

Format 1: For SCARA robots

Definition statement:

HAND Hn = <1st parameter> <2nd parameter> <3rd parameter> [R]

Selection statement:

CHANGE Hn

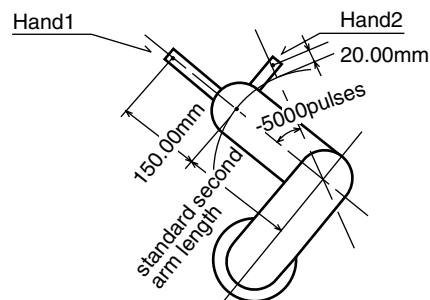
n : 0 to 3

- a. If the 4th parameter "R" is not specified, then hands installed on the second arm tip are selected (see below).

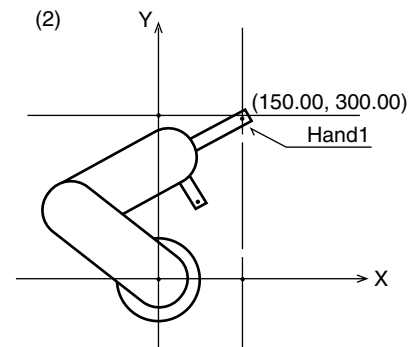
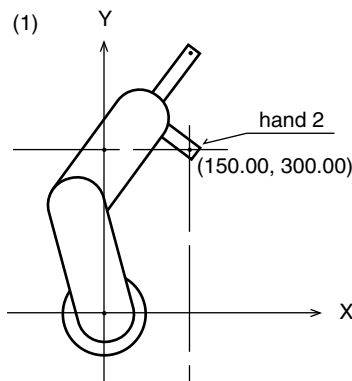
<1st parameter> : Number of offset pulses between the standard second arm position and the imaginary second arm position of hand "n". [pulse] (The "+" number indicates the counterclockwise direction.)

<2nd parameter> : Difference between the imaginary second arm length of hand "n" and the standard second arm length. [mm]

<3rd parameter> : Z-axis offset value for hand "n". [mm]



```
HAND H1= 0 150.0 0.0
HAND H2= -5000 20.00 0.0
P1=150.00 300.00 0.00 0.00 0.00 0.00
CHANGE H2
MOVE P,P1 ----- The tip of hand 2 moves to P1. (1)
CHANGE H1
MOVE P,P1 ----- The tip of hand 1 moves to P1. (2)
HALT
```

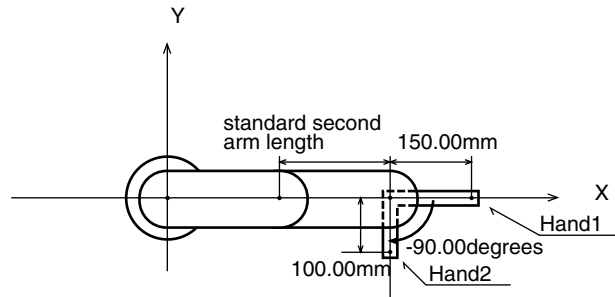


- b. When the 4th parameter “R” is specified and the R-axis uses a servo motor, those hands that are offset from the R-axis rotating center are selected (see below).

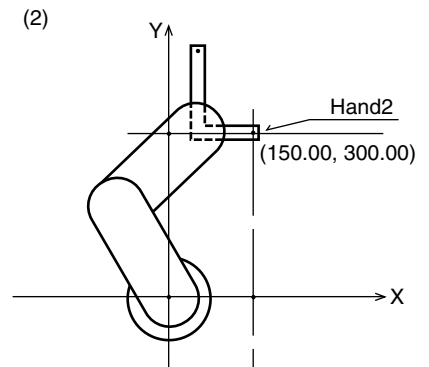
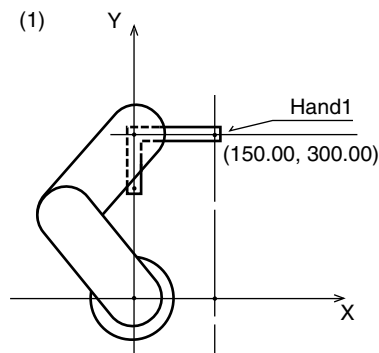
<1st parameter> : When the current position of R-axis is 0.00, this parameter shows the angle of hand “n” from the X-axis plus direction of a Cartesian coordinate system. [degree] (The “+” number indicates the counterclockwise direction.)

<2nd parameter> : Length of hand “n”. [mm] (>0)

<3rd parameter> : Z-axis offset value for hand “n”. [mm]



```
HAND H1= 0.00 150.0 0.0 R
HAND H2= -90.00 100.00 0.0 R
P1=150.00 300.00 0.00 0.00 0.00 0.00
CHANGE H1
MOVE P,P1 ----- The tip of hand 1 moves to P1. (1)
CHANGE H2
MOVE P,P1 ----- The tip of hand 2 moves to P1. (2)
HALT
```



Format 2: For Cartesian robots

Definition statement:

HAND Hn = <1st parameter> <2nd parameter> <3rd parameter> [R]

Selection statement:

CHANGE Hn

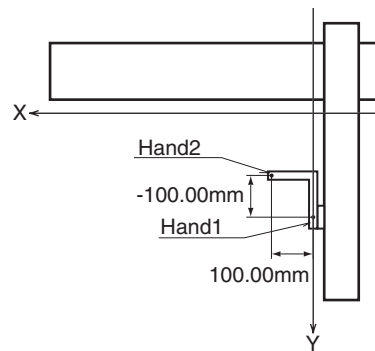
n: 0 to 3

- a. If the 4th parameter "R" is not specified, hands installed on the second arm tip are selected (see below).

<1st parameter> : X-axis offset value for hand "n". [mm]

<2nd parameter> : Y-axis offset value for hand "n". [mm]

<3rd parameter> : Z-axis offset value for hand "n". [mm]



HAND H1= 0.00 0.00 0.00

HAND H2= 100.0 -100.0 -100.0

P1=200.00 250.00 0.00 0.00 0.00 0.00

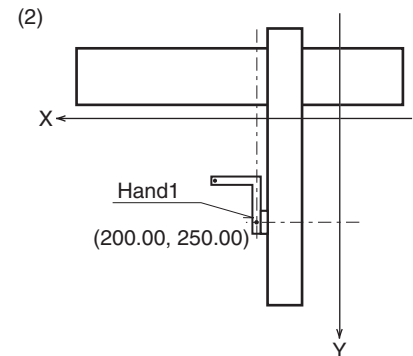
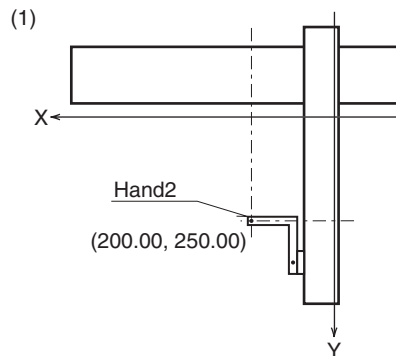
CHANGE H2

MOVE P,P1 ----- The tip of hand 2 moves to P1. (1)

CHANGE H1

MOVE P,P1 ----- The tip of hand 1 moves to P1. (2)

HALT



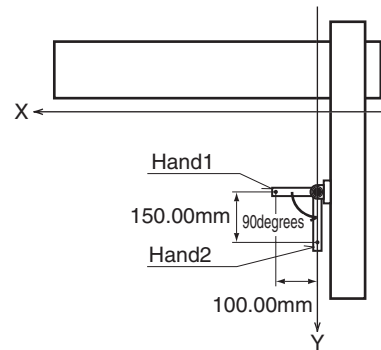
- b. When the 4th parameter “R” is specified and the R-axis uses a servo motor, those hands that are offset from the R-axis rotating center are selected (see below).

<1st parameter> : When the current position of R-axis is 0.00, this parameter shows the angle of hand “n” along the X-axis plus direction of a Cartesian coordinate system. [degree] (The “+” number indicates the counterclockwise direction.)

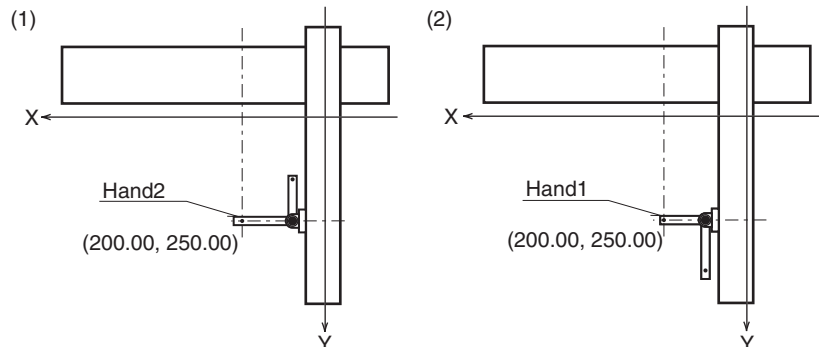
<2nd parameter> : Length of hand “n”. [mm] (> 0)

<3rd parameter> : Z-axis offset value for hand “n”. [mm]

(R-axis position is 0.00)



```
HAND H1= 0.00 100.00 0.00 R
HAND H2= 90.00 150.00 0.00 R
P1=200.00 250.00 0.00 0.00 0.00
CHANGE H2
MOVE P,P1          ----- The tip of hand 2 moves to P1. (1)
CHANGE H1
MOVE P,P1          ----- The tip of hand 1 moves to P1. (2)
```



CAUTION

If power is turned off during execution of a hand definition statement, an error “9.7: Hand data destroyed” may occur.

HAND2 definition statement, CHANGE2 statement (Sub robot hand selection)

These statements are used to define or switch the sub robot hands.

Format 1: For SCARA robots

Definition statement:

HAND 2 Hn = <1st parameter> <2nd parameter> <3rd parameter> [R]

Selection statement:

CHANGE2 Hn

n : 4 to 7



NOTE

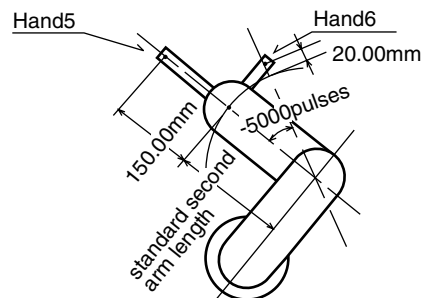
This statement can be used only when the sub group is set in the system generation.

- a. If the 4th parameter “R” is not specified, those hands installed on the second arm tip are selected (see below).

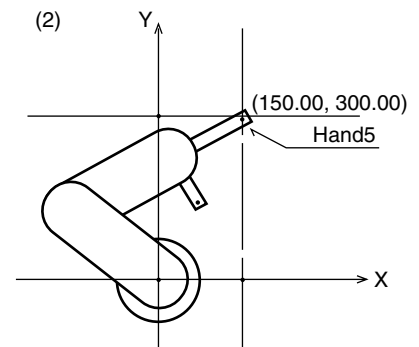
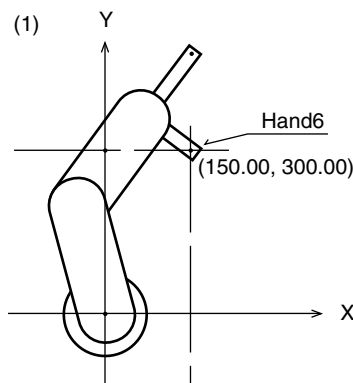
<1st parameter> : Number of offset pulses between the standard second arm position and the imaginary second arm position of hand “n”. [pulse] (The “+” number indicates the counterclockwise direction.)

<2nd parameter> : Difference between the imaginary second arm length of hand “n” and the standard second arm length. [mm]

<3rd parameter> : Z-axis offset value for hand “n”. [mm]



```
HAND2 H5= 0 150.0 0.0
HAND2 H6= -5000 20.00 0.0
P1=150.00 300.00 0.00 0.00 0.00 0.00
CHANGE2 H6
MOVE2 P,P1 ----- The tip of hand 6 moves to P1. (1)
CHANGE2 H5
MOVE2 P,P1 ----- The tip of hand 5 moves to P1. (2)
HALT
```



11. Command Statements

- b. When the 4th parameter “R” is specified and the R-axis uses a servo motor, those hands that are offset from the R-axis rotating center are selected (see below).

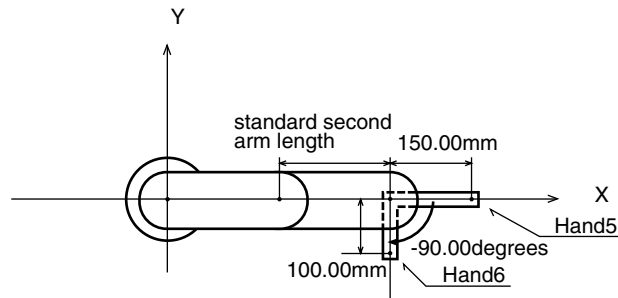
<1st parameter>: When the current position of R-axis is 0.00, this parameter shows the angle of hand “n” along the X-axis plus direction of a Cartesian coordinate system. [degree] (The “+” number indicates the counterclockwise direction.)

<2nd parameter>: Length of hand “n”. [mm] (>0)

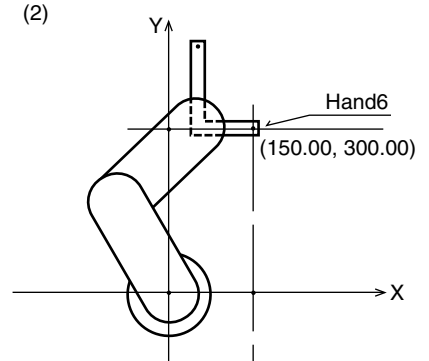
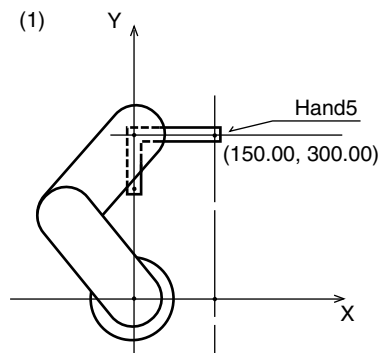
<3rd parameter>: Z-axis offset value for hand “n”. [mm]

Examples:

```
HAND2 H5= 0.00 150.00 0.0 R
HAND2 H6= -90.00 100.00 0.0 R
```



```
HAND2 H5= 0.00 150.0 0.0 R
HAND2 H6= -90.00 100.00 0.0 R
P1=150.00 300.00 0.00 0.00 0.00 0.00
CHANGE2 H5
MOVE2 P,P1 ----- The tip of hand 5 moves to P1. (1)
CHANGE2 H6
MOVE2 P,P1 ----- The tip of hand 6 moves to P1. (2)
HALT
```



Format 2: For Cartesian robots

Definition statement:

HAND 2 Hn = <1st parameter> <2nd parameter> <3rd parameter> [R]

Selection statement:

CHANGE2 Hn

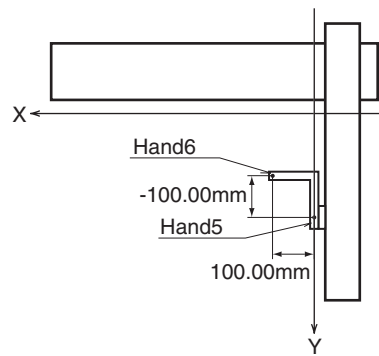
n : 4 to 7

- a. If the 4th parameter "R" is not specified, hands installed on the second arm tip are selected (see below).

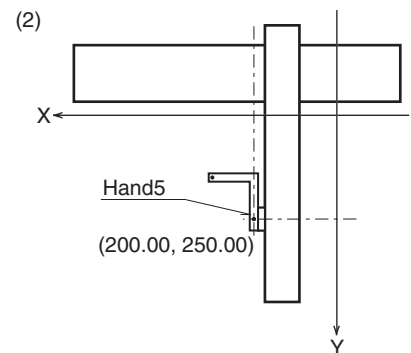
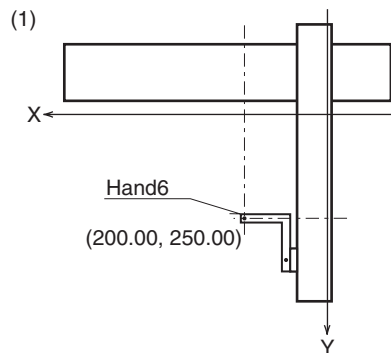
<1st parameter> : X-axis offset value for hand "n". [mm]

<2nd parameter> : Y-axis offset value for hand "n". [mm]

<3rd parameter> : Z-axis offset value for hand "n". [mm]



```
HAND2 H5= 0.00 0.00 0.00
HAND2 H6= 100.0 -100.0 -100.0
P1=200.00 250.00 0.00 0.00 0.00 0.00
CHANGE2 H6
MOVE2 P,P1 ----- The tip of hand 6 moves to P1. (1)
CHANGE2 H5
MOVE2 P,P1 ----- The tip of hand 5 moves to P1. (2)
HALT
```



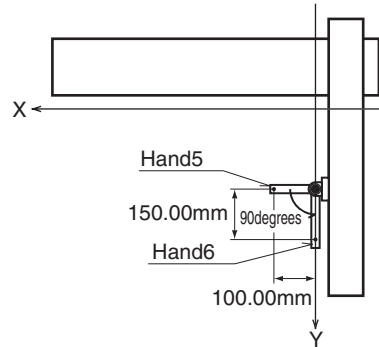
- b. When the 4th parameter “R” is specified and the R-axis uses a servo motor, those hands that are offset from the R-axis rotating center are selected (see below).

<1st parameter> : When the current position of R-axis is 0.00, this parameter shows the angle of hand “n” along the X-axis plus direction of a Cartesian coordinate system. [degree] (The “+” number indicates the counterclockwise direction.)

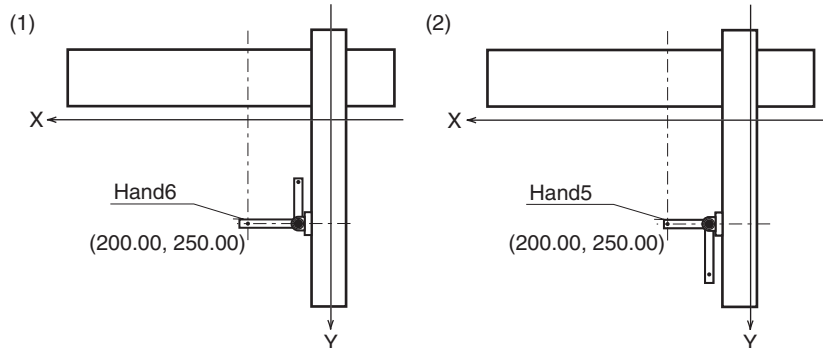
<2nd parameter> : Length of hand “n”. [mm] (> 0)

<3rd parameter> : Z-axis offset value for hand “n”. [mm]

(R-axis position is 0.00)



```
HAND2 H5= 0.00 100.00 0.00 R
HAND2 H6= 90.00 150.00 0.00 R
P1=200.00 250.00 0.00 0.00 0.00 0.00
CHANGE2 H6
MOVE2 P,P1 ----- The tip of hand 6 moves to P1. (1)
CHANGE2 H5
MOVE2 P,P1 ----- The tip of hand 5 moves to P1. (2)
HALT
```



CAUTION

If power is turned off during execution of a hand definition statement, an error “9.7: Hand data destroyed” may occur.

HOLD statement

Format:

```
HOLD [ |<expression> |  
      |<character string> | ]
```

CAUTION

In PTP movement during movement commands such as MOVE and DRIVE, the next command is executed when the axis enters within the OUT position range. So if there is a HOLD command right after a PTP movement command, that HOLD command is executed before the axis arrives within the target position tolerance range, causing the program to stop.

Likewise, in interpolation movement during MOVE/MOVE2 commands, the next command is executed right after movement starts. So if there is a HOLD command right after the interpolation movement command during MOVE/MOVE2, then that HOLD command is executed right after movement starts, causing the program to stop.

In either of the above cases, use the WAIT ARM/WAIT ARM2 command if you want to execute the HOLD command after the axis arrives within the target position tolerance range.

Explanation:

This statement temporarily pauses execution of the program. When the program is re-executed, it begins from the line following the HOLD statement.

Example:

```
'MAIN ROUTINE  
*ST:  
  MOVE P,P0,P1  
  IF DI(20)=1 THEN  
    HOLD "PROGRAM STOP"  
  ENDIF  
GOTO *ST  
HALT
```

NOTE

If an <expression> or <character string> is written in the statement, the contents of the <expression> or <character string> are displayed on the MPB/RPB screen.

IF statement

a. Simple IF statement

Format 1:

IF <conditional expression> THEN	<label 1>	[ELSE	<label 2>	]
	<statement 1>		<statement 2>	



NOTE

On controllers using the following software versions, when the conditional expression used by <conditional expression> is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

If the condition specified by <conditional expression> is met (true), this statement allows a jump to the line specified by <label 1> or executes the <statement 1> and jumps to the next line. If the condition is not met (false), the command jumps to the line specified by <label 2> or executes the <statement 2> and then jumps to the next line.

Example:

```
'MAIN ROUTINE
*ST:
  MOVE P,P0,P1
  IF DI(20)=1 THEN *L1
  DO(20)=1
  DELAY 100
*L1:
  IF DI(21)=1 THEN *ST ELSE *FIN
*FIN:
HALT
```

b. Block IF statement

Format 2:

**NOTE**

The IF statement can be used up to 48 times in succession. Note that the maximum number of times will drop if commands containing a stack such as an GOSUB statement or CALL statement are used.

```
IF <conditional expression 1> THEN
    <command block 1>
[ ELSEIF <conditional expression 2> THEN
    <command block 2>]
[ ELSE
    <command block n>]
ENDIF
```

**NOTE**

On controllers using the following software versions, when the conditional expression used by <conditional expression> is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

If the condition specified by <conditional expression 1> is met (true), this statement executes the instructions specified in <command block 1> and makes a jump to the line under ENDIF. When an ELSEIF statement is present and the condition specified by <conditional expression 2> is met (true), the instructions specified in <command block 2> are executed. If the conditions specified by all of <conditional expression n> are not satisfied (false), the instructions specified in <command block n> are executed and a jump is made to the line under ENDIF.

Example:

```
'MAIN ROUTINE
*ST:
    MOVE P,P0,P1
    IF DI(21,20)=1 THEN
        DO(20)= 1
        DELAY 100
        WAIT DI(20)=0
    ELSEIF DI(21,20)=2 THEN
        DELAY 100
    ELSE
        GOTO *FIN
    ENDIF
GOTO *ST
*FIN:
HALT
```

INPUT statement

Format:

INPUT [<prompt statement>	;	]	<variable>	[,	<variable>	, ...]
	,		<point variable>		<point variable>	
			<shift variable>		<shift variable>	

Explanation:

This statement assigns a value to the variable specified by the MPB or RPB. The input definitions are as follows:

1. When two or more variables are specified by separating them with a comma (,), the specified pieces of input data must also be separated with a comma (,).
2. In <prompt statement>, enter a character string enclosed in quotation marks (" ") that will appear as a message requiring data input. When a semicolon (;) is entered following the prompt statement, a question mark (?) and a space will appear at the end of the message. When a comma (,) is entered, nothing will be displayed following the message.
3. When <prompt statement> is omitted, only a question mark (?) and a space will be displayed.
4. The input data type must match the type of the corresponding variables. When data is input to a point variable or shift variable, insufficient elements are set to "0".
5. If only the Enter key is pressed without making any entry, the program has determined that "0" or "null string" was input. However, if specifying two or more variables, a comma (,) must be used to separate them.
6. If the specified variable is a character type and you want to enter a significant space before and after a comma (,), double quotation mark (" ") or character string, the character variable must be enclosed in double quotation marks (" "). Note in this case, you must enter two double quotation marks in succession so that they will be identified as a double quotation mark input.
7. Pressing the ESC key skips input of this statement.

NOTE

- If assignment data type differs from type of variable, a message "? Input again" appears and awaits data input.
- When assigning alphabetic characters and numbers to a character variable, enclosing the character string in double quotation marks (" ") is not necessary.

Example:

```
INPUT A
INPUT "INPUT POINT NUMBER";A1
INPUT "INPUT STRING",B$(0),B$(1)
INPUT P100
HALT
```


LET statement (Assignment statement)

Format:



NOTE

If the controller power is turned off during execution of a point assignment statement or shift assignment statement, a memory-related error such as "Point data destroyed" and "Shift data destroyed" may occur.

[LET]	<arithmetic statement > <character string assignation statement > <point data assignation statement > <shift data assignation statement >
-------	--

Explanation:

The LET statement executes the specified assignment statement.

a. Arithmetic statement

Format:

[LET]	<arithmetic variable> <point element variable> <shift element variable> <parallel output variable> <internal output variable> <arm lock output variable> <timer output variable> <serial output variable>	= <expression>
-------	--	----------------

Explanation:

The value of <expression> is assigned to the variable on the left side.

Example:

```

A!=B!+1
B%(1,2,3)=INT(10.88)
LOCZ(P0)=A!
LOCX(S1)=100.00
DO2()=&B00101101
MO(21,20)=2
LO(00)=1
TO(01)=0
SO12()=255
  
```

11. Command Statements

b. Character string assignment statement

Format:

```
[LET] <character string variable> = <character string expression>
```

Explanation:

The value of <character string expression> is assigned to the character string variable.

Example:

```
A$ = "YAMAHA"  
B$ = "ROBOT"  
D$ = A$ + "-" + B$
```

c. Point assignment statement

Format:

```
[LET] <point variable> = <point expression>
```

Explanation:

The value of <point expression> is assigned to the point variable.

Example:

```
P1=P10          ----- Point 10 is assigned to P1.  
P20=P20+P5      ----- Each element of point 20 and point 5 is summed and assigned  
                  to P20.  
P30=P30-P3      ----- Each element of point 3 is subtracted from point 30 and  
                  assigned to P30.  
P80=P70*4        ----- Each element of point 70 is multiplied by 4 and assigned to  
                  P80.  
P60=P5/3         ----- Each element of point 5 is divided by 3 and assigned to P60.
```

d. Shift assignment statement

Format:

```
[LET] <shift variable> = <shift expression>
```

Explanation:

The value of <shift expression> is assigned to the shift variable.

Example:

```
S1=S1          ----- Shift 0 is assigned to shift 1.  
S2=S1+S0      ----- Each element of shift 1 and shift 0 are summed and assigned  
                  to shift 2.
```



NOTE

Only "+" sign can be used as arithmetic operators in <character string expression>. Other arithmetic operators and parentheses cannot be used.



NOTE

Only 4 arithmetic operators "+", "-", "*", "/" can be used in <point expression>. Multiplication and division are performed only with constants or variables.
Valid examples: P15*5, P [E] / A
Invalid examples: P10*P11, 3 / P10
Multiplication and division results will differ depending on the type of point data.
When the point data is in "pulses", each element of the arithmetic result is converted to an integer and assigned to the point variable.
When the point data is in "mm", each element of the arithmetic result is converted to a real number up to the second decimal places and then assigned to the point variable.



NOTE

Only "+" and "-" signs can be used as arithmetic operators in <shift expression>. Other arithmetic operators and parentheses cannot be used.
The only element that can be used in <shift expression> is a shift element.
Invalid example: S 1 + 3

LO statement (Arm lock output)

Format:

$\left[\begin{array}{l} \text{LET} \\ \text{LO} \end{array} \right] \left(\begin{array}{l} [b, \dots, b] \\ [0b, \dots, 0b] \end{array} \right) = \text{expression}$
--

b : bit definition 0 to 7

The value of <expression> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

Explanation:

This statement outputs the specified value to the LO port to prohibit axis movement or allow axis movement.

LO(00) to LO(07) respectively correspond to axes 1 to 8. Axis movement is prohibited in ON status.



NOTE

- This statement can be used for the axis that starts to move.
- Bits must be specified in ascending order from the right.

Example:

```
LO0()=&B00001010
----- Prohibits movement of axes 2 and 4.
LO0(2,1)=&B10
----- Prohibits movement of axis 3.
```

Related commands: RESET, SET

MO statement (Internal output)

Format:

[LET]	MOm ([b, . . . , b])	= <expression>
	MO (mb, . . . , mb)	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

The value of <expression> should be an integer or it will be converted to an integer and the lower bits corresponding to the bits specified on the left side will be valid.

Explanation:

This statement outputs the specified value to the MO ports.



NOTE

- Output to the MO0 () and MO1 () ports is not allowed.
- The output status of origin sensors for axes 1 to 8 (in order from bit 0) is available to the MO0 () port. Each bit sets to "1" when the origin sensor turns ON, and to "0" when OFF.
- The HOLD status of axes 1 to 8 (in order from bit 0) is output to the MO1 () port. Each bit sets to "1" when the axis is in a HOLD status, and to "0" when not. Being in HOLD status means that the axis has already reached the target position.
- Bits must be specified in ascending order from the right.

Examples:

MO2()=&B10111000

----- MO (27, 25, 24, 23) are turned ON, and MO (26, 22, 21, 20) are turned OFF.

MO2(6,5,1)=&B010

----- MO (25) is turned ON, and MO (26, 21) are turned OFF.

MO3() = 15

----- MO (33, 32, 31, 30) are turned ON, and MO (37, 36, 35, 34) are turned OFF.

MO(37,35,27,20)=A

----- The contents of the 4 lower bits obtained when variable A is converted to an integer are output to MO (37, 35, 27, 20), respectively.

Related commands: RESET, SET

MOVE statement (Absolute position movement command)

Format:

MOVE	PTP	, <point definition> [, option [, option] ...]
	P	
	L	
	C	

Explanation:

This statement executes an absolute movement of main robot axes. This statement will not move the sub robot axes and auxiliary axes.

Movement type : PTP, linear interpolation, circular interpolation

Point data setting : Direct coordinate data input, point definition

Options : Speed setting, arch motion setting, STOPON condition setting, CONT setting, acceleration setting, deceleration setting, coordinate plane setting, port output setting

Movement type

· PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

Cautions for next command following MOVE P command

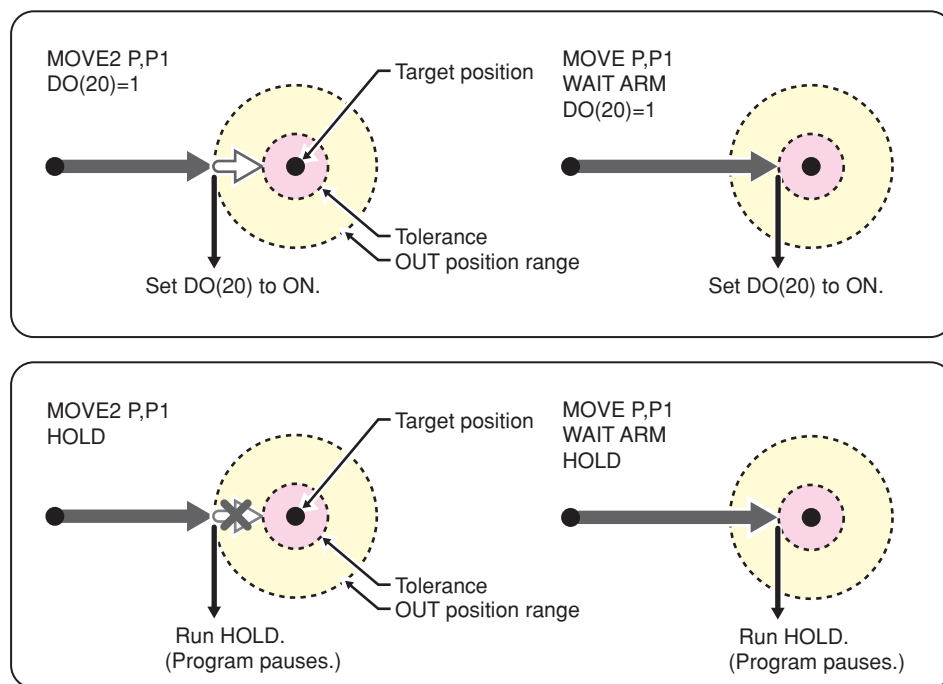
If the next command following the MOVE P command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.

11. Command Statements



Example:

```
MOVE P,P0
MOVE PTP,P0
```

----- Main robot axes move from the current position to the position specified by P0.

NOTE

- On the RCX14x controllers earlier than Ver. 8.64 and RCX22x controllers earlier than Ver. 9.11, linear interpolation can be performed by task 1 only.
- Smooth travel paths might not always be possible depending on the axis movement speed, acceleration and distance between the target points.
- Movement speed between the target points might not be constant depending on the axis movement speed, acceleration and distance between the target points.
- On robots having an R-axis, the R-axis speed may become too fast and cause an error depending on the R-axis movement distance.

· Linear interpolation movement

Linear movement starts after operations of all moving axes end (after each axis arrives within the tolerance range) and the command then ends immediately afterward (Here, "command ends immediately afterward" means a command processing transition on the program, where the movement axis moves to target position.). So the immediately following command is executed right after that axis starts moving. For example, if the immediate following command is a signal output command (such as DO), then the signal is output right after movement starts.

If the immediately prior command is a MOVE command using linear interpolation or circular interpolation, then the acceleration/deceleration sections before and after the movement path are connected, so the movement axis can move without stopping. The movement axes arrive simultaneously.

However, if a STOPON condition was specified, then the movement operation starts, and the program waits until operations of all movement axes are complete (each axis arrives within the tolerance range).

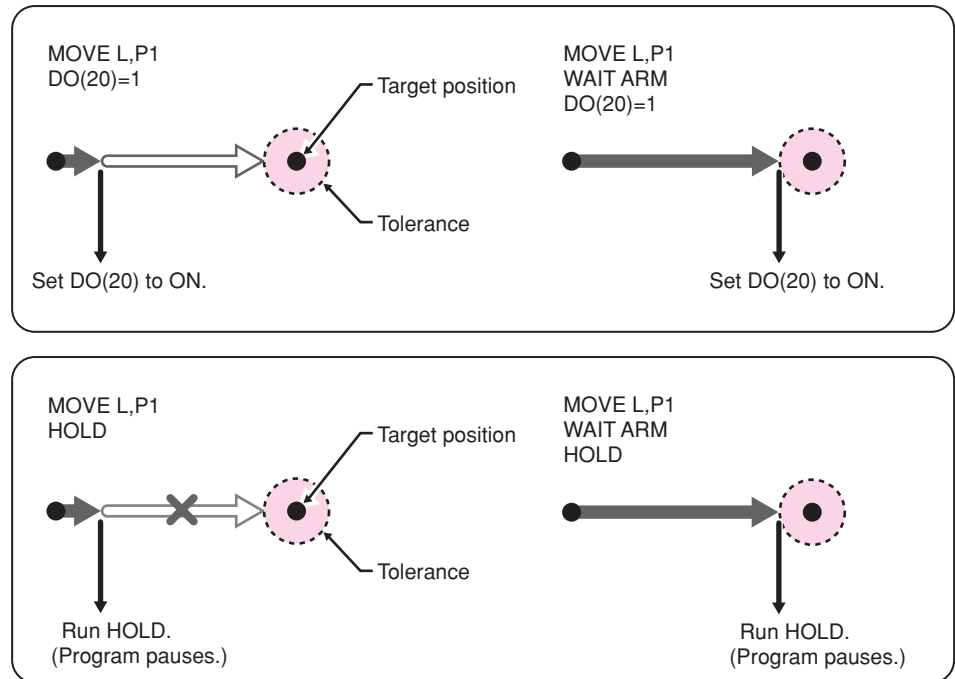
Cautions for next command following MOVE L command

If the next command following the MOVE L command is an executable command such as a signal output command, that next command will start immediately after axis movement starts.

Examples:

Signal output (DO, etc.)	Signal is output immediately after axis movement starts.
DELAY	DELAY command is executed and time standby starts, immediately after axis movement starts.
HALT	Program stops and is also reset immediately after axis movement starts. So axis movement stops.
HOLD	Program temporarily stops immediately after axis movement starts. So axis movement stops.
WAIT	WAIT command is executed immediately after axis movement starts.

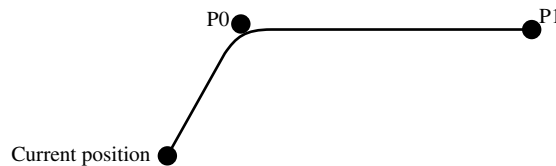
To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.



Example:

`MOVE L,P0,P1`

----- Main robot axes move from the current position to the position specified by P0 and then to the position specified by P1.



NOTE

- The movable range for circular interpolation is 5000.00mm to 1.00mm in radius.
- On the RCX14x controllers earlier than Ver. 8.64 and RCX22x controllers earlier than Ver. 9.11, linear interpolation can be performed by task 1 only.
- Circular interpolation must be specified by an even number of points.
- Circular arc may become distorted depending on the axis movement speed, acceleration and the circular arc radius.
- On robots having an R-axis, the R-axis speed may become too fast and cause an error depending on the R-axis movement distance.

• Circular interpolation movement

In circular interpolation, 3 positions consisting of the current position, middle-point position and target position form one arc.

Circular interpolation starts right after operations of all moving axes end (after each axis arrives within tolerance range), and the MOVE command ends immediately afterward (Here, "command ends immediately afterward" means a command processing transition on the program, where the movement axis moves to target position.).

So the immediately following command is executed right after that axis starts moving. For example, if the immediate following command is a signal output command (such as DO) then the signal is output right after movement starts.

If the immediately prior command is a MOVE command using linear interpolation or circular interpolation, then the acceleration/deceleration sections before and after the movement path are connected, so the movement axis can move without stopping. The movement axes arrive simultaneously.

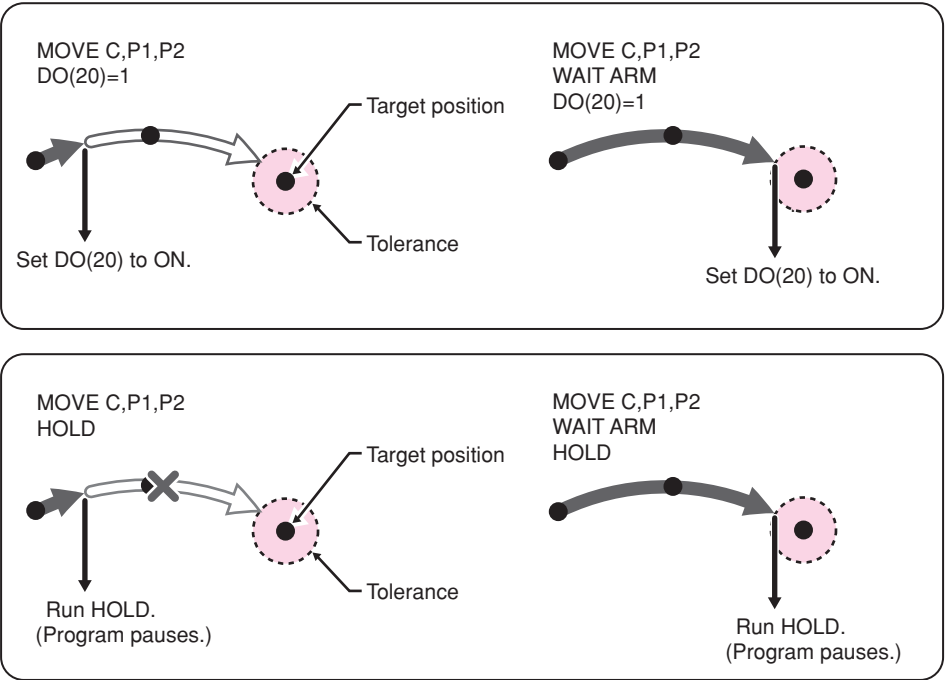
Cautions for next command following MOVE C command

If the next command following the MOVE C command is an executable command such as a signal output command, that next command will start immediately after axis movement starts.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

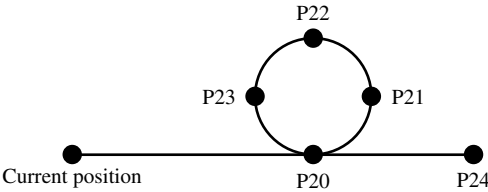
To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.



Example:

```
MOVE L,P20
MOVE C,P21,P22,P23,P20
MOVE L,P24
```

----- Moves linearly from the current position to P20, then moves along a circle formed by points P20, P21, P22, P23 and P20, and moves again linearly to P24.



Point data setting

· Direct coordinate data input

Format:

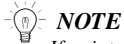
X Y Z R A B

X, Y, Z, R, A and B are coordinate values on respective axes and must be separated from each other by a space.

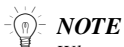
Explanation:

Directly specify the coordinate data on each axis with an integer or a real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number, the units are viewed as “mm” or “deg”.

PTP movement and linear interpolation movement can be used with this coordinate data setting.

**NOTE**

If point data is specified with both integers and real numbers in the same statement, all axes move in “mm” or “deg” units.

**NOTE**

· When a SCARA robot is used and a hand system flag is set for the target position data, that hand system has priority over the current arm type.
· Hand system flags are only available from software version 8.08 onwards.

Hand system flags can be specified when directly specifying the coordinate data in “mm” units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to move to a specified position.

2 : Left-handed system is used to move to a specified position.

**CAUTION**

When performing linear interpolation while specifying a hand system flag, make sure that the same hand system is used at the current position and target position. If the same hand system is not used, then an error will occur and robot movement will not be allowed.

Example:

MOVE P,10000 10000 1000 1000 0 0

----- Moves by PTP from the current position to the specified positions.

· Point definition

Format:

<point expression> [, <point expression> ...]

**NOTE**

· Circular interpolation must be specified by an even number of points.
· When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

Explanation:

Specify the point data in <point expression>. Two or more pieces of data can be designated by separating them with a comma (,).

Example:

MOVE P,P1

----- Moves from the current position to the position specified by P1.

MOVE P,P20,P0,P100

----- Moves in order from the current position to the positions specified by P20, P0 and P100.

**CAUTION**

When moving the robot by linear or circular interpolation to a point where a hand system flag is specified, make sure that the same hand system is used at the current position and target position. If the same hand system is not used, then an error will occur and robot movement will not be allowed.

Option types

· Speed setting

Format 1:

$\left \begin{array}{c} \text{SPEED} \\ \text{S} \end{array} \right $	$= <\text{expression}>$
--	-------------------------

The value of <expression> must be from 1 to 100. (Unit: %)

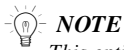
Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified MOVE statement.

Example:

MOVE P,P10,S=10

----- Moves from the current position to the position specified by P10 using a program speed of 10%.



NOTE

This option merely specifies the maximum speed and does not guarantee the specified speed.

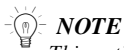
Format 2:

VEL = <expression>

The value of <expression> must be from 1 to 750 for SCARA robots, and from 1 to 1000 for Cartesian robots. (Unit: mm/sec)

Explanation:

Specify the maximum composite speed of the XYZ axes in <expression> in "mm/sec" units. This option can be used for linear interpolation and circular interpolation movements of SCARA robots or Cartesian robots and is enabled only for the specified MOVE statement.



NOTE

This option merely specifies the maximum composite speed and does not guarantee the specified speed.

Example:

MOVE L,P10,VEL=100

----- Moves at a maximum XYZ composite speed of 100mm/sec from the current position to the position specified by P10.

· Arch motion setting

Format:

$x = <\text{expression}>, [x = <\text{expression}> \dots]$
--

x : Axis X, Y, Z, R, A or B

When the value of <expression> is an integer, the movement units are viewed as "pulses". When the value is a real number containing a decimal point, the units are viewed as "mm" or "deg".

Explanation:

The specified "x" axis first moves toward the position specified in <expression>. Then, when the specified "x" axis enters the arch position range, all axes other than the "x" axis move toward the target position. Finally, all axes other than the "x" axis enter the arch position range and also the specified "x" axis enters within the tolerance range of the position specified in <expression>, the specified "x" axis moves toward the target position, and when it enters the OUT position range, the command terminates.

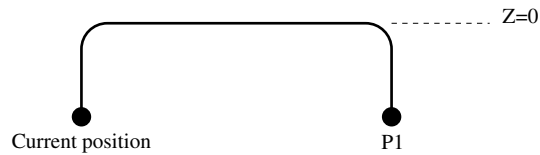
This option can be used for PTP movement.

If the specified "x" axes include an X-axis or Y-axis, the target position and the <expression> must be specified by an integer (in "pulse" units).

Example:

MOVE P,P1,Z=0

----- The Z-axis first moves from the current position to the "0 pulse" position and then other axes move to P1. Finally the Z-axis moves to P1.



· STOPON condition setting

Format:

STOPON <conditional expression>

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

MOVE P,P100,STOPON DI(20)=1

----- Moves from the current position to the position specified by P100, and slows down and stops when the condition "DI (20) = 1" is met.

· CONT setting

Format:

CONT

Explanation:

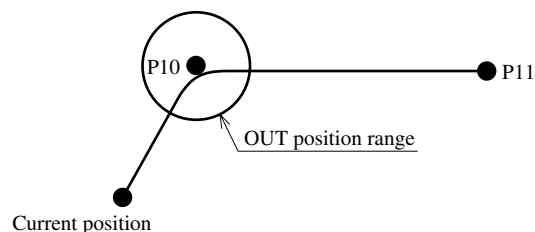
When PTP movement is executed with the CONT setting option, PTP movement (to be executed immediately after all movable axes enter the OUT position range and the command terminates) will begin without waiting for the movable axes to complete the operation (reach within target position tolerance range).

This option can be used for PTP movement and is enabled only for the specified MOVE statement.

Example:

MOVE P,P10,P11,CONT

----- Moves from the current position to the position specified by P10, and then moves to P11 without waiting for the movable axes to enter the tolerance range.



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

NOTE

- The positioning time for starting PTP movement can be reduced by CONT setting.
- The path to the target point is not guaranteed.

11. Command Statements

· Acceleration setting

Format:

`ACC = <expression>`

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the robot acceleration rate in <expression>. The actual robot acceleration is determined by the product of the acceleration coefficient parameter setting and this acceleration rate.

This option can be used for linear interpolation movement and is enabled only for the specified MOVE statement.

Example:

```
MOVE L,P100,ACC=10
```

----- Moves from the current position to the position specified by P100 at 10% acceleration rate.

· Deceleration setting

Format:

`DEC = <expression>`

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the robot deceleration rate in <expression>. The actual robot deceleration is determined by the product of the acceleration coefficient parameter setting and this deceleration rate.

This option can be used for linear interpolation movement and is enabled only for the specified MOVE statement.

Example:

```
MOVE L,P100,DEC=20
```

----- Moves from the current position to the position specified by P100 at 20% deceleration rate.

· Coordinate plane setting

Format:

XY
YZ
ZX

NOTE

- If no coordinate plane is specified, the robot moves along a 3-dimensional circle.
- When a 2-axis robot is used, it moves along a circle on the XY plane.

XY : XY coordinate plane

YZ : YZ coordinate plane

ZX : ZX coordinate plane

Explanation:

When a circular interpolation is executed by setting the coordinates, this option executes circular interpolation so that the projection on the specified coordinate plane becomes a circle.

This option can be used for circular interpolation movement and is enabled only for the specified MOVE statement.

Example:

```
P10 = 100.00 100.00 20.00 0.00 0.00 0.00
P11 = 150.00 100.00 0.00 0.00 0.00 0.00
P12 = 150.00 150.00 20.00 0.00 0.00 0.00
P13 = 100.00 150.00 40.00 0.00 0.00 0.00
MOVE P,P10
MOVE C,P11,P12
MOVE C,P13,P10
```

----- Continuously moves along a 3-dimensional circle formed by P10, P11 and P12, and also P12, P13 and P10.

```
MOVE C,P11,P12,XY
MOVE C,P13,P10,XY
```

----- Continuously moves along a circle on the XY plane, formed by P10, P11 and P12, and also P12, P13 and P10. At this point, the Z-axis moves to a position specified by P12 and P10 as the target position of the circular movement.



CAUTION

The port output setting is only available from software version 8.22 onwards.

· Port output setting

Format 1:

DO	m ([b, ... ,b]) =<expression 1>@<expression 2>
MO	
SO	

Format 2:

DO	(mb, ... ,mb) =<expression 1>@<expression 2>
MO	
SO	



NOTE

- Output to the DO0 () and DO1 () ports is not allowed.
- Output to the MO0 () and MO1 () ports is not allowed.
- Output to the SO0 () and SO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO/ SO ports not connected to hardware.

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit position 0 to 7

The value of <expression 1> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

The value of <expression 2> must be in "mm" units.

Explanation:

During linear interpolation or circular interpolation movement, this command option outputs the value of <expression 1> to the specified port when the robot reaches a position that is away from the start position by a distance specified in <expression 2>.

This command option can only be used with linear interpolation or circular interpolation movement. Up to 2 options can be set in one MOVE statement.

Example 1:

```
MOVE P,P0
MOVE L,P1,DO2( )=105@25.85
```

----- Outputs 105 (&B01101001) to DO2() at a position 25.85mm away from P0 during liner interpolation movement toward P1.

Example 2:

```
A!=10
B!=20
MOVE L,P2,MO(22)=1@A!,MO(22)=0@B!
```

----- Turns MO(22) ON at a position 10mm away from the start position and then turns MO(22) OFF at a position 20mm away from the start position during movement toward P2.

Related commands: MOVEI, DRIVE, DRIVEI, WAIT ARM

MOVE2 statement (Absolute position movement command)

Format:

MOVE2	PTP	, <point definition> [, option [, option] ...]
	P	
	L	
	C	

Explanation:

This statement executes an absolute movement of all sub robot axes. This statement will not move the main robot axes and auxiliary axes.

Movement type : PTP, linear interpolation, circular interpolation

Point data setting : Direct coordinate data input, point definition

Options : Speed setting, arch motion setting, STOPON condition setting, CONT setting, acceleration setting, deceleration setting, coordinate plane setting, port output setting

Movement type

· PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

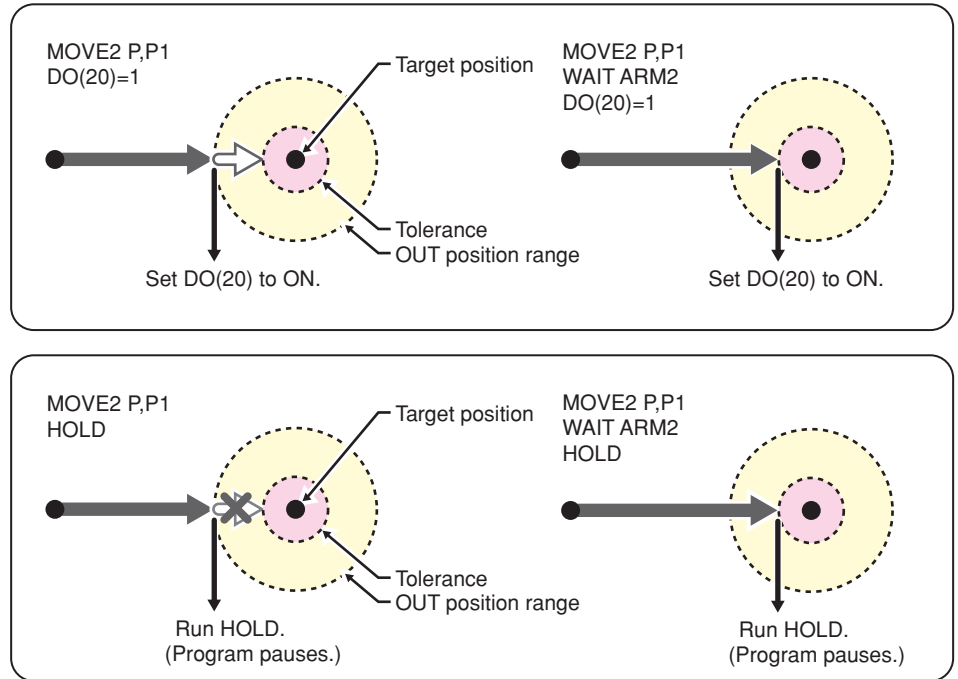
Cautions for next command following MOVE2 P command

If the next command following the MOVE2 P command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

```
MOVE2 P,P0
MOVE2 PTP,P0
```

----- Sub robot axes move from the current position to the position specified by P0.



NOTE

Linear interpolation is available from software version 8.64 onwards (RCX14x) or 9.11 onwards (RCX22x).



NOTE

- Smooth travel paths might not always be possible depending on the axis movement speed, acceleration and distance between the target points.
- Movement speed between the target points might not be constant depending on the axis movement speed, acceleration and distance between the target points.
- On robots having an R-axis, the R-axis speed may become too fast and cause an error depending on the R-axis movement distance.

Linear interpolation movement

Linear movement starts after operations of all moving axes end (after each axis arrives within the tolerance range) and the command then ends immediately afterward (Here, "command ends immediately afterward" means a command processing transition on the program, where the movement axis moves to target position.). So the immediately following command is executed right after that axis starts moving. For example, if the immediate following command is a signal output command (such as DO), then the signal is output right after movement starts.

If the immediately prior command is a MOVE2 command using linear interpolation or circular interpolation, then the acceleration/deceleration sections before and after the movement path are connected, so the movement axis can move without stopping. The movement axes arrive simultaneously.

However, if a STOPON condition was specified, then the movement operation starts, and the program waits until operations of all movement axes are complete (each axis arrives within the tolerance range).

Cautions for next command following MOVE2 L command

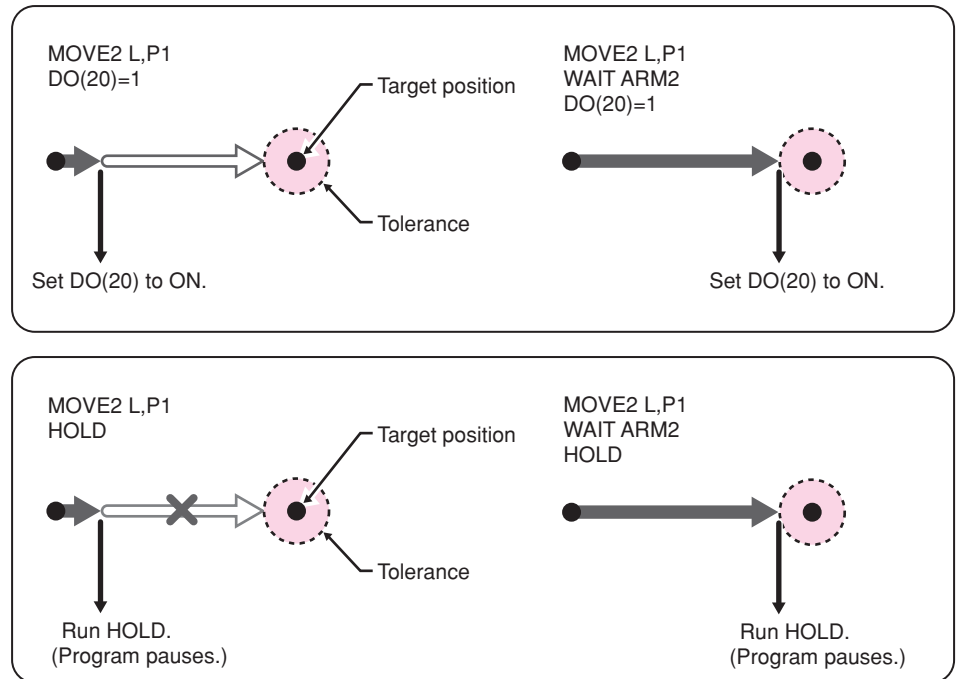
If the next command following the MOVE2 L command is an executable command such as a signal output command, that next command will start immediately after axis movement starts.

Examples:

Signal output (DO, etc.)	Signal is output immediately after axis movement starts.
DELAY	DELAY command is executed and time standby starts, immediately after axis movement starts.
HALT	Program stops and is also reset immediately after axis movement starts. So axis movement stops.
HOLD	Program temporarily stops immediately after axis movement starts. So axis movement stops.
WAIT	WAIT command is executed immediately after axis movement starts.

11. Command Statements

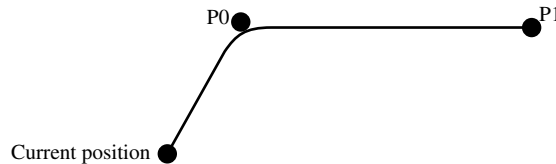
To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

MOVE2 L,P0,P1

----- Sub robot axes move from the current position to the position specified by P0 and then to the position specified by P1.



NOTE

Circular interpolation is available from software version 8.64 onwards (RCX14x) or 9.11 onwards (RCX22x).

NOTE

- The movable range for circular interpolation is 5000.00mm to 1.00mm in radius.
- Circular interpolation must be specified by an even number of points.
- Circular arc may become distorted depending on the axis movement speed, acceleration and the circular arc radius.
- On robots having an R-axis, the R-axis speed may become too fast and cause an error depending on the R-axis movement distance.

• Circular interpolation movement

In circular interpolation, 3 positions consisting of the current position, middle-point position and target position form one arc.

Circular interpolation starts right after operations of all moving axes end (after each axis arrives within tolerance range), and the MOVE2 command ends immediately afterward (Here, "command ends immediately afterward" means a command processing transition on the program, where the movement axis moves to target position.).

So the immediately following command is executed right after that axis starts moving. For example, if the immediate following command is a signal output command (such as DO) then the signal is output right after movement starts.

If the immediately prior command is a MOVE2 command using linear interpolation or circular interpolation, then the acceleration/deceleration sections before and after the movement path are connected, so the movement axis can move without stopping. The movement axes arrive simultaneously.

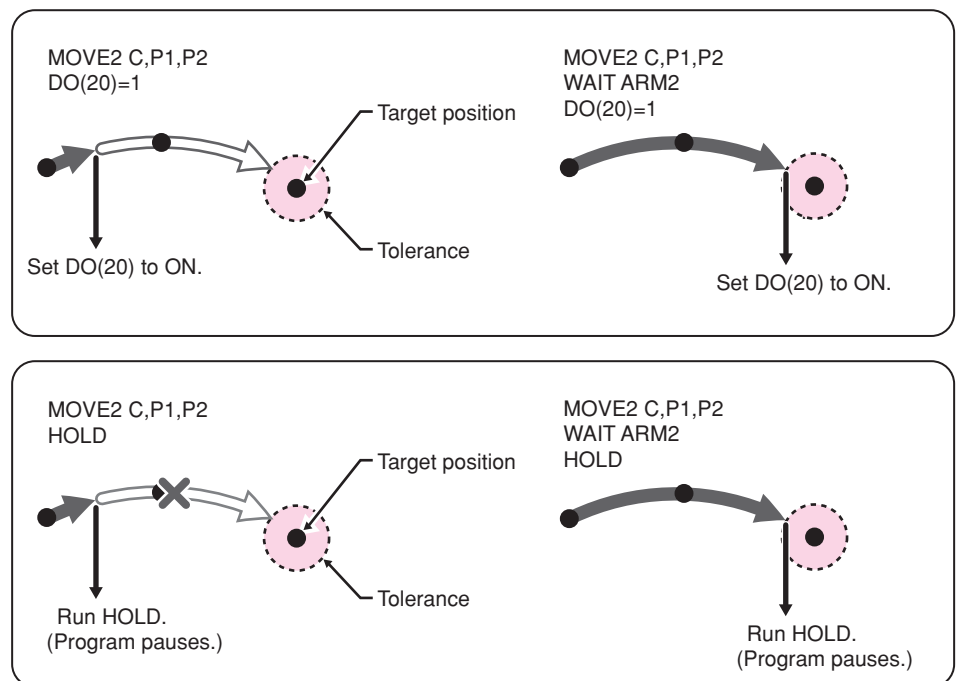
Cautions for next command following MOVE2 C command

If the next command following the MOVE2 C command is an executable command such as a signal output command, that next command will start immediately after axis movement starts.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

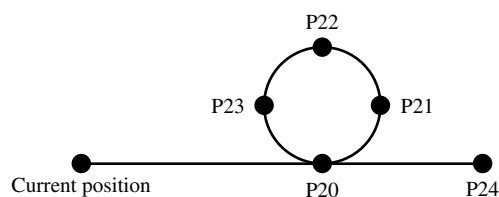
To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

```
MOVE2 L,P20
MOVE2 C,P21,P22,P23,P20
MOVE2 L,P24
```

----- Moves linearly from the current position to P20, then moves along a circle formed by points P20, P21, P22, P23 and P20, and moves again linearly to P24.



11. Command Statements

Point data setting

· Direct coordinate data input

Format:

X Y Z R A B

X, Y, Z, R, A and B are the coordinate values on respective axes, and must be separated from each other by a space.

Explanation:

Directly specify the coordinate data on each axis with an integer or a real number. When the coordinate data is an integer, the movement units are viewed as “pulses”. When the coordinate data is a real number, the units are viewed as “mm” or “deg”.

PTP movement and linear interpolation movement can be used with this coordinate data setting.

Hand system flags can be specified when directly specifying the coordinate data in “mm” units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to move to a specified position.

2 : Left-handed system is used to move to a specified position.

Example:

MOVE2 P,10000 10000 1000 1000 0 0

----- Moves by PTP from the current position to the specified positions.

· Point definition

Format:

<point expression> [, <point expression> ...]

Explanation:

Specify the point data in <point expression>. Two or more pieces of data can be designated by separating them with a comma (,).

Example:

MOVE2 P,P1

----- Moves from the current position to the position specified by P1.

MOVE2 P,P20,P0,P100

----- Moves in order from the current position to the positions specified by P20, P0 and P100.



NOTE

If point data is specified with both integers and real numbers in the same statement, all axes move in “mm” or “deg” units.



NOTE

- When a SCARA robot is used and a hand system flag is set for the target position data, that hand system has priority over the current arm type.
- Hand system flags are only available from software version 8.08 onwards.



CAUTION

When performing linear interpolation while specifying a hand system flag, make sure that the same hand system is used at the current position and target position. If the same hand system is not used, then an error will occur and robot movement will not be allowed.



NOTE

- Circular interpolation must be specified by an even number of points.
- When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.



CAUTION

When moving the robot by linear or circular interpolation to a point where a hand system flag is specified, make sure that the same hand system is used at the current position and target position. If the same hand system is not used, then an error will occur and robot movement will not be allowed.

Option types

· Speed setting

Format 1:

$\left \begin{array}{l} \text{SPEED} \\ S \end{array} \right = \langle \text{expression} \rangle$

The value of $\langle \text{expression} \rangle$ must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in $\langle \text{expression} \rangle$. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified MOVE2 statement.

**NOTE**

This option merely specifies the maximum speed and does not guarantee the specified speed.

Example:

```
MOVE2 P,P10,S=10
```

----- Moves from the current position to the position specified by P10 using a program speed of 10%.

Format 2:

VEL = $\langle \text{expression} \rangle$

The value of $\langle \text{expression} \rangle$ must be from 1 to 750 for SCARA robots, and from 1 to 1000 for Cartesian robots. (Unit: mm/sec)

**NOTE**

This option merely specifies the maximum composite speed and does not guarantee the specified speed.

Explanation:

Specify the maximum composite speed of the XYZ axes in $\langle \text{expression} \rangle$ in "mm/sec" units. This option can be used for linear interpolation and circular interpolation movements of SCARA robots or Cartesian robots and is enabled only for the specified MOVE2 statement.

Example:

```
MOVE2 L,P10,VEL=100
```

----- Moves at a maximum XYZ composite speed of 100mm/sec from the current position to the position specified by P10.

· Arch motion setting

Format:

$x = \langle \text{expression} \rangle, [x = \langle \text{expression} \rangle \dots]$
--

x : Axis X, Y, Z, R, A or B

When the value of $\langle \text{expression} \rangle$ is an integer, the movement units are viewed as "pulses". When the value is a real number containing a decimal point, the units are viewed as "mm" or "deg".

Explanation:

The specified "x" axis first moves toward the position specified in $\langle \text{expression} \rangle$. Then, when the specified "x" axis enters the arch position range, all axes other than the "x" axis move toward the target position. Finally, all axes other than the "x" axis enter the arch position range and also the specified "x" axis enters within the tolerance range of the position specified in $\langle \text{expression} \rangle$, the specified "x" axis moves toward the target position, and when it enters the OUT position range, the command terminates.

This option can be used for PTP movement.

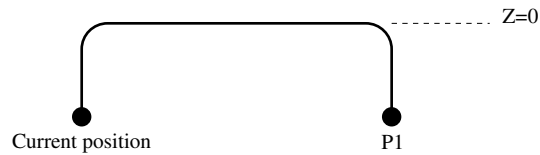
If the specified "x" axes include an X-axis or Y-axis, the target position and the $\langle \text{expression} \rangle$ must be specified by an integer (in "pulse" units).

11. Command Statements

Example:

MOVE2 P,P1,Z=0

----- The Z-axis first moves from the current position to the “0 pulse” position and then other axes move to P1. Finally the Z-axis moves to P1.



· STOPON condition setting

Format:

STOPON <conditional expression>



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

MOVE2 P,P100,STOPON DI(20)=1

----- Moves from the current position to the position specified by P100, and slows down and stops when the condition “DI (20) = 1” is met.

· CONT setting

Format:

CONT

Explanation:

When PTP movement is executed with the CONT setting option, PTP movement (to be executed immediately after all movable axes enter the OUT position range and the command terminates) will begin without waiting for the movable axes to complete the operation (reach within target position tolerance range). This option can be used for PTP movement and is enabled only for the specified MOVE2 statement.



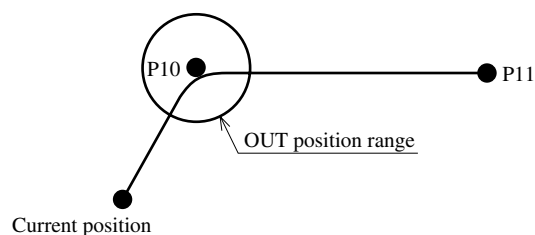
NOTE

- The positioning time for starting PTP movement can be reduced by CONT setting.
- The path to the target point is not guaranteed.

Example:

MOVE2 P,P10,P11,CONT

----- Moves from the current position to the position specified by P10, and then moves to P11 without waiting for the movable axes to enter the tolerance range.



· Acceleration setting

Format:

ACC = <expression>

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the robot acceleration rate in <expression>. The actual robot acceleration is determined by the product of the acceleration coefficient parameter setting and this acceleration rate.

This option can be used for linear interpolation movement and is enabled only for the specified MOVE2 statement.

Example:

MOVE2 L,P100,ACC=10

----- Moves from the current position to the position specified by P100 at 10% acceleration rate.

· Deceleration setting

Format:

DEC = <expression>

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the robot deceleration rate in <expression>. The actual robot deceleration is determined by the product of the acceleration coefficient parameter setting and this deceleration rate.

This option can be used for linear interpolation movement and is enabled only for the specified MOVE2 statement.

Example:

MOVE2 L,P100,DEC=20

----- Moves from the current position to the position specified by P100 at 20% deceleration rate.

· Coordinate plane setting

Format:

XY
YZ
ZX



NOTE

- If no coordinate plane is specified, the robot moves along a 3-dimensional circle.
- When a 2-axis robot is used, it moves along a circle on the XY plane.

XY : XY coordinate plane

YZ : YZ coordinate plane

ZX : ZX coordinate plane

Explanation:

When a circular interpolation is executed by setting the coordinates, this option executes circular interpolation so that the projection on the specified coordinate plane becomes a circle.

This option can be used for circular interpolation movement and is enabled only for the specified MOVE2 statement.

11. Command Statements

Example:

```
P10 = 100.00 100.00 20.00 0.00 0.00 0.00
P11 = 150.00 100.00 0.00 0.00 0.00 0.00
P12 = 150.00 150.00 20.00 0.00 0.00 0.00
P13 = 100.00 150.00 40.00 0.00 0.00 0.00
MOVE2 P,P10
MOVE2 C,P11,P12
MOVE2 C,P13,P10
```

----- Continuously moves along a 3-dimensional circle formed by P10, P11 and P12, and also P12, P13 and P10.

```
MOVE2 C,P11,P12,XY
MOVE2 C,P13,P10,XY
```

----- Continuously moves along a circle on the XY plane, formed by P10, P11 and P12, and also P12, P13 and P10. At this point, the Z-axis moves to a position specified by P12 and P10 as the target position of the circular movement.

· Port output setting

Format 1:

DO	m ([b, ... ,b]) =<expression 1>@<expression 2>
MO	
SO	

Format 2:

DO	(mb, ... ,mb) =<expression 1>@<expression 2>
MO	
SO	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit position 0 to 7

The value of <expression 1> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

The value of <expression 2> must be in "mm" units.

Explanation:

During linear interpolation or circular interpolation movement, this command option outputs the value of <expression 1> to the specified port when the robot reaches a position that is away from the start position by a distance specified in <expression 2>.

This command option can only be used with linear interpolation or circular interpolation movement. Up to 2 options can be set in one MOVE2 statement.



NOTE

- Output to the DO0 () and DO1 () ports is not allowed.
- Output to the MO0 () and MO1 () ports is not allowed.
- Output to the SO0 () and SO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO/ SO ports not connected to hardware.

Example 1:

```
MOVE2 P,P0
MOVE2 L,P1,DO2( )=105@25.85
```

----- Outputs 105 (&B01101001) to DO2() at a position 25.85mm away from P0 during linear interpolation movement toward P1.

Example 2:

```
A!=10
B!=20
MOVE2 L,P2,MO(22)=1@A!,MO(22)=0@B!
```

----- Turns MO(22) ON at a position 10mm away from the start position and then turns MO(22) OFF at a position 20mm away from the start position during movement toward P2.

Related commands: MOVEI2, DRIVE2, DRIVEI2, WAIT ARM2

**NOTE**

- If the MOVEI statement is interrupted and then re-executed in the controller versions shown below, the resumed motion can occur either to the original target position, or to a new target position referenced to the current position (stop position), in accordance with the "MOVEI/DRIVEI start position" setting selected at the other parameters.

For details, refer to the controller user's manual.

RCX14x Ver. 8.66 onwards

RCX22x Ver. 9.13 onwards

- The other parameters default "MOVEI/DRIVEI start position" setting is Keep (motion to the original target position when MOVEI is interrupted and then re-executed).
- In controller versions prior to those shown above, motion occurs to a new target position referenced to the current position when MOVEI is interrupted and then re-executed.

MOVEI statement (Relative position movement command)

Format:

```
MOVEI | PTP | , <point definition> [, option [, option] ... ]
      | P   |
```

Explanation:

This statement executes a relative movement of all main robot axes. This statement will not move the sub robot axes and auxiliary axes.

Movement type : PTP

Point data setting : Direct coordinate data input, point definition

Options : Speed setting

Movement type

- PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command then terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

Cautions for next command following MOVEI command

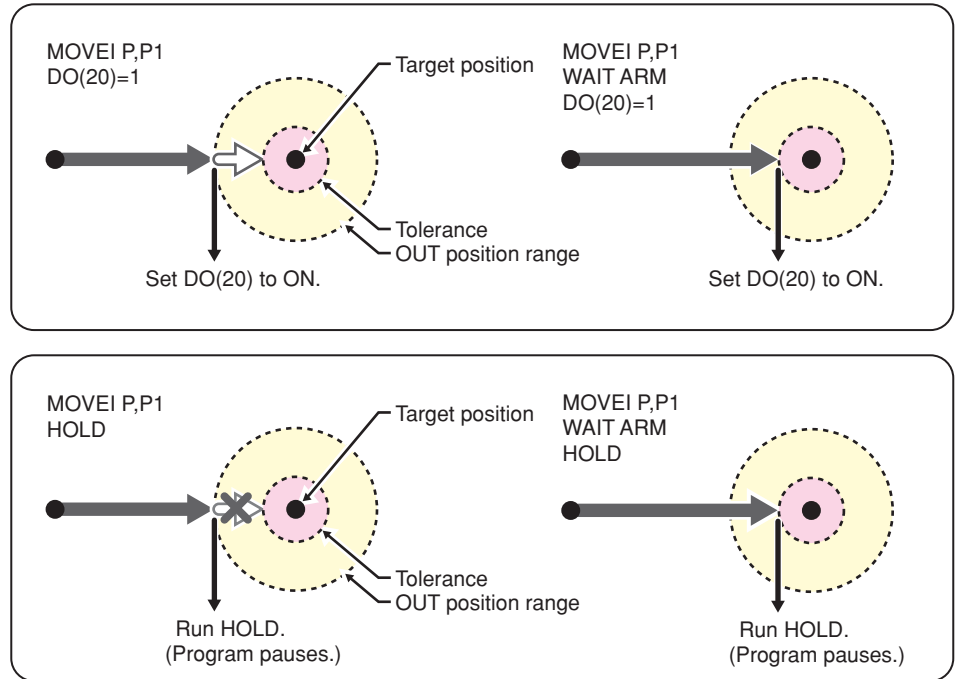
If the next command following the MOVEI command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.

11. Command Statements



Example:

```
MOVEI P,P0
MOVEI PTP,P0
```

----- Main robot axes move by PTP a distance specified by P0 from the current position.

Point data setting

· Direct coordinate data input

Format:

X Y Z R A B

X, Y, Z, R, A and B are the coordinate values on respective axes and must be separated from each other by a space.

NOTE

If point data is specified with both integers and real numbers in the same statement, all axes move in "mm" or "deg" units.

Explanation:

Directly specify the coordinate data on each axis with an integer or a real number. When the coordinate data is an integer, the movement units are viewed as "pulses". When the coordinate data is a real number, the units are viewed as "mm" or "deg".

Hand system flags can be specified when directly specifying the coordinate data in "mm" units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to move to a specified position.

2 : Left-handed system is used to move to a specified position.

Example:

```
MOVEI P, 10000 10000 1000 1000 0 0
```

----- Moves by PTP the specified distances from the current position.

· Point definition

Format:

<point expression> [, <point expression> ...]



NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

Explanation:

Specify the point data in <point expression>. Two or more pieces of data can be designated by separating them with a comma (,).

Example:

MOVEI P,P1

----- Moves by PTP a distance specified by P1 from the current position.

Option type

· Speed setting

Format 1:

| SPEED | = <expression>
| S |

The value of <expression> must be from 1 to 100. (Unit: %)



NOTE

This option merely specifies the maximum speed and does not guarantee the specified speed.

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified MOVEI statement.

Example:

MOVEI P,P10,S=10

----- Moves by PTP a distance specified by P1 from the current position using a program speed of 10%.

Related commands: MOVE, DRIVE, DRIVEI, WAIT ARM

MOVEI2 statement (Relative position movement command)



NOTE

- If the MOVEI2 statement is interrupted and then re-executed in the controller versions shown below, the resumed motion can occur either to the original target position, or to a new target position referenced to the current position (stop position), in accordance with the "MOVEI/DRIVEI start position" setting selected at the other parameters.
For details, refer to the controller user's manual.
RCX14x Ver. 8.66 onwards
RCX22x Ver. 9.13 onwards
- The other parameters default "MOVEI/DRIVEI start position" setting is Keep (motion to the original target position when MOVEI2 is interrupted and then re-executed).
- In controller versions prior to those shown above, motion occurs to a new target position referenced to the current position when MOVEI2 is interrupted and then re-executed.

Format:

```
MOVEI2 | PTP | , <point definition> [ , option [ , option ] ... ]
      | P |
```

Explanation:

This statement executes a relative movement of all sun robot axes. This statement will not move the main robot axes and auxiliary axes.

Movement type : PTP

Point data setting : Direct coordinate data input, point definition

Options : Speed setting

Movement type

- PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

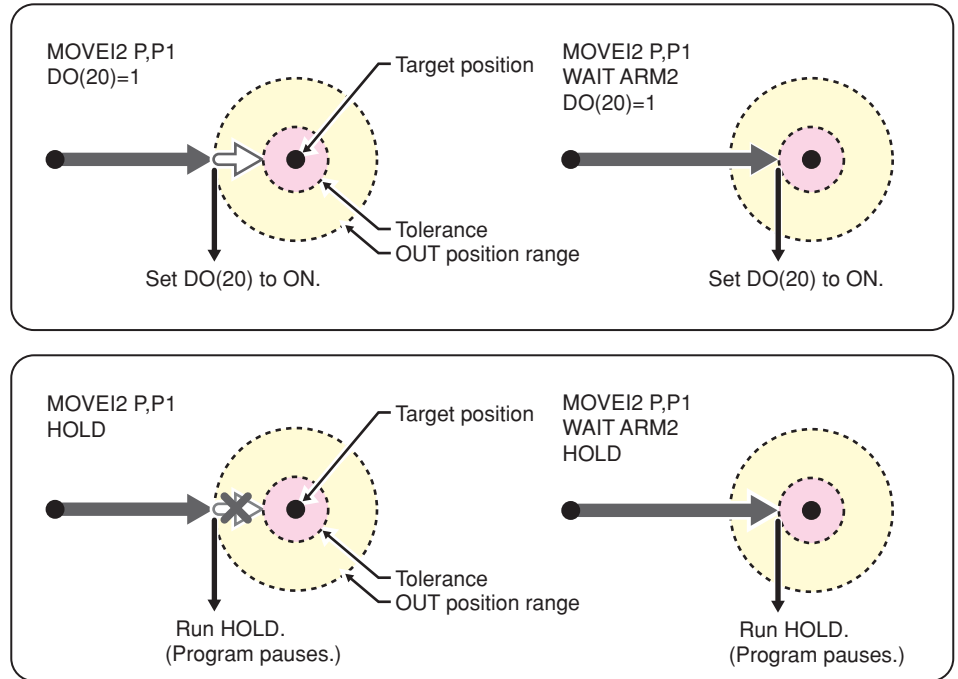
Cautions for next command following MOVEI2 command

If the next command following the MOVEI2 command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

```
MOVEI2 P,P0
MOVEI2 PTP,P0
```

----- Sub robot axes move by PTP a distance specified by P0 from the current position.

Point data setting

· Direct coordinate data input

Format:

X Y Z R A B

X, Y, Z, R, A and B are the coordinate values on respective axes and must be separated from each other by a space.

NOTE

If point data is specified with both integers and real numbers in the same statement, all axes will move in "mm" or "deg" units.

Explanation:

Directly specify the coordinate data on each axis with an integer or a real number. When the coordinate data is an integer, the movement units are viewed as "pulses". When the coordinate data is a real number, the units are viewed as "mm" or "deg".

Hand system flags can be specified when directly specifying the coordinate data in "mm" units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to move to a specified position.

2 : Left-handed system is used to move to a specified position.

Example:

```
MOVEI2 P,10000 10000 1000 1000 0 0
```

----- Moves by PTP the specified distances from the current position.

NOTE

- When a SCARA robot is used and a hand system flag is set for the target position data, that hand system has priority over the current arm type.
- Hand system flags are only available from software version 8.08 onwards.

11. Command Statements

· Point definition

Format:

`<point expression> [, <point expression> ...]`



NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

Explanation:

Specify the point data in <point expression>. Two or more pieces of data can be designated by separating them with a comma (,).

Example:

`MOVEI2 P,P1`

----- Moves by PTP a distance specified by P1 from the current position.

Option type

· Speed setting

Format 1:

SPEEDS	= <expression>
---	----------------

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified MOVEI2 statement.



NOTE

This option merely specifies the maximum speed and does not guarantee the specified speed.

Example:

`MOVEI2 P,P10,S=10`

----- Moves by PTP a distance specified by P1 from the current position using a program speed of 10%.

Related commands: MOVE2, DRIVE2, DRIVEI2, WAIT ARM2

ON ERROR GOTO statement

Format:

ON ERROR GOTO	<label>
	0

Error output information ERR : Error number
 ERL : Line number of error

Explanation:

Even if an error occurs during execution of the robot language, this statement allows the program to jump to the error processing routine specified by <label>, so that the program still continues without being suspended. (Some serious errors are excluded.) If "0" is specified for <label> in this statement, the program stops when an error is issued and the error message is displayed.

The error processing routine processes the error using the RESUME statement and handles the error output information.



NOTE

- If a serious error such as "Over load" occurs, the program execution stops.
- The last "ON ERROR GOTO <label>" statement that was used is valid.
- If an error occurs while an error processing routine is being executed, the program will stop.
- "ON ERROR GOTO <label>" statements cannot be executed within error processing routines.

Example:

```
ON ERROR GOTO *ER1
FOR A = 0 TO 9
  P[A+10] = P[A]
NEXT A
*L99: HALT
'ERROR ROUTINE
*ER1:
IF ERR = &H0604 THEN *NEXT1
----- Checks whether an error "Point doesn't exist" has occurred.
IF ERR = &H0606 THEN *NEXT2
----- Checks whether an error "Subscript out of range" has
                                         occurred.
ON ERROR GOTO 0
----- Displays the error message and stops the program execution.
*NEXT1:
  RESUME NEXT
----- Jumps to the next command line after the line where the error
                                         occurred, and continues executing the program.
*NEXT2:
  RESUME *L99
----- Jumps to the label "*L99" and continues executing the
                                         program.
```

Related commands: RESUME

ON to GOTO statement

Format:

ON <expression>	GOTO	<label 1> [, <label 2> . . .]
	GO TO	

The value of <expression> must be “0” or a positive integer.

Explanation:

In this statement, the value of <expression> determines where the program will jump. When the value of <expression> is “1”, the program will jump to <label 1>. When it is “2”, the program will jump to <label 2>. Likewise, when it is “n”, the program will jump to <label n>.

If the value of <expression> is “0” or greater than the number of labels specified, the next command is executed without making a jump.

Example:

```
'MAIN ROUTINE
*ST:
ON DI3() GOTO *L1,*L2,*L3
GOTO *ST
HALT
'SUB ROUTINE
*L1:
    MOVE P,P10,Z=0
    GOTO *ST
*L2:
    DO(30) = 1
    GOTO *ST
*L3:
    DO(30) = 0
    GOTO *ST
```

Related commands: GOTO, DECLARE

ON to GOSUB statement

Format:

ON <expression>	GOSUB	<label 1> [, <label 2>. . .]
	GO SUB	

The value of <expression> must be “0” or a positive integer.

Explanation:

In this statement, the value of <expression> determines where the program will jump. When the value of <expression> is “1”, the program will jump to <label 1>. When it is “2”, the program will jump to <label 2>. Likewise, when it is “n”, the program will jump to <label n>.

After the subroutine at the destination jumped to has been executed, the next command following the ON to GOSUB statement is executed.

If the value of <expression> is “0” or greater than the number of labels, the next command is executed without making a jump.

Example:

```
'MAIN ROUTINE
*ST:
ON DI3() GOSUB *SUB1,*SUB2,*SUB3
GOTO *ST
HALT
'SUB ROUTINE
*SUB1:
    MOVE P,P10,Z=0
    RETURN
*SUB2:
    DO(30) = 1
    RETURN
*SUB3:
    DO(30) = 0
    RETURN
```

Related commands: GOSUB, RETURN, DECLARE

ONLINE statement, OFFLINE statement

Format:

ONLINE	[	ETH	]
		CMU	
OFFLINE	[	ETH	]
		CMU	



NOTE

- An applicable port can be specified only when the software version is as follows.

RCX240 Ver. 10.20 onwards

RCX22x Ver. 9.27 onwards

In controller versions prior to those shown above, an applicable port cannot be specified and only the RS-232C can be operated.

Explanation:

These statements change the communication mode parameter of the specified port to change the communication mode (ONLINE/OFFLINE). Additionally, these statements clear the transmission or receive buffer or initialize the communication port, depending on the specified port.

An applicable port is specified using the port option following ONLINE/OFFLINE.

ETH

Changes the communication mode parameter of the Ethernet and clears the transmission and receive buffers.

CMU or No port is specified.

Changes the communication mode parameter of the RS-232C and initializes the communication port (resets the communication error and clears the receive buffer).

Example:

```
OFFLINE
SEND CMU TO A$
SEND CMU TO P10
ONLINE
HALT
```


ORGORD statement

(Return-to-origin sequence setting statement for main group)

Format:

ORGORD <expression>

The value of <expression> must be an integer from 0 to 654321.

Explanation:

This statement sets the axis sequence parameter to perform return-to-origin and absolute search operations for the main group.

Example:

```
A=3
ORGORD A
ABSRST
```

----- After return-to-origin operation on axis 3 of the main group is complete, absolute reset is executed by performing simultaneous return-to-origin on the remaining axes.

```
MOVE P,P0
HALT
```

Related commands: ABSRST, ORIGIN, ORGORD2

ORGORD2 statement **(Return-to-origin sequence setting statement for sub group)**

Format:

```
ORGORD2 <expression>
```

The value of <expression> must be an integer from 0 to 4321.

Explanation:

This statement sets the axis sequence parameter to perform return-to-origin and absolute search operations for the sub group.

Example:

```
A=1234  
ORGORD2 A  
ABSRST  
MOVE2 P,P0  
HALT
```

Related commands: ABSRST, ORIGIN, ORGORD

ORIGIN statement

Format:

ORIGIN

Explanation:

This statement performs return-to-origin on an incremental mode axis or absolute search on a semi-absolute axis.

If the movement is stopped halfway, then origin is incomplete.

When two robots are specified, return-to-origin and absolute search are first performed on the main group and then on the sub robot group.

Example:

ORIGIN

----- Performs return-to-origin on an incremental mode axis.

(Performs an absolute search on a semi-absolute mode axis.)

Related commands: ABSRST, ORGORD, ORGORD2, MCHREF, MCHREF2



CAUTION
The **OUT** statement is only available
from software version 8.22 onwards.

OUT statement

Format:

OUT	DOm ([b, . . . , b])	[, <expression>]
	DO (mb, . . . , mb)	
	MOm ([b, . . . , b])	
	MO (mb, . . . , mb)	
	SOm ([b, . . . , b])	
	SO (mb, . . . , mb)	
	LO0 ([b, . . . , b])	
	LO (0b, . . . , 0b)	
	TO0 ([b, . . . , b])	
	TO (0b, . . . , 0b)	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

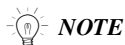
The value of <expression> must be from 1 to 3600000. (Unit: ms) Note that the minimum run time is 10ms.

Explanation:

This statement turns ON the specified port and terminates. (The program proceeds to the next line.) The output to that port is then turned OFF when automatic operation has continued for the time specified by <expression>. If the operation is stopped temporarily and then restarted, the output to the specified port is turned OFF when the remainder of the specified time has elapsed.

Figures below 10 in the <expression> value are rounded down. If a value below 10 is specified, then it is set to 10. If this expression is omitted, then the specified port is kept turned ON.

Up to 16 OUT statements using <expression> can be executed at the same time. If attempt is made to execute 17 or more OUT statements, an error "6.26: No sufficient memory for OUT" occurs.



- NOTE**
- Output to the DO0 () and DO1 () ports is not allowed.
 - Output to the MO0 () and MO1 () ports is not allowed.
 - Output to the SO0 () and SO1 () ports is not allowed.
 - Bits must be specified in ascending order from the right.
 - External output is unavailable to DO/ SO ports not connected to hardware.

Example:

```
OUT DO2(),200
----- DO (27 to 20) are turned ON and then turned OFF after 200ms.
OUT DO(37,35,27,20)
----- DO (37, 35, 27, 20) are turned ON.
```

Related commands: DO, MO, SO, TO, LO

OUTPOS statement (OUT position setting statement for main group)

Format 1:

```
OUTPOS <expression>
```

Format 2:

```
OUTPOS (<expression 1>) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression> and <expression 2> must be from 1 to 6144000. (Unit: pulses)



NOTE

If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the OUT position parameter for the main group to the value specified in <expression>. Format 1 changes all main group axes. Format 2 changes the OUT position parameter for the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH DECREASING OUTPOS
DIM SAV(3)
GOSUB *SAVE_OUTPOS
FOR A=1000 TO 10000 STEP 1000
  GOSUB *CHANGE_OUTPOS
  MOVE P,P0
  DO3(0)=1
  MOVE P,P1
  DO3(0)=0
NEXT A
GOSUB *RESTORE_OUTPOS
HALT
*CHANGE_OUTPOS:
  FOR B=1 TO 4
    OUTPOS(B)=A
  NEXT B
  RETURN
*SAVE_OUTPOS:
  FOR B=1 TO 4
    SAV(B-1)=OUTPOS(B)
  NEXT B
  RETURN
*RESTORE_OUTPOS:
  FOR B=1 TO 4
    OUTPOS(B)=SAV(B-1)
  NEXT B
  RETURN
```

OUTPOS2 statement (OUT position setting statement for sub group)

Format 1:

```
OUTPOS2 <expression>
```

Format 2:

```
OUTPOS2 (<expression 1>)=<expression 2>
```

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression> and <expression 2> must be from 1 to 6144000. (Unit: pulses)



NOTE

- This statement can be used only when the sub group is set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This statement changes the OUT position parameter for the sub group to the value specified in <expression>. Format 1 changes all sub group axes. Format 2 changes the OUT position parameter for the axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH DECREASING OUTPOS
DIM SAV(3)
GOSUB *SAVE_OUTPOS
FOR A=1000 TO 10000 STEP 1000
  GOSUB *CHANGE_OUTPOS
  MOVE2 P,P0
  DO3(0)=1
  MOVE2 P,P1
  DO3(0)=0
NEXT A
GOSUB *RESTORE_OUTPOS
HALT
*CHANGE_OUTPOS:
  FOR B=1 TO 4
    OUTPOS2(B)=A
  NEXT B
  RETURN
*SAVE_OUTPOS:
  FOR B=1 TO 4
    SAV(B-1)=OUTPOS2(B)
  NEXT B
  RETURN
*RESTORE_OUTPOS:
  FOR B=1 TO 4
    OUTPOS2(B)=SAV(B-1)
  NEXT B
  RETURN
```

PATH-related statements

What is a PATH function?

Overview

This is a function for moving the robot at a specified speed along a path made up of straight lines and circular arcs. The PATH function is ideal for applications such as sealing since there is little speed fluctuation during movement.

Features

- Moves the robot at a constant speed along the entire movement path (except the acceleration zone from stop state and the deceleration zone before the movement ends).
- Makes point teaching easy because the robot speed does not depend on how fine or coarse the point teaching positions settings were made.
- Changes the speed the robot moves along the entire path. Speed option can be used here to change the speed at just a section of the movement path.
- Using the DO option allows outputting a signal to a specified port at any desired position during movement.

How to use

The following robot language commands must be used as a set in order to use the PATH function.

- PATH statement (PATH L, PATH C)
- PATH SET statement
- PATH END statement
- PATH START statement

Example:

The PATH statements are used to set the motion path by means of sections enclosed by the PATH SET and PATH END statements. The robot does not move by just setting the motion path. It starts moving along the specified motion path when the PATH START statement is executed.

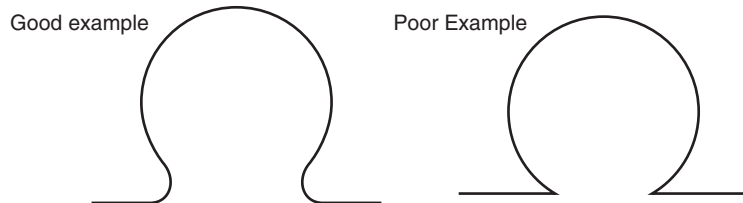
```

MOVE P,P0,Z=0
PATH SET          ----- Start of path setting
PATH L,P1,DO(20)=1@10.0
PATH L,P2
.
.
.
PATH C,P12,P13
PATH L,P14,DO(20)=0@20.0
PATH END          ----- End of path setting
MOVE P,P1,Z=0
.
.
.
MOVE P,P0,Z=0
PATH START        ----- Executes the specified PATH motion
HALT

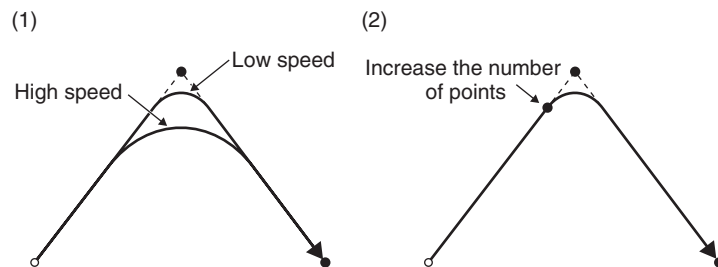
```

Cautions when using this function

- The path is a total of up to 300 segments of both straight lines and circular arcs.
- To execute PATH motion (run the PATH START statement), the robot must be positioned at the start point of the path that was set.
- At points where the arc and straight line connect, make their movement directions match as much as possible. If there is a large difference in their directions, vibrations or robot errors might occur.



- During movement, the robot moves along the inner side of the contact point where a straight line connects with another straight line. Also, the higher the speed, the more the robot shifts to the inner side (1). To avoid large path changes due to the speed, increase the number of points as shown in (2). However, in some cases, errors will occur unless the speed is lowered so use caution.



- When unable to move at the specified speed and an error occurs:
The robot will accelerate or decelerate when a PATH motion starts or stops, or when the specified speed is changed during movement along the path. When the robot cannot reach the specified speed or decelerate because the distance between points is too small at this time, an error occurs prior to starting movement. In this case, the PATH motion might be executed by lowering the specified speed. If an error still occurs even after lowering the speed, then change the points to make longer straight lines or circular arcs that include the acceleration/deceleration zones.
- During PATH motion, make the hand system the same as the hand system at the PATH start point. Do this also when using PATH motion to move to points where a hand system flag is set. If the hand system is different then errors will occur and operation will be disabled.
- If the robot stops due to an interlock during PATH motion, this is seen as termination of operation and movement on the remaining path is impossible even if re-started.

Carefully read the caution items for each command.

**NOTE**

PATH statement is only available from software version 8.38 onwards.

PATH statement (PATH-related statements)

Format:

```
PATH | L | , <point definition> [, option [, option...]]
    | C |
```

Explanation:

This command statement sets the PATH motion path for the main robot axis.

Executing this command without first running the PATH SET command will cause an error.

PATH motion type : Linear interpolation, circular interpolation
 Point data setting : Direct coordinate data input, point definition
 Options : Speed setting, coordinate plane setting (circular interpolation only), port output setting

PATH motion type

- Linear interpolation movement ----- Sets linear interpolation movement with "PATH L...".
- Circular interpolation movement ----- Sets circular interpolation movement with "PATH C...".

PATH motion is only valid for the X, Y and Z coordinate values of the specified points. The other coordinates use the coordinate value of the start point on the PATH motion path.

Movement path can be connected by repeated use of PATH commands ("PATH L", "PATH C") to allow movement without stopping.

Point data setting

· Direct coordinate data input

Format:

```
X Y Z R A B
```

X, Y, Z, R, A, and B are coordinate values on respective axes and must be separated from each other by a space.

**NOTE**

When a SCARA robot is used and a hand system flag is set for the target position data, that hand system has priority over the current arm type.

Explanation:

Directly specify the coordinate data. When the coordinate data is an integer, it is viewed as "pulses" units. When the coordinate data is a real number, it is viewed as "mm" units. If both integers and real numbers are used, then all coordinates are handled in "mm" units.

With this format, only one point can be specified as the movement destination coordinates. The only type of movement specified by this point data setting is linear interpolation.

**CAUTION**

Set the hand system for PATH motion to the same hand system as that at the start point of the PATH motion. When moving by specifying a hand system flag, also set the hand system to be the same as that at the start point for the path. If the hand system is different, then an error occurs and operation is impossible.

Hand system flags can be specified when directly specifying the coordinate data in "mm" units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

- 1 : Right-handed system is used to move to a specified position.
 2 : Left-handed system is used to move to a specified position.

11. Command Statements (PATH-related statements)

Example:

```
PATH L,10000 10000 1000 1000 0 0
```

----- Sets for movement by linear interpolation to the specified position.

```
PATH L,150.00 250.00 10.00 30.00 0.00 0.00 1
```

----- Sets for movement by linear interpolation to the position specified in the right-handed system.

· Point definition

Format:

`<point expression> [, <point expression> ...]`

NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

Explanation:

In <point expression>, specify the point to move to. Two or more points can be specified by separating them with a comma (.).

When the PATH motion type is circular interpolation, two points must be specified for each single circular arc.

Example:

```
PATH L,P1,P2,P3
```

----- Sets for movement by linear interpolation in sequence to positions specified by P1, P2 and P3.

```
PATH C P5,P6,P7,P8
```

----- Sets for movement by circular interpolation including the current position, P5, P6 and P6, P7 and P8.

CAUTION

Set the hand system for PATH motion to the same hand system as that at the start point of the PATH motion. When moving by specifying a hand system flag, also set the hand system to be the same as that at the start point of the PATH motion. If the hand system is different, then an error occurs and operation is impossible.

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value in <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression> (unit: %).

Actual speed is as follows.

$\text{Robot maximum speed (mm/sec.)} \times \text{automatic movement speed (\%)} \times \text{program speed (\%)}$

This option is enabled only for the specified PATH statement.

Example:

```
PATH L, P5, S=40
```

----- Moves to the position specified by P5 using a program speed of 40%.

Format 2:

VEL = <expression>

The value of <expression> varies according to the robot. (Unit: mm/sec)

Explanation:

Specify the movement speed in <expression> (unit: mm/sec). Error occurs if specified speed is too high.

This option is enabled only for the specified PATH statement.

Example:

PATH L,P10,VEL=150

----- Moves at speed of 150mm/sec to the position specified by P10.

· Coordinate plane setting

Format:

XY
YZ
ZX

XY: XY coordinate plane

YZ: YZ coordinate plane

ZX: ZX coordinate plane

Explanation:

Specify the coordinate plane on which to draw a circular arc for circular interpolation movement. If no coordinate plane is specified, then 3-dimensional circular interpolation movement is used.

The only type of movement specified by this coordinate plane setting is circular interpolation.

This option is enabled for only for specified PATH statement.

Example:

PATH C, P1, P2, XY

----- Moves the robot to the Z coordinate position of P2 while performing circular interpolation within XY coordinate plane.

11. Command Statements (PATH-related statements)

· Port output setting

Format 1:

DO	m ([b, ... ,b]) =<expression 1>@<expression 2>
MO	
SO	

Format 2:

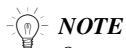
DO	(mb, ... ,mb) =<expression 1>@<expression 2>
MO	
SO	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit position 0 to 7

The value in <expression 1> is output to the specified port. Only the integer portion is valid.

The value in <expression 2> indicates the position for port output and can be specified to 2 digits below the decimal point in "mm" units.



NOTE

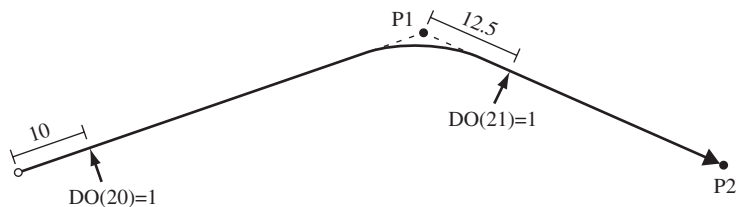
- Output to the DO0 () and DO1 () ports is not allowed.
- Output to MO0 () and MO1 () ports is not allowed.
- Output to SO0 () and SO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO/ SO ports not connected to hardware.

Explanation:

This command option outputs the value of <expression 1> to the specified output port during PATH motion at the position specified in <expression 2>.

Example:

```
PATH SET
PATH L,P1,DO(20)=1@10
PATH L,P2,DO(21)=1@12.5
PATH END
PATH START
```



Related commands: PATH SET, PATH END, PATH START

**NOTE**

PATH END statement is only available from software version 8.38 onwards.

PATH END statement (PATH-related statements)

Format:

```
PATH END
```

Explanation:

This is used to terminate the path setting for PATH motion.

The PATH END command must always be used as a pair along with the PATH SET command. The final point of the PATH command ("PATH L" or "PATH C") that was executed last between the PATH SET and PATH END commands is the end point of the PATH motion.

An error will occur if the PATH END command is executed without first running PATH SET.

Example:

See the "What is a PATH function?" description.

Related commands: PATH, PATH SET, PATH START

11. Command Statements (PATH-related statements)



NOTE

PATH SET statement is only available from software version 8.38 onwards.

PATH SET statement (PATH-related statements)

Format:

```
PATH SET [<Point definition>]
```

Explanation:

Starts the path setting for PATH motion.

Sets the position specified in <point definition> as the PATH motion start point. (This only sets the PATH motion start point and does not mean the robot actually operates.) If <point definition> is omitted, then the current robot position is set as the start point. However, during robot movement, the target position for that movement becomes the start point. (Example: OUT position range with MOVE command is wide prior to PATH SET command, so robot still moves during execution of the PATH SET command.)

The PATH SET command must always be used as a pair along with the PATH END command.

When the PATH SET command is executed, the previously set PATH motion data is deleted.

Point data setting : Direct coordinate data input, point definition

Example:

See the "What is a PATH function?" description.

Point data setting

· Direct coordinate data input

Format:

```
X Y Z R A B
```

X, Y, Z, R, A, and B are the coordinate values on respective axes and must be separated from each other by a space.



NOTE

When a SCARA robot is used and a hand system flag is set for the start point data of the PATH motion path, that hand system has priority over the current arm type.

Explanation:

Directly specify the start position coordinates for PATH motion path. When the coordinate data is an integer, it is viewed as "pulses" units. When the coordinate data is a real number, it is viewed as "mm" units (valid to 2 digits below the decimal point.) If both integers and real numbers are used, then all coordinates are handled in "mm" units.



CAUTION

Set the hand system for PATH motion to the same hand system as that at the start point of the PATH motion. If the hand system is different, then an error occurs and operation is impossible.

Hand system flags can be specified when directly specifying the coordinate data in "mm" units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of B. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Hand system at PATH motion start point is set to the right-handed system.

2 : Hand system at PATH motion start point is set to the left-handed system.

Example:

```
PATH SET 120 250.00 55.2 20.33 0 0
```

----- PATH motion start point is set to (120.00 250.00 55.20 20.33 0.00 0.00) in "mm" units.

```
PATH SET -51200 80521 7045 204410 0 0
```

----- PATH motion start point is set in "pulses" units.

· Point definition

Format:

<point expression>



NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.



CAUTION

Set the hand system for PATH motion to the same hand system as that at the start point of the PATH motion. If the hand system is different, then an error occurs and operation is impossible.

Explanation:

Specify the PATH motion start point in <point expression>.

```
PATH SET P10
```

----- Sets the PATH motion start point to P10.

```
PATH SET WHERE
```

----- Sets the PATH motion start point the current robot position.

Related commands: PATH, PATH END, PATH START



NOTE

PATH START statement is only available from software version 8.38 onwards.

PATH START statement (PATH-related statements)

Format:

PATH START

Explanation:

Starts the PATH motion. PATH motion settings must be made beforehand with the PATH SET command, PATH commands ("PATH L", "PATH C") and PATH END command before executing the PATH START command. Also, before starting PATH motion, the robot must also be positioned beforehand at the PATH motion start point specified by the PATH SET command.

The robot PATH motion speed is the product of the automatic movement speed (%) when the PATH START command is executed and the program speed (%) specified by the SPEED command or speed option (SPEED or S) for the PATH command. Robot speed does not rely on automatic movement speed, if the speed was specified with the speed option (VEL) for the PATH command.

After PATH motion starts, this command terminates when the robot reaches the PATH motion end point or when the robot stops due to an interlock.

This command can only be executed in Task 1 (main task).

Example:

See the "What is a PATH function?" description.

Related commands: PATH, PATH SET, PATH END

PDEF statement

Format:

PDEF (<expression 1>) = <expression 2>, <expression 3> [, <expression 4>]

The value of <expression 1> must be from 0 to 19.

The value of <expression 2>, <expression 3> and <expression 4> can be 1 to 32767 as long as the value of <expression 2> * <expression 3> * <expression 4> is equal to or less than 32767.



NOTE

Point data used in pallet definition uses the data area as follows.

Pallet definition 0: P[1] to P[5]
----- P3996 to P4000

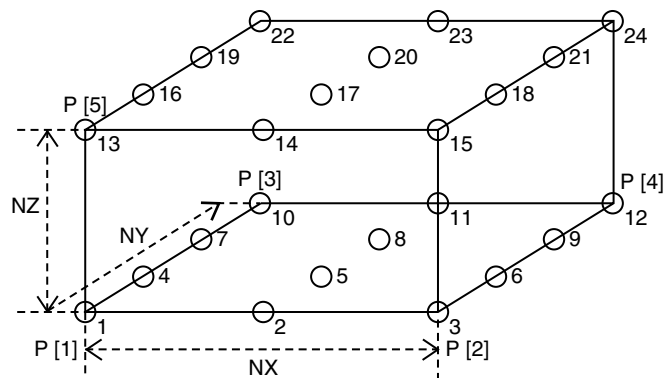
Pallet definition 1: P[1] to P[5]
----- P3991 to P3995

:

Pallet definition 19: P[1] to P[5]
----- P3901 to P3905

Explanation:

This statement defines the pallet used to execute a pallet movement command.



<expression 1> : Pallet definition number (0 to 19)

<expression 2> : Number of points between P[1] and P[2]

<expression 3> : Number of points between P[1] and P[3]

<expression 4> : Number of points between P[1] and P[5]

Example:

PDEF(1) = 3,4,2

----- Pallet definition 1 is defined as 3x4x2.

PMOVE statement (Pallet movement command)

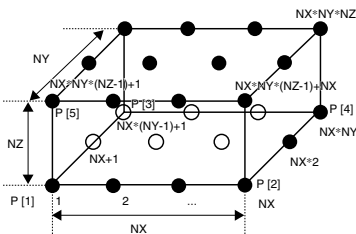
Format:

PMOVE (pallet definition number, pallet position number) [,option [,option] ...]



NOTE

- The pallet definition number must be designated to execute a PMOVE statement.
- The XYZ axes move to the positions determined by the calculated values. However, the R-axis moves to the position specified by pallet point data P[1].
- In each pallet definition, pallet position numbers are designated as follows:



Explanation:

This statement executes a pallet movement command for all main robot axes. This statement will not move the sub robot axes and auxiliary axes.

Movement type	: PTP
Pallet definition number	: Numerical expression
Pallet position number	: Numerical expression
Options	: Speed setting, arch motion setting, STOPON condition setting

Movement type

- PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

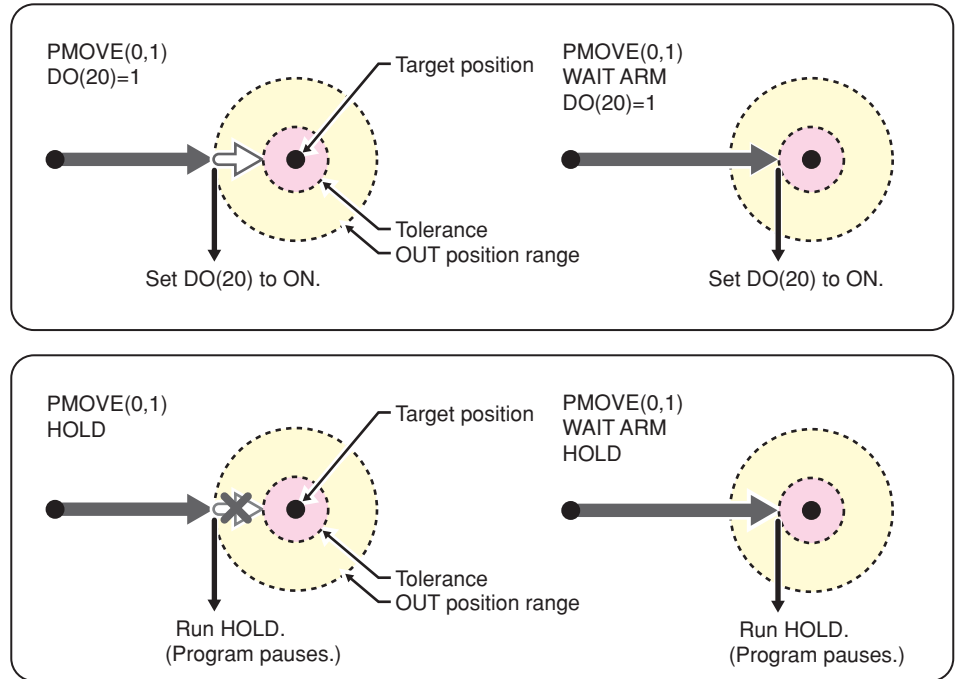
Cautions for next command following PMOVE command

If the next command following the PMOVE command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM statement.



Example:

PMOVE(1,16)

----- Main robot axes move from the current position to the position specified by pallet position number 16 of pallet definition number 1.

Pallet definition number

Format:

<expression 1>

The value of <expression 1> must be 0 to 19.

Explanation:

Specify the pallet definition number.

Pallet position number

Format:

<expression 2>

The value of <expression 2> must be from 1 to 32767.

Explanation:

Specify the pallet position number.

Example:

PMOVE(1,16)

----- Main robot axes move from the current position to the position specified by pallet position number 16 of pallet definition number 1.

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified PMOVE statement.



NOTE

This option merely specifies the maximum speed and does not guarantee the specified speed.

Example:

PMOVE(1,3),S=10

----- Moves from the current position to the position specified by pallet position number 3 of pallet definition number 1 using a program speed of 10%.

· Arch motion setting

Format:

x = <expression> [, x = <expression> ...]
--

x : Axis Z, R, A or B

When the value of <expression> is an integer, the movement units are viewed as “pulses”. When the value is a real number containing a decimal point, the units are viewed as “mm” or “deg”.

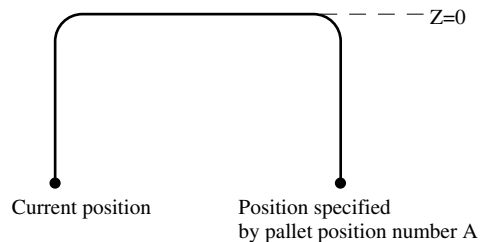
Explanation:

The specified "x" axis first moves toward the position specified in <expression>. Then, when the specified axis enters the arch position range, all axes other than the specified axis move toward the target position. Finally, when all axes other than the specified axis enter the arch position range and also when the specified axis enters within the tolerance range of the position specified in <expression>, the specified axis moves toward the target position. When it enters the OUT position range, the command terminates.

Example:

PMOVE(1,A),Z=0

----- The Z-axis first moves from the current position to the “0 pulse” position. Then the other axes move to the position specified by pallet position number A of pallet definition number 1. Finally the Z-axis moves to the position specified by pallet position number A.



· STOPON condition setting

Format:

STOPON <conditional expression>



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

PMOVE(A,16),STOPON DI(20)=1

----- Moves from the current position to the position specified by pallet position number 16 of pallet definition number A, and slows down and stops when the condition "DI (20) = 1" is met.

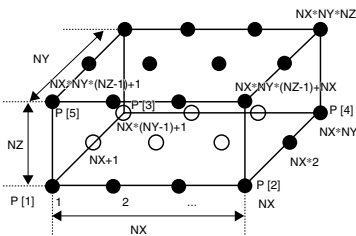
PMOVE2 statement (Pallet movement command)

Format:

PMOVE2 (pallet definition number, pallet position number) [,option [,option] ...]

NOTE

- The pallet definition number must be designated to execute a PMOVE2 statement.
- The XYZ axes move to the positions determined by the calculated values. However, the R-axis moves to the position specified by pallet point data P[1].
- In each pallet definition, pallet position numbers are designated as follows:



Explanation:

This statement executes a pallet movement command for all sub robot axes. This statement will not move the main robot axes and auxiliary axes.

Movement type	: PTP
Pallet definition number	: Numerical expression
Pallet position number	: Numerical expression
Options	: Speed setting, arch motion setting, STOPON condition setting

Movement type

- PTP (point to point) movement

PTP movement begins after positioning on all movable axes is complete (within the tolerance range), and the command terminates when all movable axes enter the OUT position range. All movable axes will reach the target positions simultaneously, but their paths are not guaranteed.

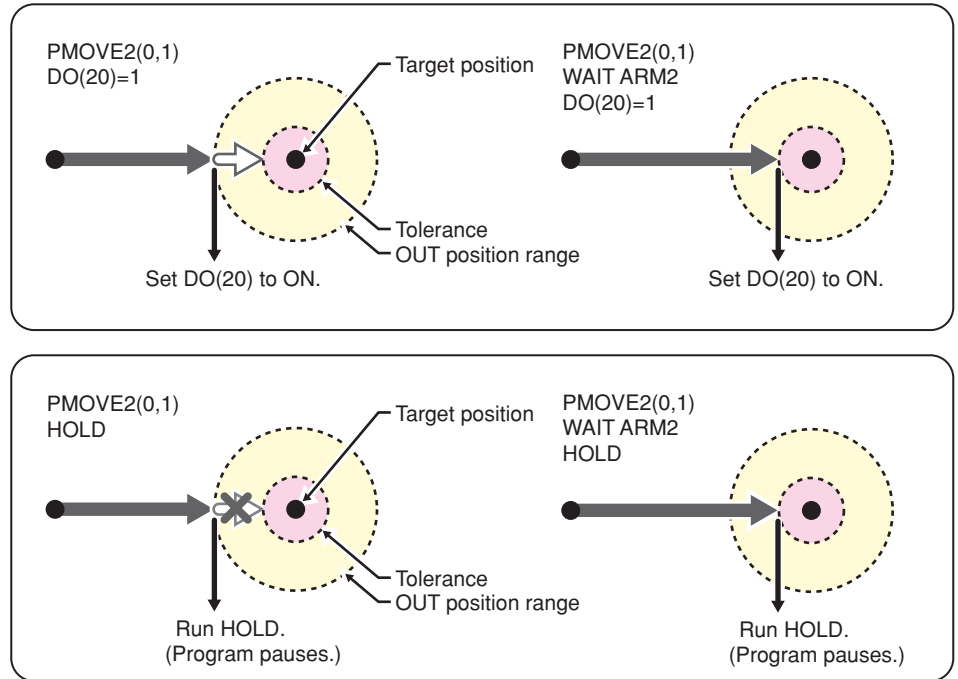
Cautions for next command following PMOVE2 command

If the next command following the PMOVE2 command is an executable command such as a signal output command, that next command will start when the movement axis enters the OUT position range. In other words, that next command starts before the axis arrives within the target position tolerance range.

Examples:

Signal output (DO, etc.)	Signal is output when axis enters within OUT position range.
DELAY	DELAY command is executed and time standby starts, when axis enters within OUT position range.
HALT	Program stops and is also reset when axis enters within OUT position range. So axis movement stops.
HOLD	Program temporarily stops when axis enters within OUT position range. So axis movement stops.
WAIT	WAIT command is executed when axis enters within OUT position range.

To execute the next command after waiting for the axes to arrive within the target position tolerance range, use the WAIT ARM2 statement.



Example:

`PMOVE2(1,16)`

----- Sub robot axes move from the current position to the position specified by pallet position number 16 of pallet definition number 1.

Pallet definition number

Format:

<expression 1>

The value of <expression 1> must be 0 to 19.

Explanation:

Specify the pallet definition number.

Pallet position number

Format:

<expression 2>

The value of <expression 2> must be from 1 to 32767.

Explanation:

Specify the pallet position number.

Example:

`PMOVE2(1,16)`

----- Sub robot axes move from the current position to the position specified by pallet position number 16 of pallet definition number 1.

Option types

· Speed setting

Format 1:

SPEED	= <expression>
S	

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

Specify the program speed in <expression>. Robot operating speed is determined by the product of the automatic movement speed and the program speed. This option is enabled only for the specified PMOVE2 statement.



NOTE

This option merely specifies the maximum speed and does not guarantee the specified speed.

Example:

PMOVE2(1,3),S=10

----- Moves from the current position to the position specified by pallet position number 3 of pallet definition number 1 using a program speed of 10%.

· Arch motion setting

Format:

x = <expression> [, x = <expression> ...]
--

x: Axis Z or R

When the value of <expression> is an integer, the unit of movement is viewed as “pulses”. When the value is a real number containing a decimal point, the unit is viewed as “mm” or “deg”.

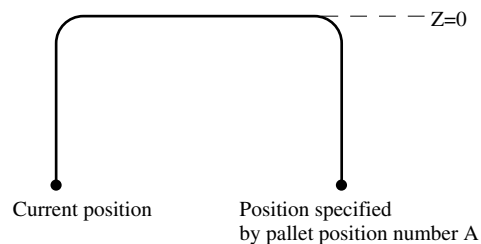
Explanation:

The specified "x" axis first moves toward the position specified in <expression>. Then, when the specified axis enters the arch position range, all axes other than the specified axis move toward the target position. Finally, when all axes other than the specified axis enter the arch position range and also when the specified axis enters within the tolerance range of the position specified in <expression>, the specified axis moves toward the target position. When it enters the OUT position range, the command terminates.

Example:

PMOVE2(1,A),Z=0

----- The Z-axis first moves from the current position to the “0 pulse” position. Then the other axes move to the position specified by pallet position number A of pallet definition number 1. Finally the Z-axis moves to the position specified by pallet position number A.



· STOPON condition setting

Format:

STOPON <conditional expression>



NOTE

On controllers using the following software versions, when the conditional expression used to designate the STOPON condition is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

This command option stops the robot movement when the conditional expression is met (becomes true). The robot will move a slight distance while slowing down before coming to a stop after the condition is met (conditional expression becomes true).

If the conditional expression is already satisfied (true) before the robot begins to move, the command will terminate without moving the robot.

This option is enabled only during program execution.

Example:

PMOVE2(A,16),STOPON DI(20)=1

----- Moves from the current position to the position specified by pallet position number 16 of pallet definition number A, and slows down and stops when the condition "DI (20) = 1" is met.

PRINT statement

Format:

```
PRINT [<expression>] [ | , | <expression>... ] [ | , | ]  
                ; ;
```

Explanation:

This statement displays a value for the variable specified on the MPB and RPB screen. Output definitions are as follows:

1. If numbers or character strings are specified in <expression>, they will be displayed as is. If variables or arrays are specified, the values assigned to the specified variables or arrays will be displayed. If no <expression> is specified, only line-feed is performed.
2. If the data length exceeds the width of the screen, line-feed is performed so that the remainder of the data appears on the following lines of the screen.
3. If a comma (,) is used as a delimiter, a space (blank) is inserted between the displayed items.
4. If a semicolon (;) is used as a delimiter, the displayed items appear in succession without being separated.
5. If the data is ended with a delimiter, no line-feed is inserted and the next PRINT statement is executed on the same display line. If not ended with a delimiter, line-feed is performed.



NOTE

Every time a PRINT statement is executed, data is sent to display the specified value on the MPB and RPB screen. The program execution time may be longer if two or more PRINT statements are executed in succession.

Examples:

```
PRINT A  
PRINT "A1 =";A1  
PRINT "B(0),B(1) = ";B(0);";";B(1)  
PRINT P100
```

Related command: INPUT

Pn (Point definition statement)

Format:

$$P_n = x \ y \ z \ r \ a \ b$$

CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified with Pn (point definition statement).

n: 0 to 9999

Input range for x, y, z, r, a, b is determined by the point data format.

Explanation:

This statement defines the point (position) to move to within the robot operation range. Make point definitions while referring to the following notes.

1. "n" is a point number.
2. Input data for "x" to "b" must be separated with a space (blank).
3. If all input data for "x" to "b" are integers (no decimal points), the movement units are viewed as "pulses". At this point, "x" through "b" correspond to axis 1 through axis 6.
4. If one of input data for "x" through "b" is a real number (containing a decimal point), the movement units are viewed as "mm". At this point, "x" to "z" correspond to the x, y and z coordinates of Cartesian coordinate system, while "r" to "b" correspond to axis 4 through axis 6.
5. Input data in "pulses" units must be in the range from -6144000 to 6144000, and input data in "mm" units must be in the range from -99999.99 to 99999.99.

NOTE

- When specifying a hand system flag for a SCARA robot and its point data, set either 1 (right-handed system) or 2 (left-handed system) at the data end of B-axis data.
- Hand system flags are only available from software version 8.08 onwards.

Hand system flags can be specified when specifying point definition data in "mm" units for SCARA robots.

When setting an extended hand system flag for SCARA robots, set either 1 or 2 at the data end of b. If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to set point data.

2 : Left-handed system is used to set point data.

NOTE

- Input data is treated as a constant.
- If power is turned off during execution of a point definition statement, a memory-related error such as "Point data destroyed" may occur.

Example:

```
P1= 0 0 0 0 0 0
P2= 100.00 200.00 50.00 0.00 0.00 0.00
P3= 10.00 0.00 0.00 0.00 0.00 0.00
P10= P2
FOR A=10 TO 15
  P[A+1]=P[A]+P3
NEXT A
FOR A=10 TO 16
  MOVE P,P1,P[A]
NEXT A
HALT
```

Related command: Point assignment statement

REM (Comment statement)

Format:

REM	,	<character string>
-----	---	--------------------

Explanation:

All characters that follow REM or an apostrophe (') are viewed as comments. This statement can only be used to make comments in the program and does not execute any command.

Example:

```
REM *** MAIN PROGRAM ***  
'*** SUBROUTINE ***  
HALT 'HALT COMMAND
```

RESET statement

Format 1:

RESET	DOm ([b, . . . , b])
	DO (mb, . . . , mb)
	MOm ([b, . . . , b])
	MO (mb, . . . , mb)
	TO0 ([b, . . . , b])
	TO (0b, . . . , 0b)
	LO0 ([b, . . . , b])
	LO (0b, . . . , 0b)
	SOM ([b, . . . , b])
	SO (mb, . . . , mb)

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

Format 2:

RESET _SYSFLG

Format 3:

RESET TCOUNTER

Explanation:

Format 1 of this statement turns OFF the bits of the specified ports.

Format 2 clears the system flag variables.

Format 3 clears 10ms counter variables.



NOTE

- Output to ports 0 and 1 of DO, MO and SO are not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO/SO ports not connected to hardware.
- System flag variables are used when making checks with the overload monitor.
- 10ms counter variables are used to measure the time in 10ms units.

Example:

```
RESET DO2()
----- DO (27 to 20) are turned OFF.
RESET DO2(6,5,1)
----- DO (26, 25, 21) are turned OFF.
RESET (37,35,27,20)
----- DO(37, 35, 27, 20) are turned OFF.
RESET _SYSFLG
----- System flag variables are cleared.
RESET TCOUNTER
----- 10ms counter variables are cleared.
```

Related commands: SET, DO, MO, SO, TO, LO

RESTART statement

Format:

```
RESTART Tn
```

n : task number 2 to 8

Explanation:

This statement restarts another task that is in pause.

Example:

```
START *SUBTASK,T2
FLAG=1
*L0:
IF FLAG=1 AND DI2(0)=1 THEN
  SUSPEND T2
  FLAG=2
  WAIT DI2(0)=0
ENDIF
IF FLAG=2 AND DI2(0)=1 THEN
  RESTART T2
  FLAG=1
  WAIT DI2(1)=0
ENDIF
MOVE P,P0
MOVE P,P1
GOTO *L0
HALT
'SUBTASK ROUTINE
*SUBTASK:
DO2(0)=1
DELAY 1000
DO2(0)=0
DELAY 1000
GOTO *SUBTASK
EXIT TASK
```

Related commands: CUT, EXIT TASK, START, SUSPEND

RESUME statement

Format:

```
RESUME [ |NEXT | ]
        |<label>|
```

Explanation:

This statement resumes the program execution after recovery from an error.

There are three ways to resume the program, depending on the location in the program.

- | | |
|-------------------|---|
| 1. RESUME | The program resumes from the command that caused the error. |
| 2. RESUME NEXT | The program resumes from the command following the command that caused the error. |
| 3. RESUME <label> | The program resumes from the command specified by <label>. |



NOTE

- The *RESUME* statement can also be executed in an error processing routine.
- Error recovery processing does not work if the error is serious.

Example:

See the description of the ON ERROR GOTO statement.

Related command: ON ERROR GOTO

RIGHTY statement, LEFTY statement

Format:

```
RIGHTY  
LEFTY
```



NOTE

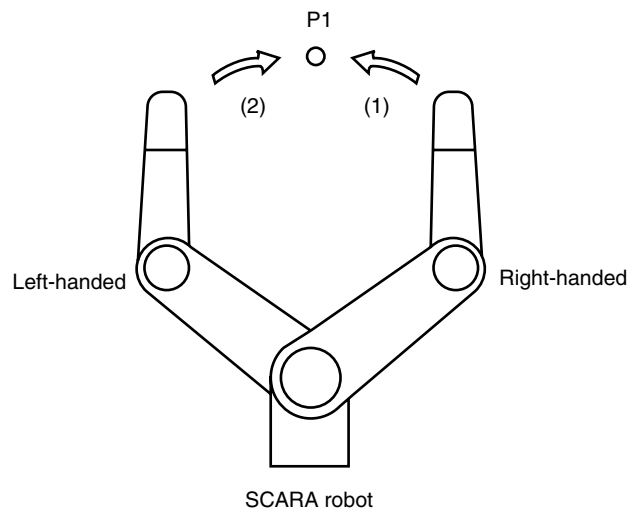
These statements are valid only for SCARA robots, and so cannot be used for Cartesian robots.

Explanation:

These statements select whether the main robot will be “right-handed” or “left-handed” when moving to a point specified on a Cartesian coordinate system. This is a just selection and does not move the robot. If the robot arm is moving, the command execution waits until movement is complete (positioned within tolerance range).

Example:

```
RIGHTY  
MOVE P,P1  
LEFTY  
MOVE P,P1  
RIGHTY  
HALT
```



RIGHTY2 statement, LEFTY2 statement

Format:

```
RIGHTY2
LEFTY2
```



NOTE

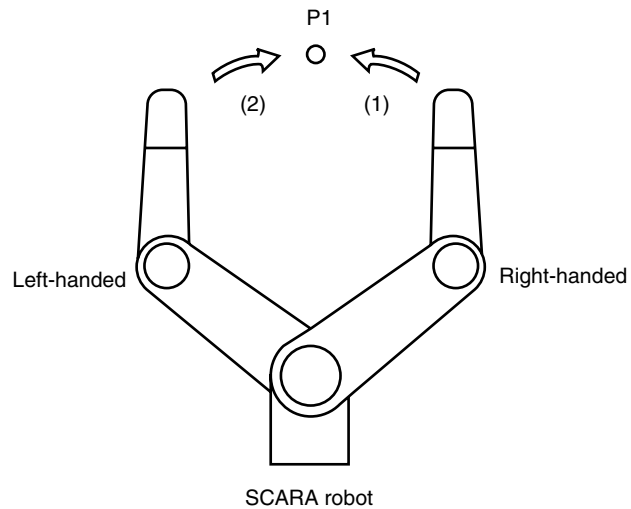
- These statements are valid only when the sub group is set in the system generation.
- These statements are valid only for SCARA robots, and so cannot be used for Cartesian robots.

Explanation:

These statements select whether the sub robot will be “right-handed” or “left-handed” when moving to a point specified on a Cartesian coordinate system. This is just a selection and does not move the robot. If the robot arm is moving, the command execution waits until movement is complete (positioned within tolerance range).

Example:

```
RIGHTY2
MOVE2 P,P1
LEFTY2
MOVE2 P,P1
RIGHTY2
HALT
```



Sn (Shift coordinate definition statement)

Format:

$$S_n = x \ y \ z \ r$$

n: 0 to 9

The values of “x”, “y”, “z” and “r” must be from –99999.99 to 99999.99.

Explanation:

This statement defines shift coordinate values in order to shift the coordinates for robot movement. Define the shift coordinate values while referring to the following notes.

1. “n” is a shift number.
2. Input data for “x” to “r” must be separated with a space (blank).
3. Input data for “x” to “r” is viewed as “mm”.
4. “x” to “z” correspond to the x, y and z coordinate shift values on a Cartesian coordinate system, and “r” to the rotating shift value on the xy coordinates.



NOTE

- Input data is treated as a constant.
- If power is turned off during execution of a shift coordinate definition statement, a memory-related error such as “Shift data destroyed” may occur.

Example:

```
S0= 0.00 0.00 0.00 0.00
S1=100.00 200.00 50.00 90.00
P3=100.00 0.00 0.00 0.00 0.00 0.00
SHIFT S0
MOVE P,P3
SHIFT S1
MOVE P,P3
HALT
```

Related commands: Shift assignment statement, SHIFT, SHIFT2

SELECT CASE statement, END SELECT statement

Format:

```
SELECT [CASE] <expression>
  CASE <expression list 1>
    [command block 1]
  [CASE <expression list 2>
    [command block 2]]
  :
  [CASE ELSE
    [command block n]]
END SELECT
```

Explanation:

These statements execute multiple command blocks according to the value in <expression>. Make the settings while referring to the following notes.

1. The <expression list> following CASE consists of multiple numerical expressions and character expressions separated from each other by a comma (,).
2. If the value of <expression> matches one of expressions contained in <expression list>, the specified command block is executed. After executing the command block, the program jumps to the next command following the END SELECT statement.
3. If the value of <expression> does not match any of the expressions contained in <expression list>, the command block written after the CASE ELSE statement is executed. After executing the command block, the program jumps to the next command following the END SELECT statement.
4. If the value of <expression> does not match any of the expressions contained in <expression list> and the CASE ELSE statement is not written, the program jumps to the next command following the END SELECT statement.

Example:

```
WHILE -1
  SELECT CASE DI3()
    CASE 1,2,3
      CALL *EXEC(1,10)
    CASE 4,5,6,7,8,9,10
      CALL *EXEC(11,20)
    CASE ELSE
      CALL *EXEC(21,30)
  END SELECT
WEND
HALT
```

11. Command Statements

CAUTION

When you restart the controller after stopping the **SEND** statement, note the following points.

1. **SEND CMU TO XXX, SEND ETH TO XXX**

When the **SEND** statement stops while data is still being read out from the receive buffer, the data acquired up until that point is discarded.

2. **SEND XXX TO CMU, SEND XXX TO ETH**

When the **SEND** statement stops while data is still being written into the transmit buffer, the data is written from the beginning.

NOTE

The point comment separate file (PCn) is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

CAUTION

On the **DO** port, **MO** port, **TO** port, **LO** port, **SO** port and **SOW** port, each entire port (**DO()**, **MO()**, etc.) cannot be specified as a write file. There are some separate files (**DOn()**, **MON()**, etc.) that cannot be specified as a write file. For details on files, refer to "Data file description" described later.

NOTE

For details on files, refer to "Data file description" described later.

SEND statement

Format:

```
SEND <read file> TO <write file>
```

Explanation:

This statement sends the data in the <read file> to the <write file>.

Type	File name	Definition format		Read	Write
		All	Separate file		
User memory	All files	ALL	—	Yes	Yes
	Program	PGM	<bbbbbb>	Yes	Yes
	Point data	PNT	Pn	Yes	Yes
	Point comment	PCM	PCn	Yes	Yes
	Parameter	PRM	/cccccc/	Yes	Yes
	Shift definition	SFT	Sn	Yes	Yes
	Hand definition	HND	Hn	Yes	Yes
	Pallet definition	PLT	PLn	Yes	Yes
Variable, constant	Variable	VAR	ab...by	Yes	Yes
	Array variable	ARY	ab...by (x)	Yes	Yes
	Constant	—	"cc...c"	Yes	No
Status	Program directory	DIR	<<bbbbbb>>	Yes	No
	Parameter directory	DPM	—	Yes	No
	Machine reference	MRF	—	Yes	No
	Error log	LOG	—	Yes	No
	Memory size	MEM	—	Yes	No
Device	DI port	DI ()	DIn ()	Yes	No
	DO port	DO ()	DOn ()	Yes	Yes
	MO port	MO ()	MON ()	Yes	Yes
	TO port	TO ()	TO n ()	Yes	Yes
	LO port	LO ()	LOn ()	Yes	Yes
	SI port	SI ()	SIn ()	Yes	No
	SO port	SO ()	SO n ()	Yes	Yes
	SIW port	SIW()	SIWn()	Yes	No
	SOW port	SOW()	SOWn()	Yes	Yes
	RS-232C	CMU	—	Yes	Yes
	Ethernet	ETH	—	Yes	Yes
Others	File end code	EOF	—	Yes	No

n: Number

a: Alphabetic character

b: Alphanumeric character or underscore (_)

c: Alphanumeric character or symbol

x: Expression

y: Format

**NOTE**

- The *SEND* statement cannot be used to write onto read-only files.

Incorrect examples:

SEND CMU TO DIR

SEND PNT TO SI()

- Even if the read and write files are correct, writing cannot be executed unless the data formats match.

Incorrect Examples:

SEND PGM TO PNT

SEND SI() TO SFT

Example:

SEND PGM TO CMU

----- All user programs are output through the RS-232C port.

SEND <<PRG1>> TO CMU

----- The program named “PRG1” is output through the RS-232C port.

SEND CMU TO PNT

----- Point data files are input through the RS-232C port.

SEND CMU TO P100

----- Data of point number 100 is input through the RS-232C port.

SEND “T1” TO CMU

----- The character string “T1” is output through the RS-232C port.

SEND CMU TO A\$

----- The character string is input to variable A\$ through the RS-232C port.

SERVO statement

Format:

SERVO	ON	[(<expression>)]	
	OFF		
	FREE		
	PWR		

The value of <expression> must be from 1 to 6 (axis number).

Explanation:

This statement controls the servo ON/OFF of the specified axes in the main group or all axes (in the main group and sub group).

If <expression> is not specified, all axes are controlled and the motor power supply simultaneously turns ON and OFF.

ON	-----	Turns the servo ON. If no axis is specified the motor power supply also turns ON.
OFF	-----	Turns the servo OFF and applies the dynamic brake. Those axes having brakes are all locked by the brake. If no axis is specified the motor power supply also turns OFF.
FREE	-----	Turns the servo OFF and releases the dynamic brake. The axes having brakes are all released. If no axis is specified the motor power supply also turns OFF.
PWR	-----	Turns ON only the motor power supply.



NOTE

- This statement is executed after positioning on all axes (in the main group and sub group) is complete (within the tolerance range).
- The servo for an axis cannot be turned ON as long as its motor power is OFF.



CAUTION

Do not perform any work within the robot movement range while the motor power is turned OFF by using the SERVO OFF statement. Always check that the emergency stop is ON when working within the robot movement area.

Example:

SERVO ON	-----	Turns ON the servos for all axes after turning ON the motor power supply.
SERVO OFF	-----	Turns OFF the motor power supply and the servos for all axes. Those axes having brakes are all locked by the brake.
SERVO FREE(3)	-----	Turns OFF the servo for axis 3 (Z-axis) and releases the brake.

SERVO2 statement

Format:

SERVO2	ON	[(<expression>)]	
	OFF		
	FREE		
	PWR		

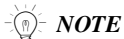
The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This statement controls the servo ON/OFF of the specified axes in the sub group or all axes (in the main group and sub group).

If <expression> is not specified, all axes are controlled and the motor power supply simultaneously turns ON and OFF.

ON	-----	Turns the servo ON. If no axis is specified the motor power supply also turns ON.
OFF	-----	Turns the servo OFF and applies the dynamic brake. Those axes having brakes are all locked by the brake. If no axis is specified the motor power supply also turns OFF.
FREE	-----	Turns the servo OFF and releases the dynamic brake. The axes having brakes are all released. If no axis is specified the motor power supply also turns OFF.
PWR	-----	Turns ON only the motor power supply.



NOTE

- This statement can be used only when the sub group is set in the system generation.
- This statement is executed after positioning on all axes (in the main group and sub group) is complete (within the tolerance range).
- The servo for an axis cannot be turned ON as long as its motor power is OFF.



CAUTION

Do not perform any work within the robot movement range while the motor power is turned OFF by using the SERVO OFF statement. Always check that the emergency stop is ON when working within the robot movement area.

Example:

SERVO2 ON	-----	Turns ON the servos for all axes after turning ON the motor power supply.
SERVO2 OFF	-----	Turns OFF the motor power supply and the servos for all axes. Those axes having brakes are all locked by the brake.
SERVO2 FREE(2)	-----	Turns OFF the servo for axis 2 (Y-axis of sub group) and releases the brake.

SET statement

Format:

SET	DOm ([b, . . . , b])	[, <expression>]
	DO (mb, . . . , mb)	
	MOm ([b, . . . , b])	
	MO (mb, . . . , mb)	
	TO0 ([b, . . . , b])	
	TO (0b, . . . , 0b)	
	LO0 ([b, . . . , b])	
	LO (0b, . . . , 0b)	
	SOm ([b, . . . , b])	
	SO (mb, . . . , mb)	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

The value of <expression> must be from 1 to 3600000. (Unit: ms) Note that the minimum run time is 10ms.

Explanation:

This statement turns ON the bits of the specified ports.

Specify the pulse output time (unit: ms) in <expression>. When the specified time has elapsed, the output turns OFF to end the command execution.

NOTE

- Output to the DO0 () and DO1 () ports is not allowed.
- Output to the MO0 () and MO1 () ports is not allowed.
- Output to the SO0 () and SO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to DO/ SO ports not connected to hardware.

Example:

```
SET DO2()
----- DO (27 to 20) are turned ON.
SET DO2(6,5,1),200
----- DO (26, 25, 21) are turned ON for 200ms.
SET DO(37,35,27,20)
----- DO (37, 35, 27, 20) are turned ON.
```

Related commands: RESET, DO, MO, SO, TO, LO

SHARED statement

Format:

```
SHARED <variable> [ ( ) ] [, <variable> [ ( ) ]. . .]
```



NOTE

- The *SHARED* statement allows variables to be shared only between the program level code and sub-procedure within the same program.
- The program level code is a program written outside of a sub-procedure.

Explanation:

This statement does not permit variables declared with a program level code to be passed on as dummy arguments, but allows them to be referred to with a sub-procedure.

As a <variable>, specify a simple variable or an array variable followed by parentheses. This variable should be at the program level used with sub-procedures. If an array is specified, that entire array is selected.

Example:

```
DIM Y!(10)
X!=2.5
Y!(10)=1.2
CALL *DISTANCE
CALL *AREA
HALT
SUB *DISTANCE
  SHARED X!,Y!()
  PRINT X!^2+Y!(10)^2          ----- Variable is shared.
END SUB
SUB *AREA
  DIM Y!(10)
  PRINT X!*Y!(10)              ----- Variable is not shared.
END SUB
```

Related commands: SUB, END SUB

SHIFT statement (Shift coordinate setting statement for main robot)

Format:

SHIFT <shift variable>



NOTE

This statement is executed after positioning on the main robot axes is complete (within the tolerance range).

Explanation:

This statement sets the shift coordinates for the main robot by using the shift data specified in <shift variable>.

Example:

```
SHIFT    S1
MOVE     P,P10
SHIFT    S[A]
MOVE     P,P20
HALT
```

Related commands: Shift definition statement, shift assignment statement

SHIFT2 statement (Shift coordinate setting statement for sub robot)

Format:

```
SHIFT2 <shift variable>
```



NOTE

- This statement can be used only when the sub robot is set in the system generation.
- This statement is executed after positioning on the sub robot axes is complete (within the tolerance range).

Explanation:

This statement sets the shift coordinates for the sub robot by using the shift data specified in <shift variable>.

Example:

```
SHIFT2      S1
MOVE2       P,P10
SHIFT2      S[A]
MOVE2       P,P20
HALT
```

Related commands: Shift definition statement, shift assignment statement

SO statement (Serial output)

Format:

[LET]	SO m ([b, . . . , b])	= <expression>
	SO (mb, . . . , mb)	

m : port number 2 to 7, 10 to 17, 20 to 27

b : bit definition 0 to 7

The value of <expression> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

Explanation:

This statement outputs the specified value to the SO port.



NOTE

- Output to the SO0 () and SO1 () ports is not allowed.
- Bits must be specified in ascending order from the right.
- External output is unavailable to SO ports not connected to hardware.

Example:

SO2()=&B10111000

----- SO (27, 25, 24, 23) are turned ON, and SO (26, 22, 21, 20) are turned OFF.

SO2(6,5,1)=&B010

----- SO (25) is turned ON, and SO (26, 21) are turned OFF.

SO3()=15

----- SO (33, 32, 31, 30) are turned ON, and SO (37, 36, 35, 34) are turned OFF.

SO(37,35,27,20)=A

----- The contents of the 4 lower bits obtained when variable A is converted to an integer are output to SO (37, 35, 27, 20), respectively.

Related commands: RESET, SET

SPEED statement

(Speed setting statement for main group)

Format:

SPEED <expression>

The value of <expression> must be from 1 to 100. (Unit: %)

Explanation:

This statement changes the program speed for the main group to the value specified in <expression>.



NOTE

- This statement changes all the movement speed parameters for the main axes and auxiliary axes of the robot.
- Robot operating speed is determined by the product of the automatic movement speed (specified by the MPB/RPB or ASPEED command) and the program speed (specified by the SPEED command or the SPEED option for movement commands).

Example:

When the automatic movement speed is 80% and the program speed is 50%, then: Robot operating speed = $80\% \times 50\% = 40\%$

- If the speed is specified by the DSPEED option in the DRIVE or DRIVEI command, the operating speed is determined only by the DSPEED option without using the automatic movement speed and SPEED command.

Example:

```
ASPEED 100
SPEED 70
MOVE P,P0
```

----- Moves at 70% ($=100 \times 70$) of maximum speed from the current position to P0.

```
SPEED 50
MOVE P, P1
```

----- Moves at 50% ($=100 \times 50$) of maximum speed from the current position to P1.

```
MOVE P,P2, S=10
```

----- Moves at 10% ($=100 \times 10$) of maximum speed from the current position to P2.

```
HALT
```

Related commands: ASPEED, ASPEED2, SPEED2

SPEED2 statement (Speed setting statement for sub group)

Format:

SPEED2 <expression>

The value of <expression> must be from 1 to 100. (Unit: %)



NOTE

- This statement changes all the movement speed parameters for the main axes and auxiliary axes of the robot.
- Robot operating speed is determined by the product of the automatic movement speed (specified by the MPB/RPB or ASPEED2 command) and the program speed (specified by the SPEED2 command or the SPEED option for movement commands).
- Example:**
When the automatic movement speed is 80% and the program speed is 50%, then: Robot operating speed = $80\% \times 50\% = 40\%$
- If the speed is specified by the DSPEED option in the DRIVE2 or DRIVEI2 command, the operating speed is determined only by the DSPEED option without using the automatic movement speed and SPEED2 command.
- This statement can be used only when the sub group is set in the system generation.

Explanation:

This statement changes the program speed for the sub group to the value specified in <expression>.

Example:

```
ASPEED2 50
SPEED2 70
MOVE2 P,P0
```

----- Moves at 35% ($=50 \times 70$) of maximum speed from the current position to P0.

```
SPEED2 50
MOVE2 P,P1
```

----- Moves at 25% ($=50 \times 50$) of maximum speed from the current position to P1.

```
MOVE2 P,P2,S=10
```

----- Moves at 5% ($=50 \times 10$) of maximum speed from the current position to P2.

```
HALT
```

Related commands: ASPEED, ASPEED2, SPEED

START statement

Format:

START <label>, Tn [, p]

n : task number 2 to 8
p : task priority 17 to 47

Explanation:

This statement starts the task specified by <label> as task “n” with priority “p”.
If the task priority “p” is omitted, the priority will be 32.

Example:

```
START *SUBTASK,T2,33
*ST:
  MOVE P,P0,P1
GOTO *ST
HALT
'SUBTASK ROUTINE
*SUBTASK:
  P100 = WHERE
  IF LOCZ(P100) > 10000 THEN
    DO(20) = 1
  ELSE
    DO(20) = 0
  ENDIF
GOTO *SUBTASK
EXIT TASK
```



NOTE

- The priority of task 1 (main task) is 32.
- The smaller the priority number, the higher the priority level.
- When a task with a higher priority is set “READY”, all tasks with lower priority levels also stay “READY”.

Related commands: CUT, EXIT TASK, RESTART, SUSPEND, CHGPRI

SUB statement, END SUB statement

Format:

```
SUB <label> [ ( <dummy argument> [, <dummy argument> . . . ] ) ]  
  <command block>  
END SUB
```

NOTE

- Sub-procedures cannot be defined within a sub-procedure.
- The *DECLARE* statement cannot be used within a sub-procedure.
- A label can be defined within a sub-procedure, but it cannot jump with a *GOTO* or *GOSUB* statement to a label outside the sub-procedure.
- Sub-procedures cannot be used within an expression.
- Local variables cannot be used with *PRINT* and *SEND* statements.

Explanation:

The SUB statement and END SUB statement define a sub-procedure that can be executed with the CALL statement. When the END SUB statement is executed, the program jumps to the next command in the CALL statement that was called. Defines are shown below.

1. All variables declared within the sub-procedure are local variables so they are valid only within the sub-procedure. Local variables are initialized each time the sub-procedure is called up.
2. Use a SHARED statement when using global variables (program level).
3. Use <dummy argument> when variables are to be passed on. If two or more dummy arguments are used, separate them by a comma (,).
4. A valid <dummy argument> consists of a name of variable and an entire array (array name followed by parentheses).

Example 1:

```
I=1
CALL *TEST
PRINT I
HALT
'SUB ROUTINE: TEST
SUB *TEST
    I=50
END SUB
```

In the above example, the program level variable I does not have any relation to variable I within the sub-procedure. The value displayed with the PRINT statement on the third line will be “1”.

Example 2:

```
X% = 4
Y% = 5
CALL *COMPARE( REF X%, REF Y% )
PRINT X%,Y%
Z% = 7
W% = 2
CALL *COMPARE( REF Z%, REF W% )
PRINT Z%,W%
HALT
'SUB ROUTINE: COMPARE
SUB *COMPARE( A%, B% )
    IF A% < B% THEN
        TEMP% = A%
        A% = B%
        B% = TEMP%
    ENDIF
END SUB
```

In the above example, different variables are passed on as arguments to call up the sub-procedure twice.

Related commands: CALL, DECLARE, EXIT SUB, SHARED

SUSPEND statement

Format:

SUSPEND Tn

n: task number 2 to 8

Explanation:

This statement temporarily stops another task being executed.

Examples:

```
START *SUBTASK,T2
SUSFLG=0
*L0:
  MOVE P,P0
  MOVE P,P1
  WAIT SUSFLG=1
  SUSPEND T2
  SUSFLG=0
GOTO *L0
HALT
'SUBTASK ROUTINE
*SUBTASK:
  WAIT SUSFLG=0
  DO2(0)=1
  DELAY 1000
  DO2(0)=0
  DELAY 1000
  SUSFLG=1
  GOTO *SUBTASK
EXIT TASK
```

Related commands: CUT, EXIT TASK, RESTART, SUSPEND

SWI statement

This statement switches the execution program.

Format:

```
SWI <program name>
```



NOTE

- If there is no program to switch to, then the message "3.3: Program doesn't exist" appears and operation stops.
- If an error occurs during compiling, an error message line is displayed and the program stops.
- The SWI statement can only be executed within task 1 (main task). If used within tasks 2 through 8, the message "6.1:Illegal command" appears and operation stops.
- The Stop key does not function during compiling.
- Execution of a SWI statement is always accompanied by compiling, and the time required for this compiling depends on the size of program to switch to.

Explanation:

This statement switches from the current program to the specified program, starting from the first line after executing compiling. The output variable status does not change when the program was switched. However, dynamic variables and array variables are cleared. If an error occurs during compiling, the operation stops. The program name you want to switch to must be enclosed in < >.

Example:

```
SWI <ABC>
```

----- The execution program switches to <ABC>.

TO statement

Format:

[LET]	TO0 ([b, . . . , b])	= <expression>
	TO (0 b, . . . , 0 b)	

b : bit definition 0 to 7

The value of <expression> should be an integer or it will be converted to an integer. The lower bits corresponding to the bits specified on the left side will be valid.

Explanation:

This statement outputs the specified value to the TO port.

Example:

TO0() = &B00000110

Related commands: RESET, SET



NOTE

Bits must be specified in ascending order from the right.

TOLE statement (Tolerance setting statement for main group)

Format 1:

```
TOLE <expression>
```

Format 2:

```
TOLE ( <expression 1> ) = <expression 2>
```

The value of <expression 1> must be from 1 to 6 (axis number).

The values of <expression> and <expression 2> differ depending on the motor. (Unit: pulses)



NOTE

- If an axis that is set to “no axis” in the system generation is specified, then an error message “Specification mismatch” appears and execution of the command will stop.
- This statement is executed after positioning on the specified axes is complete (within the tolerance range).

Explanation:

This statement changes the tolerance parameter for the main group to the value specified in <expression>. Format 1 changes all axes in the main group. Format 2 changes the tolerance parameter for the main group axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH DECREASING TOLERANCE
FOR A=100 TO 20 STEP -20
GOSUB *CHANGE_TOLE
MOVE P,P0
MOVE P,P1
NEXT A
HALT
*CHANGE_TOLE:
FOR B=1 TO 4
TOLE(B)=A
NEXT B
RETURN
```

TOLE2 statement (Tolerance setting statement for sub group)

Format 1:

```
TOLE2 <expression>
```

Format 2:

```
TOLE2 ( <expression 1> ) = <expression 2>
```

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression> and <expression 2> differs depending on the motor setting.
(Unit: pulses)



NOTE

- This statement can be used only when the sub group is set in the system generation.
- If an axis that is set to “no axis” in the system generation is specified, then an error message “Specification mismatch” appears and execution of the command will stop.
- This statement is executed after positioning on the specified axes is complete (within the tolerance range).

Explanation:

This statement changes the tolerance parameter for the sub group to the value specified in <expression>. Format 1 changes all axes in the sub group. Format 2 changes the tolerance parameter for the sub group axis specified in <expression 1> to the value specified in <expression 2>.

Example:

```
'CYCLE WITH DECREASING TOLERANCE
FOR A=100 TO 20 STEP -20
GOSUB *CHANGE_TOLE
MOVE2 P,P0
MOVE2 P,P1
NEXT A
HALT
*CHANGE_TOLE:
FOR B=1 TO 4
TOLE2(B)=A
NEXT B
RETURN
```

TORQUE statement

Format

TORQUE (<expression 1>) = <expression 2>

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression 2> must be from 1 to 100. (Unit: %)



NOTE

If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.



CAUTION

- The timing that enables a maximum torque instruction changed by the TORQUE statement (mentioned on the right) is only valid from driver version 2.17 onwards. On earlier version controllers, the maximum torque instruction is enabled at the point when it was changed by the TORQUE statement.
- If the specified torque limit is too small, the axis might not move. In this case, press the emergency stop button and then continue operation.
- If the specified value is less than the rated torque, then an error might not occur even if the robot bumps into an obstacle.

Explanation:

This statement changes the maximum torque instruction for the main group axis specified in <expression 1> to the value specified in <expression 2>. The new value is enabled after the next movement command such as a MOVE or DRIVE statement is executed. The torque parameter value does not change.

The maximum torque instruction by this statement remains valid until any of the following operations are performed.

- When another TORQUE command for the same axis is executed, and the next movement command such as a MOVE or DRIVE statement is then executed.
- When a torque limit option is executed in the DRIVE statement for the same axis.
- When controller power is turned off and then turned back on.
- When parameters are changed or initialized.
- When return-to-origin or absolute search accompanying return-to-origin is performed.
- When return-to-origin accompanying absolute reset is performed.
- When servo is turned off.

Example:

```

TRQTIME(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 seconds.

DRIVE(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 (pushing action).

IF TRQSTS(3)=1 THEN
----- Checks if time-out is up.
  DO(21)=1
----- Time-out is up (pushing is complete).
  ----- (Result is output to DO(21) in this example.)
ELSE
  DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to complete pushing.)
  ----- (Result is output to DO(21) in this example.)
ENDIF
TORQUE(3)=100
----- Returns the maximum torque instruction to original value (100%).

DRIVE(3,P0)
----- Ends the torque limit and torque control, and moves to P0.
  
```

Related commands: DRIVE, TRQTIME, TRQSTS functions

TORQUE2 statement

Format

```
TORQUE2 ( <expression 1> ) = <expression 2>
```

The value of <expression 1> must be from 1 to 4 (axis number).

The value of <expression 2> must be from 1 to 100. (Unit: %)



NOTE

- This statement is enabled only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.



CAUTION

- The timing that enables a maximum torque instruction changed by the TORQUE2 statement (mentioned on the right) is only valid from driver version 2.17 onwards. On earlier version controllers, the maximum torque instruction is enabled at the point when it was changed by the TORQUE2 statement.
- If the specified torque limit is too small, the axis might not move. In this case, press the emergency stop button and then continue operation.
- If the specified value is less than the rated torque, then an error might not occur even if the robot bumps into an obstacle.

Explanation:

This statement changes the maximum torque instruction for the sub group axis specified in <expression 1> to the value specified in <expression 2>. The new value is enabled after the next movement command such as a MOVE2 or DRIVE2 statement is executed. The torque parameter value does not change.

The maximum torque instruction by this statement remains valid until any of the following operations are performed.

- When another TORQUE2 command for the same axis is executed, and the next movement command such as a MOVE2 or DRIVE2 statement is then executed.
- When a torque limit option is executed in the DRIVE2 statement for the same axis.
- When controller power is turned off and then turned back on.
- When parameters are changed or initialized.
- When return-to-origin or absolute search accompanying return-to-origin is performed.
- When return-to-origin accompanying absolute reset is performed.
- When servo is turned off.

Example:

```
TRQTIME2(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 seconds.

DRIVE2(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 (pushing action).

IF TRQSTS2(3)=1 THEN
----- Checks if time-out is up.
  DO(21)=1
----- Time-out is up (pushing is complete).
  ----- (Result is output to DO(21) in this example.)
ELSE
  DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to complete pushing.)
  ----- (Result is output to DO(21) in this example.)
ENDIF
TORQUE2(3)=100
----- Returns the maximum torque instruction to original value (100%).

DRIVE2(3,P0)
----- Ends the torque limit and torque control, and moves to P0.
```

Related commands: DRIVE2, TRQTIME2, TRQSTS2 functions

TRQTIME statement



NOTE

- The **TRQTIME** statement is only available from software version 8.45 onwards.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Format:

```
TRQTIME ( <expression 1> ) = <expression 2>
```

The value in <expression 1> must be from 1 to 6 (main group axis number).

The value in <expression 2> is the time-out period and must be from 0 to 10000.

(Unit: ms)

Explanation:

This statement sets the time-out period for the current limit when a torque limit option is used in the **DRIVE** statement. This statement sets the time-out period for the main group axis specified in <expression 1> to the value specified in <expression 2>.

After the **DRIVE** statement is executed with a torque limit option, that **DRIVE** command ends when the axis has reached the target position or when the time (time-out period) specified by the **TRQTIME** statement has elapsed while the torque has reached the specified limit value. At this point, a value is set in the **TRQSTS** function, depending on whether this command has ended due to a time-out during which the torque has reached the specified torque limit value.

When the controller power is turned on, the time-out period is set to 1 second.

Although the time-out period is specified in "ms" (milliseconds) units, it actually works in "10ms" units. So the setting is rounded up to the next multiple of 10ms. For example, if the setting is a value from 1 to 9, it is viewed as 10ms. However, if "0" is specified, it will be 1 second (1000ms).

Example:

```
TRQTIME(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 seconds.

DRIVE(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 (pushing action).

IF TRQSTS(3)=1 THEN
----- Checks if time-out is up.
  DO(21)=1
----- Time-out is up (pushing is complete).
  (Result is output to DO(21) in this example.)
ELSE
  DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to complete pushing.)
  (Result is output to DO(21) in this example.)
ENDIF
TORQUE(3)=100
----- Returns the maximum torque instruction to original value (100%).

DRIVE(3,P0)
----- Ends the torque limit and torque control, and moves to P0.
```

Related commands: **DRIVE**, **TRQSTS** functions

TRQTIME2 statement



NOTE

- The TRQTIME2 statement is only available from software version 8.45 onwards.
- This statement can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

Format:

```
TRQTIME2 ( <expression 1> ) = <expression 2>
```

The value in <expression 1> must be from 1 to 6 (sub group axis number).
The value in <expression 2> is the time-out period and must be from 0 to 10000.
(Unit: ms)

Explanation:

This statement sets the time-out period for the current limit when a torque limit option is used in the DRIVE2 statement. This statement sets the time-out period for the sub group axis specified in <expression 1> to the value specified in <expression 2>.

After the DRIVE2 statement is executed with a torque limit option, that DRIVE command ends when the axis has reached the target position or when the time (time-out period) specified by the TRQTIME2 statement has elapsed while the torque has reached the specified limit value. At this point, a value is set in the TRQSTS2 function, depending on whether this command has ended due to a time-out during which the torque has reached the specified torque limit value.

When the controller power is turned on, the time-out period is set to 1 second. Although the time-out period is specified in "ms" (milliseconds) units, it actually works in "10ms" units. So the setting is rounded up to the next multiple of 10ms. For example, if the setting is a value from 1 to 9, it is viewed as 10ms. However, if "0" is specified, it will be 1 second (1000ms).

Example:

```
TRQTIME2(3)=2500
----- Sets the torque control time-out period on axis 3 to 2.5 seconds.
DRIVE2(3,P1),T=(20,15)
----- Sets the maximum torque value to 20% of rated torque and the torque offset to 15, and moves axis 3 from the current position to the point specified by P1 (pushing action).
IF TRQSTS2(3)=1 THEN
----- Checks if time-out is up.
DO(21)=1
----- Time-out is up (pushing is complete).
              (Result is output to DO(21) in this example.)
ELSE
DO(21)=0
----- Time-out is not yet up. (Reached target position but failed to complete pushing.)
              (Result is output to DO(21) in this example.)
ENDIF
TORQUE2(3)=100
----- Returns the maximum torque instruction to original value (100%).
DRIVE2(3,P0)
----- Ends the torque limit and torque control, and moves to P0.
```

Related commands: DRIVE2, TRQSTS2 functions

WAIT statement

Format 1:

```
WAIT < DI / DO conditional expression> [, <expression>]
```

The value of <expression> must be from 1 to 3600000. (Unit: ms)



NOTE

On controllers using the following software versions, when the conditional expression is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.

RCX14x Ver. 8.63 onwards,

RCX22x Ver. 9.08 onwards

On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

This statement allows waiting until the condition in <DI/DO conditional expression> is met (becomes true). Specify the time-out period (milliseconds) in the second <expression>.

If the wait condition is not met within the time-out period, the command terminates. Note that the minimum wait time is 10 ms. If the value in <expression> is set to 1 to 9, then the time-out period is viewed as 10ms.

Format 2:

```
WAIT | ARM [ (<expression 1> ) ] |
      | ARM2 [ (<expression 2> ) ] |
```

The value of <expression 1> must be from 1 to 6 (axis number).

The value of <expression 2> must be from 1 to 4 (axis number).

Explanation:

This statement allows waiting until positioning on the robot axes is complete (within the tolerance range).

If the <expression 1> and <expression 2> are not specified, then all axes (ARM) of the main robot or all axes (ARM2) of the sub robot are subject to this command. When the <expression 1> and <expression 2> are specified, only the specified axes are subject to this command.

Examples:

WAIT A=10

----- Waits until variable A is set to 10.

WAIT DI2()=&B01010110

----- Waits until DI(21), (22), (24), (26) are turned ON, and DI(20), (23), (25), (27) are turned OFF.

WAIT DI2(4,3,2)=&B101

----- Waits until DI(22) and DI(24) are turned ON, and DI(23) is turned OFF.

WAIT DI(31)=1 OR DO(21)=1

----- Waits until DI(31) or DO(21) is turned ON.

WAIT DI(20)=1,1000

----- Waits until DI(20) is ON. If not turned ON after 1 second, the command will end.

WAIT ARM

----- Waits until the main robot movement is complete.

WAIT ARM2(2)

----- Waits until movement on axis 2 of the sub robot is complete.

Related commands: DRIVE, DRIVE2, DRIVEI, DRIVEI2, MOVE, MOVE2, MOVEI, MOVEI2

WEIGHT statement (Tip weight parameter setting statement for main robot)

Format:

```
WEIGHT <expression>
```

The range of <expression> differs depending on the robot setting.

Explanation:

This statement changes the tip weight parameter of the main robot to the value of <expression>.



NOTE

This statement changes the robot tip weight parameter but does not have any effect on auxiliary axes.

Example:

```
A=5  
B=2  
C=WEIGHT  
WEIGHT A  
MOVE P,P0  
WEIGHT B  
MOVE P,P1  
WEIGHT C  
HALT
```

WEIGHT2 statement

(Tip weight parameter setting statement for sub robot)

Format:

```
WEIGHT2 <expression>
```

The range of <expression> differs depending on the robot setting.

Explanation:

This statement changes the tip weight parameter of the sub robot to the value of <expression>.

Example:

```
A=5
B=2
C=WEIGHT2
WEIGHT2 A
MOVE2 P,P0
WEIGHT2 B
MOVE2 P,P1
WEIGHT2 C
HALT
```



NOTE

- This statement can be used only when the sub robot is set in the system generation.
- This statement changes the robot tip weight parameter but does not have any effect on auxiliary axes.

WHILE statement, WEND statement

The WHILE statement allows the same operation to repeat as long as the condition specified by <expression> is met.

Format:

```
WHILE <expression>
    <command block>
WEND
```



NOTE

- The WHILE and WEND statements must be used together as a set.
 - If the value of <expression> is -1, the condition is judged to be satisfied.
 - On controllers using the following software versions, when the conditional expression is a numeric expression, the "TRUE condition" parameter of "Other parameters" (SYSTEM > PARAM > OTHERS mode) can be set to change the conditions for determining "true" and "false" (-1 is "true" and 0 is "false", or a value other than 0 is "true" and 0 is "false"). Refer to the controller user's manual for detailed information.
- RCX14x Ver. 8.63 onwards,
RCX22x Ver. 9.08 onwards
- On earlier version controllers, -1 is "true" and a value other than -1 is "false".

Explanation:

This statement executes the command block placed between the WHILE and WEND statements when the condition specified by <expression> is met (true), and then returns to the WHILE statement to repeat the same operation. When the condition specified by <expression> is not met (becomes false), the program jumps to the next command following the WEND statement.

If the condition specified by <expression> is not satisfied (false) from the beginning, then no commands between the WHILE and WEND statements are executed and a jump is made to the next statement following the WEND statement.

To jump outside of the WHILE and WEND statement loop, use a GOTO statement, etc.

Examples 1:

```
A=0
WHILE DI3(0)=0
    A=A+1
    MOVE P,P0
    MOVE P,P1
    PRINT "COUNTER=";A
WEND
HALT
```

Examples 2:

```
A=0
WHILE -1 ----- This will be an endless loop because the conditional expression is
                    always true (-1).

    A=A+1
    MOVE P,P0
    IF DI3(0)=1 THEN *END
    MOVE P,P1
    PRINT "COUNTER=";A
    IF DI3(0)=1 THEN *END
WEND
*END
HALT
```

Label statement

Format:

```
<label>:
```

Explanation:

This statement defines <Label> in the program lines. Labels must begin with an asterisk (*).

Examples:

```
*SUB2:  
*PROG_END:
```

12. Functions

12.1 Arithmetic functions

These functions are used as elements in arithmetic expressions.

■ ABS

Format:

ABS (<expression>)

Explanation:

This function gives the absolute value of the value specified in <expression>.

Example:

A=ABS(-326.54)

----- The absolute value of -326.54 (=326.54) is assigned to variable A.

■ ABSRPOS

Format:

ABSRPOS (<expression>)

The value in <expression> must be from 1 to 6 (main group axis number).

Explanation:

This function gives the machine reference (unit: %) of the main group axis specified in <expression>. This function can only be used for axes whose return-to-origin method is set to mark method. This function is ignored in cases where the return-to-origin method is set to the sensor or stroke end (torque detection) method.

Example:

A=ABSRPOS(1)

----- The machine reference of axis 1 in the main group is assigned to variable A.

■ ABSRPOS2

Format:

ABSRPOS2 (<expression>)

The value in <expression> must be from 1 to 6 (sub group axis number).

Explanation:

This function gives the machine reference (unit: %) of the sub group axis specified in <expression>. This function can only be used for axes whose return-to-origin method is set to mark method. This function is ignored in cases where the return-to-origin method is set to the sensor or stroke end (torque detection) method.

Example:

A=ABSRPOS2(1)

----- The machine reference of axis 1 in the sub group is assigned to variable A.

NOTE

- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.
- On axes whose return-to-origin method is set to mark method, absolute reset is possible when the ABSRPOS value is in a range from 44 to 56%.

NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.
- On axes whose return-to-origin method is set to mark method, absolute reset is possible when the ABSRPOS2 value is in a range from 44 to 56%.

■ ACCEL

Format:

ACCEL (<expression>)

The value of <expression> must be from 1 to 6 (axis number).

NOTE

If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.

Explanation:

This function gives the acceleration coefficient parameter for the main group axis specified in <expression>.

Example:

A=ACCEL(3)

----- The acceleration coefficient parameter for axis 3 in the main group is assigned to variable A.

■ ACCEL2

Format:

ACCEL2 (<expression>)

The value of <expression> must be from 1 to 4 (axis number).

NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.

Explanation:

This function gives the acceleration coefficient parameter for the sub group axis specified in the <expression>.

Example:

A=ACCEL2(2)

----- The acceleration coefficient parameter for axis 2 in the sub group is assigned to variable A.

■ ARCH

Format:

ARCH (<expression>)

The value of <expression> must be from 1 to 6 (axis number).

NOTE

If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.

Explanation:

This function gives the arch position parameter for the main group axis specified in <expression>.

Example:

A=ARCH(1)

----- The arch position parameter for axis 1 in the main group is assigned to variable A.

■ ARCH2

Format:

```
ARCH2 (<expression>)
```

The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This function gives the arch position parameter for the sub group axis specified in the <expression>.

Example:

```
A=ARCH2(1)
```

----- The arch position parameter for axis 1 in the sub group is assigned to variable A.

■ ARMCND

Format:

```
ARMCND
```

Explanation:

This function gives the current arm status used for the SCARA robot in the main group. The arm status is “1” for left-handed condition and “0” for right-handed condition. This function is enabled only when a SCARA robot is used.

Example:

```
A=ARMCND
```

----- The current arm status of the main robot is assigned to variable A.

```
IF A=0 THEN
```

----- If right-handed system is used...

```
    MOVE P,P100,Z=0
```

```
ELSE
```

----- If left-handed system is used...

```
    MOVE P,P200,Z=0
```

```
ENDIF
```

■ ARMCND2

Format:

```
ARMCND2
```

Explanation:

This function gives the current arm status for the SCARA robot in the sub group. The arm status is “1” for left-handed condition and “0” for right-handed condition. This function is enabled only when a SCARA robot is used.

Example:

```
A=ARMCND2
```

----- The current arm status of the sub robot is assigned to variable A.

```
IF A=0 THEN
```

----- If right-handed system is used...

```
    MOVE2 P,P100,Z=0
```

```
ELSE
```

----- If left-handed system is used...

```
    MOVE2 P,P200,Z=0
```

```
ENDIF
```



NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.



NOTE

This function can be used only when the sub group has been set in the system generation.

■ ARMTYPE

Format:

ARMTYPE

Explanation:

This function gives the hand system currently selected for the SCARA robot in the main group.

The arm type is “1” for left-handed system and “0” for right-handed system.

This function is enabled only when a SCARA robot is used.

Example:

```
A=ARMTYPE          ----- The arm type of the main robot is assigned to variable A.
IF A=0 THEN        ----- If the arm type is a right-handed system ...
    MOVE P,P100,Z=0
ELSE               ----- If the arm type is a left-handed system...
    MOVE P,P200,Z=0
ENDIF
```

■ ARMTYPE2

Format:

ARMTYPE2

Explanation:

This function gives the hand system currently selected for the SCARA robot in the sub group.

The arm type is “1” for left-handed system and “0” for right-handed system.

This function is enabled only when a SCARA robot is used.

Example:

```
A=ARMTYPE2          ----- The arm type of the sub robot is assigned to variable A.
IF A=0 THEN        ----- If the arm type is a right-handed system ...
    MOVE2 P,P100,Z=0
ELSE               ----- If the arm type is a left-handed system...
    MOVE2 P,P200,Z=0
ENDIF
```

■ ATN

Format:

ATN (<expression>)

Explanation:

This function gives the arctangent of the value of <expression>. The resulting value obtained will be from $-\pi/2$ rad to $+\pi/2$ rad.

Examples:

```
A(0)=A*ATN(Y/X)
----- The product of the arctangent of the expression (Y/X) and
         variable A is assigned to array A (0).

A(0)=ATN(0.5)
----- The arctangent of 0.5 is assigned to array A (0).
```

Related functions: COS, DEGRAD, RADDEG, SIN, TAN



NOTE

This function can be used only when the sub group has been set in the system generation.

■ AXWGHT

Format:

AXWGHT (<expression>)

The value of <expression> must be from 1 to 6 (axis number).

Explanation:

This function gives the axis tip weight parameter for the main group axis specified in <expression>.

Example:

A=AXWGHT(1)

----- The axis tip weight parameter for axis 1 in the main group is assigned to variable A.

■ AXWGHT2

Format:

AXWGHT2 (<expression>)

The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This function gives the axis tip weight parameter for the sub group axis specified in <expression>.

Example:

A=AXWGHT2(1)

----- The axis tip weight parameter for axis 1 in the sub group is assigned to variable A.

■ COS

Format:

COS (<expression>)

Explanation:

This function gives the cosine of the value of <expression>. The <expression> units are in radians.

Examples:

A(0)=B*COS(C)

----- The product of the cosine of variable C and variable B is assigned to array A (0).

A(1)=COS(DEGRAD(20))

----- The cosine of 20.0° is assigned to array A (1).

Related functions: ATN, DEGRAD, RADDEG, SIN, TAN



NOTE

- This function can be used only for MULTI type robot axes or auxiliary axes.
Robot type and auxiliary axes are set prior to shipment.
- If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.



NOTE

- This function can be used only when the sub group has been set in the system generation.
- This function can be used only for MULTI type robot axes or auxiliary axes.
Robot type and auxiliary axes are set prior to shipment.
- If an axis that is set to “no axis” in the system generation is specified in <expression>, then an error message “Specification mismatch” appears and execution of the command will stop.

**CAUTION**

The **DECEL** statement is only available from software version 8.15 onwards.

**NOTE**

- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

DECEL

Format:

DECEL (<expression>)

The value of <expression> must be from 1 to 6 (axis number).

Explanation:

This function gives the deceleration rate parameter for the main group axis specified in the <expression>.

Example:

A = DECEL (3)

----- The deceleration rate parameter for axis 3 in the main group is assigned to variable A.

**CAUTION**

The **DECEL2** statement is only available from software version 8.15 onwards.

**NOTE**

- This function can be used only when the sub group is set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

DECEL2

Format:

DECEL2 (<expression>)

The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This function gives the deceleration rate parameter for the sub group axis specified in the <expression>.

Example:

A = DECEL2 (2)

----- The deceleration rate parameter for axis 2 in the sub group is assigned to variable A.

DEGRAD

Format:

DEGRAD (<expression>)

Explanation:

This function converts the value of <expression> into radians. The <expression> units are in degrees.

Example:

A=COS(DEGRAD(30))

----- The cosine of 30° is assigned to variable A.

Related functions: ATN, COS, RADDEG, SIN, TAN

■ DIST

Format:

```
DIST (<point expression 1>, <point expression 2>)
```

Explanation:

This function gives the distance between the 2 points (X, Y, Z) specified in <point expression 1> and <point expression 2>. An error is issued when the 2 points specified by each <point expression> are not in the Cartesian coordinate system.

Example:

```
A=DIST(P0,P1)
```

----- The distance between P0 and P1 is assigned to variable A.

■ ERR, ERL

Format:

```
ERR  
ERL
```

Explanation:

These are used for in error processing routines specified by the ON ERROR GOTO statement. ERR gives the error code of the error that has occurred, and ERL gives the line number in which the error occurred.

Examples:

```
IF ERR<>&H604 THEN HALT  
IF ERL=20 THEN RESUME NEXT
```

Related commands: ON ERROR GOTO, RESUME

■ INT

Format:

```
INT (<expression>)
```

Explanation:

This function gives an integer that does not exceed the value of <expression>, by truncating all decimal fractions.

Examples:

```
A=INT(A(0))  
B=INT(-1.233)
```

----- “-2” is assigned to B.

■ LEN

Format:

```
LEN (<character string expression>)
```

Explanation:

This function gives the character string length (number of bytes) specified in <character string expression>.

Example:

```
B=LEN(A$)
```

■ LSHIFT

Format:

```
LSHIFT (<expression 1>, <expression 2>)
```

Explanation:

This function shifts the value of <expression 1> to the left, by the number of bits in <expression 2>. Here, a "0" is inserted in spaces open to the right due to shifting.

Example:

```
A=LSHIFT(&B10111011,2)
```

----- "&B1011101100" is assigned to A.

Related functions: RSHIFT

■ MCHREF

Format:

```
MCHREF (<expression>)
```

The value of <expression> must be 1 to 6 (axis number).

Explanation:

This function gives the return-to-origin or absolute-search machine reference (unit: %) of the main group axis specified in <expression>.

This function can only be used for axes whose return-to-origin method is set to the sensor or stroke end (torque detection) method.

Example:

```
A=MCHREF(1)
```

----- The return-to-origin or absolute-search machine reference of axis 1 in the main group is assigned to variable A.



NOTE

If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.



NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

■ MCHREF2

Format:

MCHREF2 (<expression>)

The value of <expression> must be 1 to 4 (axis number).

Explanation:

This function gives the return-to-origin or absolute-search machine reference (unit: %) of the sub group axis specified in <expression>.

This function can only be used for axes whose return-to-origin method is set to the sensor or stroke end (torque detection) method.

Example:

A=MCHREF2(3)

----- The return-to-origin or absolute-search machine reference of axis 3 in the sub group is assigned to variable A.

■ ORD

Format:

ORD (<character string expression>)

Explanation:

This function gives the character code of the first character specified in <character string expression>.

Example:

A=ORD("B")

----- "66 (= &H42)" is assigned to A.

Related functions: CHR\$

■ ORGORD

Format:

ORGORD

Explanation:

This function gives the axis sequence parameter for performing return-to-origin and absolute search operations for the main group.

Example:

A=ORGORD

----- The return-to-origin sequence parameter for the main group is assigned to variable A.

■ ORGORD2

Format:

ORGORD2

Explanation:

This function gives the axis sequence parameter for performing return-to-origin and absolute search operations for the sub group.

Example:

A=ORGORD2

----- The return-to-origin sequence parameter for the sub group is assigned to variable A.



NOTE

This function can be used only when the sub group has been set in the system generation.

■ OUTPOS

Format:

OUTPOS (<expression>)

The value of <expression> must be 1 to 6 (axis number).

Explanation:

This function gives the Out position parameter for the main group axis specified in the <expression>.

Example:

A=OUTPOS(3)

----- The Out position parameter for axis 3 in the main group is assigned to A.



NOTE

If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

■ OUTPOS2

Format:

OUTPOS2 (<expression>)

The value of <expression> must be 1 to 4 (axis number).

Explanation:

This function gives the Out position parameter for the sub group axis specified in the <expression>.

Example:

A=OUTPOS2(3)

----- The Out position parameter for axis 3 in the sub group is assigned to A.



NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

■ RADDEG

Format:

RADDEG (<expression>)

Explanation:

This function converts the value of <expression> into degrees. The <expression> units are in radians.

Example:

LOC(RP0)=RADDEG(ATN(B))

----- The arctangent of variable B is converted into a value in degrees and assigned to R data for the P0.

Related functions: ATN, COS, DEGRAD, SIN, TAN

■ RSHIFT

Format:

RSHIFT (<expression 1>, <expression 2>)

Explanation:

This function shifts the value of <expression 1> to the right, by the number of bits in <expression 2>. Here a "0" is inserted in space open to the left due to shifting.

Example:

A=RSHIFT(&B10111011,2)

----- "&B00101110" is assigned to A.

Related functions: LSHIFT

■ SIN

Format:

SIN (<expression>)

Explanation:

This function gives the sine of the value of <expression>. The <expression> units are in radians.

Examples:

A(0)=SIN(B*2+C)

----- The sine of the expression "B*2+C" is assigned to array A (0).

A(1)=SIN(DEGRAD(30))

----- The sine of 30.0° is assigned to array A (1).

Related functions: ATN, COS, DEGRAD, RADDEG, TAN

■ SQR

Format:

SQR (<expression>)

The value of <expression> must be a positive integer or “0”.



NOTE

An error occurs if a negative value is specified in <expression>.

Explanation:

This function gives the square root of the value in <expression>.

Example:

$A = \text{SQR}(X^2 + Y^2)$

----- The square root of $X^2 + Y^2$ will be assigned to A.

■ TAN

Format:

TAN (<expression>)

Explanation:

This function gives the tangent of the value of <expression>. The <expression> units are in radians.

Examples:

$A(0) = B - \text{TAN}(C)$

----- The tangent of variable C is subtracted from variable B and the result is assigned to array A (0).

$A(1) = \text{TAN}(\text{DEGRAD}(20))$

----- The tangent of 20.0° is assigned to array A (1).

Related functions: ATN, COS, DEGRAD, RADDEG, SIN

■ TIMER

Format:

TIMER

Explanation:

This function gives the current time in seconds, counting from 12:00 midnight as "0:00". It is used to measure operation time during program execution, as well as for other uses.



NOTE

The time can be set in "SYSTEM>INIT>CLOCK" mode.



CAUTION

The clock used in the controller might differ from the correct time.

Example:

```
A%=TIMER
FOR B=1 TO 10
  MOVE P,P0
  MOVE P,P1
NEXT
A%=TIMER-A%
PRINT A%/60;"",A% MOD 60
HALT
```

Related functions: TIME\$

■ TOLE

Format:

TOLE (<expression>)

The value of <expression> must be from 1 to 6 (axis number).



NOTE

If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

Explanation:

This function gives the tolerance parameter for the main group axis specified in <expression>.

Example:

```
A=TOLE(2)
```

----- The tolerance parameter for axis 2 in the main group is assigned to variable A.

■ TOLE2

Format:

TOLE2 (<expression>)

The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This function gives the tolerance parameter for the sub group axis specified in <expression>.

Example:

A=TOLE2(2)

----- The tolerance parameter for axis 2 in the sub group is assigned to variable A.

■ TORQUE

Format:

TORQUE (<expression>)

The value of <expression> must be from 1 to 6 (axis number).

Explanation:

This function gives the torque setting value for the main group axis specified in <expression>.

Example:

A=TORQUE (2)

----- The torque setting value for axis 2 in the main group is assigned to variable A.

■ TORQUE2

Format:

TORQUE2 (<expression>)

The value of <expression> must be from 1 to 4 (axis number).

Explanation:

This function gives the torque setting value for the sub group axis specified in <expression>.

Example:

A=TORQUE2 (2)

----- The torque setting value for axis 2 in the sub group is assigned to variable A.

NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

NOTE

- If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

NOTE

- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified in <expression>, then an error message "Specification mismatch" appears and execution of the command will stop.

■ TRQSTS

Format:

TRQSTS (<expression>)

The value in <expression 1> must be from 1 to 6 (main group axis number).

Explanation:

This function gives the status at the time that the DRIVE statement with a torque limit option was executed on the main group axis specified in <expression> and completed. The status is expressed as either 0 or 1.

- 0 ... The DRIVE statement with a torque limit setting option that was executed on the main group axis specified in <expression> ended for a reason other than torque limit time-out.
- 1 ... The DRIVE statement with a torque limit setting option that was executed on the main group axis specified in <expression> ended by torque limit time-out.

Example:

```
DRIVE(3,P1),T=20
----- Axis 3 in the main group moves under torque limit.
IF TRQSTS(3)=1 THEN
----- Axis 3 in the main group reached the torque limit value and
      command ended in a time-out.
      GOTO *OK
ELSE
----- Axis 3 in the main group stopped without timing out under
      torque limit.
      GOTO *NG
ENDIF
```

Related commands: DRIVE, TRQTIME



NOTE

- TRQSTS is only available from software version 8.45 onwards.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

■ TRQSTS2



NOTE

- *TRQSTS2 is only available from software version 8.45 onwards.*
- *This function can be used only when the sub group has been set in the system generation.*
- *If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.*

Format:

TRQSTS2 (<expression>)

The value in <expression 1> must be from 1 to 6 (sub group axis number).

Explanation:

This function gives the status at the time that the DRIVE2 statement with a torque limit option was executed on the sub group axis specified in <expression> and completed.

The status is expressed as either 0 or 1.

0 ... The DRIVE2 statement with a torque limit setting option that was executed on the sub group axis specified in <expression> ended for a reason other than torque limit time-out.

1 ... The DRIVE2 statement with a torque limit setting option that was executed on the sub group axis specified in <expression> ended by torque limit time-out.

Example:

```
DRIVE2(3,P1),T=20
----- Axis 3 in the sub group moves under torque limit.
IF TRQSTS2(3)=1 THEN
----- Axis 3 in the sub group reached the torque limit value and
      GOTO *OK
      command ended in a time-out.
ELSE
----- Axis 3 in the sub group stopped without timing out under
      GOTO *NG
      torque limit.
ENDIF
```

Related commands: DRIVE2, TRQTIME2

■ TRQTIME

Format:

TRQTIME (<expression>)

The value in <expression> must be from 1 to 6 (main group axis number).

Explanation:

This function gives the torque control time-out period for the main group axis specified in <expression>. The <expression> units are in ms (milliseconds)

Example:

A%=TRQTIME(3)

----- The torque control time-out period (unit: ms) for axis 3 in the main group is assigned to variable A.

■ TRQTIME2

Format:

TRQTIME2 (<expression>)

The value in <expression> must be from 1 to 6 (sub group axis number).

Explanation:

This function gives the torque control time-out period for the sub group axis specified in <expression>. The <expression> units are in ms (milliseconds)

Example:

A%=TRQTIME2(3)

----- The torque control time-out period (unit: ms) for axis 3 in the sub group is assigned to variable A.



NOTE

- TRQTIME is only available from software version 8.45 onwards.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.



NOTE

- TRQTIME2 is only available from software version 8.45 onwards.
- This function can be used only when the sub group has been set in the system generation.
- If an axis that is set to "no axis" in the system generation is specified, then an error message "Specification mismatch" appears and execution of the command will stop.

■ VAL

Format:

```
VAL (<character string expression>)
```

Explanation:

This function converts the numeric value of the character string specified in <character string expression> into an actual numeric value.

The value may be expressed in integer format (binary, decimal, hexadecimal), or real number format (decimal point format, exponential format).

If the first character of the character string is “+”, “-”, “&” or anything other than a numeric character, then the VAL function value becomes 0. If there are non-numeric characters or spaces elsewhere in the character string, then all subsequent characters are ignored by this function. However, in the case of hexadecimal numbers, A to F are considered part of the numeric character.

Example:

```
A=VAL("&B100001")
```

Related functions: STR\$

■ WEIGHT

Format:

```
WEIGHT
```

Explanation:

This function gives the tip weight parameter for the main robot.

Example:

```
A=WEIGHT
```

----- The tip weight parameter of the main robot is assigned to variable A.

■ WEIGHT2

Format:

```
WEIGHT2
```

Explanation:

This function gives the tip weight parameter for the sub robot.

Example:

```
A=WEIGHT2
```

----- The tip weight parameter of the sub robot is assigned to variable A.



NOTE

This function can be used only when the sub group has been set in system generation.

■ TCOUNTER

Format:

TCOUNTER

Explanation:

This function outputs count-up values at 10ms intervals starting from the point when the TCOUNTER variable is reset.



NOTE

The 10ms COUNTER variable counts up from 0 to 65535, and then returns to 0 to continue count-up.

Example:

A=TCOUNETR

Related functions: RESET

■ _SYSFLG

Format:

_SYSFLG

Explanation:

This is used as a flag that monitors the axis status according to the value specified by the _SYSFLG variable. This should not be used during normal operation.



NOTE

When using this function as a flag for monitoring the axis status, please consult us for proper support.

Example:

_SYSFLG = 1

Related functions: RESET

12.2 Character string functions

Character string functions are only used as elements in character string expressions.

■ CHR\$

Format:

```
CHR$ (<expression>)
```

Explanation:

This function gives the character having a character code specified in <expression>. The value of <expression> must be between 0 and 255, otherwise an error will result.

Example:

```
A$=CHR$(65)
```

----- A character "A" is assigned to A\$.

Related functions: ORD

■ DATE\$

Format:

```
DATE$
```

Explanation:

This function gives the date in the format "yy/mm/dd" (last two digits of the year, month, day).

Examples:

```
A$=DATE$
```

```
PRINT DATE$
```

Related functions: TIME\$



NOTE

The date can be set in
"SYSTEM>INIT>CLOCK" mode.

■ LEFT\$

Format:

```
LEFT$ (<character string expression>, <expression>)
```

The value of <expression> must be 0 to 75.

Explanation:

This function extracts a character string with the digits specified by <expression> from the left end of the character string specified by <character string expression>. The value of <expression> must be between 0 and 75, otherwise an error will result. If the value of <expression> is 0, then LEFT\$ will be a null string. If the value of <expression> is greater than the length of <character string expression>, then the LEFT\$ string will be the same as the <character string expression>.

Example:

```
B$=LEFT$(A$,4)
```

Related functions: MID\$, RIGHT\$

■ MID\$

Format:

```
MID$ (<character string expression>, <expression 1>[, <expression 2>])
```

The value of <expression 1> must be 1 to 75.

The value of <expression 2> must be 0 to 75.

Explanation:

This function extracts a character string with the desired length (number of characters) from the character string specified by <character string expression>. The value of <expression 1> specifies the first character to be extracted, and the value of <expression 2> the number of characters. The value of <expression 1> must be between 1 and 75 and <expression 2> must be between 0 and 75, otherwise an error will result. If <expression 2> is omitted, or if the number of characters to the right of the character of <expression 1> is less than the value of <expression 2>, then all characters to the right of the character expressed by <expression 1> will be extracted. If <expression 1> is longer than the character string, MID\$ will be a null string.

Example:

```
B$=MID$(A$,2,4)
```

Related functions: LEFT\$, RIGHT\$

■ RIGHT\$

Format:

RIGHT\$ (<character string expression>, <expression>)

The value of <expression> must be 0 to 75.

Explanation:

This function extracts a character string with the digits specified by <expression> from the right end of the character string specified by <character string expression>. The value of <expression> must be between 0 and 75, otherwise an error will result. If the value of <expression> is 0, then RIGHT\$ will be a null string. If the value of <expression> is greater than the length of <character string expression>, then the RIGHT\$ string will be the same as the <character string expression>.

Example:

B\$=RIGHT\$(A\$,4)

Related functions: LEFT\$, MID\$

■ STR\$

Format:

STR\$ (<expression>)

Explanation:

This function changes the value of <expression> into a character string. The value of <expression> may be either an integer or real number.

Example:

B\$=STR\$(10.01)

Related functions: VAL

■ TIMES\$

Format:

TIMES\$

Explanation:

This function gives the time in the format “hh:mm:ss” (hours, minutes, seconds).

Examples:

A\$=TIMES\$
PRINT TIMES\$

Related functions: DATE\$, TIMER



NOTE

The clock can be set in
“SYSTEM>INIT>CLOCK” mode.

12.3 Point functions

Point functions are only used as elements for point expressions.

■ JTOXY

Format:

JTOXY (<point expression>)

Explanation:

This function converts the joint coordinate data (unit: pulses) specified by <point expression> into Cartesian coordinate data (unit: mm, deg.) for the main group.

Example:

P10=JTOXY(WHERE)

----- The current position data is converted into Cartesian coordinate data.

Related functions: XYTOJ

■ JTOXY2

Format:

JTOXY2 (<point expression>)

Explanation:

This function converts the joint coordinate data (unit: pulses) specified by <point expression> into Cartesian coordinate data (unit: mm, deg.) for the sub group.

Example:

P10=JTOXY2(WHERE2)

----- The current position data is converted into Cartesian coordinate data.

Related functions: XYTOJ2

■ PPNT

Format:

PPNT (pallet definition number, pallet position number)

Explanation:

This function creates point data specified by a pallet definition number and pallet position number.

Example:

P10=PPNT(1, 24)

----- Creates, at point P10, point data specified by pallet position number 24 of pallet definition number 1.

Related functions: PDEF, PMOVE, PMOVE2



NOTE

When the command is executed, the data is converted based on the standard coordinate, shift coordinate and hand definition that are set.



NOTE

- This function can be used only when the sub group has been set in the system generation.
- When the command is executed, the data is converted based on the standard coordinate, shift coordinate and hand definition that are set.

■ WHERE

Format:

WHERE

Explanation:

This function reads out the current position of the main group robot arm in joint coordinates (pulses).

Example:

P10=WHERE

----- The current position is read and assigned to point 10. Point 10 is defined in joint coordinates (pulses).

Related command: WHRXY

■ WHERE2

Format:

WHERE2

Explanation:

This function reads out the current position of the sub group robot arm in joint coordinates (pulses).

Example:

P10=WHERE2

----- The current position is read and assigned to point 10. Point 10 is defined in joint coordinates (pulses).

Related command: WHRXY2

■ WHRXY

Format:

WHRXY

Explanation:

This function reads out the current XY coordinate position (mm or degrees) of the main group arm.

Example:

P10=WHRXY

----- Reads the current position and registers it in P10. P10 is defined as a Cartesian coordinate (mm or degrees).

Related command: WHERE



NOTE

This function can be used only when the sub group has been set in the system generation.



NOTE

WHRXY is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards



NOTE

- *WHRXY2 is only available on the following software versions.*
 - RCX14x Ver. 8.64 onwards
 - RCX22x Ver. 9.11 onwards
- *This function can be used only when the sub group has been set in the system generation*

■ WHRXY2

Format:

WHRXY2

Explanation:

This function reads out the current XY coordinate position (mm or degrees) of the sub group arm.

Example:

P10=WHRXY2

----- Reads the current position and registers it in P10. P10 is defined as a Cartesian coordinate (mm or degrees).

Related command: WHERE2

■ XYTOJ

Format:

XYTOJ (<point expression>)

Explanation:

This function converts the Cartesian coordinate data (unit: mm, deg.) in the main group specified by <point expression> into joint coordinate data (unit: pulses).

Example:

P10=XYTOJ(P10)

----- Point data at P10 is converted into joint coordinate data.

Related functions: JTOXY

■ XYTOJ2

Format:

XYTOJ2 (<point expression>)

Explanation:

This function converts the Cartesian coordinate data (unit: mm, deg.) in the sub group specified by <point expression> into joint coordinate data (unit: pulses).

Example:

P10=XYTOJ2(P10)

—— Point data at P10 is converted into joint coordinate data.

Related functions: JTOXY2



NOTE

- *When the command is executed, the data is converted based on the standard coordinate, shift coordinate and hand definition that were set. On SCARA robots, the converted result differs depending on whether right-handed or left-handed is specified.*



NOTE

- *This function can be used only when the sub group has been set in the system generation.*
- *When the command is executed, the data is converted based on the standard coordinate, shift coordinate and hand definition that are set. On SCARA robots, the converted result differs depending on whether right-handed or left-handed is specified.*

13. Multi-tasking

Multi-tasking is used to create more sophisticated programs.

Before using the multi-tasking function, read this section thoroughly and make sure that you fully understand its contents.

13.1 Outline

Multi-tasking allows executing two or more tasks in parallel. However, this does not mean that multiple tasks are executed simultaneously because the controller has only one CPU to execute the tasks. In multi-tasking, the CPU time is shared among multiple tasks by assigning a priority to each task so that they can be executed efficiently.

A maximum of 8 tasks (task 1 to task 8) can be executed in one program. The priority level of task 1 is fixed at 32, while the priority of task 2 to task 8 can be set to any level between 17 and 47.

13.2 Task definition

A task is a set of instructions written in a program, which are executed in sequence. In a program, tasks are defined by placing a label as shown below.

```
'MAIN TASK(TASK1)
START *IOTASK,T2
*ST1:
  MOVE P,P1,P0
  IF DI(20)= 1 THEN
    HALT
  ENDIF
GOTO *ST
HALT
'SUB TASK(TASK2)
*IOTASK:
  IF DI(21)=1 THEN
    DO(30)=1
  ELSE
    DO(30)=0
  ENDIF
GOTO *IOTASK
EXIT TASK
```

Label *IOTASK: to EXIT TASK is a subtask.

13.3 Task status and transition

There are 6 types of task status.

(1) STOP status

A task is present but the task processing is stopped.

(2) RUN status

A task is present and the task processing is being executed by the CPU.

(3) READY status

A task is present and ready to be allocated to the CPU for task processing.

(4) WAIT status

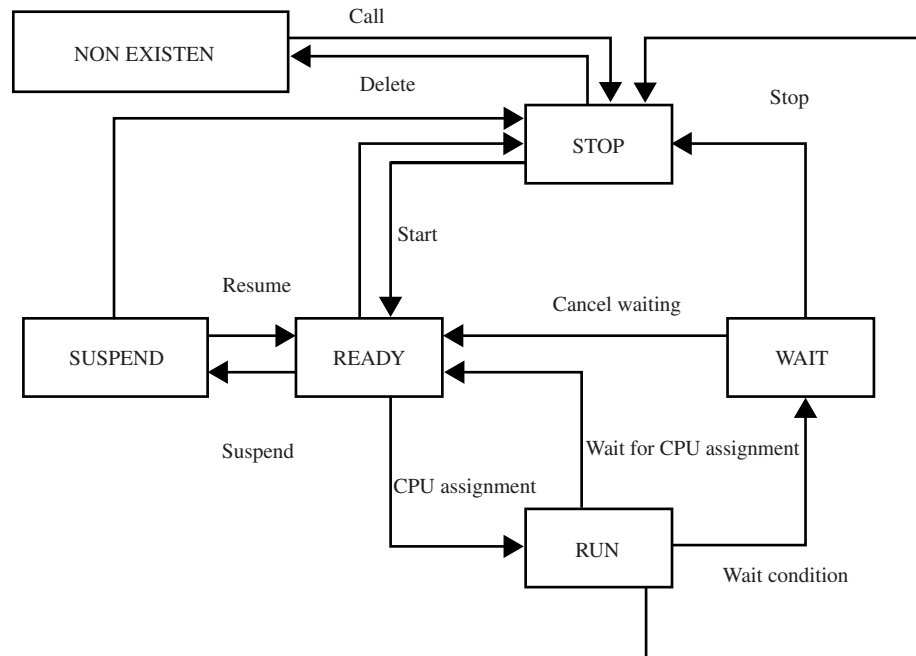
A task is present and waiting for an event to begin the task processing.

(5) SUSPEND status

A task is present but suspended while waiting to begin the task processing.

(6) NON EXISTEN status

No tasks exist in the program.



Task state transition

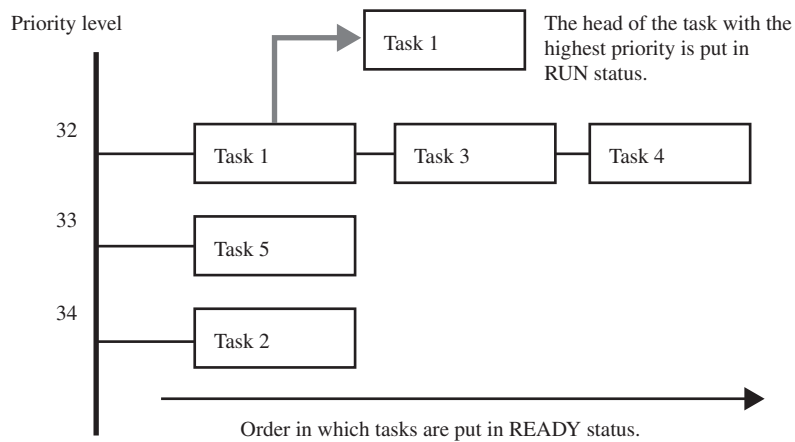
13.4 Starting tasks

Task 1 is called up by a program if present, and placed in RUN status when the program has started in AUTO mode. To call up the other tasks and place them in the READY status, use the START command.

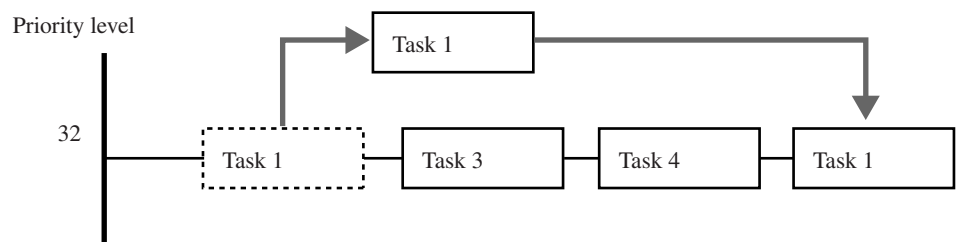
13.5 Task scheduling

Task scheduling determines the priority to be used in allocating tasks in the READY status to the CPU and executing them. One task is selected from among tasks in the READY status and placed in RUN status.

When there are two or more tasks which are put in the READY status, ready queues for CPU allocation are used to determine the priority for executing the tasks. The number of ready queues equal the number of task priority levels and are supervised by FCFS (First Come First Service). The smaller the number, the higher the task priority level.



The task in the RUN status moves to the end of the ready queue when the CPU occupation time exceeds a specified time or a task with a higher priority level is put in READY status.



13.6 Condition wait in task

A task is put in the WAIT state (waiting for an event) when a command causing a wait state is executed for that task. At this time, the transition to READY status does not take place until the wait condition is canceled.

1. When a command causing a wait state is executed, the following transition happens.

Task for which a command causing a wait state is executed - > WAIT

Task at the head of the ready queue with higher priority - > RUN

2. When an event waited by the task in the WAIT status occurs, the following status transition takes place by task scheduling.

Task in the WAIT status for which the awaited event occurred- > READY

However, if the task put in the READY status was at the head of the ready queue with the highest priority, the following transition takes place.

Task that is currently in RUN status - > READY

Task at the head of the ready queue with higher priority - > RUN



NOTE

If multiple tasks are in WAIT status awaiting the the same condition event, or different condition events occur simultaneously, all tasks for which the waited events occur are put in READY status.

Tasks are put in WAIT status by the following commands.

Event		Command
Wait for axis movement to complete	Axis movement command	ABSRST, DRIVE, DRIVE2, DRIVEI, DRIVEI2, MOVE, MOVE2, MOVEI, MOVEI2, ORIGIN, PATH START, PMOVE, PMOVE2, SERVO, SERVO2, WAIT ARM, WAIT ARM2
	Parameter command	ACCEL, ACCEL2, ARCH, ARCH2, AXWGHT, AXWGHT2, DECEL, DECEL2, ORGORD, ORGORD2, OUTPOS, OUTPOS2, TOLE, TOLE2, TORQUE, TORQUE2, WEIGHT, WEIGHT2
	Robot status change command	ASPEED, ASPEED2, CHANGE, CHANGE2, LEFTY, LEFTY2, RIGHTY, RIGHTY2, SHIFT, SHIFT2, SPEED, SPEED2
Wait for time to elapse		DELAY, SET (Time should be specified.), WAIT (Time should be specified.)
Wait for condition to be met		WAIT
Wait for data to send or to be received		SEND
Wait for print buffer to become empty		PRINT
Wait for key input		INPUT

The tasks are not put in WAIT status if the event has been established before the above commands are executed.

13.7 Suspending and restarting tasks

The SUSPEND command temporarily stops tasks other than task 1 and places them in SUSPEND status. The SUSPEND command cannot be used for task 1.

When the SUSPEND command is executed, the status transition takes place as follows.



NOTE

When the SUSPEND command is executed for a task that is in WAIT status, the command being executed with the specified task is terminated.

Task that executed the SUSPEND command	- >	RUN
Specified task	- >	SUSPEND

Tasks in the SUSPEND status can be restarted with the RESTART command. However, the RESTART command cannot be used for task 1.

When the RESTART command is executed, the status transition takes place as follows.

Task for which the RESTART command was executed	- >	RUN
Specified task	- >	READY

13.8 Deleting tasks

Tasks can delete themselves by using the EXIT TASK command and set to the NON EXISTEN (no task registration) status. The EXIT TASK command cannot be used for task 1.

When the EXIT TASK command is executed, the status transition takes place as follows.

Task that executed the EXIT TASK command	- >	NON EXISTEN
Task at the head of the ready queue with higher priority	- >	RUN

A task can also be deleted and put in the NON EXISTEN (no task registration) status by the other tasks using the CUT command. The CUT command cannot be used for task 1.

When the CUT command is executed, the status transition takes place as follows.

Task that executed the CUT command	- >	RUN
Specified task	- >	NON EXISTEN

13.9 Stopping tasks

All tasks stop if any of the following cases occurs.

- (1) HALT command is executed.

The program is reset and all tasks other than task 1 are put in the NON EXISTEN status. Task 1 is put in the STOP status.

- (2) HOLD command is executed.

All tasks are put in the STOP status. When the program is restarted, the tasks in the STOP status set to the READY or SUSPEND status.

- (3) STOP key on the programming box (MPB or RPB) is pressed or the interlock signal is cut off.

Just as in the case where the HOLD command is executed, all tasks are put in the STOP status. When the program is restarted, the tasks in the STOP status set to the READY or SUSPEND status.

- (4) When the emergency stop switch on the programming box (MPB or RPB) is pressed or the emergency stop signal is cut off.

All tasks are put in STOP status. At this point, the power to the robot is shut off and the servo sets to the non-hold state. After the canceling emergency stop, when the program is restarted, the tasks in STOP status are set to the READY or SUSPEND status. To turn back on the power to the robot, other processing is required.



NOTE

When the program is restarted without being reset after the tasks have been stopped by a cause other than (1), then each task is processed from the status in which the task stopped. This holds true when the power to the controller is turned off and then turned on.

13.10 Multi-task program example

Each task in a multi-task program is processed according to the task scheduling.

```
'TASK1
START *ST2,T2
START *ST3,T3
*ST1:
    DO(20) = 1
    WAIT MO(20) = 1
    MOVE P,P1,P2,Z=0
    IF MO(21)=1 THEN *FIN
GOTO *ST1
*FIN:
CUT T2
HALT
'TASK2
*ST2:
    IF DI(20) = 1
        MO(20) = 1
        DELAY 100
    ELSE
        MO(20) = 0
    ENDIF
GOTO *ST2
EXIT TASK
'TASK3
*ST3:
    IF DI(21) = 0 THEN *ST3
    IF DI(30) = 0 THEN *ST3
    IF DI(33) = 0 THEN *ST3
    MO(21) = 1
EXIT TASK
```



CAUTION

When the same variables and data are used in two or more tasks, use sufficient caution when rewriting the variable and data. Improper changes may cause trouble in the task processing.

13.11 Sharing the data

Point data, shift coordinate definition data, hand definition data, pallet definition data, all global variables and other variables are shared between all tasks.

Execution of each task can be controlled while using the same variables and data shared with the other tasks. In this case, however, use sufficient caution when rewriting the variable and data because improper changes may cause trouble in the task processing.

14. Data file description

This section explains data files used with a SEND statement and READ/WRITE online commands.

There are 27 different types of data files.



NOTE

- All data is handled as character strings conforming to ASCII character codes.
- No lower case alphabetic characters may be used in command statements.

1. Program file	15. DO file
2. Point file	16. MO file
3. Point comment file	17. LO file
4. Parameter file	18. TO file
5. Shift coordinate definition file	19. SI file
6. Hand definition file	20. SO file
7. Pallet definition file	21. Error message history file
8. All file	22. Machine reference file
9. Program directory file	23. EOF file
10. Parameter directory file	24. Serial port communication file
11. Variable file	25. SIW file
12. Constant file	26. SOW file
13. Array variable file	27. Ethernet port communication file
14. DI file	



NOTE

"27. Ethernet port communication file" is only available from software version 8.22 onwards.

14.1 Program file

14.1.1 All programs

Format:

PGM

- Expresses all programs.
- When used as a readout file, all programs currently stored are read out.
- When used as a write file, writing is performed with a program name.

Data format:

```
NAME=program name[cr/lf]
aaaa.....aaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
:
aaaa.....aaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
NAME=program name[cr/lf]
:
NAME=program name[cr/lf]
aaaa.....aaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
:
aaaa.....aaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
[cr/lf]
```

- Program names are shown with 8 characters or less consisting of alphanumeric characters and underscore (_).
- “a” indicates a character code.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.
- A TAB code is converted to a space.

Example:

SEND PGM TO CMU

----- Outputs all programs from communication port as shown below.

SEND CMU TO PGM

----- Inputs all programs from communication as shown below.

Response:

```
NAME=TEST1[cr/lf]
A=1[cr/lf]
RESET DO2()[cr/lf]
:
NAME=TEST2[cr/lf]
:
NAME=PGM1[cr/lf]
:
HALT[cr/lf]
[cr/lf]
```

14.1.2 One program

Format:

```
<program name>
```

- Expresses a specified program.
- Program name may be up to 8 characters consisting of alphanumeric characters and underscore (_) and must be enclosed by "<" and ">".
- If no program name is specified, the currently selected program is specified.
- An error occurs if the specified program name differs from the program name on the data.

Data format:

```
NAME=program name[cr/lf]
aaaaa.....aaaaaaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
:
aaaaa.....aaaaaaaaaaaaaaaaaaaaaaaaaaaaa[cr/lf]
[cr/lf]
```

- The program name is shown with 8 characters or less consisting of alphanumeric characters and underscore (_).
- "a" indicates a character code.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.
- A TAB code is converted to a space.

Example:

```
SEND <TEST1> TO CMU
```

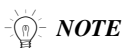
----- Outputs the program "TEST1" from communication port as shown below.

```
SEND CMU TO <TEST1>
```

----- Inputs the program "TEST1" from communication port as shown below.

Response:

```
NAME=TEST1[cr/lf]
A=1[cr/lf]
RESET DO2()[cr/lf]
:
HALT[cr/lf]
[cr/lf]
```



NOTE

- When the current mode is "AUTO" or "PROGRAM" mode, and writing into the currently selected program is not possible.
- When a sequence program is being executed, writing into the program name "SEQUENCE" is not possible.
- If there is no NAME statement used to specify a program, writing into the program is not possible.

14.2 Point file

14.2.1 All points

Format:

PNT

- Expresses all point data.
- When used as a readout file, all points currently stored are read out.
- When used as a write file, writing is performed with a point number.

Data format:

```
Pmmmm= fxxxxxx fyyyyy fzxxxx frrrrr faaaaaa fbbbbb t [cr/lf]
Pmmmm= fxxxxxx fyyyyy fzxxxx frrrrr faaaaaa fbbbbb t [cr/lf]
:
Pmmmm= fxxxxxx fyyyyy fzxxxx frrrrr faaaaaa fbbbbb t [cr/lf]
Pmmmm= fxxxxxx fyyyyy fzxxxx frrrrr faaaaaa fbbbbb t [cr/lf]
[cr/lf]
```



CAUTION

In controllers whose software version is earlier than 8.28, "mmmm" represents a number from 0 to 4000.



CAUTION

Hand system flags are only available from software version 8.08 onwards.

- "mmmm" represents a number from 0 to 9999.
- "f" represents a +, - or space.
- "xxxxxx", "yyyyyy" ... "bbbbbb" represent a numeric value of 8 digits or less. When a dot (decimal point) is included, this is treated as point data in "mm" units. Each piece of data is separated by one or more spaces.
- "t" is an extended hand system flag setting for SCARA robots. Hand system flags are enabled only when specifying the coordinate data in "mm" units for SCARA robots.
If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.
1 : Right-handed system is used to set point data.
2 : Left-handed system is used to set point data.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND PNT TO CMU

----- Outputs all points from communication port as shown below.

SEND CMU TO PNT

----- Inputs all points from communication port as shown below.

Response:

P0	=	1	2	3	4	5	6[cr/lf]
P1	=	1.00	2.00	3.00	4.00	5.00	6.00[cr/lf]
P2	=	1.00	0.00	0.00	0.00	0.00	0.00[cr/lf]
:							
P9999	=	-1.00	0.00	0.00	0.00	0.00	0.00[cr/lf]
[cr/lf]							

14.2.2 One point

Format:

```
Pmmmm
```



CAUTION

In controllers whose software version is earlier than 8.28, "mmmm" represents a number from 0 to 4000.

- Expresses a specified point.
- "mmmm" must be from 0 to 9999

Data format:

```
Pmmmm= fxxxxxx fyyyyyy fzzzzz frrrrr faaaaa fbbbbb t[cr/lf]
```

- "mmmm" is from 0 to 9999.
- "f" represents a +, - or space.
- "xxxxxx", "yyyyyy" ... "bbbbbb" represent a numeric value of 8 digits or less. When a dot (decimal point) is included, this is treated as point data in "mm" units. Each piece of data is separated by one or more spaces.
- "t" is an extended hand system flag setting for SCARA robots. Hand system flags are enabled only when specifying the coordinate data in "mm" units for SCARA robots.

If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to set point data.

2 : Left-handed system is used to set point data.

- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line may be up to 75 characters long.



CAUTION

Hand system flags are only available from software version 8.08 onwards.



NOTE

Integers indicate point data in "pulse" units, and real numbers (containing a decimal point) in "mm" units.

Example:

SEND P100 TO CMU

----- Outputs the specified point from communication port as shown below.

SEND CMU TO P100

----- Inputs the specified point from communication port as shown below.

Response:

```
P100      =      1      2      3      4      5      6[cr/lf]
```

14.3 Point comment file

14.3.1 All point comments

Format:

PCM

- Expresses all point comments.
- When used as a readout file, all point comments currently stored are read out.
- When used as a write file, writing is performed with a point comment number.

Data format:

```
PCmmmm= sssssssssss[cr/lf]
PCmmmm= sssssssssss[cr/lf]
      :
PCmmmm= sssssssssss[cr/lf]
PCmmmm= sssssssssss[cr/lf]
[cr/lf]
```



CAUTION

In controllers whose software version is earlier than 8.28, "mmmm" represents a number from 0 to 4000.

- "mmmm" represents a number from 0 to 9999.
- "ss...ss" represents command data, which can be up to 15 characters. If comment data exceeds 15 characters, then the 16th character onwards will be deleted.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND PCM TO CMU

----- Outputs all point comments from communication port as shown below.

SEND CMU TO PCM

----- Inputs all point comments from communication port as shown below.

Response:

```
PC1  =ORIGIN POS[cr/lf]
PC3  =WAIT POS[cr/lf]
      :
PC3999= WORK100[cr/lf]
[cr/lf]
```

14.3.2 Individual point comment


NOTE

Handling of individual point comment is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

Format:

```
PCmmmm
```

- Expresses a specified point comment.
- “mmmm” represents a number from 0 to 9999. (point comment number)

Data format:

```
PCmmmm=ssssssssss[cr/lf]
```

- “mmmm” represents a number from 0 to 9999. (point comment number)
- “ss...ss” represents command data, which can be up to 15 characters. If comment data exceeds 15 characters, then the 16th character onwards will be deleted.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line may be up to 75 characters long.

Example:

SEND PC1 TO CMU

----- Outputs the specified point comment from communication port as shown below.

SEND CMU TO PC1

----- Inputs the specified point comment from communication port as shown below.

Response:

PC1 =ORIGIN POS[cr/lf]

14.4 Parameter file

14.4.1 All parameters

Format:

PRM

- Expresses all parameters (including settings in “UTILITY” mode).
- When used as a readout file, all parameters currently stored are read out.
- When used as a write file, only the parameters specified by parameter labels are written.

Data format:

```
/parameter label/ ' <comment> [cr/lf]
RC= xxxxxx [cr/lf]
/parameter label/ ' <comment> [cr/lf]
R1= xxxxxx R2= yyyyyy [cr/lf]
/parameter label/ ' <comment> [cr/lf]
A1= xxxxxx A2= yyyyyy A3= zzzzzz A4= rrrrrr[cr/lf]
/parameter label/ ' <comment> [cr/lf]
:
[cr/lf]
```

- Parameter labels are shown with 6 alphabetic characters.
- The description of each parameter is shown in <comment>.
- “RC=” indicates the entire controller.
- “R1=” indicates the main robot, and “R2=” the sub robot.
- “A1=” to “An=” indicate axis numbers from axis 1 to axis n.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.



NOTE

- Parameters can be written only when the axis servo is off.
- Parameters are already compatible with upper versions. However, parameters might not always be compatible with lower versions.
- When you attempt to load a parameter file of new version into a controller of an earlier version, an error “10.14 : Undefined parameter found” may appear. If this happens, you may load the parameter by setting the “Skip undefined parameters” parameter to “VALID”. For more details, refer to the “SYSTEM mode” – “Other parameters” section in the robot controller user's manual.

Example:

SEND PRM TO CMU

----- Outputs all parameters from communication port as shown below.

SEND CMU TO PRM

----- Inputs all parameters from communication port as shown below.

Response:

```

/RBTNUM/ 'Robot number(V8.01/R1001)[cr/lf]
R1= 3000 R2= 3010[cr/lf]
/AXES / 'Number of axes[cr/lf]
R1= 2 R2= 2[cr/lf]
/AXSNUM/ 'Axis number(V1.01/V1.01/V1.01/V1.01/-----/-----/-----/-----)[cr/lf]
A1= 5000 A2= 5001 A3= 5010 A4= 5011[cr/lf]
A5= 0 A6= 0 A7= 0 A8= 0[cr/lf]
/ATTRIB/ 'Axis attribute[cr/lf]
A1= 33792 A2= 33792 A3= 33792 A4= 33792[cr/lf]
A5= 256 A6= 256 A7= 256 A8= 256[cr/lf]
/WEIGHT/ 'Tip weight[kg][cr/lf]
R1= 2 R2= 12[cr/lf]
:
/CURPNO/ 'Port number of output[cr/lf]
RC= 20[cr/lf]
/CURPT1/ 'Compare point number1[cr/lf]
RC= 0[cr/lf]
/CURPT2/ 'Compare point number2[cr/lf]
RC= 0[cr/lf]
[cr/lf]

```

14.4.2 One parameter

Format:

```
/parameter label/
```

- Expresses a parameter label with 6 alphabetic characters.
- When used as a readout file, only the parameter specified by a parameter label is read out.
- When used as a write file, only the parameter specified by a parameter label is written.

Data format:

```
/parameter label/ ' <comment> [cr/lf]
RC= xxxxxx [cr/lf]
[cr/lf]
or
/parameter label/ ' <comment> [cr/lf]
R?= xxxxxx [cr/lf]
[cr/lf]
or
/parameter label/ ' <comment> [cr/lf]
A?= xxxxxx[cr/lf]
[cr/lf]
```

- The parameter label is shown with 6 alphabetic characters.
- The description of each parameter is shown in <comment>.
- “RC=” indicates the entire controller.
- “R1=” indicates the main robot, and “R2=” the sub robot.
- “A1=” to “An=” indicate axis numbers from axis 1 to axis n.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

NOTE

- Parameters can be written only when the axis servo is off.
- Parameters are already compatible with upper versions. However, parameters might not always be compatible with lower versions.
- When you attempt to load a parameter file of new version into a controller of an earlier version, an error "10.14 : Undefined parameter found" may appear. If this happens, you may load the parameter by setting the "Skip undefined parameters" parameter to "VALID". For more details, refer to the "SYSTEM mode" – "Other parameters" section in the robot controller user's manual.

Example:

```
SEND /ACCEL / TO CMU
```

----- Outputs the acceleration parameter from communication port as shown below.

```
SEND CMU TO /ACCEL /
```

----- Inputs the acceleration parameter from communication port as shown below.

Response:

```
/ACCEL / 'Accel coefficient[%]
A1= 100 A2= 100 A3= 100 A4= 100[cr/lf]
[cr/lf]
```

14.5 Shift coordinate definition file

14.5.1 All shift data

Format:

SFT

- Expresses all shift data.
- When used as a readout file, all shift data currently stored are read out.
- When used as a write file, writing is performed with a shift number.

Data format:

```
Sm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]
{SPm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]}
{SMm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]}
:
Sm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]
{SPm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]}
{SMm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]}
[cr/lf]
```

- “m” represents a number from 0 to 9.
- “f” represents a +, - or space.
- “xxxxxx”, “yyyyyy” ... “rrrrrr” represent a numeric value of 8 digits or less, having 2 or less places below the decimal point.
- Data enclosed by { } is shown depending on the data input.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND SFT TO CMU

----- Outputs all shift data from communication port as shown below.

SEND CMU TO SFT

----- Inputs all shift data from communication port as shown below.

Response:

```
S0 = 0.00 0.00 0.00 0.00[cr/lf]
SP0 = 0.00 0.00 0.00 0.00[cr/lf]
SM0 = 0.00 0.00 0.00 0.00[cr/lf]
S1 = 1.00 1.00 1.00 1.00[cr/lf]
:
SM9 = 9.00 9.00 9.00 9.00[cr/lf]
[cr/lf]
```

14.5.2 One shift definition

Format:

Sm

- Expresses a specified shift definition.
- “m” must be from 0 to 9.

Data format:

Sm = fxxxxxx fyyyyyy fzzzzzz frrrrrr [cr/lf]

- “m” is from 0 to 9.
- “f” represents a +, - or space.
- “xxxxxx”, “yyyyyy” ... “rrrrrr” represent a numeric value of 8 digits or less, having 2 or less places below the decimal point.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line may be up to 75 characters long.

Example:

SEND S0 TO CMU

----- Outputs the specified shift coordinate definition data from communication port as shown below.

SEND CMU TO S0

----- Inputs the specified shift coordinate definition data from communication port as shown below.

Response:

S0 =	0.00	0.00	0.00	0.00[cr/lf]
SP0 =	0.00	0.00	0.00	0.00[cr/lf]
SM0 =	0.00	0.00	0.00	0.00[cr/lf]
[cr/lf]				

14.6 Hand definition file

14.6.1 All hand data

Format:

HND

- Expresses all hand data.
- When used as a readout file, all hand data currently stored are read out.
- When used as a write file, writing is performed with a hand number.

Data format:

```
Hm = fxxxxxx fyzyyyy fzzyzzz {R} [cr/lf]
      :
Hm = fxxxxxx fyzyyyy fzzyzzz {R} [cr/lf]
[cr/lf]
```

- “m” represents a number from 0 to 7. 0 to 3 are used for the main robot, and 4 to 7 for the sub robot.
- “f” represents a +, - or space.
- “xxxxxx”, “yyzyyy”, “zzyzzz” represent a numeric value of 8 digits or less, having 2 or less places below the decimal point, or an integer of 7 digits or less. (This numeric format depends on the robot type setting and hand definition type.)
- {R} indicates whether a hand is attached to the R-axis.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND HND TO CMU

----- Outputs all hand data from communication port as shown below.

SEND CMU TO HND

----- Inputs all hand data from communication port as shown below.

Response:

```
H0 = 0.00 0.00 0.00[cr/lf]
H1 = 1.00 1.00 1.00[cr/lf]
H2 = 2.00 2.00 2.00[cr/lf]
H3 = 3.00 3.00 3.00[cr/lf]
H4 = 4.00 4.00 4.00[cr/lf]
H5 = 5.00 5.00 5.00[cr/lf]
H6 = 6.00 6.00 6.00[cr/lf]
H7 = 7.00 7.00 7.00[cr/lf]
[cr/lf]
```

14.6.2 One hand definition

Format:

Hm

- Expresses a specified hand definition.
- “m” must be from 0 to 7.

Data format:

Hm = fxxxxxx fyzyyyy fzzzzzz {R} [cr/lf]

- “m” is from 0 to 7. 0 to 3 are used for the main robot, and 4 to 7 for the sub robot.
- “f” represents a +, - or space.
- “xxxxxx”, “yyzyyy”, “zzzzzz” represent a numeric value of 8 digits or less, having 2 or less places below the decimal point, or an integer of 7 digits or less. (This numeric format depends on the robot type setting and hand definition type.)
- {R} indicates whether a hand is attached to the R-axis.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line may be up to 75 characters long.

Example:

SEND H3 TO CMU

----- Outputs the specified hand definition data from communication port as shown below.

SEND CMU TO H3

----- Inputs the specified hand definition data from communication port as shown below.

Response:

H3 = 3.00 3.00 3.00[cr/lf]

14.7 Pallet definition file

14.7.1 All pallet definitions

Format:

PLT

- Expresses all pallet definitions.
- When used as a readout file, all pallet definitions currently stored are read out.
- When used as a write file, writing is performed with a pallet number.

Data format:

```
PLm [cr/lf]
PLN = XY [cr/lf]
NX = nnn [cr/lf]
NY = nnn [cr/lf]
NZ = nnn [cr/lf]
P[1] = fxxxxxx fyxxxxx fzxxxxx frxxxxx faaaaaa fbbbbb t [cr/lf]
      :
P[5] = fxxxxxx fyxxxxx fzxxxxx frxxxxx faaaaaa fbbbbb t [cr/lf]
PLm [cr/lf]
      :
[cr/lf]
```

- “m” represents a number from 0 to 19.
- “nnn” represents a positive integer.
- “f” represents a +, - or space.
- “xxxxxx”, “yyxxxx” ... “bbbbb” represent a numeric value of 8 digits or less. When a dot (decimal point) is included, this is treated as point data in “mm” units. Each piece of data is separated by one or more spaces.
- “t” is an extended hand system flag setting for SCARA robots. Hand system flags are enabled only when specifying the coordinate data in “mm” units for SCARA robots. However, hand system flags are ignored during operation using pallet definitions.

If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to set point data.

2 : Left-handed system is used to set point data.

- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

CAUTION

- Hand system flags are only available from software version 8.08 onwards.
- Hand system flags are ignored during operation using pallet definitions.

14. Data file description

Example:

SEND PLT TO CMU

----- Outputs all pallet definitions from communication port as shown below.

SEND CMU TO PLT

----- Inputs all pallet definitions from communication port as shown below.

Response:

PL0[cr/lf]

PLN=XY[cr/lf]

NX= 3[cr/lf]

NY= 4[cr/lf]

NZ= 2[cr/lf]

P[1]=	0.00	0.00	0.00	0.00	0.00	0.00[cr/lf]
-------	------	------	------	------	------	-------------

P[2]=	100.00	0.00	0.00	0.00	0.00	0.00[cr/lf]
-------	--------	------	------	------	------	-------------

P[3]=	0.00	100.00	0.00	0.00	0.00	0.00[cr/lf]
-------	------	--------	------	------	------	-------------

P[4]=	100.00	100.00	0.00	0.00	0.00	0.00[cr/lf]
-------	--------	--------	------	------	------	-------------

P[5]=	0.00	0.00	50.00	0.00	0.00	0.00[cr/lf]
-------	------	------	-------	------	------	-------------

PL1[cr/lf]

PLN=XY[cr/lf]

NX= 3[cr/lf]

NY= 4[cr/lf]

NZ= 2[cr/lf]

P[1]=	0.00	0.00	0.00	0.00	0.00	0.00[cr/lf]
-------	------	------	------	------	------	-------------

P[2]=	100.00	100.00	0.00	0.00	0.00	0.00[cr/lf]
-------	--------	--------	------	------	------	-------------

P[3]=	0.00	200.00	0.00	0.00	0.00	0.00[cr/lf]
-------	------	--------	------	------	------	-------------

P[4]=	100.00	200.00	0.00	0.00	0.00	0.00[cr/lf]
-------	--------	--------	------	------	------	-------------

P[5]=	0.00	0.00	100.00	0.00	0.00	0.00[cr/lf]
-------	------	------	--------	------	------	-------------

[cr/lf]

14.7.2 One pallet definition

Format:

PLm

- Expresses a specified pallet definition.
- “m” must be from 0 to 19.

Data format:

```
PLm [cr/lf]
PLN = XY [cr/lf]
NX = nnn [cr/lf]
NY = nnn [cr/lf]
NZ = nnn [cr/lf]
P[1] = fxxxxxx fyxxxxx fzxxxxx frxxxxx faaaaaa fbbbbb t [cr/lf]
      :
P[5] = fxxxxxx fyxxxxx fzxxxxx frxxxxx faaaaaa fbbbbb t [cr/lf]
[cr/lf]
```

- “m” is from 0 to 19.
- “nnn” represents a positive integer.
- “f” represents a +, - or space.
- “xxxxxx”, “yyxxxx” ... “bbbbb” represent a numeric value of 8 digits or less. When a dot (decimal point) is included, this is treated as point data in “mm” units. Each piece of data is separated by one or more spaces.
- “t” is an extended hand system flag setting for SCARA robots. Hand system flags are enabled only when specifying the coordinate data in “mm” units for SCARA robots. However, hand system flags are ignored during operation using pallet definitions.

If a number other than 1 or 2 is set, or if no number is designated, then 0 will be set to indicate that there is no hand system flag.

1 : Right-handed system is used to set point data.

2 : Left-handed system is used to set point data.

- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND PL2 TO CMU

----- Outputs the specified pallet definition from communication port as shown below.

SEND CMU TO PL2

----- Inputs the specified pallet definition from communication port as shown below.

Response:

```
PL2[cr/lf]
PLN=XY[cr/lf]
NX= 3[cr/lf]
NY= 3[cr/lf]
NZ= 2[cr/lf]
P[1]= 100.00      100.00      50.00      90.00      0.00      0.00[cr/lf]
P[2]= 200.00      100.00      50.00      90.00      0.00      0.00[cr/lf]
P[3]= 100.00      200.00      50.00      90.00      0.00      0.00[cr/lf]
P[4]= 200.00      200.00      50.00      90.00      0.00      0.00[cr/lf]
P[5]= 100.00      10.00      100.00      90.00      0.00      0.00[cr/lf]
[cr/lf]
```



CAUTION

- Hand system flags are only available from software version 8.08 onwards.
- Hand system flags are ignored during operation using pallet definitions.



NOTE

Integers indicate point data in “pulse” units, and real numbers (containing a decimal point) in “mm” units.

14.8 All file

14.8.1 All files

Format:

ALL

- Expresses all files essential for the entire system.
- When used as a readout file, all files of the entire system are read out.
- When used as a write file, a specified file is written.

Data format:

[PGM]
All program format
[PNT]
All point format
[PCM]
All point comment format
[PRM]
All parameter format
[SFT]
All shift data format
[HND]
All hand data format
[PLT]
All pallet data format
[END]



NOTE

- For more details on each file, see the description of each file.
- The [xxx] identifiers are used to determine the file format from the next line.
- [END] indicates the end of all files.

Example:

```
SEND ALL TO CMU
----- Outputs all files of the entire system from communication
port.
SEND CMU TO ALL
----- Inputs all files of the entire system from communication port.
```

14.9 Program directory file

14.9.1 Entire program directory

Format:

DIR

- Expresses entire program directory.
- When used as a readout file, information on entire program directory is read out.
- Cannot be used as a write file.

Data format:

```
No. Name Line Byte RW/RO Date Time[cr/lf]
nnnfgsssssss llll bbbbbb xx yy/mm/dd hh:mm[cr/lf]
:
nnnfgsssssss llll bbbbbb xx yy/mm/dd hh:mm[cr/lf]
END[cr/lf]
```

- “nnn” (3 digits) represents a program directory number.
- “f” (1 digit) shows “o” when a program object is present and “s” when a sequence object is present.
- “g” (1 digit) shows an asterisk (*) for the currently selected program.
- “sssssss” (8 digits) represents a program name.
- “llll” (4 digits) represents the number of program lines.
- “bbbbbb” (6 digits) represents the byte size of the program.
- “xx” (2 digits) shows “RW” when the program is readable/writable and “RO” when not rewritable.
- “yy/mm/dd” (8 digits) is the date when the program was updated.
- “hh:mm” (5 digits) is the time when the program was updated.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).

Example:

SEND DIR TO CMU

----- Outputs information on all program directory from communication port as shown below.

Response:

```
No. Name Line Byte RW/RO Date Time[cr/lf]
1o* 12345678 5 21 RW 01/06/20 10:35[cr/lf]
2 PGM1 5 66 RW 01/06/20 10:35[cr/lf]
3 PGM2 5 66 RW 01/06/20 10:35[cr/lf]
4 PGM3 5 66 RW 01/06/20 10:35[cr/lf]
5 PGM4 5 66 RW 01/06/20 10:35[cr/lf]
6 PGM5 5 66 RW 01/06/20 10:35[cr/lf]
7 PGM6 5 66 RW 01/06/20 10:35[cr/lf]
8s SEQUENCE 1 15 RW 01/06/20 10:35[cr/lf]
END[cr/lf]
```

14.9.2 One program

Format:

<<program name>>

- Expresses information on one program.

Data format:

No.	Name	Line	Byte	RW/RO	Date	Time[cr/lf]
nnnfgsssssss	llll	bbbbbb	xx	yy/mm/dd	hh:mm[cr/lf]	

- “nnn” (3 digits) represents a program directory number.
- “f” (1 digit) shows “o” when a program object is present and “s” when a sequence object is present.
- “g” (1 digit) shows an asterisk (*) for the currently selected program.
- “sssssss” (8 digits) represents a program name.
- “llll” (4 digits) represents the number of program lines.
- “bbbbbb” (6 digits) represents the byte size of the program.
- “xx” (2 digits) shows “RW” when the program is readable/writable and “RO” when not rewritable.
- “yy/mm/dd” (8 digits) is the date when the program was updated.
- “hh:mm” (5 digits) is the time when the program was updated.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).

Example:

SEND <<TEST>> TO CMU

----- Outputs information on the specified program from communication port as shown below.

Response:

No.	Name	Line	Byte	RW/RO	Date	Time[cr/lf]
30*	PGM2	5	66	RW	01/06/20	10:35[cr/lf]

14.10 Parameter directory file

14.10.1 Entire parameter directory

Format:

DPM

- Expresses entire parameter directory.
- When used as a readout file, information on entire parameter directory is read out.
- Cannot be used as a write file.

Data format:

```
/parameter label/ '<comment>[cr/lf]
/parameter label/ '<comment>[cr/lf]
:
/parameter label/ '<comment>[cr/lf]
[cr/lf]
```

- Parameter labels are shown with 6 alphabetic characters.
- The description of each parameter is shown in <comment>.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND DPM TO CMU

----- Outputs information on all parameter directory from communication port as shown below.

Response:

```
/RBTNUM/ 'Robot number(V8.01/R1001)[cr/lf]
/AXES / 'Number of axes[cr/lf]
/AXSNUM/ 'Axis number (V1.01/V1.01/V1.01/V1.01/-----/-----/-----/-----)[cr/lf]
/ATTRIB/ 'Axis attribute[cr/lf]
/WEIGHT/ 'Tip weight[kg][cr/lf]
/ORIGIN/ 'Origin sequence[cr/lf]
/RORIEN/ 'R axis orientation[cr/lf]
:
/CURPNO/ 'Port number of output[cr/lf]
/CURPT1/ 'Compare point number1[cr/lf]
/CURPT2/ 'Compare point number2[cr/lf]
[cr/lf]
```

14.11 Variable file

14.11.1 All variables

Format:

```
VAR
```

- Expresses all global variables.
- When used as a readout file, all global variables currently stored are read out.
- When used as a write file, a specified global variable is written.

Data format:

```
<variable name>t = xxxxxx [cr/lf]
<variable name>t = xxxxxx [cr/lf]
      :
<variable name>t = xxxxxx [cr/lf]
[cr/lf]
```

- <Variable name> is a global variable defined in the program.
- <Variable name> is shown with 16 characters or less consisting of alphanumeric characters and underscore (_).
- “t” indicates a type of variable. (!: real type, %: integer type, \$: character type)
- “xxxxxx” differs depending on the type of variable.
 - Integer type: integer of 7 digits or less
 - Real type: real number of 7 digits or less including decimal fractions
 - Character type: character string of 70 characters or less
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND VAR TO CMU

----- Outputs all global variables from communication port as shown below.

Response:

```
SGI0=0[cr/lf]
SGI1=1111[cr/lf]
SGI2=2222[cr/lf]
SGI3=3333[cr/lf]
SGI4=4444[cr/lf]
SGI5=5555[cr/lf]
SGI6=6666[cr/lf]
SGI7=7777[cr/lf]
SGR0=0[cr/lf]
SGR1=1.1111E3[cr/lf]
SGR2=2.2222E3[cr/lf]
SGR3=3.3333E3[cr/lf]
SGR4=4.4444E3[cr/lf]
SGR5=5.5555E3[cr/lf]
SGR6=6.6666E3[cr/lf]
SGR7=7.7777E3[cr/lf]
B1%=111[cr/lf]
B2%=222[cr/lf]
C1$="CNS_1"[cr/lf]
C2$="CNS_2"[cr/lf]
[cr/lf]
```

14.11.2 One variable

Format:

```
<variable name>t
```

- Expressed one variable.

Data format:

```
xxxxxx [cr/lf]
```

- <Variable name> is a global variable defined in the program.
- <Variable name> is shown with 16 characters or less consisting of alphanumeric characters and underscore (_).
- “t” indicates a type of variable. (!: real type, %: integer type, \$: character type)
- “xxxxxx” differs depending on the type of variable.
 - Integer type: integer of 8 digits or less
 - Real type: real number of 7 digits or less including decimal fractions
 - Character type: character string of 70 characters or less
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line may be up to 75 characters long.



NOTE

- SGIx indicates an integer type static variable.
- SGRx indicates a real type static variable.
- Dynamic global variables are registered during compiling. Variables cannot be referred to unless they are registered.

Example:

```
SEND SGI6 TO CMU[cr/lf]
```

----- Outputs the specified variable SGI6 from communication port as shown below.

Response:

```
6666[cr/lf]
```


14.12 Constant file

14.12.1 One character string

Format:

```
"<character string>"
```

- Expresses a specified character string.
- When used as a readout file, the specified character string is read out.
- Cannot be used as a write file.

Data format:

```
sssss...sssss[cr/lf]
```

- "sssss...sssss" shown with 61 characters or less.
- Output of a double quotation (") is shown with two successive double quotations.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example:

```
SEND""""YAMAHA ROBOT"""" TO CMU
```

----- Outputs the specified character string from communication port as shown below.

Response:

```
""YAMAHA ROBOT""[cr/lf]
```

14.13 Array variable file

14.13.1 All array variables

Format:

ARY

- Expresses all array variables.
- When used as a readout file, all array variables are read out.
- When used as a write file, writing is performed with a specified array variable.

Data format:

```
<variable name>t(l{,m{n}}) = xxxxxx [cr/lf]
<variable name>t(l{,m{n}}) = xxxxxx [cr/lf]
:
<variable name>t(l{,m{n}}) = xxxxxx [cr/lf]
[cr/lf]
```

- <Variable name> is a global array variable defined by the DIM statement in the program.
- <Variable name> is shown with 16 characters or less consisting of alphanumeric characters and underscore (_).
- “t” indicates a type of variable. (!: real type, %: integer type, \$: character type)
- “l”, “m” and “n” indicate array arguments.
- “xxxxxx” differs depending on the type of variable.
 - Integer type: integer of 8 digits or less
 - Real type: real number of 7 digits or less including decimal fractions
 - Character type: character string of 70 characters or less
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.
- A line may be up to 75 characters long.

Example:

SEND ARY TO CMU

----- Outputs all global array variables from communication port
as shown below.

Response:

```
A!(0)=0[cr/lf]
A!(1)=1.E2[cr/lf]
A!(2)=2.E2[cr/lf]
B%(0,0)=0[cr/lf]
B%(0,1)=1 1 1 1[cr/lf]
B%(1,0)=2222[cr/lf]
B%(1,1)=3333[cr/lf]
C$(0,0,0)="ARY1"[cr/lf]
C$(0,0,1)="ARY2"[cr/lf]
C$(0,1,0)="ARY3"[cr/lf]
C$(0,1,1)="ARY4"[cr/lf]
C$(1,0,0)="ARY5"[cr/lf]
C$(1,0,1)="ARY6"[cr/lf]
C$(1,1,0)="ARY7"[cr/lf]
C$(1,1,1)="ARY8"[cr/lf]
[cr/lf]
```

14.13.2 One array variable

Format:

```
<variable name>t(l{,m{n}})
```

- Expresses one array variable.

Data format:

```
xxxxxx [cr/lf]
```

- <Variable name> is a global array variable defined by the DIM statement in the program.
- <Variable name> is shown with 16 characters or less consisting of alphanumeric characters and underscore (_).
- “t” indicates a type of variable. (!: real type, %: integer type, \$: character type)
- “l”, “m” and “n” indicate array arguments.
- “xxxxxx” differs depending on the type of variable.
 - Integer type: integer of 8 digits or less
 - Real type: real number of 7 digits or less including decimal fractions
 - Character type: character string of 70 characters or less
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example:

```
SEND CMU TO C1$(2)[cr/lf]
```

----- Inputs the specified array variable C1\$(2) from communication port as shown below.

Response:

```
YAMAHA ROBOT[cr/lf]
```



NOTE

Array variables defined by the DIM statement are registered during compiling. Array variables cannot be referred to unless they are registered.

14.14 DI file

14.14.1 All DI information

Format:

DI()

- Expresses all DI information.
- When used as a readout file, all DI information is read out.
- Cannot be used as a write file.

Data format:

```
DI0()=&Bnnnnnnnn [cr/lf]
DI1()=&Bnnnnnnnn [cr/lf]
      :
DI27()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right. (“m” is a port number.)
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

SEND DI() TO CMU

----- Outputs all DI information from communication port as shown below.

Response:

```
DI0()=&B10001001[cr/lf]
DI1()=&B00000010[cr/lf]
DI2()=&B00000000[cr/lf]
      :
DI7()=&B00000000[cr/lf]
DI10()=&B00000000[cr/lf]
DI11()=&B00000000[cr/lf]
DI12()=&B00000000[cr/lf]
      :
DI17()=&B00000000[cr/lf]
DI20()=&B00000000[cr/lf]
      :
DI26()=&B00000000[cr/lf]
DI27()=&B00000000[cr/lf]
[cr/lf]
```

14.14.2 One DI port

Format:

DI m ()

- Expresses the status of one DI port.
- When used as a readout file, the specified DI port status is read out.
- Cannot be used as a write file.

Data format:

DI m ()=&Bnnnnnnnn[cr/lf]

- “m” is from 0 to 7, 10 to 17 or 20 to 27.
- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example:

SEND DI5() TO CMU

----- Outputs the DI5 port status from communication port as shown below.

Response:

DI5()=&B00000000[cr/lf]

14.15 DO file

14.15.1 All DO information

Format:

```
DO()
```

- Expresses all DO information.
- When used as a readout file, all DO information is read out.
- Cannot be used as a write file.

Data format:

```
DO0()=&Bnnnnnnnn [cr/lf]
DO1()=&Bnnnnnnnn [cr/lf]
      :
DO27()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right. (“m” is a port number.)
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND DO() TO CMU
```

----- Outputs all DO information from communication port as shown below.

Response:

```
DO0()=&B10001001[cr/lf]
DO1()=&B00000010[cr/lf]
DO2()=&B00000000[cr/lf]
      :
DO7()=&B00000000[cr/lf]
DO10()=&B00000000[cr/lf]
DO11()=&B00000000[cr/lf]
DO12()=&B00000000[cr/lf]
      :
DO17()=&B00000000[cr/lf]
DO20()=&B00000000[cr/lf]
      :
DO26()=&B00000000[cr/lf]
DO27()=&B00000000[cr/lf]
[cr/lf]
```

14.15.2 One DO port

Format:

DOm()

- Expresses the status of one DO port.
- When used as a readout file, the specified DO port status is read out.
- When used as a write file, the value is written to the specified DO port. However, writing to DO0() and DO1() is prohibited.

Data format:

Readout file

DOm()=&Bnnnnnnnn[cr/lf]

Write file

&Bnnnnnnnn[cr/lf] or k[cr/lf]

- “m” is from 0 to 7, 10 to 17 or 20 to 27.
- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right.
- “k” is an integer from 0 to 255.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).



NOTE

Writing to DO0() and DO1() is prohibited.

Example 1:

SEND DO5() TO CMU

----- Outputs the DO5 port status from communication port as shown below.

Response:

DO5()=&B00000000[cr/lf]

Example 2:

SEND CMU TO DO5()

----- Inputs the DO5 port status from communication port as shown below.

&B00000111

Response:

OK[cr/lf]

14.16 MO file

14.16.1 All MO information

Format:

```
MO()
```

- Expresses all MO information.
- When used as a readout file, all MO information is read out.
- Cannot be used as a write file.

Data format:

```
MO0()=&Bnnnnnnnn [cr/lf]
MO1()=&Bnnnnnnnn [cr/lf]
      :
MO27()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right. (“m” is a port number.)
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND MO() TO CMU
```

----- Outputs all MO information from communication port as shown below.

Response:

```
MO0()=&B10001001[cr/lf]
MO1()=&B00000010[cr/lf]
MO2()=&B00000000[cr/lf]
      :
MO7()=&B00000000[cr/lf]
MO10()=&B00000000[cr/lf]
MO11()=&B00000000[cr/lf]
MO12()=&B00000000[cr/lf]
      :
MO17()=&B00000000[cr/lf]
MO20()=&B00000000[cr/lf]
      :
MO26()=&B00000000[cr/lf]
MO27()=&B00000000[cr/lf]
[cr/lf]
```

14.16.2 One MO port

Format:

MOm()

- Expresses the status of one MO port.
- When used as a readout file, the specified MO port status is read out.
- When used as a write file, the value is written to the specified MO port. However, writing to MO0() and MO1() is prohibited.

Data format:

Readout file

MOm()=&Bnnnnnnnn[cr/lf]

Write file

&Bnnnnnnnn[cr/lf] or k[cr/lf]

- “m” is from 0 to 7, 10 to 17 or 20 to 27.
- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right.
- “k” is an integer from 0 to 255.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).



NOTE

Writing to MO0() and MO1() is prohibited.

Example 1:

SEND MO5() TO CMU

----- Outputs the MO5 port status from communication port as shown below.

Response:

MO5()=&B00000000[cr/lf]

Example 2:

SEND CMU TO MO5()

----- Inputs the MO5 port status from communication port as shown below.

&B00000111

Response:

OK[cr/lf]

14.17 LO file

14.17.1 All LO information

Format:

LO()

- Expresses all LO information.
- When used as a readout file, all LO information is read out.
- Cannot be used as a write file.

Data format:

LO0()=&Bnnnnnnnn [cr/lf]
[cr/lf]

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to 00, 01 ... 07 starting from the right.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

SEND LO() TO CMU

----- Outputs all LO information from communication port as shown below.

Response:

LO0()=&B10001001[cr/lf]
[cr/lf]

14.17.2 One LO port

Format:

```
LO0()
```

- Expresses the status of one LO port.
- When used as a readout file, the specified LO port status is read out.
- When used as a write file, the value is written to the specified LO port.

Data format:

Readout file

```
LO0()=&Bnnnnnnnn[cr/lf]
```

Write file

```
&Bnnnnnnnn[cr/lf] or k[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to 00, 01 ... 07 starting from the right.
- “k” is an integer from 0 to 255.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example 1:

```
SEND LO0() TO CMU
```

----- Outputs the LO0 port status from communication port as shown below.

Response:

```
LO0()=&B00000000[cr/lf]
```

Example 2:

```
SEND CMU TO LO0()
```

----- Inputs the LO0 port status from communication port as shown below.

```
&B00000111
```

Response:

```
OK[cr/lf]
```

14.18 TO file

14.18.1 All TO information

Format:

```
TO()
```

- Expresses all TO information.
- When used as a readout file, all TO information is read out.
- Cannot be used as a write file.

Data format:

```
TO0()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to 00, 01 ... 07 starting from the right.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND TO() TO CMU
```

----- Outputs all TO information from communication port as shown below.

Response:

```
TO0()=&B10001001[cr/lf]
[cr/lf]
```

14.18.2 One TO port

Format:

TO0()

- Expresses the status of one TO port.
- When used as a readout file, the specified TO port status is read out.
- When used as a write file, the value is written to the specified TO port.

Data format:

Readout file

TO0()=&Bnnnnnnnn[cr/lf]

Write file

&Bnnnnnnnn[cr/lf] or k[cr/lf]

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to 00, 01 ... 07 starting from the right.
- “k” is an integer from 0 to 255.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example 1:

SEND TO0() TO CMU

----- Outputs the TO0 port status from communication port as shown below.

Response:

TO0()=&B00000000[cr/lf]

Example 2:

SEND CMU TO TO0()

----- Inputs the TO0 port status from communication port as shown below.

&B00000111

Response:

OK[cr/lf]

14.19 SI file

14.19.1 All SI information

Format:

```
SI()
```

- Expresses all SI information.
- When used as a readout file, all SI information is read out.
- Cannot be used as a write file.

Data format:

```
SI0()=&Bnnnnnnnn [cr/lf]
SI1()=&Bnnnnnnnn [cr/lf]
      :
SI27()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right. (“m” is a port number.)
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND SI() TO CMU
```

----- Outputs all SI information from communication port as shown below.

Response:

```
SI0()=&B10001001[cr/lf]
SI1()=&B00000010[cr/lf]
SI2()=&B00000000[cr/lf]
      :
SI7()=&B00000000[cr/lf]
SI10()=&B00000000[cr/lf]
SI11()=&B00000000[cr/lf]
SI12()=&B00000000[cr/lf]
      :
SI17()=&B00000000[cr/lf]
SI20()=&B00000000[cr/lf]
      :
SI26()=&B00000000[cr/lf]
SI27()=&B00000000[cr/lf]
[cr/lf]
```

14.19.2 One SI port

Format:

```
SIm()
```

- Expresses the status of one SI port.
- When used as a readout file, the specified SI port status is read out.
- Cannot be used as a write file.

Data format:

```
SIm()=&Bnnnnnnnn[cr/lf]
```

- “m” is from 0 to 7, 10 to 17 or 20 to 27.
- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m₀, m₁ ... m₇ starting from the right.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example:

```
SEND SI5() TO CMU
```

----- Outputs the SI5 port status from communication port as shown below.

Response:

```
SI5()=&B00000000[cr/lf]
```


14.20 SO file

14.20.1 All SO information

Format:

```
SO()
```

- Expresses all SO information.
- When used as a readout file, all SO information is read out.
- Cannot be used as a write file.

Data format:

```
SO0()=&Bnnnnnnnn [cr/lf]
SO1()=&Bnnnnnnnn [cr/lf]
      :
SO27()=&Bnnnnnnnn [cr/lf]
[cr/lf]
```

- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right. (“m” is a port number.)
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND SO() TO CMU
```

----- Outputs all SO information from communication port as shown below.

Response:

```
SO0()=&B10001001[cr/lf]
SO1()=&B00000010[cr/lf]
SO2()=&B00000000[cr/lf]
      :
SO7()=&B00000000[cr/lf]
SO10()=&B00000000[cr/lf]
SO11()=&B00000000[cr/lf]
SO12()=&B00000000[cr/lf]
      :
SO17()=&B00000000[cr/lf]
SO20()=&B00000000[cr/lf]
      :
SO26()=&B00000000[cr/lf]
SO27()=&B00000000[cr/lf]
[cr/lf]
```

14.20.2 One SO port

Format:

SOm()

- Expresses the output status of one SO port.
- When used as a readout file, the specified SO port status is read out.
- When used as a write file, the value is written to the specified SO port. However, writing to SO0() and SO1() is prohibited.

Data format:

Readout file

SOm()=&Bnnnnnnnn[cr/lf]

Write file

&Bnnnnnnnn[cr/lf] or k[cr/lf]

- “m” is from 0 to 7, 10 to 17 or 20 to 27.
- “n” is a binary number of 0 or 1, comprising an 8-digit number corresponding to m0, m1 ... m7 starting from the right.
- “k” is an integer from 0 to 255.
- [cr/lf] shows CR code (0Dh) + LF code (0Ah).

Example 1:

SEND SO5() TO CMU

----- Outputs the SO5 port status from communication port as shown below.

Response:

SO5()=&B00000000[cr/lf]

Example 2:

SEND CMU TO SO5()

----- Inputs the SO5 port status from communication port as shown below.

&B00000111

Response:

OK[cr/lf]



NOTE

Writing to SO0() and SO1() is prohibited.

14.21 Error message history file

14.21.1 All error message history

Format:

LOG

- Expresses all error message history.
- When used as a readout file, all error message history is read out.
- Cannot be used as a write file.

Data format:

```
nnn:yy/mm/dd,hh:mm:ss gg.bb.msg[cr/lf]
nnn:yy/mm/dd,hh:mm:ss gg.bb.msg[cr/lf]
:
nnn:yy/mm/dd,hh:mm:ss gg.bb.msg[cr/lf]
[cr/lf]
```

- “nnn” represents an error history serial number and may be up to 500.
- “yy/mm/dd” is the year/month/day when the error occurred.
- “hh”, “mm” and “ss” represent hour, minute and second.
- “gg” represents an error message group.
- “bb” represents an error message category.
- “msg” represents an error message.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

SEND LOG TO CMU

----- Outputs all error message history from communication port as shown below.

```
1 :01/06/14,13:19:20 14.22:No start code (@)[cr/lf]
2 :01/06/14,13:18:34 22.3:DC24V power low[cr/lf]
:
498 :01/06/12,21:49:54 5.39:Illegal identifier[cr/lf]
499 :01/06/12,21:49:14 14.22:No start code (@)[cr/lf]
500 :01/06/12,21:49:00 22.3:DC24V power low[cr/lf]
[cr/lf]
```

14.22 Machine reference file

14.22.1 All machine reference data

Format:

MRF

- Expresses the machine reference data obtained after robots have performed re-turn-to-origin and absolute search.
- Reads out all machine reference data when used as a readout file.
- Cannot be used as a write file.

Data format:

```
M1=nnn% M2=nnn% . . . nnn%[cr/lf]
S1=nnn% S2=nnn% . . . nnn%[cr/lf]
[cr/lf]
```

- “nnn” is an integer from 0 to 100.
- “M1”, “M2” ... indicate the main robot group axes.
- “S1”, “Sn” ... indicate the sub robot group axes.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

SEND MRF TO CMU

----- Outputs all machine reference data from communication port as shown below.

Response:

```
M1= 55% M2= 23% M3= 33% M4= 26%[cr/lf]
[cr/lf]
```

14.23 EOF file

14.23.1 EOF data

Format:

EOF

- This file is a special file consisting only of a ^Z (=1Ah) code.
- When used as a readout file, ^Z (=1Ah) is read out.
- Cannot be used as a write file.

Data format:

^ Z (=1Ah)

Example:

```
SEND PGM TO CMU
SEND EOF TO CMU
```

----- Outputs EOF data from communication port as shown below.



NOTE

When transmitting data to an external device through the communication port, the EOF data can be used to add a ^Z code at the end of file.

```
NAME=TEST1[cr/lf]
A=1[cr/lf]
:
HALT[cr/lf]
[cr/lf]
^Z
```

14.24 Serial port communication file

14.24.1 Serial port communication file

Format:

CMU

- Expresses the serial communication port.
- Depends on the various data formats.

Example:

```
SEND PNT TO CMU
----- Outputs all point data from communication port.
SEND CMU TO PNT
----- Inputs all point data from communication port.
```

14.25 SIW file

14.25.1 All SIW


CAUTION

SIW is only available from software version 8.08 onwards.

Format:

```
SIW()
```

- Expresses all SIW data.
- Reads out all SIW information in hexadecimal digit when used as a readout file.
- Cannot be used as a write file.

Data format:

```
SIW( 0)=&Hnnnn [cr/lf]
SIW( 1)=&Hnnnn [cr/lf]
      :
SIW(15)=&Hnnnn [cr/lf]
[ cr/lf]
```

- “n” represents a 4-digit number from 0 to 9 and A to F.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

```
SEND SIW() TO CMU
```

----- Outputs all SIW data from communication port as shown below.

Response:

```
SIW( 0)=&H1001[cr/lf]
SIW( 1)=&H0010[cr/lf]
SIW( 2)=&H0000[cr/lf]
      :
SIW(15)=&H0000[cr/lf]
[cr/lf]
```

14.25.2 One SIW data

Format:



CAUTION

SIW is only available from software version 8.08 onwards.

SIW(m)

- Expresses one SIW state.
- Reads out the designated SIW information in hexadecimal when used as a readout file.
- Cannot be used as a write file.

Data format:

SIW(m)=&Hnnnn [cr/lf]

- “m” represents a number from 0 to 15.
- “n” represents a 4-digit number from 0 to 9 and A to F.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).

Example:

SEND SIW(5) TO CMU

----- Outputs SIW (5) from communication port as shown below.

Response:

SIW(5)=&H1001[cr/lf]

14.26 SOW file

14.26.1 All SOW

Format:



CAUTION

SOW is only available from software version 8.08 onwards.

SOW()

- Expresses all SOW data.
- Reads out all SOW information in hexadecimal when used as a readout file.
- Cannot be used as a write file.

Data format:

```
SOW( 0)=&Hnnnn [cr/lf]
SOW( 1)=&Hnnnn [cr/lf]
      :
SOW(15)=&Hnnnn [cr/lf]
[ cr/lf]
```

- “n” represents a 4-digit number from 0 to 9 and A to F.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).
- A line containing only [cr/lf] is added at the end of the file, indicating the end of the file.

Example:

SEND SOW() TO CMU

----- Outputs all SOW data from communication port as shown below.

Response:

```
SOW( 0)=&H1001[cr/lf]
SOW( 1)=&H0010[cr/lf]
SOW( 2)=&H0000[cr/lf]
      :
SOW(15)=&H0000[cr/lf]
[cr/lf]
```

14.26.2 One SOW data

Format:



CAUTION

SOW is only available from software version 8.08 onwards.

SOW(m)

- Expresses one SOW state.
- Reads out the designated SOW information in hexadecimal when used as a readout file.
- When used as a write file, the value is written to the designated SOW. However, writing to SOW (0) and SOW (1) is prohibited.

Data format:

SOW(m)=&Hnnnn [cr/lf]

- “m” represents a number from 2 to 15.
- “n” represents a 4-digit number from 0 to 9 and A to F.
- [cr/lf] indicates CR code (0Dh) + LF code (0Ah).



NOTE

Writing to SOW (0) and SOW (1) is prohibited.

Example1:

SEND SOW(5) TO CMU

----- Outputs SOW (5) from communication port as shown below.

Response:

SOW(5)=&H1001[cr/lf]

Example2:

SEND CMU TO SOW(5)

----- Outputs SOW (5) from communication port as shown below.

&H1001

Response:

OK[cr/lf]

14.27 Ethernet port communication file

14.27.1 Ethernet port communication file

Format:



CAUTION

ETH is only available from software version 8.22 onwards.

ETH

- Expresses the Ethernet port.
- Depends on the various data formats.

Example:

SEND PNT TO ETH

----- Outputs all point data from the Ethernet port.

SEND ETH TO PNT

----- Inputs all point data from the Ethernet port.

15. User program examples

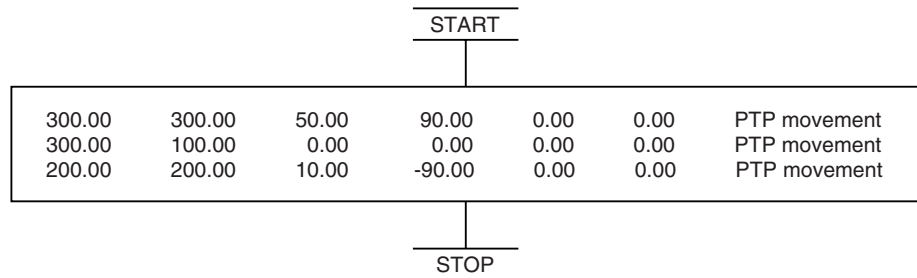
15.1 Basic operation

15.1.1 Directly writing point data in program

Overview

The robot arm can be moved by PTP (point-to-point) motion by directly specifying point data in the program.

Processing flow



Program example

```
MOVE P, 300.00 300.00 50.00 90.00 0.00 0.00
MOVE P, 300.00 100.00 0.00 0.00 0.00 0.00
MOVE P, 200.00 200.00 10.00 -90.00 0.00 0.00
HALT
```

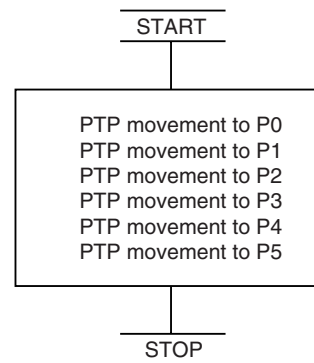
15.1.2 Using point numbers

Overview

Coordinate data can be specified by using point numbers in a program. Coordinate data should be entered beforehand in “MANUAL>POINT” mode, for example as shown below.

P0=	0.00	0.00	0.00	0.00	0.00	0.00
P1=	100.00	0.00	150.00	30.00	0.00	0.00
P2=	0.00	100.00	50.00	0.00	0.00	0.00
P3=	300.00	300.00	0.00	0.00	0.00	0.00
P4=	300.00	100.00	100.00	90.00	0.00	0.00
P5=	200.00	200.00	0.00	0.00	0.00	0.00

Processing flow



Program example

```

MOVE P, P0    FOR J=0 TO 5
MOVE P, P1    MOVE P,P[J]
MOVE P, P2    NEXT J
MOVE P, P3    or  HALT
MOVE P, P4
MOVE P, P5
HALT
  
```

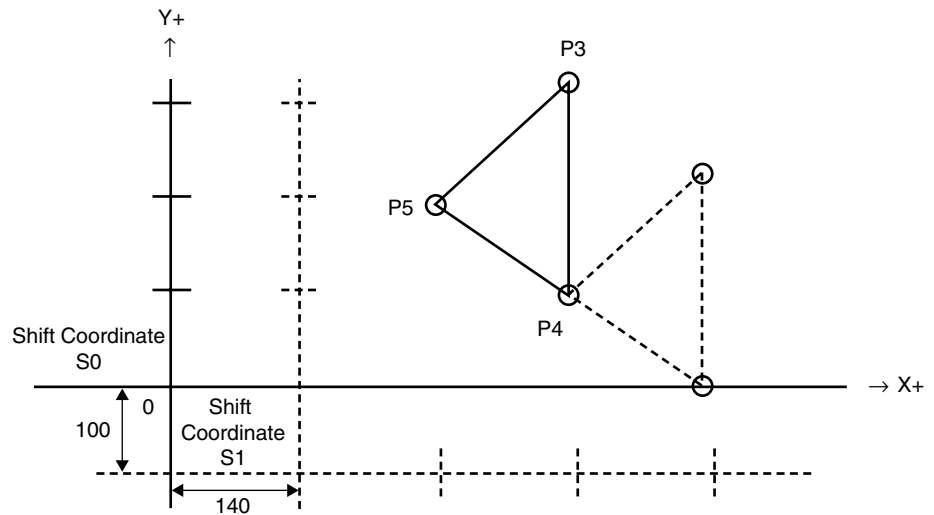
15.1.3 Using shift coordinates

Overview

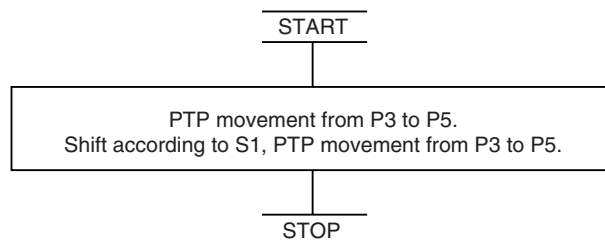
In the example shown below, after PTP movement from P3 to P5, the coordinate system is shifted +140 along the X-axis and -100 along the Y-axis, and the robot then moves from P3 to P5 again. The shift coordinate data is set in S1 and P3, P4, P5 are set as described in the previous section ("15.1.2 Using point numbers").

S0= 0.00 0.00 0.00 0.00

S1= 140.00 -100.00 0.00 0.00



Processing flow



Program example

```

SHIFT S0
FOR J=3 TO 5
  MOVE P, P[J]
NEXT J
SHIFT S1
FOR K=3 TO 5
  MOVE P, P[K]
NEXT K
HALT
  
```

15.1.4 Palletizing

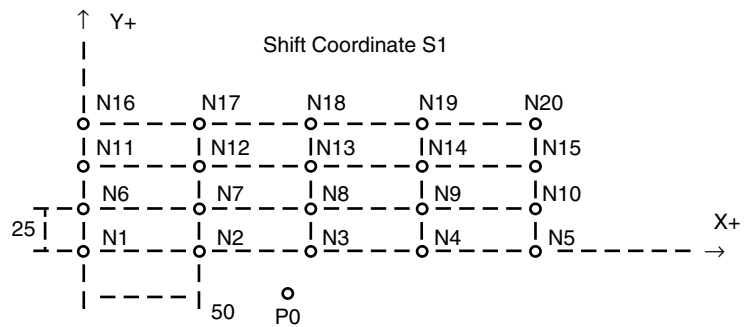
15.1.4.1 Utilizing the shift coordinates

Overview

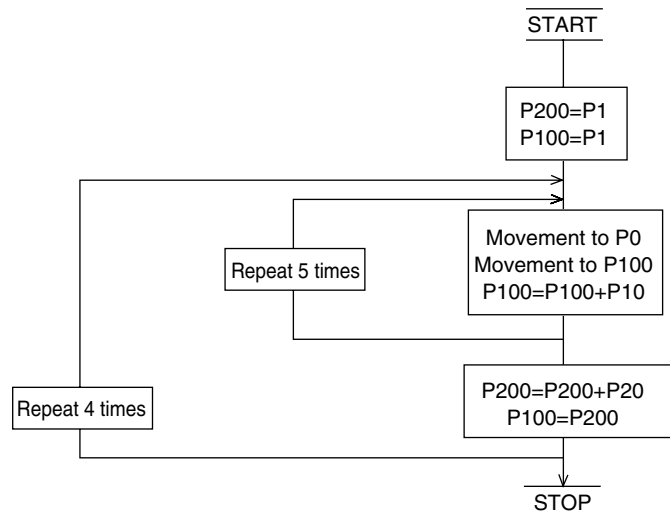
Repetitive movement between a fixed work supply position P0 and each of the equally spaced points on a pallet can be performed with the following program. In the drawing below, points N1 to N20 are on Cartesian coordinates, consisting of 5 points positioned at a 50mm pitch in the X-axis direction and 4 points at a 25mm pitch in the Y-axis direction. The robot arm moves from point to point in the order of P0-N1-P0-N2...N5-P0-N6-P0... while repeatedly moving back and forth between point P0 and each pallet.

Work supply position	: P0=	0.0	0.0	0.0	0.0	0.0	0.0
X-axis pitch	: P10=	50.0	0.0	0.0	0.0	0.0	0.0
Y-axis pitch	: P20=	0.0	25.0	0.0	0.0	0.0	0.0
N1 position	: P1=	100.0	50.0	0.0	0.0	0.0	0.0

The above data should be entered beforehand in “MANUAL>POINT” mode.



Processing flow



Program Example

```

SHIFT S1
P100=P1
P200=P1
FOR J=1 TO 4
  FOR K=1 TO 5
    SHIFT S0
    MOVE P,P0
    SHIFT S1
    MOVE P,P100
    P100=P100+P10
  NEXT K
  P200=P200+P20
  P100=P200
NEXT J

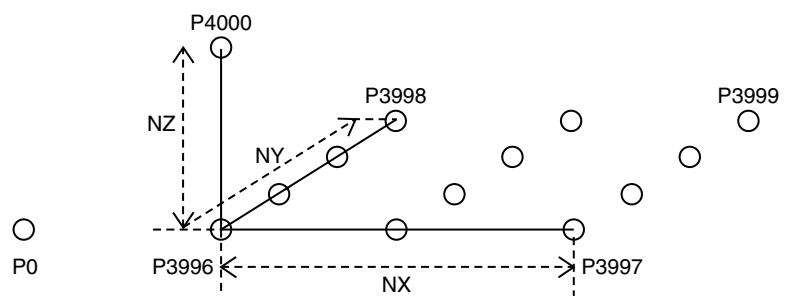
```

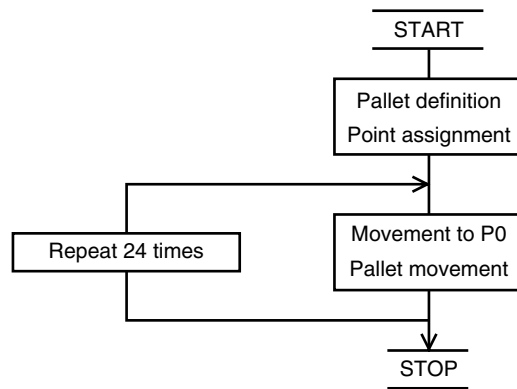
15.1.4.2 Utilizing pallet movement

Overview

Repetitive movement between a fixed work supply position P0 and each of the equally spaced points on a pallet can be performed with the following program. In the drawing below, points N1 to N24 are on Cartesian coordinates, consisting of 3 points positioned at a 50mm pitch in the X-axis direction, 4 points at a 50mm pitch in the Y-axis direction, and 2 points at 100mm pitch in the Z-axis direction. The robot arm moves from point to point in the order of P0-N1-P0-N2...-N5-P0-N6... while repeatedly moving back and forth between point P0 and each pallet.

Work supply position : P0 = 0.00 0.00 100.00 0.00 0.00 0.00
Pallet definition : PL0 (P3996 to P4000 are used)
NX= 3
NY= 4
NZ= 2
P3996= 100.00 50.00 100.00 0.00 0.00 0.00
P3997= 200.00 50.00 100.00 0.00 0.00 0.00
P3998= 100.00 200.00 100.00 0.00 0.00 0.00
P3999= 200.00 200.00 100.00 0.00 0.00 0.00
P4000= 100.00 50.00 100.00 0.00 0.00 0.00



Processing flow**Program example**

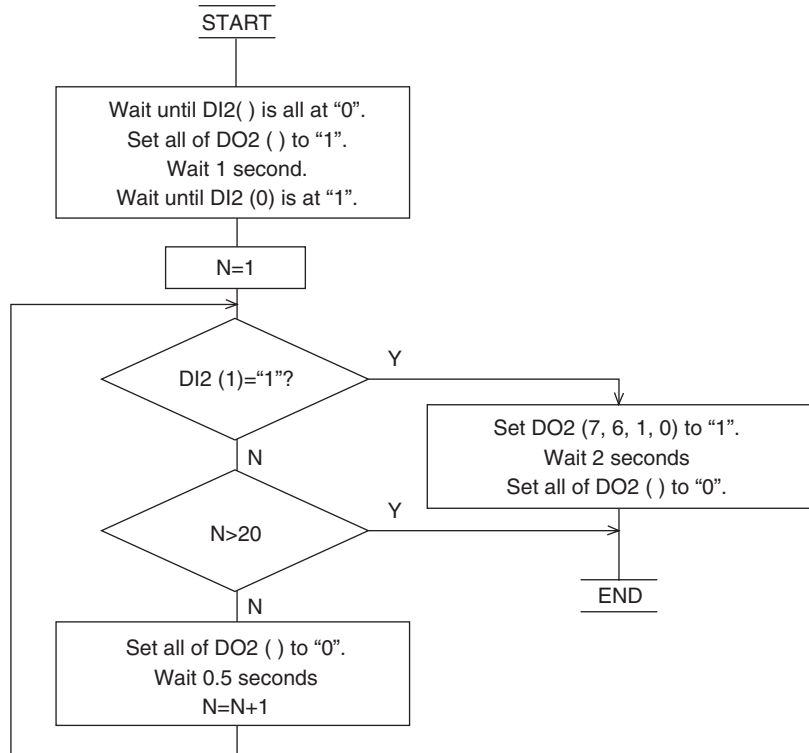
```
FOR I=1 TO 24
  MOVE P,P0,Z=0.00
  PMOVE (0,I),Z=0.00
NEXT I
MOVE P,P0,Z=0.00
HALT
```

15.1.5 DI/DO (digital input and output) operation

Overview

The following example shows general-purpose signal input and output operations through the STD. I/O connector.

Processing flow



Program example

```

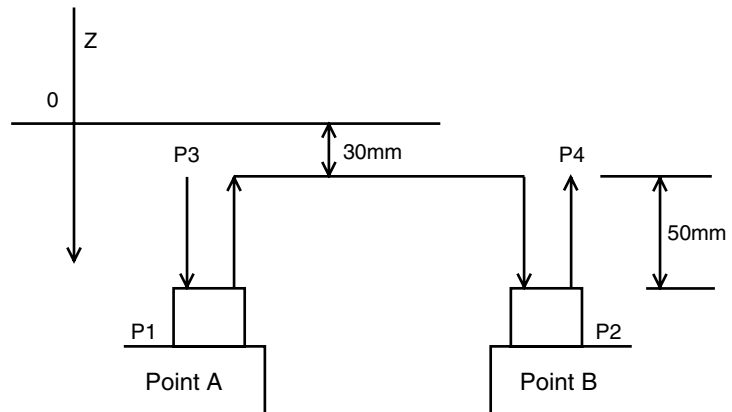
WAIT DI2( )=0
DO2( )=&B11111111
DELAY 1000
WAIT DI2(0)=1
N=1
*LOOP1:
IF DI2(1)=1 THEN *PROGEND
IF N>20 THEN *ALLEND
DO2( )=0
DELAY 500
N=N+1
GOTO *LOOP1
'END ROUTINE
*PROGEND:
DO2(7,6,1,0)=&B1111
DELAY 2000
DO2( )=0
*ALLEND:
HALT
  
```

15.2 Application

15.2.1 Pick and place between 2 points

Overview

The following is an example for picking up a part at point A and placing it at point B.



Precondition

1. Set the robot movement path.

$P3 \rightarrow P1 \rightarrow P3 \rightarrow P4 \rightarrow P2 \rightarrow P4$

Locate P3 and P4 respectively at a position 50mm above P1 and P2 and set the P1 and P2 positions by teaching.

2. I/O signal

DO2 (0) Chuck (gripper) open/close = 0: open, 1: close

A 0.1 second wait time is set during chuck open and close.

Program examples

(1) When calculating to find P3 and P4

```
P3=P1
P4=P2
LOCZ(P3)=LOCZ(P3)-50.0
LOCZ(P4)=LOCZ(P4)-50.0
MOVE P,P3
GOSUB *OPEN
MOVE P,P1
GOSUB *CLOSE
MOVE P,P3
MOVE P,P4
MOVE P,P2
GOSUB *OPEN
MOVE P,P4
HALT
*OPEN:
DO2(0)=0
DELAY 100
RETURN
*CLOSE:
DO2(0)=1
DELAY 100
RETURN
```

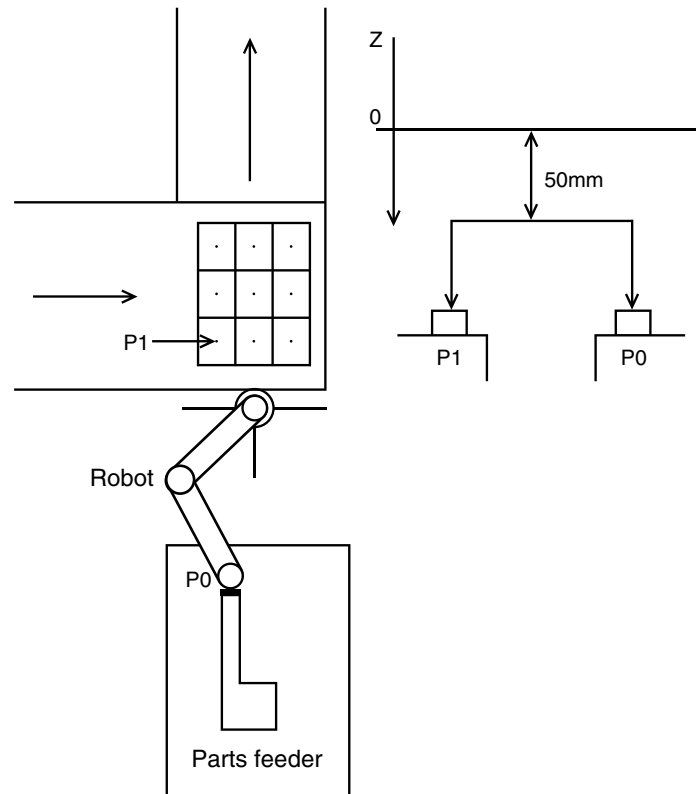
(2) When using arch motion

```
P4=P2
LOCZ(P4)=LOCZ(P4)-50.0
GOSUB *OPEN
MOVE P,P1,Z=30.0
GOSUB *CLOSE
MOVE P,P2,Z=30.0
GOSUB *OPEN
MOVE P,P4
HALT
*OPEN:
DO2(0)=0
DELAY 100
RETURN
*CLOSE:
DO2(0)=1
DELAY 100
RETURN
```

15.2.2 Palletizing

Overview

The following is an example for picking up parts supplied from the parts feeder and placing them on a pallet on the conveyor. The pallet is ejected when full.



Precondition

1. I/O signal

DI(30)	= 1: Parts are supplied
DI(31)	= 1: Pallet is loaded

DO(30)	Robot hand open/close = 0: open, =1: close
DO(31)	= 1: Pallet eject

Robot hand open/close time is 0.1 second and pallet eject time is 0.5 seconds.

2. The points below should be input beforehand as point data.

P0	Part supply position
P1	Pallet reference position
P10	X direction pitch
P11	Y direction pitch

3. Vertical movement is performed to a position Z=50mm above the pallet and parts feeder.

Program example 1

```
WHILE -1
FOR A=0 TO 2
  FOR B=0 TO 2
    WAIT DI(31)=1
    WAIT DI(30)=1
    DO(30)=0
    DELAY 100
    MOVE P,P0,Z=50.0
    DO(30)=1
    DELAY 100
    P100=P1+P10*B+P11*A
    MOVE P,P100,Z=50.0
    DO(30)=0
    DELAY 100
  NEXT
NEXT
DRIVE (3,0)
DO(31)=1
DELAY 500
DO(31)=0
WEND
HALT
```

Program example 2

* When defined in pallet 0

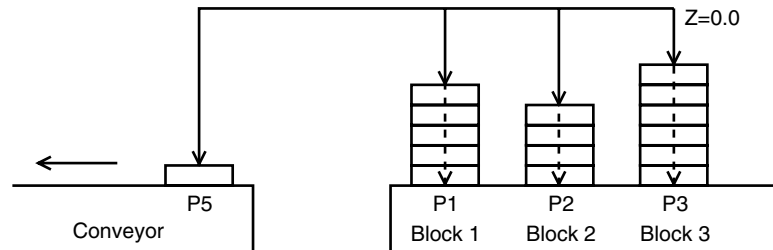
```
WHILE -1
FOR A=1 TO 9
  WAIT DI(31)=1
  WAIT DI(30)=1
  DO(30)=0
  DELAY 100
  MOVE P,P0,Z=50.0
  DO(30)=1
  DELAY 100
  PMOVE(0,A),Z=50.0
  DO(30)=0
  DELAY 100
NEXT
DRIVE(3,0)
DO(31)=1
DELAY 500
DO(31)=0
WEND
HALT
```

15.2.3 Pick and place of stacked parts

Overview

The following is an example for picking up parts stacked in a maximum of 6 layers and 3 blocks and placing them on the conveyor.

The number of parts per block may differ from others. Parts are detected with a sensor installed on the robot hand.



Precondition

1. I/O signal

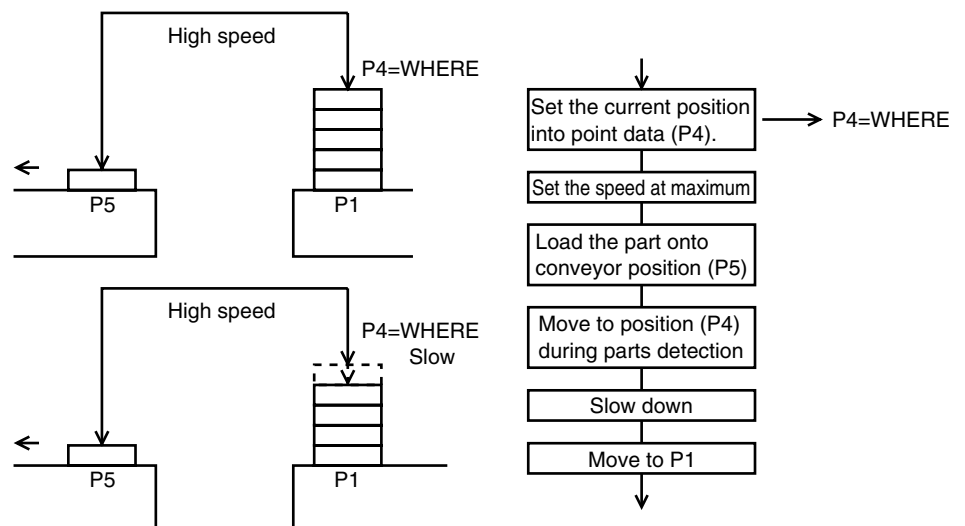
DI(30)	Part detection sensor = 1: Parts are supplied
DO(30)	Robot hand open/close = 0: open, =1: close

Robot hand open/close time is 0.1 seconds.

2. The points below should be input beforehand as point data.

P1	Bottom of block 1
P2	Bottom of block 2
P3	Bottom of block 3
P5	Position on conveyor

3. Movement proceeds at maximum speeds but slows down when in proximity to the part.



4. Use a STOPON condition in the MOVE statement for sensor detection during movement.

Program example

```
FOR A=1 TO 3
  SPEED 100
  GOSUB *OPEN
  P6=P[A]
  LOCZ(P6)=0.00
  MOVE P,P6,Z=0.0
  WHILE -1
    SPEED 20
    MOVE P,P[A],STOPON DI3(0)=1
    IF DI3(0)=0 THEN *L1
    'SENSOR ON
    P4=JTOXY(WHERE)
    GOSUB *CLOSE
    SPEED 100
    MOVE P,P5,Z=0.0
    GOSUB *OPEN
    MOVE P,P4,Z=0.0
  WEND
*L1: 'SENSOR OFF
NEXT A
SPEED 100
DRIVE (3,0)
HALT
*OPEN:
DO3(0)=0
DELAY 100
RETURN
*CLOSE:
DO3(0)=1
DELAY 100
RETURN
```

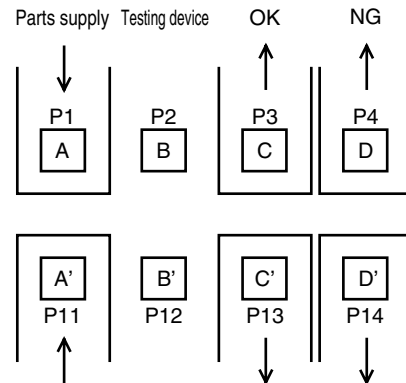

15.2.4 Parts inspection (Multi-tasking example)

Overview

One robot is used to inspect two different parts and sort them according to the OK/NG results.

The robot picks up the part at point A and moves it to the testing device at point B. The testing device checks the part and sends it to point C if OK or to point D if NG.

The part at point A' is picked up and moved to the testing device at point B' in the same way. The testing device checks the part and sends it to point C' if OK or to point D' if NG. It is assumed that 10 to 15 seconds are required for the testing device to issue the OK/NG results.



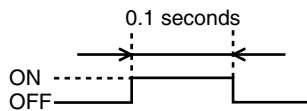
Precondition

1. I/O signal



NOTE

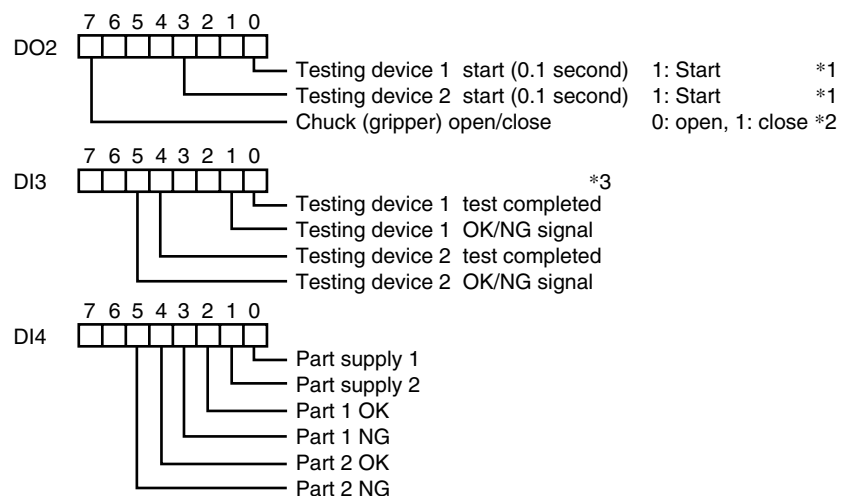
*1: As the start signal, supply a 0.1 second pulse signal to the testing device.



*2: Chuck open and close time is 0.1 seconds.

*3: Each time a test is finished, the test completion signal and OK/NG signal are sent from the testing device.

After testing, the test completion signal turns ON (=1), and the OK/NG signal turns ON (=1) when the result is OK and turns OFF (=0) when NG.



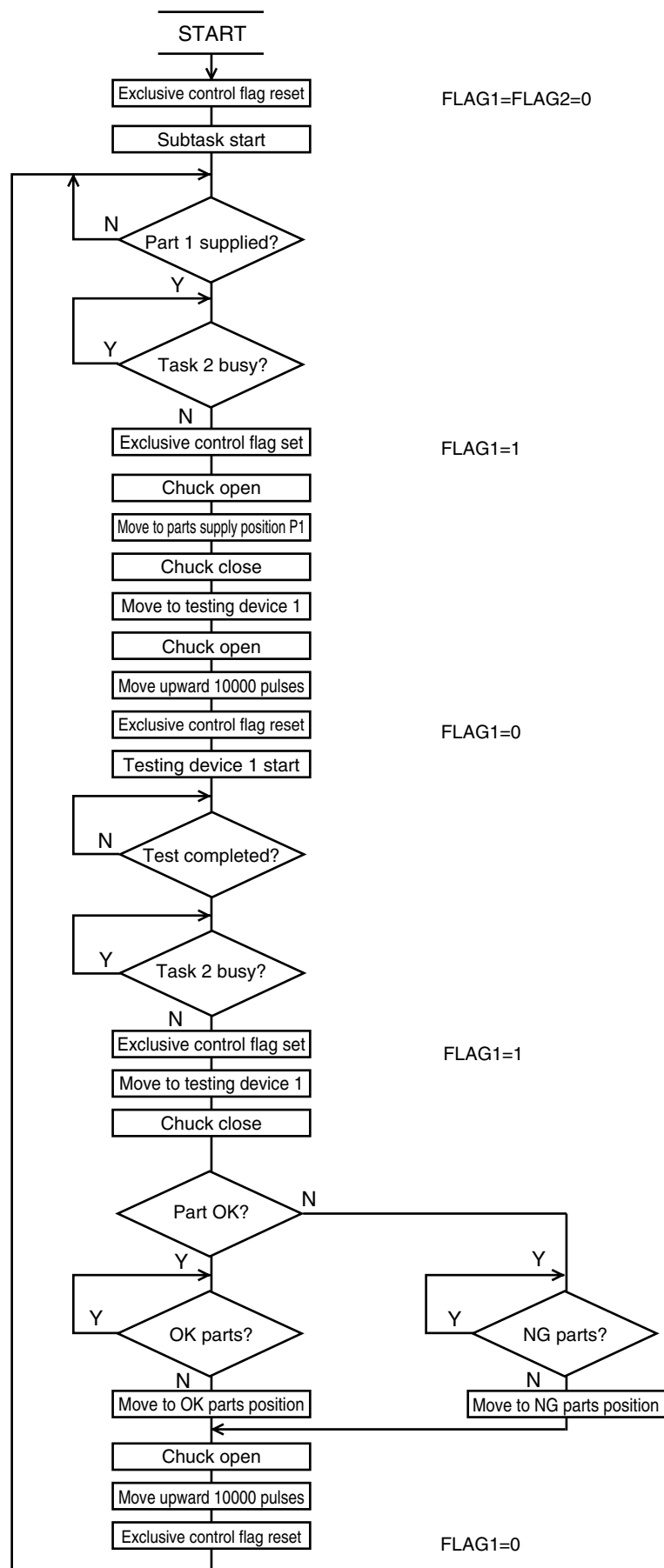
2. The main task (task 1) is used to test part 1 and the subtask (task 2) is used to test part 2.

3. An exclusive control flag is used to allow other tasks to run while waiting for the test completion signal from the testing device.

FLAG1 = 1 : Task 1 busy (Task 2 is kept waiting)
= 0 : Task 1 ready (Task 2 operable)

FLAG2 = 1 : Task 2 busy (Task 1 is kept waiting)
= 0 : Task 2 ready (Task 1 operable)

4. Flow chart



Program example

```

FLAG1=0
FLAG2=0
UPPOS=0.0
START *S1,T2          Subtask Start

*L1:
WAIT DI4(0)=1          Part supply standby
WAIT FLAG2=0          Task completion standby
FLAG1=1               Exclusive control flag set
GOSUB *OPEN           Chuck open
MOVE P,P1,Z=UPPOS     Move to part supply position
GOSUB *CLOSE          Chuck close
MOVE P,P2,Z=UPPOS     Move to testing device
GOSUB *OPEN           Chuck open
DRIVEI (3,-10000)     Move Z-axis upward Z 10,000 pulses
FLAG1=0               Exclusive control flag reset
DO2(0)=1              Testing device start
DELAY 100
DO2(0)=0
WAIT DI3(0)=1          Test completion standby
WAIT FLAG2=0          Task completion standby
FLAG1=1               Exclusive control flag set
MOVE P,P2,Z=UPPOS     Move to testing device
GOSUB *CLOSE          Chuck close
IF DI3(1)=1 THEN      Test
'GOOD
WAIT DI4(2)=0          Part movement standby
MOVE P,P3,Z=UPPOS     Move to OK parts position
ELSE
'NG
WAIT DI4(3)=0          Part movement standby
MOVE P,P4,Z=UPPOS     Move to NG parts position
ENDIF
GOSUB *OPEN           Chuck open
DRIVEI (3,-10000)     Move Z-axis upward 10,000 pulses
FLAG1=0               Exclusive control flag reset
GOTO *L1

*OPEN:
DO2(7)=0
DELAY 100
RETURN

*CLOSE:
DO2(7)=1
DELAY 100
RETURN

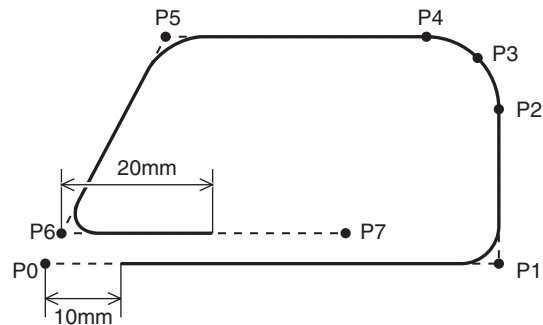
*S1:
WAIT DI4(1)=1
WAIT FLAG1=0
FLAG2=1
GOSUB *OPEN
MOVE P,P11,Z=UPPOS
GOSUB *CLOSE
MOVE P,P12,Z=UPPOS
GOSUB *OPEN
DRIVEI (3,-10000)
FLAG2=0
DO2(3)=1
DELAY 100
DO2(3)=0
WAIT DI3(4)=1
WAIT FLAG1=0
FLAG2=1
MOVE P,P12,Z=UPPOS
GOSUB *CLOSE
IF DI3(5)=1 THEN
'GOOD
WAIT DI4(4)=0
MOVE P,P13,Z=UPPOS
ELSE
'NG
WAIT DI4(5)=0
MOVE P,P14,Z=UPPOS
ENDIF
GOSUB *OPEN
DRIVEI (3,-10000)
FLAG2=0
GOTO *S1

```

15.2.5 Sealing

Overview

This section shows an example of the parts sealing operation.



Precondition

1. I/O signal

DO(20)	Valve open/close = 1: open, =0: close
--------	---------------------------------------

2. Set P0 - P7 by teaching.

Program example

MOVE P,P0,Z=0		
SPEED 40		
PATH SET	Start of path setting	} Path setting (Robot does not operate.)
PATH L,P1,DO(20)=1@10.0	Start of coating at 10 mm position	
PATH L,P2		
PATH C,P3,P4		
PATH L,P5		
PATH L,P6,S=30		
PATH L,P7,DO(20)=0@20.0	End of coating at 20 mm position	
PATH END	End of path setting	
PATH START	Execute PATH motion (Robot starts from P0 and stops at P7.)	
HALT		

15.2.6 Connection to an external device through RS-232C (example 1)

Overview

Point data can be written in a program by using an external device connected to the RCX series controller via the RS-232C port.

Precondition

1. Input to the external device from the controller
SDATA/X/Y [cr]
2. Output to the controller from the external device
P10=156.42 243.91 0.00 0.00 0.00 0.00 [cr]
* The [cr] is cr (=0Dh).



NOTE

- "SEND xxx TO CMU" outputs the contents specified by "xxx" through the RS-232C.
- "SEND CMU TO xxx" sends data into the files specified by "xxx" through the RS-232C.

Program example

```
'INIT
VCMD$="SDATA/X/Y"
P0 = 0.00 0.00 0.00 0.00 0.00 0.00
'MAIN ROUTINE
MOVE P, P0
*ST:
SEND VCMD$ TO CMU
SEND CMU TO P10
MOVE P, P10
GOTO *ST
```

15.2.7 Connection to an external device through RS-232C (example 2)

Overview

Point data can be created from the desired character strings and written in a program by using an external device connected to the RCX series controller via the RS-232C port.

Precondition

1. Input to the external device from the controller
SDATA/X/Y [cr]
2. Output to the controller from the external device
X=156.42, Y=243.91 [cr]
* The [cr] is cr (=0Dh).



NOTE

- "SEND xxx TO CMU" outputs the contents specified by "xxx" through the RS-232C.
- "SEND CMU TO xxx" sends data into the files specified by "xxx" through the RS-232C.
- The LEN () function obtains the length of the character string.
- The MID\$ () function obtains the specified character string from among the character strings.
- The VAL () function obtains the value from the character string.

Program example

```
'INT
VCMD$="SDATA/X/Y"
VIN$=""
VX$=""
VY$=""
P0 = 0.00 0.00 0.00 0.00 0.00 0.00
P11 = 100.00 100.00 0.00 0.00 0.00 0.00
'MAIN ROUTINE
MOVE P, P0
*ST:
SEND VCMD$ TO CMU
SEND CMU TO VIN$
I=1
VMAX=LEN(VIN$)
*LOOP:
IF I>VMAX THEN GOTO *E_LOOP
C$=MID$(VIN$,I,1)
IF C$="X" THEN
I=I+2
J=I
*X_LOOP:
C$=MID$(VIN$, J, 1)
IF C$="," THEN
*X1_LP:
L=J-I
VX$=MID$(VIN$, I, L)
I=J+1
GOTO *LOOP
ENDIF
J=J+1
IF J>VMAX THEN GOTO *X1_LP
GOTO *X_LOOP
ENDIF
IF C$="Y" THEN
I=I+2
J=I
*Y_LOOP:
C$=MID$(VIN$, J, 1)
IF C$="," THEN
*Y1_LP:
L=J-I
VY$=MID$(VIN$, I, L)
I=J+1
GOTO *LOOP
ENDIF
J=J+1
IF J>VMAX THEN GOTO *Y1_LP
GOTO *Y_LOOP
ENDIF
I=I+1
GOTO *LOOP
*E_LOOP:
WX=VAL(VX$)
WY=VAL(VY$)
LOCX(P11)=WX
LOCY(P11)=WY
MOVE P, P11
GOTO *ST
```

16. Sequence function

Besides normal robot programs, this controller can execute high-speed processing programs (sequence programs) in response to the robot input/output (DI, DO, MO, LO TO, SI, SO) signals. This means that two different programs (robot program and sequence program) can be executed at the same time.

The sequence program runs according to its own cycle in "AUTO" or "MANUAL" mode, regardless of the execution or stopping of a robot program. The sequence program starts running as soon as the controller is turned on (that is, in "MANUAL" mode), so it can be used to monitor the status of sensors, push button switches, solenoid valves, etc.

The sequence program can be written in the same robot language used for robot programs. This eliminates the need to learn a new language and making it easier to program.

16.1 Creating a sequence program

16.1.1 Programming method

The following explains how to create a sequence program in order to make use of the sequence function.

First, enter "PROGRAM" mode and create a file with the file name "SEQUENCE". The controller automatically recognizes that a file with this name is a sequence program.

Fig. 16-1-1-1 Naming a sequence program file

```
PROGRAM>DIR                                     <TEST10  >
-----
No.      Name      Line      Byte  RW/RO
1       TEST10      12       145    RW
2       LOCATE20     25       320    RW

Enter program name >SEQUENCE
```

Next, create a sequence program the same way as you create a robot program. The commands that may be used are explained later.

Fig. 16-1-1-2 Creating a sequence program

```
PROGRAM>EDIT                                     <SEQUENCE>
-----+-----+-----+-----+-----+-----+
1  DO(20)=DI(21) AND DI(22)
2  MO(30)=DO(23) OR DI(22)
3  MO(31)= ~ MO(30)
4  DO(21)=(DI(36) OR DI(25))AND DI(2
5  DO(30)=MO(30) OR DI_

SELECT COPY CUT PASTE BS
```

16.1.2 Compiling

After editing the program, it must be compiled as a sequence program. Compiling is performed in the same way as for robot programs. Press the **F 5** (COMPILE) key on the highest-level screen in “PROGRAM” mode (Fig. 16-1-2-1).

Fig. 16-1-2-1 Sequence program

PROGRAM		<SEQUENCE>	
		1	2
1	DO(20)=DI(21) AND DI(22)		
2	MO(30)=DO(23) OR DI(22)		
3	MO(31)= ~ MO(30)		
4	DO(21)=(DI(36) OR DI(25))AND DI(2		
5	DO(30)=MO(30) OR DI(27)		
EDIT		DIR	COMPILE

A check message appears asking if you want to compile the sequence program. (Fig. 16-1-2-2) Press the **F 4** (YES) key to compile the program. To cancel this compiling, press the **F 5** (NO) key. The display changes to the compiling screen for normal robot programs.

Fig. 16-1-2-2 Compiling the sequence program

PROGRAM>COMPILE		<SEQUENCE>	
		1	2
1	DO(20)=DI(21) AND DI(22)		
2	MO(30)=DO(23) OR DI(22)		
3	MO(31)= ~ MO(30)		
4	DO(21)=(DI(36) OR DI(25))AND DI(2		
5	DO(30)=MO(30) OR DI(27)		
Compile for SEQUENCE OK?		YES	NO

If there is a syntax error in the program, an error message appears and the program will be listed from the line with the error (Fig. 16-1-2-3)
When the compiling ends without any error, the program will be listed from its first line.

Fig. 16-1-2-3 Compiling error

PROGRAM		<SEQUENCE>	
		1	2
5.1: Syntax error			
3	MO(31)= ~ MO(30)* AB		
4	DO(21)=MO(36) OR DI(27)		
5	DO(30)=MO(30) OR DI(27)		
6	DO(25)=DI(26) AND DO(32)		
7	DO(30)=MO(30) OR DI(27)		
EDIT		DIR	COMPILE

-  **CAUTION**
The sequence execution program is erased and the letter “s” disappears in the following cases. In these cases the sequence function cannot be used in “UTILITY” mode.
1. When the sequence program was erased

2. When the sequence program was edited

3. When normal robot program compiling was performed for the sequence program

4. Program data was initialized.

5. A “Sequence object check sum” error occurred.

When you display the directory after the compiling the sequence program, a letter “s” appears to the left of the program name “SEQUENCE”. This means that the sequence program has been compiled successfully and is ready for use (Fig. 16-1-2-4).

Fig. 16-1-2-4 Sequence execution program after compiling

PROGRAM>DIR					<TEST10 >	
No.	Name	Line	Byte	RW/RO		
1	TEST10	12	145	RW		
2	LOCATE20	25	320	RW		
3	s SEQUENCE	8	141	RW		
NEW					INFO	

16.2 Executing a sequence program

The following conditions must be satisfied to execute a sequence program. If any of these conditions is not met, the sequence program cannot be executed.

1. The sequence execution program has been created by compiling.
2. The sequence function is enabled in “UTILITY” mode.
3. The external sequence control input (DI10) contact is closed.
4. The current operation mode is “MANUAL” or “AUTO”.

When all of the above conditions are met, the sequence program can now be executed. While the program is running, the letter “s” will appear at the left end of the second line of the screen. (Fig. 16-2-1)

Fig. 16-2-1 Sequence program execution in progress

MANUAL		50% [MG] [S0H0J]			
S _____					
Current position					
M1=	0	M2=	0	*M3=	0
*M4	0				
POINT		PALLET		VEL+	VEL-

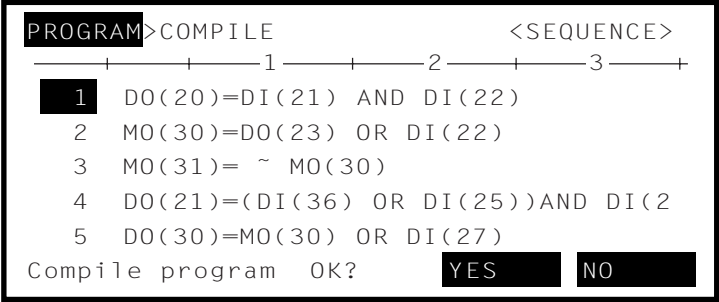
16.2.1 Sequence program STEP execution

The sequence program may be executed line by line while checking one command line at a time.

To do this, press the **F 5** (NO) key on the compile screen (Fig. 16-1-2-2). Sequence program compiling is canceled and the normal robot compile screen (Fig. 16-2-2) then appears.

Press the **F 4** (YES) key to compile and create a normal execution program. Then, execute this program with the STEP statement in “AUTO” mode to check the operation.

Fig. 16-2-2 Sequence program STEP execution



16.3 Creating a sequence program

When creating a sequence program, you may use only assignment statements comprised of input/output variables and logical operators. An error will occur during compiling if any statement other than assignment statements is used in the program, and the compiling cannot be completed.

16.3.1 Assignment statements used with sequence program

<output variable> <internal auxiliary output variable> <arm lock output variable> <timer output variable>	=<expression>
--	--------------------------------

<expression> may only be a logic operation using parallel input/output variables, internal auxiliary output variables, arm lock output variables, timer output variables or serial input/output variables.

16.3.2 Input/output variables used in sequence program

a. Parallel input variables

These variables show the status of the parallel input signal.

DI(mb)	m: Port number 0 to 7, 10 to 17, 20 to 27 b: bit definition 0 to 7
--------	---

b. Parallel output variables

These variables specify or refer to the status of parallel output signal.

DO(mb)	m: port number 0 to 7, 10 to 17, 20 to 27 b: bit definition 0 to 7
--------	---

Output to ports 0 and 1 is not possible.

c. Internal auxiliary output variables

These variables are used within the controller and are not output externally.

MO(mb)	m: port number 0 to 7, 10 to 17, 20 to 27 b: bit definition 0 to 7
--------	---

Output to ports 0 and 1 is not possible.

d. Arm lock output variables

These variables are used to prohibit the arm movement. An axis movement is prohibited by ON status.

LO(mb)	m: port number 0 b: bit definition 0 to 7
--------	--

LO(00) to LO(07) corresponds to arm 1 to arm 8.

e. Timer output variables

TO(mb)	m: port number 0 b: bit definition 0 to 7
--------	--

There are a total of 8 timer output variables: TO(00) to TO(07). The timer of each variable is defined by the timer definition statement TIM00 to 07.

f. Serial input variables

These variables show the status of serial input signal.

SI(mb)	m: Port number 0 to 7, 10 to 17, 20 to 27 b: bit definition 0 to 7
--------	---

g. Serial output variables

These variables show the status of serial output signal.

SO(mb)	m: port number 0 to 7, 10 to 17, 20 to 27 b: bit definition 0 to 7
--------	---

Output to ports 0 and 1 is not possible.

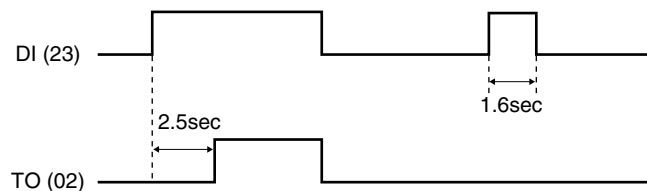
Timer example:

TIM02=2500
TO(02)=DI(23)

When DI(23) is ON, after 2.5 seconds, TO(02) is set ON.

When DI(23) is OFF, TO(02) is also OFF.

When DI(23) isn't ON after 2.5 second or more, TO(02) does not change to ON.



Variables may only be defined in the 1 bit format.

Examples:

DO(35)
MO(24)
DI(16)

Incorrect examples:

DO(37,24)
DI3(4)
MO3()

16.3.3 Timer definition statement

TIMmb=<expression> m: port number 0 b: bit definition 0 to 7
--

The value of <expression> must be from 100 to 999900msec (=0.1 to 999.9 second). However, since the units are set every 100msec, values less than 99msec are truncated

Examples:

TIM00 = 650	 0.6 seconds
TIM03 = 2480	 2.4 seconds

The timer definition statement sets the timer value of the timer output variable. This definition statement may be anywhere in the program. When the timer definition statement is omitted, the timer setting value of the variable is 0.
TIM00 to 07 correspond to the timer output variables TO(00) to (07).

16.3.4 Logical operators used with sequence program

OR,	Logical OR
AND, &	Logical AND
NOT, ~	Logical NOT

16. Sequence function

⚠ CAUTION

1. NOT cannot be used prior to the first parenthesis “ (“ or on the left of an expression. For example, the following commands cannot be used.
 $DO(21)=\sim(DI(30) \mid DI(32))$
 $\sim DO(30)=DI(22) \& DI(27)$

2. Numeric values cannot be assigned on the right of an expression. The following examples cannot be used:
 $MO(35)=1$
 $DO(26)=0$

3. There is no need to define a “HALT” or “HOLD” statement at the end of the program.

4. The I/O and internal auxiliary output variables used in sequence programs are shared with robot programs, so be careful not to make improper changes when using the same variables between them.

16.3.5 Priority of logic operations

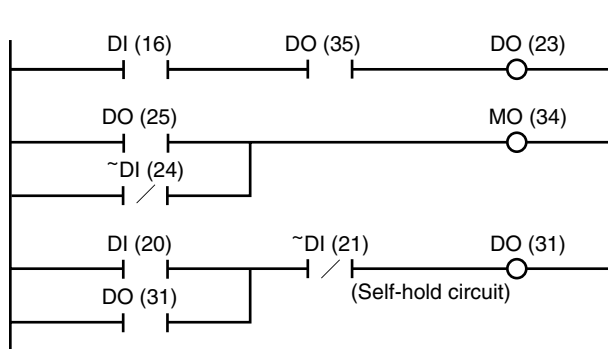
1. Expressions in parentheses
2. NOT, $\sim$
3. AND, $\&$
4. OR, $\mid$

Assignment statement example:

A ladder diagram is also shown merely for your reference.

$DO(23)=DI(16)\&DO(35)$
 $MO(34)=DO(25)\mid DI(24)$
 $DO(31)=(DI(20)\mid DO(31))\& DI(21)$

Ladder diagram



Sequence program specifications

Commands	Logical OR, AND, NOT
I/O	Same as robot language
Program capacity	4096 bytes (A maximum of 512 variables can be specified.)
Scan time	10 to 30ms depending on the number of steps (This changes automatically.)

17. Online commands

This chapter explains online commands that can be input to the robot controller through the RS-232C or Ethernet (option) port to execute specific instructions. See the RCX controller user's manual for information on methods for connecting to the RS-232C port. For details on Ethernet, refer to the "Ethernet for RCX series" user's manual.

■ About termination codes

During data transmission, the controller adds the following codes to the end of a line of transmission data.

●RS-232C

- CR (0Dh) and LF (0Ah) are added to the end of the line when the "Termination code" parameter of communication parameters is set to CRLF.
- CR (0Dh) is added to the end of the line when the "Termination code" parameter of communication parameters is set to CR.

●Ethernet (option)

- CR (0Dh) and LF (0Ah) are added to the end of the line.

When data is received, then the data up to CR (0Dh) is treated as one line regardless of the "Termination code" parameter setting, so LF (0Ah) is ignored.

The termination code is expressed as [cr/lf] in the detailed description of each online command in "17.1 Key operation" onwards.

Online command list

Operation type	Command	Option	Meaning	Condition
1. Key operation	AUTO		Change mode	3
	PROGRAM		AUTO mode	
	MANUAL		PROGRAM mode	
	SYSTEM		MANUAL mode	
	RESET		SYSTEM mode	4
	RUN		Reset program	
	STEP		Execute program	
	SKIP		Execute one line	
	NEXT		Skip one line	
	STOP		Execute to next line	
			Stop program	2
	BREAK	m, n (m: break point No., n: line)	Set break point	4
	CHGTSK		Switch execution task	3
	MSPEED	k (k : 1-100)	Change manual speed	3
	MSPEED2	k (k : 1-100)	Main robot	
	ABSADJ	k, 0 or k, 1 (k : 1-6)	Sub robot	3
	ABSADJ2	k, 0 or k, 1 (k : 1-6)	Move to absolute reset position	
	ABSRESET	k (k : 1-6)	Absolute reset on each axis	
	ABSRESET2	k (k : 1-6)	Main robot	
	ORGRTN	k (k : 1-6)	Sub robot	3
	ORGRTN2	k (k : 1-6)	Return-to-origin	
	INCH	k+ or k- (k : X,Y,Z,R,A,B)	Manual movement (inching)	3
	INCH2	k+ or k- (k : X,Y,Z,R,A,B)	Main robot	
	JOG	k+ or k- (k : X,Y,Z,R,A,B)	Sub robot	3
	JOG2	k+ or k- (k : X,Y,Z,R,A,B)	Manual movement (jog)	
	TEACH	m(m : point number)	Point data teaching	3
	TEACH2	m(m : point number)	Main robot	
2. Utility	PADDR		Sub robot	4
	COPY	<program 1> TO <program 2>	Acquire program execution status	3
		Pm-Pn TO Pk	Copy program 1 to program 2	
		PCm-PCn TO PCk	Copy points "m - n" to point "k"	
	ERA	<program>	Copy point comments "m - n" to point comment "k"	3
			Delete program	
		Pm-Pn	Delete points "m - n"	
		PCm-PCn	Delete point comments "m - n"	
		PLm	Delete pallet "m"	
	REN	<program 1> TO <program 2>	Rename "program 1" to "program 2"	3
	ATTR	<program 1> TO s (s: RW/RO)	Change program attribute	3

(continued to next page)

17. Online commands

Operation type	Command	Option	Meaning	Condition
2. Utility	INIT	PGM	Initialize data Program	3
		PNT	Point	
		SFT	Shift	
		HND	Hand	
		PLT	Pallet	
		PCM	Point comment	
		MEM	Above data	
		PRM	Parameter	
		ALL	MEM+PRM	
		CMU	Initialize data Communication parameter	3
		LOG	Initialize data Error log	
	LANGUAGE	k	Setting Display language	3
	UNIT	k	Setting Point units	3
	MSGCLR		Clear line message	1
	ACCESS	k	Setting Access level	3
	EXELVL	k	Setting Execution level	3
3. Data handling	?	SEQUENCE	Setting Sequence execution flag	3
		ARMTYP	Setting Main hand system	3
		ARMTYP2	Sub hand system	
		EMGRST	Reset internal emergency stop flag	1
		DATE	Check or set date	2
		TIME	Check or set time	2
		LANGUAGE	Acquiring status Display language	1
		ACCESS	Acquiring status Access level	
		ARM	Acquiring status Arm status	
		BREAK	Acquiring status Break point status	
		CONFIG	Acquiring status Controller configuration	
		EXELVL	Acquiring status Execution level	
		MOD	Acquiring status Mode status	
		MSG [m, n]	Acquiring status Error message	
		ORIGIN	Acquiring status Return-to-origin status	
		ABSRST	Acquiring status Absolute reset status	
		SERVO	Acquiring status Servo status	
		SEQUENCE	Acquiring status Sequence execution flag status	
		SPEED	Acquiring status AUTO/MANUAL speed status	
		UNIT	Acquiring status Point unit status	
		VER	Acquiring status Version information	
		WHERE	Acquiring status Current main robot position (pulse coordinate)	
		WHERE2	Current sub robot position (pulse coordinate)	
		WHRXY	Acquiring status Current main robot position (XY coordinate)	
		WHRXY2	Current sub robot position (XY coordinate)	
		TASKS	Acquiring status Task number	
		TSKMON	Acquiring status Task operation status	
		SHIFT	Acquiring status Selected shift status	
		HAND	Acquiring status Selected hand status	
		MEM	Acquiring status Remaining memory capacity	
		EMG	Acquiring status Emergency stop status	
		SELFCHK	Acquiring status Error status by self-diagnosis	
		OPSLLOT	Acquiring status Option slot status	
		Numerical expression	Acquiring status Numerical data	
		Character string expression	Acquiring status Character string data	
		Point expression	Acquiring status Point data	
		Shift expression	Acquiring status Shift data	
	READ	Data file (PGM, PNT, ...)	Data readout	2
	WRITE	Data file (PGM, PNT, ...)	Data write	2
4. Robot language independent execution	SWI	<program>	Program switching	4
	Robot language executable independently.			4
5. Control code	^C(=03H)		Execution language interruption	1

Conditions:

1. Always executable.
2. Executable except when MPB and RPB is used for input/output operation.
3. Executable except when in condition 2 or program is being executed.
4. Executable except for condition 3 or specific limitation.

Online command description

17.1 Key operation

**NOTE**

Basically, a response “OK” appears when an instruction from key operation online command is received.
An error message responds if online commands cannot be executed due to error.

17.1.1 Changing the mode

Command format:

```
@AUTO[cr/lf]
@PROGRAM[cr/lf]
@MANUAL[cr/lf]
@SYSTEM[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Changes the mode.

AUTO	: Changes to AUTO mode.
PROGRAM	: Changes to PROGRAM mode.
MANUAL	: Changes to MANUAL mode.
SYSTEM	: Changes to SYSTEM mode.

Example:

Command	@AUTO[cr/lf]
Response	OK[cr/lf]

17.1.2 AUTO mode operation

17.1.2.1 Program execution

Command format:

```
@RESET[cr/lf]
@RUN[cr/lf]
@STEP[cr/lf]
@SKIP[cr/lf]
@NEXT[cr/lf]
@STOP[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Executes or stops the current program.

RESET	: Resets the program.
RUN	: Executes the program.
STEP	: Executes one line of the program. (Enters the subroutine.)
SKIP	: Skips one line of program. (Skips one line of the subroutine.)
NEXT	: Executes to the next line. (Executes the subroutine as one line.)
STOP	: Stops the program.



NOTE

Programs can be executed only in AUTO mode.

Example:

Command	@RUN[cr/lf]
Response	OK[cr/lf]

17.1.2.2 Setting a break point

Command format:

@BREAK m,n[cr/lf]

m : Break point number (m = 1 to 4)

n : Line number to set a break point. (n = 1 to 9999)

When setting a break point in the COMMON program, the line number should be +10000.

Break point is cleared when 0 is specified as the line number.

Response format:

OK[cr/lf]



NOTE

Break points can be set only in AUTO mode.

Meaning:

- Sets a break point used to temporarily stop execution of the program.

Example:

Command	@BREAK 1, 28[cr/lf]
Response	OK[cr/lf]

17.1.2.3 Switching the execution task

Command format:

```
@CHGTSK[cr/lf]
```

Response format:

```
OK[cr/lf]
```



NOTE

Tasks can be switched only in AUTO mode.

Meaning:

- Switches the selected task while program execution is stopped.
- The ongoing task is switched to another task not being executed in order from task 1 -> 2 -> ... -> 8 -> 1.

Example:

Command	@CHGTSK[cr/lf]
Response	OK[cr/lf]

17.1.3 MANUAL mode operation

17.1.3.1 Changing the MANUAL mode speed

Command format:

```
@MSPEED k[cr/lf]
@MSPEED2 k[cr/lf]
```

k : Manual movement speed (k = 1 to 100)

Response format:

```
OK[cr/lf]
```



NOTE

The MANUAL mode speed can be changed only in MANUAL mode.

Meaning:

- Changes the MANUAL mode movement speed.

MSPEED	: Changes manual movement speed for main robot.
MSPEED2	: Changes manual movement speed for sub robot.

Example:

Command	@MSPEED 50[cr/lf]
Response	OK[cr/lf]

17.1.3.2 Absolute reset

Command format:

```
@ABSADJ k,f[cr/lf]
@ABSADJ2 k,f[cr/lf]
@ABSRESET k[cr/lf]
@ABSRESET2 k[cr/lf]
```

k : Designated axis (k = 1 to 6)

f : Movement direction (f = 0: + direction, 1: - direction)

Response format:

```
OK[cr/lf]
```



NOTE

Absolute reset can be performed only in MANUAL mode.

Meaning:

- Performs absolute reset.

ABSADJ : Moves the specified main robot axis to an absolute reset position.
ABSADJ2 : Moves the specified sub robot axis to an absolute reset position.
ABSRESET : Performs absolute reset on the specified main robot axis.
ABSRESET2 : Performs absolute reset on the specified sub robot axis.

Example:

Command	@ABSADJ 1,0[cr/lf]
Response	OK[cr/lf]

17.1.3.3 Return-to-origin operation

Command format:

```
@ORGRTN k[cr/lf]
@ORGRTN2 k[cr/lf]
```

k : Specified axis (k = 1 to 6)

Response format:

```
OK[cr/lf]
```



NOTE

This command can be executed only in MANUAL mode.

Meaning:

- Performs return-to-origin on the specified axis.
- Performs return-to-origin on an incremental mode axis when return-to-origin is executed.
- Performs absolute search on a semi-absolute mode axis when return-to-origin is executed.

ORGRTN : Performs return-to-origin on the specified main robot axis.
ORGRTN2 : Performs return-to-origin on the specified sub robot axis.

Example:

Command	@ORGRTN 1[cr/lf]
Response	OK[cr/lf]

17.1.3.4 Manual movement (inching)

Command format:

```
@INCH km[cr/lf]
@INCH2 km[cr/lf]
```

k : Specified axis (k = X, Y, Z, R, A, or B; any one of these)
m : Direction (m = + or -)

Response format:

```
OK[cr/lf]
```



NOTE

This command can be executed only in MANUAL mode.

Meaning:

- Manually moves (inching motion) the specified axis. The robot performs the same motion as when moved manually in inching mode with the MPB/RPB jog keys (moves a fixed distance each time a jog key is pressed).

INCH : Moves the specified main robot axis in inching mode.
INCH2 : Moves the specified sub robot axis in inching mode.



NOTE

Response is transmitted after movement is complete.

Example:

Command	@INCH X+[cr/lf]
Response	OK[cr/lf]

17.1.3.5 Manual movement (jog)

Command format:

```
@JOG km[cr/lf]
@JOG2 km[cr/lf]
```

k : Specified axis (k = X, Y, Z, R, A, or B; any one of these)
m : Direction (m = + or -)

Response format:

```
OK[cr/lf]
```



NOTE

This command can be executed only in MANUAL mode.

Meaning:

- Manually moves (jog motion) the specified axis. The robot performs the same motion as when holding down the MPB/RPB jog keys in manual mode.

After the robot has started moving, it will stop when any of the following occurs.

- When software limit was reached.
- When interlock signal was turned off.
- When STOP key on the MPB/RPB was pressed.
- When an online command (^C (=03H)) to interrupt execution was input.

JOG : Moves the specified main robot axis in jog mode.
JOG2 : Moves the specified sub robot axis in jog mode.



NOTE

Response is transmitted after movement is complete.

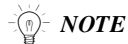
Example:

Command	@JOG X+[cr/lf]
Response	OK[cr/lff]

17.1.3.6 Point data teaching

Command format:

```
@TEACH mmmm[cr/lf]
@TEACH2 mmmm[cr/lf]
```



NOTE

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified.

mmmm : Point number for registering point data (mmmm = 0 to 9999)

Response format:

```
OK[cr/lf]
```



NOTE

This command can be executed only in MANUAL mode.

Meaning:

- Registers the current robot position as point data for the specified point number. If point data is already registered in the specified point number, then that point data will be overwritten. Point data is registered in the same format as the currently selected unit system.

TEACH : Registers the current position of the main group as point data for the specified point number.

TEACH2 : Registers the current position of the sub group as point data for the specified point number.

Example:

Command @TEACH 100[cr/lf]

Response OK[cr/lf]

17.2 Utility operation

17.2.1 Acquiring the program execution status

Command format:

```
@PADDR[cr/lf]
```

Response format:

```
<program name>, Tn,m,k[cr/lf]
```

Program name : Currently selected program name

Tn: Current task number (n = 1 to 8)

m : Current program line number (m = 1 to 9999)

k : Current task priority level (k = 17 to 47)

If the COMMON program is used, the response format might become as follows.

```
<COMMON>/<program name>, Tn,m,k[cr/lf]
```

Meaning:

- Acquires the current program execution status.

Example:

Command	@PADDR[cr/lf]
Response	<TEST>,T3,134,32[cr/lf]



NOTE

The current program execution status can be acquired only when the program is stopped during AUTO mode.

17.2.2 Copy

17.2.2.1 Copying a program

Command format:

```
@COPY <program name 1> TO <program name 2>[cr/lf]
```

Program name 1 : Program name in copy source (8 characters or less consisting of alphanumeric characters and underscore)

Program name 2 : Program name in copy destination (8 characters or less consisting of alphanumeric characters and underscore)

Response format:

```
OK[cr/lf]
```

Meaning:

- Copies the contents of program name 1 under program name 2.

Example:

Command	@COPY <TEST1> TO <TEST2>[cr/lf]
Response	OK[cr/lf]

17.2.2.2 Copying point data

Command format:

```
@COPY Pmmmm-Pnnnn TO Pkkkk[cr/lf]
```

m m m m : Top point number in copy source (m m m m = 0 to 9999)
n n n n : Last point number in copy source (n n n n = 0 to 9999)
k k k k : Top point number in copy destination (k k k k = 0 to 9999)

Response format:

```
OK[cr/lf]
```

Meaning:

- Copies the point data between Pmmmm and Pnnnn to Pkkkk and subsequent point numbers.

Example:

Command	@COPY P101-P200 TO P1101[cr/lf]
Response	OK[cr/lf]

17.2.2.3 Copying point comments

Command format:

```
@COPY PCmmmm-PCnnnn TO PCkkkk[cr/lf]
```

m m m m : Top point comment number in copy source (m m m m = 0 to 9999)
n n n n : Last point comment number in copy source (n n n n = 0 to 9999)
k k k k : Top point comment number in copy destination (k k k k = 0 to 9999)

Response format:

```
OK[cr/lf]
```

Meaning:

- Copies the point comments between PCmmmm and PCnnnn to PCkkkk and subsequent point comment numbers.

Example:

Command	@COPY PC101-PC200 TO PC1101[cr/lf]
Response	OK[cr/lf]



CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified as the point copy source and destination.



CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified as the point comment copy source and destination.

17.2.3 Erase

17.2.3.1 Erasing a program

Command format:

```
@ERA <program name>[cr/lf]
```

Program name : Program name to be erased (8 characters or less consisting of alpha-numeric characters and underscore)

Response format:

```
OK[cr/lf]
```

Meaning:

- Erases the designated program.

Example:

Command	@ERA <TEST1>[cr/lf]
Response	OK[cr/lf]

17.2.3.2 Erasing point data

Command format:

```
@ERA Pmmmm-Pnnnn[cr/lf]
```

mmmm : Top point number to be erased (mmmm = 0 to 9999)

nnnn : Last point number to be erased (nnnn = 0 to 9999)

Response format:

```
OK[cr/lf]
```

Meaning:

- Erases the point data between Pmmmm and Pnnnn.

Example:

Command	@ERA P101-P200[cr/lf]
Response	OK[cr/lf]



CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be used to specify the top and last point numbers of the point range to be erased.

17.2.3.3 Erasing point comments

Command format:

```
@ERA PCmmmm-PCnnnn[cr/lf]
```

mmmm : Top point comment number to be erased (mmmm = 0 to 9999)

nnnn : Last point comment number to be erased (nnnn = 0 to 9999)

Response format:

```
OK[cr/lf]
```

Meaning:

- Erases the point comments between PCmmmm and PCnnnn.

Example:

Command @ERA PC101-PC200[cr/lf]

Response OK[cr/lf]

17.2.3.4 Erasing pallet data

Command format:

```
@ERA PLm[cr/lf]
```

m : Pallet number to be erased (m = 0 to 19)

Response format:

```
OK[cr/lf]
```

Meaning:

- Erases the designated pallet data.

Example:

Command @ERA PL1[cr/lf]

Response OK[cr/lf]

CAUTION

In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be used to specify the top and last point numbers of the point comment range to be erased.

17.2.4 Rename

Command format:

```
@REN <program name 1> TO <program name 2>[cr/lf]
```

Program name 1 : Program name before renaming (8 characters or less consisting of alphanumeric characters and underscore)

Program name 2 : Program name after renaming (8 characters or less consisting of alphanumeric characters and underscore)

Response format:

```
OK[cr/lf]
```

Meaning:

- Changes program name 1 to program name 2.

Example:

Command @REN <TEST1> TO <TEST2>[cr/lf]

Response OK[cr/lf]

17.2.5 Changing the attribute

Command format:

```
@ATTR <program name> TO s[cr/lf]
```

Program name : Program name to change the attribute (8 characters or less consisting of alphanumeric characters and underscore)

s: Attribute (s = RW: read & write, RO: read only)

Response format:

```
OK[cr/lf]
```

Meaning:

- Changes the attribute of the designated program.

Example:

Command @ATTR <TEST1> TO RO[cr/lf]

Response OK[cr/lf]

17.2.6 Initialize

17.2.6.1 Initializing the memory

Command format:

```
@INIT PGM[cr/lf]
@INIT PNT[cr/lf]
@INIT SFT[cr/lf]
@INIT HND[cr/lf]
@INIT PLT[cr/lf]
@INIT PCM[cr/lf]
@INIT MEM[cr/lf]
@INIT PRM[cr/lf]
@INIT ALL[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Initializes the memory.

PGM : Initializes the program area.
PNT : Initializes the point data area.
SFT : Initializes the shift data area.
HND : Initializes the hand data area.
PLT : Initializes the pallet data area.
PCM : Initializes the point comment area.
MEM : Initializes the above areas (PGM+PNT+SFT+HND+PLT+PCM).
PRM : Initializes the parameter area.
ALL : Initializes all areas (MEM+PRM).

Example:

Command	@INIT PGM[cr/lf]
Response	OK[cr/lf]

17.2.6.2 Initializing the communication port

Command format:

```
@INIT CMU[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Initializes the communication port parameters.
- For information about the communication port initial settings, refer to the communication port manual.

Example:

Command	@INIT CMU[cr/lf]
Response	OK[cr/lf]

17.2.6.3 Initializing the error log

Command format:

```
@INIT LOG[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Initializes the error log.

Example:

Command	@INIT LOG[cr/lf]
Response	OK[cr/lf]

17.2.7 Setting the display language

Command format:

```
@LANGUAGE k[cr/lf]
```

k : Display language (k = 0: Japanese, 1: English)

Response format:

```
OK[cr/lf]
```

Meaning:

- Sets the controller display language.

Example:

Command	@ LANGUAGE 1[cr/lf]
Response	OK[cr/lf]

17.2.8 Setting the coordinates and units in MANUAL mode

Command format:

```
@UNIT k[cr/lf]
```

k : Unit definition (k=0 : pulses, 1 : mm, 2 : mm or degrees in tool coordinate mode)

Response format:

```
OK[cr/lf]
```

Meaning:

- Select the display unit to indicate current position.
- k=2 (tool coordinate mode) can be selected only when the hand attached to the R-axis of a Cartesian or SCARA robot is selected.

Example:

Command	@UNIT 1[cr/lf]
Response	OK[cr/lf]

17.2.9 Clearing the MPB/RPB error message

Command format:

```
@MSGCLR[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Clears the error messages displayed on the MPB or RPB.

Example:

Command	@MSGCLR[cr/lf]
Response	OK[cr/lf]

 **CAUTION**
k=2 (mm or degree units in tool coordinate mode) is only available from software version 8.19 onwards.

17.2.10 Setting the UTILITY mode

17.2.10.1 Setting the access level

Command format:

```
@ ACCESS k[cr/lf]
```

k : Access level (k = 0 to 3)

Response format:

```
OK[cr/lf]
```



NOTE

For a detailed description of the access level, refer to the robot controller user's manual.

Meaning:

- Sets the access level.

Example:

Command	@ACCESS 1[cr/lf]
Response	OK[cr/lf]

17.2.10.2 Setting the execution level

Command format:

```
@EXELVL k[cr/lf]
```

k : Execution level (k = 0 to 8)

Response format:

```
OK[cr/lf]
```



NOTE

For a detailed description of the execution level, refer to the robot controller user's manual.

Meaning:

- Sets the execution level.

Example:

Command	@ EXELVL 1[cr/lf]
Response	OK[cr/lf]

17.2.10.3 Setting the sequence program execution flag

Command format:

```
@SEQUENCE k[cr/lf]
```

k : Execution flag (k = 0: disable, 1: enable, 3: enable (DO reset))

Response format:

```
OK[cr/lf]
```

Meaning:

- Sets the sequence program execution flag.

Example:

Command	@ SEQUENCE 1[cr/lf]
Response	OK[cr/lf]

17.2.10.4 Setting the SCARA robot hand system

Command format:

```
@ARMTYP m,k[cr/lf]  
@ARMTYP2 m,k[cr/lf]
```

m : Current hand system (m = 0: right-handed system, 1: left-handed system)

k : Hand system at program reset (k = 0: right-handed system, 1: left-handed system)

Response format:

```
OK[cr/lf]
```

Meaning:

- Sets the SCARA robot hand system.
- | | |
|---------|---------------------------------------|
| ARMTYP | : Changes the main robot hand system. |
| ARMTYP2 | : Changes the sub robot hand system. |

Example:

Command	@ ARMTYP 0, 0 [cr/lf]
Response	OK[cr/lf]

17.2.10.5 Resetting the internal emergency stop flag

Command format:

```
@EMGRST[cr/lf]
```

Response format:

```
OK[cr/lf]
```

Meaning:

- Resets the internal emergency stop flag.

Example:

Command	@ EMGRST[cr/lf]
Response	OK[cr/lf]

17.2.11 Checking and setting the date

Command format:

```
@DATE[cr/lf]
```

Response format:

```
current date: yy/mm/dd[cr/lf]
enter new date: (YY/MM/DD)[cr/lf]
```

yy/mm/dd : Current date (year, month, day)
 yy : Lower 2 digits of the year (00 to 99)
 mm : Month (01 to 12)
 dd : Day (01 to 31)

Transmission format:

```
aa/bb/cc[cr/lf]
```

aa/bb/cc : Date to be set. (year/month/day)
 aa : Lower 2 digits of the year (00 to 99) *This can be omitted.
 bb : Month (01 to 12) *This can be omitted.
 cc : Day (01 to 31) *This can be omitted.

* The currently set values are used for the omitted items.

<Example> To change only the day, enter as follows:

```
//15[cr/lf] ..... Day is set to 15th.
```

If only [cr/lf] is transmitted, then the date remains unchanged.

Response format:

```
OK[cr/lf] or "error message" [cr/lf]
```

Explanation:

- Acquires the current date in the controller and sets a new date.

Example:

Command	@DATE[cr/lf]
Response	current date: 07/05/10[cr/lf] enter new date: (YY/MM/DD)[cr/lf]
Transmission	07/05/11[cr/lf]
Response	OK[cr/lf]



NOTE

The date can be checked and set only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards



CAUTION

If an improbable date is entered, then "5.2: Data error" occurs.



NOTE

To change only the year or month, the slash (/) following it can be omitted.

Example:

To set the year to 2007, enter 07[cr/lf].

To set the month to June, enter /06[cr/lf].

17.2.12 Checking and setting the time



NOTE

The time can be checked and set only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

Command format:

```
@TIME[cr/lf]
```

Response format:

```
current time: hh:mm:ss[cr/lf]
enter new time: (HH:MM:SS)[cr/lf]
```

hh:mm:ss : Current time
 hh : hour (00 to 23)
 mm : minute (00 to 59)
 ss : second (00 to 59)

Transmission format:

```
aa:bb:cc[cr/lf]
```



CAUTION

If an improbable time is entered, then "5.2: Data error" occurs.



NOTE

To change only the hour or minute, the slash (/) following it can be omitted.
 Example:

To set the hour to 13, enter 13[cr/lf].

To set the minute to 35, enter /35[cr/lf].

aa:bb:cc : Time to be set.
 aa : hour (00 to 23) *This can be omitted.
 bb : minute (00 to 59) *This can be omitted.
 cc : second (00 to 59) *This can be omitted.
 * The currently set values are used for the omitted items.
 <Example> To change only the minute, enter as follows
 :20:[cr/lf] Minute is set to 20 minutes.
 If only [cr/lf] is transmitted, then the time remains unchanged.

Response format:

```
OK[cr/lf] or "error message" [cr/lf]
```

Meaning:

- Acquires the current time in the controller and sets a new time.

Example:

Command	@TIME[cr/lf]
Response	current time: 10:21:35 [cr/lf] enter new time: (HH:MM:SS)[cr/lf]
Transmission	10:25:00[cr/lf]
Response	OK[cr/lf]

17.3 Data handling

17.3.1 Acquiring the display language

Command format:

```
@?LANGUAGE[cr/lf]
```

Response format:

```
m[cr/lf]
```

m : Display language (m = JAPANESE or ENGLISH)

Meaning:

- Acquires the language for displaying messages on the MPB.

Example:

Command @?LANGUAGE[cr/lf]

Response JAPANESE[cr/lf]

17.3.2 Acquiring the access level

Command format:

```
@?ACCESS[cr/lf]
```

Response format:

```
LEVELk[cr/lf]
```

k : Access level (k = 0 to 3)

Meaning:

- Acquires the access level.

Example:

Command @?ACCESS[cr/lf]

Response LEVEL2[cr/lf]



NOTE

For a detailed description of the access level, refer to the robot controller user's manual.

17.3.3 Acquiring the arm status

Command format:

```
@?ARM[cr/lf]
```

Response format:

```
m1/s1,m2/s2[cr/lf]
```

Main robot

m1 : Current arm setting status (m1 = RIGHTY: right-handed system, LEFTY: left-handed system)

m2 : Arm setting status at program reset (m2 = RIGHTY: right-handed system, LEFTY: left-handed system)

Sub robot

s1 : Current arm setting status (s1 = RIGHTY: right-handed system, LEFTY: left-handed system)

s2 : Arm setting status at program reset (s2 = RIGHTY: right-handed system, LEFTY: left-handed system)



NOTE

- This command is valid only for SCARA robot setting.
- “s1” and “s2” are not displayed when sub robot is not set.

Meaning:

- Acquires the arm setting status.

Example:

Command	@?ARM[cr/lf]
Response	RIGHTY,RIGHTY[cr/lf]

17.3.4 Acquiring the break point status

Command format:

```
@?BREAK[cr/lf]
```

Response format:

```
k1,k2,k3,k4[cr/lf]
```

kn: Line number on which break point “n” is set (kn = 1 to 9999)

- When kn is 0, this means no break point is set.
- When a break point is set in the COMMON program, the line number shows +10000.

Meaning:

- Acquires the break point status.

Example:

Command	@?BREAK[cr/lf]
Response	12,35,0,0[cr/lf]

17.3.5 Acquiring the controller configuration status

Command format:

```
@?CONFIG[cr/lf]
```

Response format:

```
mr/sr-ma/sa-r-o1-o2[cr/lf]
```

mr : Main robot name

sr : Sub robot name

ma : Main group axis setting (Auxiliary axes are shown separated by "+".)

sa : Sub group axis setting (Auxiliary axes are shown separated by "+".)

r : Standard interface unit

o1 : Option unit

o2 : Other setting



NOTE

"sr" and "sa" are not displayed when sub robot is not set.

Meaning:

- Acquires the controller configuration status.

Example:

Command @?CONFIG[cr/lf]

Response YK250X-XYZR-SRAM/196kB,DIO_N-DIO_N(1/2)[cr/lf]

17.3.6 Acquiring the execution level

Command format:

```
@?EXELVL[cr/lf]
```

Response format:

```
LEVELk[cr/lf]
```

k : Execution level (k = 0 to 8)



NOTE

For a detailed description of the execution level, refer to the robot controller user's manual.

Meaning:

- Acquires the execution level.

Example:

Command @?EXELVL[cr/lf]

Response LEVEL2[cr/lf]

17.3.7 Acquiring the mode status

Command format:

```
@?MOD[cr/lf]
```

Response format:

```
s[cr/lf]
```

s : Mode status

s		Meaning
English	Japanese	
AUTO	ｼﾞﾄﾞｳ	AUTO mode
PROGRAM	ﾌﾟﾛｸﾞﾗﾑ	PROGRAM mode
MANUAL	ﾏﾆｭｱﾙ	MANUAL mode
SYSTEM	ｼｽﾃﾑ	SYSTEM mode

Meaning:

- Acquires the controller mode status.

Example:

Command @?MOD[cr/lf]

Response AUTO[cr/lf]

17.3.8 Acquiring the message

Command format 1:

```
@?MSG[cr/lf]
```

Response format 1:

```
gg,bb:msg[cr/lf]
or
OK[cr/lf]
```

gg : Error group

bb : Error category

msg : Error message

Command format 2:

```
@?MSG m,n[cr/lf]
```

m : Top number to be acquired (m = 1 to 500)

n : Last number to be acquired (n = 1 to 500)

Response format 2:

```
yy/mm/dd,hh:mm:ss gg.bb:msg[cr/lf]
yy/mm/dd,hh:mm:ss gg.bb:msg[cr/lf]
:
OK[cr/lf]
```

yy/mm/dd : Date (year/month/day) when error occurred
 hh:mm:ss : Time (hour:minute:second) when error occurred
 gg : Error group
 bb : Error category
 msg : Error message

Meaning:

- Command format 1 acquires information on the message line displayed on the MPB or RPB.
- Command format 2 acquires error history message.

Example:

Command	@?MSG[cr/lf]
Response	5.30: Undefined identifier[cr/lf] or OK[cr/lf]

Example:

Command	@?MSG 1,5[cr/lf]
Response	01/10/28,14:20:20 5.30: Undefined identifier[cr/lf] 01/10/28,14:18:34 5.1: Syntax error[cr/lf] 01/10/28,14:10:54 5.30: Undefined identifier[cr/lf] 01/10/28,14:05:40 14.22: No start code(@)[cr/lf] 01/10/28,14:05:00 5.52: Command doesn't exist[cr/lf] OK[cr/lf]

17.3.9 Acquiring return-to-origin status

Command format:

```
@?ORIGIN[cr/lf]
```

Response format:

```
COMPLETE[cr/lf] or INCOMPLETE[cr/lf]
```

COMPLETE : Return-to-origin is complete.
 INCOMPLETE : Return-to-origin is incomplete.

Meaning:

- Acquires return-to-origin status.

Example:

Command	@?ORIGIN[cr/lf]
Response	COMPLETE[cr/lf]

17.3.10 Acquiring the absolute reset status

Command format:

```
@?ABSRST[cr/lf]
```

Response format:

```
COMPLETE[cr/lf] or INCOMPLETE,xxxxxxx[cr/lf]
```

COMPLETE : Absolute reset is complete.

INCOMPLETE : Absolute reset is incomplete.

xxxxxxx : Absolute reset status of each axis (axis 1 to axis 8 from the right)

“0”: Incomplete

“1”: Complete

“9”: Not applicable

Meaning:

- Acquires the absolute reset status.

Example:

Command	@?ABSRST[cr/lf]
Response	INCOMPLETE,99991011[cr/lf]

17.3.11 Acquiring the servo status

Command format:

```
@?SERVO[cr/lf]
```

Response format:

```
OFF,xxxxxxx [cr/lf] or ON,xxxxxxx[cr/lf]
```

- Response outputs are defined as follows:

ON : Motor power is ON.

OFF : Motor power is OFF.

xxxxxxx : Status of each axis (axis 1 to axis 8 from the right)

“0”: Mechanical break ON + dynamic break ON

“1”: Servo ON

“2”: Mechanical break OFF + dynamic break OFF

“9”: Not applicable

Meaning:

- Acquires the servo status.

Example:

Command	@?SERVO[cr/lf]
Response	OFF,99991011[cr/lf]

17.3.12 Acquiring the sequence program execution status

Command format:

```
@?SEQUENCE[cr/lf]
```

Response format:

```
ENABLE,s[cr/lf] or ENABLE(RST.DO),s[cr/lf] or DISABLE[cr/lf]
```

- Response output means as follows:

ENABLE : Enabled

ENABLE(RST.DO): Enabled and output is cleared at emergency stop

DISABLE : Disabled

s : Program execution is in progress or stopped. (s = RUNNING or STOP)

Meaning:

- Acquires the sequence program execution status.

Example:

Command @? SEQUENCE[cr/lf]

Response DISABLE[cr/lf]

17.3.13 Acquiring the speed setting status

Command format:

```
@?SPEED[cr/lf]
```

Response format:

```
ma/sa,mm/sm[cr/lf]
```

Main group

ma : Automatic movement speed setting status (ma = 1 to 100)

mm : Manual movement speed setting status (mm = 1 to 100)

Sub group

sa : Automatic movement speed setting status (sa = 1 to 100)

sm : Manual movement speed setting status (sm = 1 to 100)



NOTE

"sa" and "sm" are not displayed unless the sub robot is set.

Meaning:

- Acquires the speed setting status.

Example:

Command @?SPEED[cr/lf]

Response 100,50[cr/lf]

17.3.14 Acquiring the point coordinates and units

Command format:

```
@?UNIT[cr/lf]
```

Response format:

```
s [cr/lf]
```

s : Coordinates and units (s = PULSE: joint coordinate in “pulse” units, MM: Cartesian coordinate in “mm” or “deg.” units)

Meaning:

- Acquires the coordinates and units for point data.

Example:

Command	@?UNIT[cr/lf]
Response	PULSE[cr/lf]

17.3.15 Acquiring the version information

Command format:

```
@?VER[cr/lf]
```

Response format:

```
cv,cr-mv-d1/d2/d3/d4/d5/d6/d7/d8{-ov}[cr/lf]
```

- Response outputs are defined as follows:

cv	: Host version number (RCX14x: V8.xx, RCX22x: V9.xx, RCX240: V10.xx)
cr	: Host revision number (Rxxxx)
mv	: MPB/RPB version number (V5.xx)
d?(?:1-8)	: Driver version number (Vx.xx)
ov	: Option unit version number (Vx.xx)

Meaning:

- Acquires the version information.

Example:

Command	@?VER[cr/lf]
Response	V8.02,R1021-V5.10-V1.01/V1.01/V1.01/V1.01/-----/-----/-----/-----/[cr/lf]

17.3.16 Acquiring the current positions

17.3.16.1 Acquiring the current positions on pulse unit coordinates

Command format:

```
@?WHERE[cr/lf]
@?WHERE2[cr/lf]
```

Response format:

```
[POS]xxxxxx yyyyyy zzzzzz rrrrrr aaaaaa bbbbbb[cr/lf]
```

xxxxxx : Current position of axis 1 in “pulse” units

yyyyyy : Current position of axis 2 in “pulse” units

:

bbbbbb : Current position of axis 6 in “pulse” units

Meaning:

- Acquires the current positions.

WHERE : Acquires the current positions of main group axes.

WHERE2 : Acquires the current positions of sub group axes.

Example:

Command @?WHERE[cr/lf]

Response [POS] 1000 2000 3000 -40000 0 0[cr/lf]

17.3.16.2 Acquiring the current positions on XY coordinates

Command format:

```
@?WHRXY[cr/lf]
@?WHRXY2[cr/lf]
```

Response format:

```
[POS] xxxxxx yyyyyy zzzzzz rrrrrr aaaaaa bbbbbb[cr/lf]
```

xxxxxx : Current position of axis 1 in “mm” units

yyyyyy : Current position of axis 2 in “mm” units

:

bbbbbb : Current position of axis 6 in “mm” units

Meaning:

- Acquires the current positions.

WHRXY : Acquires the current positions of main group axes.

WHRXY2 : Acquires the current positions of sub group axes.

Example:

Command @?WHRXY[cr/lf]

Response [POS] 100.00 200.00 300.00 -40.00 0.00 0.00[cr/lf]



NOTE

WHERE2 cannot be used unless the sub robot is set.



NOTE

WHRXY2 cannot be used unless the sub robot is set.

17.3.17 Acquiring the tasks in RUN or SUSPEND status

Command format:

```
@?TASKS[cr/lf]
```

Response format:

```
n{n,{...}}[cr/lf]
```

n : Task number currently run or suspended (n = 1 to 8)

Meaning:

- Acquires the tasks in RUN or SUSPEND status.

Example:

Command	@?TASKS[cr/lf]
Response	1,3,4,6[cr/lf]

17.3.18 Acquiring the tasks operation status

Command format:

```
@?TSKMON[cr/lf]
```

Response format:

```
nfp,{nfp},{nfp},{nfp},{nfp},{nfp},{nfp},{nfp}[cr/lf]
```

n : Line number being executed in each task (n = 1 to 9999)

f : Status of each task (f = R: RUN, U: SUSPEND, S: STOP)

p : Priority level of each task (p = 17 to 47)

Meaning:

- Acquires the status of each task in order from Task 1 to Task 8.

Example:

Command	@?TSKMON[cr/lf]
Response	11R32,,43U32,,,,129R31,[cr/lf]

17.3.19 Acquiring the shift status

Command format:

```
@?SHIFT[cr/lf]
```

Response format:

```
m/s[cr/lf]
```

m : Shift number selected for main robot (m = 0 to 9)

s : Shift number selected for sub robot (s = 0 to 9)



NOTE

"s" is not displayed unless the sub robot is set.

Meaning:

- Acquires the shift status.

Example:

Command @?SHIFT[cr/lf]

Response 1[cr/lf]

17.3.20 Acquiring the hand status

Command format:

```
@?HAND[cr/lf]
```

Response format:

```
m/s[cr/lf]
```

m : Hand number selected for main robot (m = 0 to 3)

s : Hand number selected for sub robot (s = 4 to 7)



NOTE

"s" is not displayed unless the sub robot is set.

Meaning:

- Acquires the hand status.

Example:

Command @?HAND[cr/lf]

Response 1[cr/lf]

17.3.21 Acquiring the remaining memory capacity

Command format:

```
@?MEM[cr/lf]
```

Response format:

```
k/m[cr/lf]
```

k : Remaining source area (unit: bytes)

m : Remaining object area (unit: bytes)

Meaning:

- Acquires the remaining memory capacity.

Example:

Command	@?MEM[cr/lf]
Response	102543/1342[cr/lf]

17.3.22 Acquiring the emergency stop status

Command format:

```
@?EMG[cr/lf]
```

Response format:

```
k[cr/lf]
```

k : Emergency stop status (k = 0: normal operation, 1: emergency stop)

Meaning:

- Acquires the emergency stop status by checking the internal emergency stop flag.

Example:

Command	@?EMG[cr/lf]
Response	1[cr/lf]

17.3.23 Acquiring the error status by self-diagnosis

Command format:

```
@?SELFCHK[cr/lf]
```

Response format:

- If an error occurred

```
m.n: "message" [cr/lf]
:
END [cr/lf]
```

m : Shows error group
n : Shows error category
"message" : Show error message

- When no error was found

```
OK [cr/lf]
```

Meaning:

- Acquires the error status by self-diagnosis that checks for errors inside the controller.

Example:

Command	@?SELFCHK[cr/lf]
Response	12.1: Emg.stop on[cr/lf] END[cr/lf]

17.3.24 Acquiring the option slot status

Command format:

```
@?OPSLOT[cr/lf]
```

Response format:

```
OP.1 : "option board name" [cr/lf]
OP.2 : "option board name" [cr/lf]
OP.3 : "option board name" [cr/lf]
OP.4 : "option board name" [cr/lf]
```

“option board name” : Name of option board installed in the controller.

DIO_Nm : DIO board with NPN specifications (m: board ID)

DIO_Pm : DIO board with PNP specifications (m: board ID)

CCLnk : CC-Link compatible board

D_Net : DeviceNet compatible board

Profi : Profibus compatible board

E_Net : Ethernet compatible board

YCLnk : YC-Link compatible board

no board : No option board is installed.

illegal board : Incompatible board is installed.

Meaning:

- Acquires the option slot status by checking the option boards.

Example:

Command	@?OPSLOT[cr/lf]
Response	OP.1 : DIO_N2[cr/lf]
	OP.2 : DIO_N1[cr/lf]
	OP.3 : no board[cr/lf]
	OP.4 : CCLnk [cr/lf]

17.3.25 Acquiring various values

17.3.25.1 Acquiring the value of a numerical expression


NOTE

Numerical expression values can be acquired only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

Command format:

```
@? "numerical expression" [cr/lf]
```

Response format:

```
"numerical value" [cr/lf]
```

Explanation:

- Acquires the value of the specified numerical expression.
- The value of the numerical expression is returned in decimal integer or real number format.

Example 1:

```
Command    @?SQR(100*5)[cr/lf]
Response    2.23606E01[cr/lf]
```

Example 2:

```
Command    @?LOCX(WHERE)[cr/lf]
Response    102054[cr/lf]
```

17.3.25.2 Acquiring the value of a character string expression


NOTE

Character string expression values can be acquired only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

Command format:

```
@? "character string expression" [cr/lf]
```

Response format:

```
"character string expression" [cr/lf]
```

Meaning:

- Acquires the value (character string) of the specified character string expression.

Example:

```
If A$="ABC" and B$="DEF"
Command    @?A$+B$+"123"[cr/lf]
Response    ABCDEF123[cr/lf]
```



NOTE

Point expression values can be acquired only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

17.3.25.3 Acquiring the value of a point expression

Command format:

```
@? "point data expression" [cr/lf]
```

Response format:

```
"point data" [cr/lf]
```

Meaning:

- Acquires the value (point data) of the specified point expression.

Example:

Command	@?P1+WHRXY[cr/lf]
Response	10.41 -1.60 52.18 3.00 0.00 0.00 0 [cr/lf]

17.3.25.4 Acquiring the value of a shift expression

Command format:

```
@? "shift expression" [cr/lf]
```

Response format:

```
"shift data" [cr/lf]
```

Meaning:

- Acquires the value (shift data) of the specified shift expression.

Example:

Command	@?S1[cr/lf]
Response	25.00 12.60 10.00 0.00 [cr/lf]



NOTE

Shift expression values can be acquired only on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

17.3.26 Data readout processing

Command format:

```
@READ "readout file" [cr/lf]
```

"readout file": Designate a readout file name.

Response format:

- Response output depends on the designated readout file.

Meaning:

- Reads out the data from the designated file.
- Online commands that are input through the RS-232C port have the same meaning as the following command.

SEND <readout file> TO CMU

Commands via Ethernet have the same meaning as the following command.

SEND <readout file> TO ETH



NOTE

For more information about files, refer to the earlier section "Data file description".



NOTE

The point comment separate format (PCn) of readout files is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards

Type	Readout file name	Definition format	
		All	Separate file
User memory	All files	ALL	—
	Program	PGM	<bbbbbb>
	Point data	PNT	Pn
	Point comment	PCM	PCn
	Parameter	PRM	/ccccc/
	Shift definition	SFT	Sn
	Hand definition	HND	Hn
	Pallet definition	PLT	PLn
Variable, constant	Variable	VAR	ab...by
	Array variable	ARY	ab...by (x)
	Constant	—	"cc...c"
Status	Program directory	DIR	<<bbbbbb>>
	Parameter directory	DPM	—
	Machine reference	MRF	—
	Error history (log)	LOG	—
	Memory size	MEM	—
Device	DI port	DI()	DIn()
	DO port	DO()	DOOn()
	MO port	MO()	MOOn()
	TO port	TO()	TOOn()
	LO port	LO()	LOOn()
	SI port	SI()	SIn()
	SO port	SO()	SOOn()
	SIW port	SIW()	SIWn()
Others	SOW port	SOW()	SOWn()
	File end code	EOF	—

n: Number
a: Alphabetic character
b: Alphanumeric character or underscore (_)
c: Alphanumeric character or symbol
x: Expression
y: Format

Example:

Command @READ PGM[cr/lf]

Example:

Command @READ P100[cr/lf]

17.3.27 Data write processing

Command format:

```
@WRITE "write file" [cr/lf]
```

"write file" : Designate a write file name.

Response format:

- The following response appears asking you to enter the data to write.

```
*** Please enter ![cr/lf]
```

- The following response appears when writing to the designated file has been finished.

```
OK[cr/lf]
```



NOTE

For more information about files, refer to the earlier section "Data file description".

Meaning:

- Writes the data in the designated file.
- Online commands that are input through the RS-232C port have the same meaning as the following commands.

SEND CMU TO <write file>

Commands via Ethernet have the same meaning as the following command.

SEND ETH TO <write file>



NOTE

The point comment separate format (PCn) of write files is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards



CAUTION

On the DO port, MO port, TO port, LO port, SO port and SOW port, each entire port (DO(), MO(), etc.) cannot be specified as a write file. There are some separate files (DOn(), MOn(), etc.) that cannot be specified as a write file. For more details, refer to the earlier section "Data file description".

Type	Write file name	Definition format	
		All	Separate file
User memory	All files	ALL	—
	Program	PGM	<bbbbbbbbb>
	Point data	PNT	Pn
	Point comment	PCM	PCn
	Parameter	PRM	/ccccc/
	Shift definition	SFT	Sn
	Hand definition	HND	Hn
	Pallet definition	PLT	PLn
Variable, constant	Variable	VAR	ab...by
	Array variable	ARY	ab...by (x)
Device	DO port	—	DOn()
	MO port	—	MOn()
	TO port	—	TOn()
	LO port	—	LOn()
	SO port	—	SOn()
	SOW port	—	SOWn()

n: Number

a: Alphabetic character

b: Alphanumeric character or underscore (_)

c: Alphanumeric character or symbol

x: Expression

y: Format

Example:

Command @WRITE PRM[cr/lf]

Example:

Command @WRITE P100[cr/lf]

17.4 Executing the robot language independently

17.4.1 Switching the program

Command format:

```
@SWI <program name>[cr/lf]
```

Response format:

```
OK[cr/lf] or LINEx,m,n:"message"[cr/lf]
```

OK	: Program was switched correctly.
LINEx,m,n	: "message" Error occurred during compiling.
x	: Shows the line number on which error occurred.
m	: Shows error group
n	: Shows error category
"message"	: Show error message

Meaning:

- Switches the program.
 - In AUTO mode, the program that is switched to will be compiled.
 - In other modes, the program is only switched.
- However, when "SEQUENCE" program is designated, a sequence object is created.

Example: AUTO mode

Command	@SWI <TEST1>[cr/lf]
Response	Line2,5.39:Illegal identifier[cr/lf]

Example: Other modes

Command	@SWI <TEST1>[cr/lf]
Response	OK[cr/lf]

17.4.2 Other robot language command processing

Command format:

```
@"robot language"[cr/lf]
```

Response format:

```
OK[cr/lf] or "error message" or ***Aborted [cr/lf]
```

- Response output means as follows:

OK	: Command ended correctly.
"Error message"	: An error occurred.
***Aborted	: An interruption occurred.

Meaning:

- Robot language commands can be executed.
- Independently executable commands can only be executed.
- Command format depends on each command to be executed.

Example:

Command	@SET DO(20) [cr/lf]
Response	OK[cr/lf]

Example:

Command	@MOVE P,P100,S=20[cr/lf]
Response	OK[cr/lf]

17.5 Control codes

17.5.1 Interrupting the command execution

Command format:

```
^C (=03H)
```

Response format:

```
*** Aborted[cr/lf]
```

Meaning:

- Interrupts execution of the current command.

Example:

Command	@MOVE P,P100,S=20[cr/lf]
	^C
Response	*** Aborted[cr/lf]

18. IO commands

Using bit information (DI/DO port) for general-purpose input/output allows issuing commands directly from the PLC. It is now possible to execute commands such as the MOVE command that were impossible to execute up until now without using the robot program or RS-232C port.

18.1 IO command format

CAUTION

- IO commands are only available from software version 8.18 onwards.
- To use IO commands, the following parameters in SYSTEM mode must be set to "VALID" beforehand. For more details, refer to the robot controller user's manual.

RCX240

"IO cmd (DI05) on STD.DIO"
of other parameters

RCX22x

"IO cmd (DI05)" of option
board parameters

NOTE

General-purpose outputs DO26 and DO27 are used when executing an IO command. General-purpose outputs DO26 and DO27 (dedicated outputs DO16 and DO17 for RCX22x) are used when executing an IO command. Pay attention to this point when you are using them for other purposes.

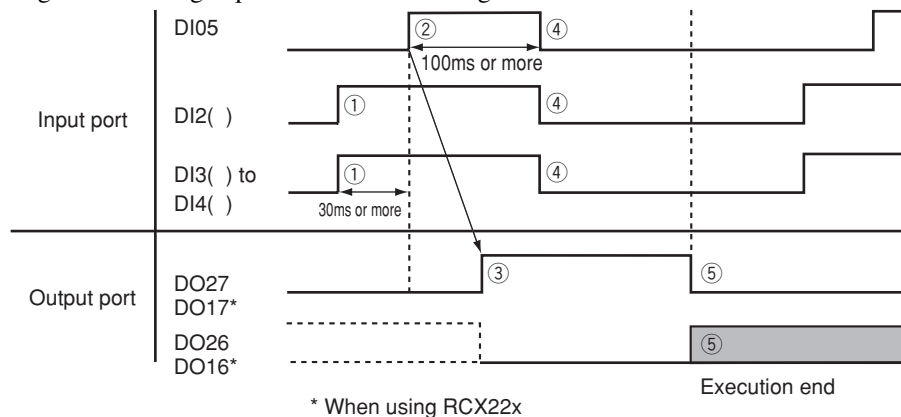
The following features are assigned to each IO.

Output (Controller → PLC)			Input (Controller ← PLC)	
	Output port	Contents	Input port	Contents
RCX240	DO26	Execution check output	DI05	IO command execution trigger input
	DO27	Execution in-progress output	DI2()	Command code
RCX22x	DO16	Execution check output	DI3()	Command data
	DO17	Execution in-progress output	DI4()	

- IO commands can be executed by using part of the general-purpose input and output. When no connection is made to the option DIO, then DI4() is always recognized as being OFF.
- IO commands cannot be executed while program execution is in progress (DO13 is ON).
- IO commands cannot be executed simultaneously with online commands.
- IO commands assign command codes to be executed to DI2(), and command data to DI3() and DI4(). These are executed when the DI05 is changed from OFF to ON. The controller processes the IO commands when they are received and sends execution check results and execution in-progress information to the PLC via DO26 and DO27 (DO16 and DO17 for RCX22x).
- Command data added to the IO commands will differ according to the IO command. See detailed information available on the IO commands. Command data settings must always be made before attempting to set the IO commands.
- Data is set in binary code. If the data size is greater than 8 bits, set the upper bit data into the higher address. (little endian)
For example, to set 0x0F9F [hexadecimal] (=3999) in the DI3 () and DI4 () ports, set 0x0F [hexadecimal] in DI4 () and set 0x9F [hexadecimal] in DI3 ().
- The IO command execution trigger input is not accepted when the execution in-progress output is ON.
- The execution in-progress output is ON in the following cases.
 - When an IO command is running after receiving an IO command execution trigger input.
 - When an IO command is terminated after receiving an IO command execution trigger input yet a maximum of 100ms state is maintained when IO command trigger input is ON.
- The IO command trigger input pulse must always be maintained for 100ms or more during input. Commands cannot be accepted unless this statement is maintained.
- Sometimes 20ms or more is needed for the execution in-progress output to turn ON after startup (rising edge) of the IO command trigger input pulse. The IO command trigger input might not be accepted during this period.
- After inputting the IO command trigger input pulse and the in-progress output turns OFF, at least a 100ms time period must always elapse before executing the next command. If this elapsed time period is too small, the IO command execution trigger input might not be accepted.
- The execution check output turns OFF when an IO command is received.
- The execution check output turns ON when an IO command ends normally, but it remains OFF if the IO command ends abnormally.

18.2 Sending and receiving IO commands

Sending and receiving is performed in the IO register as shown below.



CAUTION

To use IO commands, the following parameters in SYSTEM mode must be set to "VALID" beforehand. For more details, refer to the robot controller user's manual.

RCX240

"IO cmd (DI05) on STD.DIO" of other parameters

RCX22x

"IO cmd (DI05)" of option board parameters

CAUTION

Output port for the execution check output is as follows:

RCX240 : DO26

RCX22x : DO16

Output port for the execution in-progress output is as follows:

RCX240 : DO27

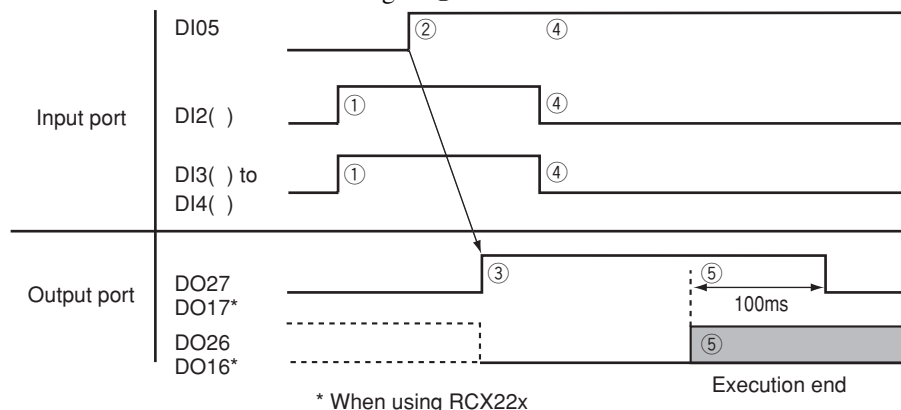
RCX22x : DO17

- ① Set command code and command data (Time interval between ① and ②: 30ms or more)
- ② Set IO command execution trigger input (Pulse width: 100ms or more)
- ③ Transition to execute
- ④ Clear the IO command execution trigger input and command code and command data
- ⑤ Set termination of IO command and execution check output

Example : Follow these steps when sending and receiving IO commands to execute the PTP movement command to point 19.

- ① Set the following values in the register to execute the PTP movement command by designating a point.
 - DI2 () : Command code (0x01)
 - DI3 () : Lower point setting (0x13= point 19)
 - DI4 () : Upper point setting (0x00=point 19)
- ② Set DI05 from ON to OFF.
- ③ The controller receives the IO command and executes it if the command and command data are acceptable. The execution in-progress output turns ON and the execution check output turns OFF at this time. The robot moves to the position specified by point 19.
- ④ Clear DI2 () through DI4 () after checking that execution in-progress output is ON.
- ⑤ The command has ended so the execution in-progress output turns OFF and execution check output turns ON if command ended correctly, and turns OFF if the command failed.

* If DI05 was not set to OFF in ④, the execution in-progress output remains ON for a maximum of 100ms from the timing in ⑤.



18.3 IO command list

IO commands are expressed with hexadecimal codes.



NOTE

In the MOVE command, linear interpolation (0x83) for sub robot is only available on the following software versions.

- RCX14x Ver. 8.64 onwards
- RCX22x Ver. 9.11 onwards



NOTE

On controllers whose software version is earlier than 8.45, the return-to-origin command is executed on all axes (main robot + sub robot) and command code 0x32 can only be use.

No.	Command contents		Command code (DI2())	
			Main robot	Sub robot
1	MOVE command	PTP designation	0x01	0x81
		Linear interpolation	0x03	0x83
2	MOVEI command	PTP designation	0x09	0x89
3	Pallet movement command	PTP designation at pallet 0	0x18	0x98
4	Jog movement command		0x20	0xA0
5	Inching movement command		0x24	0xA4
6	Point teaching command		0x28	0xA8
7	Absolute reset movement command		0x30	0xB0
8	Absolute reset command		0x31	0xB1
9	Return-to-origin command		0x32	0xB2
10	Servo command	On	0x34	0xB4
		Off	0x35	0xB5
		Free	0x36	0xB6
		Power-on	0x37	
11	Manual movement speed change command		0x38	0xB8
12	Auto movement speed change command		0x39	0xB9
13	Program speed change command		0x3A	0xBA
14	Shift designation change command		0x3B	0xBB
15	Hand designation change command		0x3C	0xBC
16	Arm designation change command		0x3D	0xBD
17	Point display unit designation command		0x3E	

- * 3. The pallet movement command is only valid for pallet 0.
- * 4, 5. The movement methods on the jog movement command and inching movement command differ according to the point units that were specified.
- * 6. The point teaching command uses different point units according to the point units that were specified.
- * 8. If no axis is specified, the absolute reset command is executed on all axes (main robot + sub robot) in either case of command code 0x31 or 0xB1.
- * 9. If no axis is specified, the return-to-origin command is executed on all axes (main robot + sub robot) in either case of command code 0x32 or 0xB2.
- * 17. The point display unit designation command is for use on the controller.

18.4 IO command description

18.4.1 MOVE command

Execute this command group to move the robot to an absolute position.

18.4.1.1 PTP designation

This command moves the robot to a target position in PTP motion by specifying the point number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x01
		For sub robot	0x81
DI3()	Point number		0xpppp
DI4()			

pppp: Specify the point number in 16 bits.
Specified range: 0 (=0x0000) to 9999(=0x270F)

18.4.1.2 Linear interpolation

This command moves the robot to a target position by linear interpolation by specifying the point number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x03
		For sub robot	0x83
DI3()	Point number		0xpppp
DI4()			

pppp: Specify the point number in 16 bits.
Specified range: 0 (=0x0000) to 9999(=0x270F)

CAUTION

- In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified.
- The point number setting range is 0 to 255 when there is no DI4().

NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

CAUTION

- In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified.
- The point number setting range is 0 to 255 when there is no DI4().
- When moving the robot by linear interpolation to a point where a hand system flag is specified, make sure that the same hand system is used at the current position and target position. If the same hand system is not used, then an error will occur and robot movement will not be allowed.

NOTE

- In the MOVE command, linear interpolation (0x83) for sub robot is only available on the following software versions.
 - RCX14x Ver. 8.64 onwards
 - RCX22x Ver. 9.11 onwards
- When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

18.4.2 MOVEI command

Execute this command group to move the robot to a relative position.

18.4.2.1 PTP designation

This command moves the robot a specified distance in PTP motion by specifying the point number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x09
		For sub robot	0x89
DI3() DI4()	Point number		0xpppp

pppp: Specify the point number in 16 bits.
Specified range: 0 (=0x0000) to 9999(=0x270F)

NOTE

- If the MOVEI command is interrupted and then re-executed in the controller versions shown below, the resumed motion can occur either to the original target position, or to a new target position referenced to the current position, in accordance with the "MOVEI/DRIVEI start position" setting selected at the other parameters. For details, refer to the controller user's manual.
 - RCX14x Ver. 8.66 onwards
 - RCX22x Ver. 9.13 onwards
- The other parameters default "MOVEI/DRIVEI start position" setting is Keep (motion to the original target position when MOVEI is interrupted and then re-executed).
- In controller versions prior to those shown above, motion occurs to a new target position referenced to the current position when MOVEI is interrupted and then re-executed.

CAUTION

- In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified.
- The point number setting range is 0 to 255 when there is no DI4().

NOTE

When a SCARA robot is used and a hand system flag is set for the point data you specify, that hand system has priority over the current arm type.

18.4.3 Pallet movement command

Execute this command group to move the robot to a position with respect to pallet 0.

18.4.3.1 PTP designation

This command moves the robot to a target position in PTP motion by specifying the work position number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x18
		For sub robot	0x98
DI3() DI4()	Work position number		0xwww

www : Specify the work position number in 16 bits.
Specified range: 1 (=0x0001) to 32767 (=0x7FFF)

CAUTION

The work position number setting range is 0 to 255 when there is no DI4().

18.4.4 Jog movement command

This command moves the robot in jog mode while in MANUAL mode. This command is only valid in MANUAL mode.

This command is linked with the controller point display units. The robot axis moves in PTP motion when display units are in pulses, and moves by linear interpolation on Cartesian coordinates when units are in millimeters.

Jog speed is determined by the manual movement speed.

To stop the jog movement command, set the dedicated input interlock signal to OFF. After checking that jog movement has stopped, set the interlock signal back to ON.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x20
		For sub robot	0xA0
DI3()	Axis to move and direction	bit 0	Axis 1
		bit 1	Axis 2
		bit 2	Axis 3
		bit 3	Axis 4
		bit 6-bit 4	(0:Fixed)
		bit 7	Direction
DI4()	Not used		0x00

tt: Specify the axis to move in 0 to 3 bits. Only one axis can be specified.

d: Specify the movement direction in 1 bit.

Value	Meaning
0	+ direction
1	- direction

18.4.5 Inching movement command

Execute this command to move the robot by inching in MANUAL mode. Inching movement distance is linked to the manual movement speed. The inching command can only be executed in MANUAL mode.

This command is linked with the controller's point display unit system. So when display units are in pulses, the axis moves a certain number of pulses at the manual speed setting. When display units are in millimeters, the axis moves on Cartesian coordinates by linear interpolation at the manual speed setting divided by 100.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x24
		For sub robot	0xA4
DI3()	Axis to move and direction	bit 0	Axis 1
		bit 1	Axis 2
		bit 2	Axis 3
		bit 3	Axis 4
		bit 6-bit 4	(0:Fixed)
		bit 7	Direction
DI4()	Not used		0x00

tt: Specify the axis to move in 0 to 3 bits. Only one axis can be specified.

d: Specify the movement direction in 1 bit.

Value	Meaning
0	+ direction
1	- direction

CAUTION

- In controllers whose software version is earlier than 8.28, point numbers from 0 to 4000 can be specified.
- The point number setting range is 0 to 255 when there is no DI4().

18.4.6 Point teaching command

Execute this command to teach the current robot position to the specified point number. Point data units of this command are linked to the controller's point display unit system.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x28
		For sub robot	0xA8
DI3()	Point number		0xpppp
DI4()			

pppp: Specify the point number in 16 bits.
Specified range: 0 (=0x0000) to 9999(=0x270F)

18.4.7 Absolute reset movement command

When absolute reset of the specified axis uses the mark method, this command moves the axis to the nearest position where absolute reset can be executed.

Positions capable of absolute reset are located at every 1/4 rotation of the motor.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x30
		For sub robot	0xB0
DI3()	Axis to move and direction	bit 0	Axis 1
		bit 1	Axis 2
		bit 2	Axis 3
		bit 3	Axis 4
		bit 6-bit 4	(0:Fixed)
		bit 7	Direction
DI4()	Not used		0x00

tt : Specify the axis to move in 0 to 3 bits. Only one axis can be specified.

d : Specify the movement direction in 1 bit.

Value	Meaning
0	+ direction
1	- direction

18.4.8 Absolute reset command

This command executes absolute reset of the specified axis.

When absolute reset of the specified axis uses the mark method, the axis must be at a position where absolute reset can be executed.

If no axis is specified, the absolute reset command is executed on all axes (main robot + sub robot) in either case of command code 0x31 or 0xB1. However, this command cannot be executed if return-to-origin is not yet complete on the axis using the mark method. In this case, perform absolute reset individually on each axis.

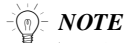
Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x31
		For sub robot	0xB1
DI3()	Specify the axis	bit 0	Axis 1
		bit 1	Axis 2
		bit 2	Axis 3
		bit 3	Axis 4
		bit 7-bit 4	(0:Fixed)
DI4()	Not used		0x00

tt : Specify the axis to perform absolute reset in 0 to 3 bits.

Only one axis can be specified.

If no axis is specified then absolute reset is performed on all axes (main robot + sub robot).

**NOTE**

An axis can be specified with the return-to-origin command from software version 8.45 onwards. On earlier version controllers, return-to-origin is performed on all axes and command code 0x32 can only be used.

18.4.9 Return-to-origin command

This command executes return-to-origin on a specified axis.

When this command is executed on an incremental mode axis, that axis returns to its origin. When executed on a semi-absolute mode axis, an absolute search is performed on that axis.

If no axis is specified (DI3() is 0), this command is executed on all axes (main robot + sub robot) in either case of 0x32 or 0xB2.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x32
		For sub robot	0xB2
DI3()	Specify the axis	bit 0	Axis 1
		bit 1	Axis 2
		bit 2	Axis 3
		bit 3	Axis 4
		bit 7-bit 4	(0:Fixed)
DI4()	Not used		0x00

tt : Specify the axis to perform return-to-origin in 0 to 3 bits.

Only one axis can be specified.

If no axis is specified then this command is executed on all axes (main robot + sub robot).

18.4.10 Servo command

Execute this command group to operate the robot servos.

Operation	Contents
Servo ON	Execute this command to turn on the servo of a specified axis. The motor power must be turned on when specifying the axis. All controller servos are turned on if no axis is specified.
Servo OFF	Execute this command to turn off the servo of a specified axis. All controller servos are turned off if no axis is specified.
Servo Free	Execute this command to turn off the mechanical brake and dynamic brake after turning off the servo of a specified axis. Servo OFF and Free are repeated when this command is consecutively executed. All controller servos will be free if no axis is specified.
Power ON	Execute this command to turn on the motor power. No axis can be specified.

Command

DI port	Contents		Value
DI2()	Command code	Servo ON	For main robot
			For sub robot
			0x34
		Servo OFF	For main robot
			For sub robot
			0xB4
DI3()	Specify the axis	Servo Free	For main robot
			For sub robot
			0x35
			0xB5
		Power ON	All controller servos
DI4()	Not used	0x36	
		0xB6	
		0x37	
		0x00	
		0x00	

tt : Specify the axis to move in 0 to 3 bits. All controller servos are processed if no axis is specified. No axis can be specified when executing Power ON.

18.4.11 Manual movement speed change command

Execute this command to change the manual movement speed in MANUAL mode.
This command can only be executed in MANUAL mode.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x38
		For sub robot	0xB8
DI3()	Specified speed		0xss
DI4()	Not used		0x00

ss : Specify the manual movement speed in 8 bits.
Specified range: 1 (=0x01) to 100 (=0x64)

18.4.12 Auto movement speed change command

Execute this command to change the auto movement speed in AUTO mode.
This command can only be executed in AUTO mode.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x39
		For sub robot	0xB9
DI3()	Specified speed		0xss
DI4()	Not used		0x00

ss : Specify the auto movement speed in 8 bits.
Specified range: 1 (=0x01) to 100 (=0x64)

18.4.13 Program speed change command

Execute this command to change the program speed in AUTO mode.
The program speed changed with this command is reset to 100% when the program is reset or changed.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x3A
		For sub robot	0xBA
DI3()	Specified speed		0xss
DI4()	Not used		0x00

ss : Specify the program speed in 8 bits.
Specified range: 1 (=0x01) to 100 (=0x64)

18.4.14 Shift designation change command

Execute this command to change the selected shift to a specified shift number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x3B
		For sub robot	0xBB
DI3()	Specified shift number		0xss
DI4()	Not used		0x00

ss : Specify the shift number in 8 bits.
Specified range: 0 (=0x00) to 9 (0x09)

18.4.15 Hand designation change command

Execute this command to change the selected hand to a specified hand number.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x3C
		For sub robot	0xBC
DI3()	Specified hand number		0xss
DI4()	Not used		0x00

ss : Specify the hand number in 8 bits.
Specified range for main robot : 0 (=0x00) to 3 (0x03)
Specified range for sub robot : 4 (=0x04) to 7 (0x07)

18.4.16 Arm designation change command

Execute this command to change the arm designation status.

Command

DI port	Contents		Value
DI2()	Command code	For main robot	0x3D
		For sub robot	0xBD
DI3()	Status of specified arm		0xss
DI4()	Not used		0x00

ss : Specify the arm designation status in 8 bits.

Value	Meaning
0x00	Right-handed system
0x01	Left-handed system

18.4.17 Point display unit designation command

Execute this command to change the point display unit.
This command is for the controller.

Command

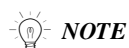
DI port	Contents		Value
DI2()	Command code	For controller	0x3E
DI3()	Display units for specified point		0xss
DI4()	Not used		0x00

ss : Specify the point display unit in 8 bits.

Value	Meaning
0x00	Pulse units
0x01	Millimeter units

A. Reserved word list

ABSADJ	DEFPOS	IRET	PADDR	SOW
ABSADJ2	DEGRAD	JTOXY	PATH	SPEED
ABSRESET	DELAY	JTOXY2	PC	SPEED2
ABSRESET2	DI	JOG	PCM	SQR
ABSRPOS	DIM	JOG2	PDEF	START
ABSRPOS2	DIR	LANGUAGE	PLN	STEP
ABSRST	DIST	LEFT	PLT	STOP
ABOVE	DO	LEFTY	PMOVE	STOPON
ABS	DPM	LEFTY2	PMOVE2	STR
ACC	DRIVE	LEN	PRINT	SUB
ACCEL	DRIVE2	LET	PROGRAM	SUSPEND
ACCEL2	DRIVEI	LOCA	PGM	SWI
ACCESS	DRIVEI2	LOCB	PNT	SYS
ALL	DS	LOCF	POS	SYSTEM
AND	DSPEED	LOCR	PPNT	TAN
ARCH	ELSE	LOCX	PRM	TASK
ARCH2	ELSEIF	LOCY	PTP	TASKS
ARM	EMGRST	LOCZ	PWR	TCOUNTER
ARM2	EMG	LOG	RADDEG	TEACH
ARMTYPE	END	LOOP	READ	TEACH2
ARMTYPE2	ENDIF	LSHIFT	REF	THEN
ARY	EOF	MANUAL	REN	TIME
ASPEED	ERA	MCHREF	RELESE	TIMER
ASPEED2	ERL	MCHREF2	REM	TO
ATN	ERR	MEM	REMOTE	TOLE
ATN2	ERROR	MID	RESET	TOLE2
ATTR	ETH	MIRROR	RESTART	TORQUE
AUTO	EXELVL	MOD	RESUME	TORQUE2
AXWGHT	EXIT	MOVE	RETURN	TRQTIME
AXWGHT2	EXITTASK	MOVE2	RIGHT	TRQTIME2
BELOW	FDD	MOVEI	RIGHTY	TRQSTS
BIN	FLIP	MOVEI2	RIGHTY2	TRQSTS2
BIT	FN	MRF	RO	TSKMON
BREAK	FOR	MSG	ROTATE	UNIT
BYTE	FREE	MSGCLR	RSHIFT	UNTIL
CALL	FUNCTION	MSPEED	RUN	VAL
CASE	GASP	MSPEED2	RW	VAR
CHANGE	GEN	NAME	S	VER
CHANGE2	GO	NEXT	SELECT	VEL
CHGPRI	GOHOME	NONFLIP	SEND	WAIT
CHGTSK	GOSUB	NOT	SEQUENCE	WEIGHT
CHR	GOTO	OFF	SERVO	WEIGHT2
CMU	HALT	OFFLINE	SERVO2	WEND
CMU1	HAND	ON	SET	WHERE
CONFIG	HAND2	ONLINE	SFT	WHERE2
CONT	HND	OR	SGI	WHILE
COO	HEX	ORD	SGR	WHRXY
COPY	HOME	ORGORD	SHARED	WHRXY2
COS	HOLD	ORGORD2	SHIFT	WORD
CUT	IF	ORGRTN	SHIFT2	WRITE
DATE	IN	ORGRTN2	SI	XOR
DEC	INCH	ORIGIN	SID	XY
DECEL	INCH2	OUT	SIN	XYTOJ
DECEL2	INIT	OUTPOS	SIW	XYTOJ2
DECLARE	INPUT	OUTPOS2	SKIP	YZ
DEF	INT	P	SO	ZX
DEFIO	INTEGER	PASS	SOD	_SYSFLG



NOTE

The above words are reserved for robot language and cannot be used as identifiers (variables, etc.).

B. Robot language lists

How to see table

- The "Cond." column in each table indicates the conditions that allow the robot language commands to be executed. The numbers shown in the "Cond." column correspond to the following 6 conditions.
 - Condition 1: Commands that can be executed with both direct commands and on-line commands.
 - Condition 2: In addition to condition 1, commands that execute task 1 (main task) only.
 - Condition 3: In addition to condition 1, commands containing operands that cannot be executed with direct commands and on-line commands.
 - Condition 4: Same as condition 5, except that STOPON option cannot be executed with direct commands and online commands.
 - Condition 5: In addition to condition 1, commands that are executed after positioning is complete.
 - Condition 6: Commands that cannot be executed with direct commands and on-line commands.
- The numbers shown in the "Ref. page" column indicate the page numbers in this manual.

■ General commands

Language	Type	Function	Cond.	Ref. page
		Format		
DECLARE	Command	Declares that a label or sub-procedure is in an external program.	6	44
		DECLARE <label>[,<label>...]		
		DECLARE SUB <label>[(<dummy argument>[,<dummy argument>...])]		
DEF FN	Command	Defines a function that is available to the user.	6	46
		DEF FN<name>[%] [(<dummy argument>[,<dummy argument>...])] =<function definition expression>		
		!		
DIM	Command	Declares the name of an array variable and the number of elements.	6	48
		DIM <array definition>[,<array definition>,...]		
EXIT FOR	Command	Terminates a FOR statement to NEXT statement loop.	6	66
		EXIT FOR		
FOR to NEXT	Command	Controls repetitive operations. Repeatedly executes the FOR statement to NEXT statement until the specified value is reached.	6	69
		FOR <control variable>=<expression 1> TO <expression 2> [STEP <expression 3>]		
		: NEXT [<control variable>]		
GOSUB to RETURN	Command	Jumps to a subroutine with the label specified by a GOSUB statement and executes the subroutine. Then, with RETURN in the subroutine being executed, jumps to the next line of the GOSUB statement.	6	70
		GOSUB <label>		
		GO SUB : <label>: : RETURN		
GOTO	Command	Unconditionally jumps to the line specified by a label.	6	71
		GOTO <label>		
		GO TO		
HALT	Command	Stops a program and resets it.	6	72
		HALT [<expression>]		
		<character string>		
HOLD	Command	Pauses a program.	6	81
		HOLD [<expression>]		
		<character string>		

Language	Type	Function	Cond.	Ref. page
		Format		
IF	Command	Allows control flow to branch according to conditions.	6	82
		IF <conditional expression> THEN <label 1> [ELSE <label 2>] <statement 1> <statement 2> IF <conditional expression 1> THEN : [ELSEIF <conditional expression 2> THEN :] [ELSE :] ENDIF		
LET	Command	Executes a specified assignment statement.	1	85
		[LET] <arithmetic variable> =<expression> <point element variable> <shift element variable> <parallel output variable> <internal output variable> <arm lock output variable> <timer output variable> <serial output variable> [LET] <character string variable> =<character string expression>		
ON to GOSUB	Command	Jumps to a subroutine with each label specified by a GOSUB statement according to conditions and executes the subroutine. Then, with RETURN in the subroutine being executed, jumps to the next line of the GOSUB statement.	6	115
		ON <expression> GOSUB <label 1>[,<label 2>...] GO SUB		
ON to GOTO	Command	Jumps to each line specified by a label according to conditions.	6	114
		ON <expression> GOTO <label 1>[,<label 2>...] GO TO		
REM	Command	All characters that follow REM or an apostrophe (') are viewed as comments.	6	144
		REM <character string> '		
SELECT CASE to END SELECT	Command	Allows control flow to branch according to conditions. Executes a block (bunch of commands) that meets conditions.	6	151
		SELECT [CASE] <expression> CASE <expression list 1> [command block 1] [CASE <expression list 2> [command block 2]] : [CASE ELSE [command block n]] END SELECT		
SWI	Command	Switches the currently executed program to a specified program, and executes from the first line after compiling.	2	167
		SWI <program name>		
WHILE to WEND	Command	Controls repetitive operations. Repeats the same operation as long as the conditions are met.	6	178
		WHILE <expression> : WEND		
Label statement	Command	Defines "labels" in program lines. Labels must begin with an asterisk (*).	6	179
		*<label>:		

■ Robot operation

Language	Type	Function	Cond.	Ref. page
		Format		
ABSRST	Command	Performs return-to-origin along robot absolute motor axes. ABSRST	5	30
DRIVE	Command	Performs an absolute movement of each axis in the main group. Only PTP movement on specified axes is possible. Command options allow conditional stop by I/O or variables during arm movement and maximum torque limit. DRIVE (<expression 1>, <expression 2>) [, (<expression 1>, <expression 2> ...)] [option] <point expression> <point expression>	5	50
DRIVE2	Command	Performs an absolute movement of each axis in the sub group. Only PTP movement on specified axes is possible. Command options allow conditional stop by I/O or variables during arm movement and maximum torque limit. DRIVE2 (<expression 1>, <expression 2>) [, (<expression 1>, <expression 2> ...)] [option] <point expression> <point expression>	5	55
DRIVEI	Command	Performs a relative movement of each axis in the main group. Only PTP movement on specified axes is possible. Command options allow conditional stop by I/O or variables during arm movement. DRIVEI (<expression 1>, <expression 2>) [, (<expression 1>, <expression 2> ...)] [option] <point expression> <point expression>	5	60
DRIVEI2	Command	Performs a relative movement of each axis in the sub group. Only PTP movement on specified axes is possible. Command options allow conditional stop by I/O or variables during arm movement. DRIVEI2 (<expression 1>, <expression 2>) [, (<expression 1>, <expression 2> ...)] [option] <point expression> <point expression>	5	63
MOVE	Command	Performs an absolute movement of the main robot axes. Three movements of PTP, linear interpolation and circular interpolation are possible. Command options allow arch motion, conditional stop by I/O or variables during arm movement, and circular interpolation so that a movement projected on a specified coordinate plane is circular. MOVE PTP , <point definition>[, option[, option]...] P L C	4	89
MOVE2	Command	Performs an absolute movement of the sub robot axes. Three movements of PTP, linear interpolation and circular interpolation are possible. Command options allow arch motion, conditional stop by I/O or variables during arm movement, and circular interpolation so that a movement projected on a specified coordinate plane is circular. MOVE2 PTP , <point definition>[, option[, option]...] P L C	4	98
MOVEI	Command	Performs a relative movement of the main robot axes. Only PTP movement is possible. MOVEI PTP , <point definition>[, option[, option]...] P	5	107
MOVEI2	Command	Performs a relative movement of the sub robot axes. Only PTP movement is possible. MOVEI2 PTP , <point definition>[, option[, option]...] P	5	110
ORIGIN	Command	Performs return-to-origin on an incremental mode axis or absolute search on a semi-absolute mode axis. ORIGIN	5	119

Language	Type	Function			Cond.	Ref. page
		Format				
PMOVE	Command	Performs a pallet movement of the main robot axes. Only PTP movement is possible. Command options allow arch motion and conditional stop by I/O or variables during arm movement.			5	134
		PMOVE(pallet definition number, pallet position number)[.option[.option]...]				
PMOVE2	Command	Performs a pallet movement of the sub robot axes. Only PTP movement is possible. Command options allow arch motion and conditional stop by I/O or variables during arm movement.			5	138
		PMOVE2(pallet definition number, pallet position number)[.option[.option]...]				
SERVO	Command	Controls the servo ON/OFF of the specified axes in the main group or all axes (in main group and sub group).			5	154
		SERVO	ON OFF FREE PWR	[(<expression>)]		
SERVO2	Command	Controls the servo ON/OFF of the specified axes in the sub group or all axes (in main group and sub group).			5	155
		SERVO2	ON OFF FREE PWR	[(<expression>)]		

■ I/O control

Language	Type	Function	Cond.	Ref. page
		Format		
DELAY	Command	Waits for the specified length of time (ms) .	6	47
		DELAY <expression>		
DO	Command	Outputs the specified value to the DO ports.	1	49
		[LET] DOm([b, ..., b]) =<expression> DO(mb, ..., mb)		
LO	Command	Outputs the specified value to the LO port to prohibit axis movement or permit axis movement.	1	87
		[LET] LO0([b, ..., b]) =<expression> LO(0b, ..., 0b)		
MO	Command	Outputs the specified value to the MO ports. The origin sensor status and axis hold status can also be checked with an MO statement.	1	88
		[LET] MOm([b, ..., b]) =<expression> MO(mb, ..., mb)		
OUT	Command	Turns ON the bits of the specified output ports and the command statement ends. Then, turns OFF the bits of the specified ports after automatic operation has continued for the specified time.	6	120
		OUT DOm([b, ..., b]) [, <expression> DO(mb, ..., mb) MOm([b, ..., b]) MO(mb, ..., mb) SOM([b, ..., b]) SO(mb, ..., mb) LO0([b, ..., b]) LO(0b, ..., 0b) TO0([b, ..., b]) TO(0b, ..., 0b)		
RESET	Command	Turns OFF the bits of the specified output ports. Also clears the system flag variables and/or 10ms counter variables.	1	145
		RESET DOm([b, ..., b]) DO(mb, ..., mb) MOm([b, ..., b]) MO(mb, ..., mb) TO0([b, ..., b]) TO(0b, ..., 0b) LO0([b, ..., b]) LO(0b, ..., 0b) SOM([b, ..., b]) SO(mb, ..., mb)		
SET	Command	Turns ON the bits of the specified output ports.	3	156
		SET DOm([b, ..., b]) [, <expression> DO(mb, ..., mb) MOm([b, ..., b]) MO(mb, ..., mb) TO0([b, ..., b]) TO(0b, ..., 0b) LO0([b, ..., b]) LO(0b, ..., 0b) SOM([b, ..., b]) SO(mb, ..., mb)		

Language	Type	Function	Cond.	Ref. page
		Format		
SO	Command	Outputs the specified value to the SO port.	1	160
		[LET] SOm([b, ..., b]) =<expression> SO(mb, ..., mb)		
TO	Command	Outputs the specified value to the TO port.	1	168
		[LET] TO0([b, ..., b]) =<expression> TO(0b, ..., 0b)		
WAIT	Command	① Waits until the condition in DI/DO conditional expression are met. The time-out period can also be specified.	6	175
		② Waits until positioning on the robot axes is complete (within the tolerance range).		
		① WAIT <DI/DO conditional expression> [, <expression>] ② WAIT ARM [(<expression 1>)] ARM2 [(<expression 2>)]		

■ Coordinate control

Language	Type	Function	Cond.	Ref. page
		Format		
CHANGE	Command	Switches the hand of the main robot.	5	73
		CHANGE Hn		
CHANGE2	Command	Switches the hand of the sub robot.	5	77
		CHANGE2 Hn		
HAND	Command	Defines the hand of the main robot.	5	73
		HAND Hn=<1st parameter> <2nd parameter> <3rd parameter> [R]		
HAND2	Command	Defines the hand of the sub robot.	5	77
		HAND2 Hn=<1st parameter> <2nd parameter> <3rd parameter> [R]		
RIGHTY / LEFTY	Command	Selects whether the main robot will be "right-handed" or "left-handed" when moving to a point specified on a Cartesian coordinate system.	5	148
		RIGHTY LEFTY		
RIGHTY2 / LEFTY2	Command	Selects whether the sub robot will be "right-handed" or "left-handed" when moving to a point specified on a Cartesian coordinate system.	5	149
		RIGHTY2 LEFTY2		
SHIFT	Command	Sets the shift coordinates for the main robot by using the shift data specified by a shift variable.	5	158
		SHIFT <shift variable>		
SHIFT2	Command	Sets the shift coordinates for the sub robot by using the shift data specified by a shift variable.	5	159
		SHIFT2 <shift variable>		

■ Condition change

Language	Type	Function	Cond.	Ref. page
		Format		
ACCEL	Command	Changes the acceleration coefficient parameter of the main group.	5	31
		ACCEL <expression>		
		ACCEL(<expression 1>)=<expression 2>		
ACCEL2	Command	Changes the acceleration coefficient parameter of the sub group.	5	32
		ACCEL2 <expression>		
		ACCEL2(<expression 1>)=<expression 2>		
ARCH	Command	Changes the arch position parameter of the main group.	5	33
		ARCH <expression>		
		ARCH(<expression 1>)=<expression 2>		
ARCH2	Command	Changes the arch position parameter of the sub group.	5	34
		ARCH2 <expression>		
		ARCH2(<expression 1>)=<expression 2>		
ASPEED	Command	Changes the automatic movement speed of the main group.	5	35
		ASPEED <expression>		
ASPEED2	Command	Changes the automatic movement speed of the sub group.	5	36
		ASPEED2 <expression>		
AXWGHT	Command	Changes the axis tip weight parameter of the main group.	5	37
		AXWGHT(<expression 1>)=<expression 2>		
AXWGHT2	Command	Changes the axis tip weight parameter of the sub group.	5	38
		AXWGHT2(<expression 1>)=<expression 2>		
DECEL	Command	Changes the deceleration rate parameter of the main group.	5	42
		DECEL <expression>		
		DECEL(<expression 1>)=<expression 2>		
DECEL2	Command	Changes the deceleration rate parameter of the sub group.	5	43
		DECEL2 <expression>		
		DECEL2(<expression 1>)=<expression 2>		
ORGORD	Command	Sets the axis sequence parameter to perform return-to-origin and absolute search in the main group.	5	117
		ORGORD <expression>		
ORGORD2	Command	Sets the axis sequence parameter to perform return-to-origin and absolute search in the sub group.	5	118
		ORGORD2 <expression>		
OUTPOS	Command	Changes the OUT position parameter of the main group.	5	121
		OUTPOS <expression>		
		OUTPOS(<expression 1>)=<expression 2>		
OUTPOS2	Command	Changes the OUT position parameter of the sub group.	5	122
		OUTPOS2 <expression>		
		OUTPOS2(<expression 1>)=<expression 2>		
PDEF	Command	Defines the pallet used to execute a pallet movement command.	1	133
		PDEF(pallet definition number)=nx,ny[,nz]		
SPEED	Command	Changes the program speed for the main group.	5	161
		SPEED <expression>		
SPEED2	Command	Changes the program speed for the sub group.	5	162
		SPEED2 <expression>		
TOLE	Command	Changes the tolerance parameter of the main group.	5	169
		TOLE <expression>		
		TOLE(<expression 1>)=<expression 2>		
TOLE2	Command	Changes the tolerance parameter of the sub group.	5	170
		TOLE2 <expression>		
		TOLE2(<expression 1>)=<expression 2>		

Language	Type	Function	Cond.	Ref. page
		Format		
WEIGHT	Command	Changes the tip weight parameter of the main robot. WEIGHT <expression>	5	176
WEIGHT2	Command	Changes the tip weight parameter of the sub robot. WEIGHT2 <expression>	5	177

■ Parameter reference

Language	Type	Function	Cond.	Ref. page
		Format		
ACCEL	Arithmetic function	Gives the acceleration coefficient parameter of the specified main group axis. ACCEL(<expression>)	–	181
ACCEL2	Arithmetic function	Gives the acceleration coefficient parameter of the specified sub group axis. ACCEL2(<expression>)	–	181
ARCH	Arithmetic function	Gives the arch position parameter of the specified main group axis. ARCH(<expression>)	–	181
ARCH2	Arithmetic function	Gives the arch position parameter of the specified sub group axis. ARCH2(<expression>)	–	182
AXWGHT	Arithmetic function	Gives the axis tip weight parameter of the specified main group axis. AXWGHT(<expression>)	–	184
AXWGHT2	Arithmetic function	Gives the axis tip weight parameter of the specified sub group axis. AXWGHT2(<expression>)	–	184
DECEL	Arithmetic function	Gives the deceleration rate parameter of the specified main group axis. DECEL(<expression>)	–	185
DECEL2	Arithmetic function	Gives the deceleration rate parameter of the specified sub group axis. DECEL2(<expression>)	–	185
ORGORD	Arithmetic function	Gives the axis sequence parameter to perform return-to-origin and absolute search in the main group. ORGORD	–	188
ORGORD2	Arithmetic function	Gives the axis sequence parameter to perform return-to-origin and absolute search in the sub group. ORGORD2	–	189
OUTPOS	Arithmetic function	Gives the OUT position parameter of the specified main group axis. OUTPOS(<expression>)	–	189
OUTPOS2	Arithmetic function	Gives the OUT position parameter of the specified sub group axis. OUTPOS2(<expression>)	–	189
TOLE	Arithmetic function	Gives the tolerance parameter of the specified main group axis. TOLE(<expression>)	–	192
TOLE2	Arithmetic function	Gives the tolerance parameter of the specified sub group axis. TOLE2(<expression>)	–	193
WEIGHT	Arithmetic function	Gives the tip weight parameter of the main robot. WEIGHT	–	197
WEIGHT2	Arithmetic function	Gives the tip weight parameter of the sub robot. WEIGHT2	–	197

■ Condition reference

Language	Type	Function	Cond.	Ref. page
		Format		
ABSRPOS	Arithmetic function	Gives the machine reference of the specified main group axis. This function can only be used for axes whose return-to-origin method is set to mark method.	–	180
		ABSRPOS (<expression>)		
ABSRPOS2	Arithmetic function	Gives the machine reference of the specified sub group axis. This function can only be used for axes whose return-to-origin method is set to mark method.	–	180
		ABSRPOS2 (<expression>)		
ARMCND	Arithmetic function	Gives the current arm status of the SCARA robot in the main group.	–	182
		ARMCND		
ARMCND2	Arithmetic function	Gives the current arm status of the SCARA robot in the sub group.	–	182
		ARMCND2		
ARMTYPE	Arithmetic function	Gives the hand system currently selected for the SCARA robot in the main group.	–	183
		ARMTYPE		
ARMTYPE2	Arithmetic function	Gives the hand system currently selected for the SCARA robot in the sub group.	–	183
		ARMTYPE2		
MCHREF	Arithmetic function	Gives the return-to-origin or absolute-search machine reference of the specified main group axis. This function can only be used for axes whose return-to-origin method is set to the sensor or stroke end (torque detection) method.	–	187
		MCHREF(<expression>)		
MCHREF2	Arithmetic function	Gives the return-to-origin or absolute-search machine reference of the specified sub group axis. This function can only be used for axes whose return-to-origin method is set to the sensor or stroke end (torque detection) method.	–	188
		MCHREF2(<expression>)		

■ Communication control

Language	Type	Function	Cond.	Ref. page
		Format		
ONLINE / OFFLINE	Command	Changes the communication mode of the specified port, also clears the transmission or receive buffer or initializes the communication port, depending on the specified port.	1	116
		ONLINE [ETH] CMU OFFLINE [ETH] CMU		
SEND	Command	Sends the read file data into a write file.	1	152
		SEND <read file> TO <write file> SEND CMU TO ~ (Inputs data from RS-232C port) SEND ~ TO CMU (Outputs data from RS-232C port)		

■ Screen control

Language	Type	Function	Cond.	Ref. page
		Format		
PRINT	Command	Displays the value of specified variable on the MPB/RPB screen.	1	142
		PRINT [<expression>[, <expression>...][,] ; ;]		

■ Key control

Language	Type	Function	Cond.	Ref. page
		Format		
INPUT	Command	Assigns a value to the variable specified from the MPB/RPB.	1	84
		INPUT [<prompt statement> ;] <variable> [, <variable> , ...] ; , <point variable> <point variable> ; <shift variable> <shift variable>		

■ Procedure

Language	Type	Function	Cond.	Ref. page
		Format		
CALL	Command	Calls up sub-procedures defined by the SUB and END SUB statements.	6	39
		CALL <label>[(<actual argument>[,<actual argument>...])]		
EXIT SUB	Command	Terminates the sub-procedure defined by the SUB and END SUB statements.	6	67
		EXIT SUB		
SHARED	Command	Does not permit variables declared with a program written outside a sub-procedure (SUB to END SUB) to be passed on as dummy arguments, but allows them to be referred to with a sub-procedure.	6	157
		SHARED <variable>[()],[<variable>[()]]...		
SUB to END SUB	Command	Defines a sub-procedure.	6	164
		SUB <label>[(<dummy argument> [,<dummy argument>...])] : END SUB		

■ Task control

Language	Type	Function	Cond.	Ref. page
		Format		
CHGPRI	Command	Changes the priority of the specified task.	6	40
		CHGPRI T _n ,p		
CUT	Command	Terminates a task currently being executed or temporarily stopped.	6	41
		CUT T _n		
EXIT TASK	Command	Terminates its own task currently being executed.	6	68
		EXIT TASK		
RESTART	Command	Restarts a task that is temporarily stopped.	6	146
		RESTART T _n		
START	Command	Sets the task number and priority of the specified task and starts that task.	6	163
		START <label>,T _n ,[p]		
SUSPEND	Command	Temporarily stops another task being executed.	6	166
		SUSPEND T _n		

■ Error control

Language	Type	Function	Cond.	Ref. page
		Format		
ON ERROR GOTO	Command	If an error occurs during program execution, this command allows the program to jump to the error processing routine specified by the label without stopping the program, or stops the program and displays the error message.	6	113
		ON ERROR GOTO <label> 0		
RESUME	Command	Resumes the program execution after recovery from an error. This command is used in the error processing routine.	6	147
		RESUME [NEXT] <label>		
ERL	Arithmetic function	Gives the line number where an error occurred.	–	186
		ERL		
ERR	Arithmetic function	Gives the error code number when an error occurred.	–	186
		ERR		

■ PATH control

Language	Type	Function	Cond.	Ref. page
		Format		
PATH	Command	Sets the PATH motion on the main robot axis. Command options allow circular interpolation so that a movement projected on a specified coordinate plane is circular and also specifying port output at a specified position during robot movement.	6	125
		PATH L , <point definition> [, option [, option...]] C		
PATH END	Command	Terminates the path setting for PATH motion.	6	129
		PATH END		
PATH SET	Command	Starts the path setting for PATH motion.	6	130
		PATH SET [<point definition>]		
PATH START	Command	Starts the PATH motion.	6	132
		PATH START		

■ Torque control

Language	Type	Function	Cond.	Ref. page
		Format		
DRIVE (with torque limit option)	Command	Executes an absolute movement command on each axis in the main group. Only PTP movement on a specified axis is possible. Attaching a torque limit setting option as a command option allows movement under torque control.	5	50
		DRIVE (<expression 1>, <expression 2>), T=<expression 1> <point expression> (<expression 1> [, <expression 2>])		
DRIVE2 (with torque limit option)	Command	Executes an absolute movement command on each axis in the sub group. Only PTP movement on a specified axis is possible. Attaching a torque limit setting option as a command option allows movement under torque control.	5	55
		DRIVE2 (<expression 1>, <expression 2>), T=<expression 1> <point expression> (<expression 1> [, <expression 2>])		
TORQUE	Command	Changes the maximum torque instruction for the specified main group axis.	5	171
		TORQUE (<expression 1>) = <expression 2>		
TORQUE2	Command	Changes the maximum torque instruction for the specified sub group axis.	5	172
		TORQUE2 (<expression 1>) = <expression 2>		
TRQTIME	Command	Sets the current limit time-out period on the specified main group axis when using a torque limit setting option in the DRIVE statement.	1	173
		TRQTIME (<expression 1>) = <expression 2>		
TRQTIME2	Command	Sets the current limit time-out period on the specified sub group axis when using a torque limit setting option in the DRIVE2 statement.	1	174
		TRQTIME2 (<expression 1>) = <expression 2>		
TORQUE	Arithmetic function	Gives the torque setting on the specified main group axis.	–	193
		TORQUE(<expression>)		
TORQUE2	Arithmetic function	Gives the torque setting on the specified sub group axis.	–	193
		TORQUE2(<expression >)		
TRQSTS	Arithmetic function	Gives the status at the time that the DRIVE statement with a torque limit option was executed on the specified main group axis.	–	194
		TRQSTS (<expression>)		
TRQSTS2	Arithmetic function	Gives the status at the time that the DRIVE2 statement with a torque limit option was executed on the specified sub group axis.	–	195
		TRQSTS2 (<expression>)		
TRQTIME	Arithmetic function	Gives the torque control time-out period on the specified main group axis.	–	196
		TRQTIME (<expression>)		
TRQTIME2	Arithmetic function	Gives the torque control time-out period on the specified sub group axis.	–	196
		TRQTIME2 (<expression>)		

■ Point arithmetic operation

Language	Type	Function	Cond.	Ref. page
		Format		
LET	Command	Executes the specified assignment statement.	1	85
		[LET] <point variable>=<point expression>		
Pn	Command	Defines a point in a program.	1	143
		Pn=x y z r a b		
JTOXY	Point function	Converts the joint coordinate data (unit: pulses) at the specified point into Cartesian coordinate data (unit: mm, deg) in the main group.	–	202
		JTOXY(<point expression>)		
JTOXY2	Point function	Converts the joint coordinate data (unit: pulses) at the specified point into Cartesian coordinate data (unit: mm, deg) in the sub group.	–	202
		JTOXY2(<point expression>)		
LOCx	Point element variable	Handles point data for each axis and hand system flag.	–	14
		LOCx(<point expression>)		
PPNT	Point function	Creates point data specified by a pallet definition number and pallet position number.	–	202
		PPNT(pallet definition number, pallet position number)		
WHERE	Point function	Reads out the current position of the main group robot arm in joint coordinates (pulses).	–	203
		WHERE		
WHERE2	Point function	Reads out the current position of the sub group robot arm in joint coordinates (pulses).	–	203
		WHERE2		
WHRXY	Point function	Reads out the current XY coordinate position (mm or degrees) of the main group arm.	–	203
		WHRXY		
WHRXY2	Point function	Reads out the current XY coordinate position (mm or degrees) of the sub group arm.	–	204
		WHRXY2		
XYTOJ	Point function	Converts the Cartesian coordinate data (unit: mm, deg.) in the main group at the specified point into joint coordinate data (unit: pulses).	–	204
		XYTOJ(<point expression>)		
XYTOJ2	Point function	Converts the Cartesian coordinate data (unit: mm, deg.) in the sub group at the specified point into joint coordinate data (unit: pulses).	–	204
		XYTOJ2(<point expression>)		

■ Shift arithmetic operation

Language	Type	Function	Cond.	Ref. page
		Format		
LET	Command	Executes the specified assignment statement.	5	85
		[LET] <shift variable>=<shift expression>		
Sn	Command	Defines shift coordinates in a program.	5	150
		Sn=x y z r		
LOCx	Shift element variable	Handles shift data for each element.	–	15
		LOCx(<shift expression>)		

■ Arithmetic functions

Language	Type	Function	Cond.	Ref. page
		Format		
ABS	Arithmetic function	Gives the absolute value of the specified value.	–	180
		ABS(<expression>)		
ATN	Arithmetic function	Gives the arctangent of the specified value.	–	183
		ATN(<expression>)		
COS	Arithmetic function	Gives the cosine of the specified value.	–	184
		COS(<expression>)		
DEGRAD	Arithmetic function	Converts the specified value into radians.	–	185
		DEGRAD(<expression>)		
DIST	Arithmetic function	Gives the distance between the specified 2 points.	–	186
		DIST(<point expression 1>,<point expression 2>)		
INT	Arithmetic function	Gives an integer of the specified value by truncating all decimal fractions.	–	186
		INT(<expression>)		
LSHIFT	Arithmetic function	Shifts the specified value to the left by the specified number of bits.	–	187
		LSHIFT(<expression 1>,<expression 2>)		
RADDEG	Arithmetic function	Converts the specified value into degrees.	–	190
		RADDEG(<expression>)		
RSHIFT	Arithmetic function	Shifts the specified value to the right by the specified number of bits.	–	190
		RSHIFT(<expression 1>,<expression 2>)		
SIN	Arithmetic function	Gives the sine of the specified value.	–	190
		SIN(<expression>)		
SQR	Arithmetic function	Gives the square root of the specified value.	–	191
		SQR(<expression>)		
TAN	Arithmetic function	Gives the tangent of the specified value.	–	191
		TAN(<expression>)		
TCOUNTER	Arithmetic function	Outputs count-up values at 10ms intervals starting from the point when the TCOUNTER variable is reset.	–	198
		TCOUNTER		
_SYSFLG	Arithmetic function	This is used as a flag that monitors the axis status according to the value specified by the _SYSFLG variable. This should not be used in normal operation.	–	198

■ Character string operation

Language	Type	Function	Cond.	Ref. page
		Format		
CHR\$	Character string function	Gives the character having a specified character code.	–	199
		CHR\$(<expression>)		
LEFT\$	Character string function	Extracts a character string with the specified digits from the left end of the specified character string.	–	200
		LEFT\$(<character string expression>, <expression>)		
LEN	Arithmetic function	Acquires the length (number of bytes) of the specified character string.	–	187
		LEN(<character string expression>)		
MID\$	Character string function	Extracts a character string with the desired length (number of characters) from the specified character string.	–	200
		MID\$(<character string expression>, <expression 1>[, <expression 2>])		
ORD	Arithmetic function	Acquires the character code of the first character of the specified character string.	–	188
		ORD(<character string expression>)		
RIGHT\$	Character string function	Extracts a character string with the specified digits from the right end of the specified character string.	–	201
		RIGHT\$(<character string expression>, <expression>)		
STR\$	Character string function	Converts the specified value into a character string.	–	201
		STR\$(<expression>)		
VAL	Arithmetic function	Converts the numeric value of the specified character string into an actual numeric value.	–	197
		VAL(<character string expression>)		

■ Date and time control

Language	Type	Function	Cond.	Ref. page
		Format		
DATE\$	Character string function	Gives the date in the "yy/mm/dd" format character string.	–	199
		DATE\$		
TIME\$	Character string function	Gives the time in the format "hh:mm:ss" format character string.	–	201
		TIME\$		
TIMER	Arithmetic function	Gives the current time in seconds, counting from 12:00 midnight as "0:00".	–	192
		TIMER		

■ Variables

Language	Type	Function	Cond.	Ref. page
		Format		
Array variable	Array variable	—	—	12
		<Variable name>[%] (<expression>,[<expression>,<expression>]) ! \$		
Pn	Point variable	Specifies a point number.	—	14
		Pnnnn P["<expression>"]		
Sn	Shift variable	Specifies a shift number.	—	14
		Sn S["<expression>"]		
DI	Parallel input variable	Indicates the parallel input signal status.	—	15
		DIm([b,...,b]) DI(mb,...,mb)		
DO	Parallel output variable	Specifies the parallel output signal and indicates the output status.	—	16
		DOm([b,...,b]) DO(mb,...,mb)		
MO	Internal output variable	This variable is used for signal exchange with a sequence program.	—	17
		MOm([b,...,b]) MO(mb,...,mb)		
TO	Timer output variable	This variable is used for the timer function of a sequence program.	—	19
		TOm([b,...,b]) TO(mb,...,mb)		
LO	Arm lock output variable	This variable is used to prohibit axis movement.	—	18
		LOm([b,...,b]) LO(mb,...,mb)		
SI	Serial input variable	Indicates the serial input signal status.	—	20
		SIIm([b,...,b]) SI(mb,...,mb)		
SO	Serial output variable	Specifies the serial output signal and indicates the output status.	—	21
		SOM([b,...,b]) SO(mb,...,mb)		
SIW	Serial word input variable	Indicates the status of serial input word information.	—	22
		SIW(m)		
SID	Serial double-word input variable	Indicates the status of serial input word information in a double word.	—	22
		SID(m)		
SOW	Serial word output variable	Outputs to the serial output word information and indicate the output status.	—	23
		SOW(m)		
SOD	Serial double-word output variable	Outputs the status of serial output word information in a double word and indicate the output status.	—	23
		SOD(m)		

* For point element variable (LOCx), see "Point arithmetic operation".

* For shift element variable (LOCx), see "Shift arithmetic operation".

Revision record

Manual version	Issue date	Description
Ver. 2.08	Aug. 2009	CAUTION about the execution of the commands following the PTP or interpolation movement was added.
Ver. 2.09	Dec. 2009	The address and other information were added to the front cover.
Ver. 2.10	Aug. 2010	ETH and CMU options were added to the ONLINE/OFFLINE statements.
Ver. 2.11	Sep. 2010	Clerical error corrections.

Programming Manual

YAMAHA **RCX** series
Robot Controller

Sep. 2010

Ver. 2.11

This manual is based on Ver. 3.12 of Japanese manual.

© YAMAHA MOTOR CO., LTD.

IM Operations

All rights reserved. No part of this publication may be reproduced in any form without the permission of YAMAHA MOTOR CO., LTD. Information furnished by YAMAHA in this manual is believed to be reliable. However, no responsibility is assumed for possible inaccuracies or omissions. If you find any part unclear in this manual, please contact YAMAHA or YAMAHA sales representatives.